ESCUELA POLITÉCNICA
SUPERIOR DE CÓRDOBA
Universidad de Córdoba

TRABAJO FIN DE GRADO
Grado en Ingeniería Informática

Diseño y Desarrollo de un Sistema IoT para la Gestión Inteligente del Riego
Design and Development of an IoT System for Smart Irrigation Management

Autor: Miguel Ángel Corredor Ferrer
Directores: Ezequiel Gómez Herruzo
Rafael M. Navarro Cerrillo

septiembre, 2025

UNIVERSIDAD
DE
CÓRDOBA

Resumen

El Trabajo de Fin de Grado que se presenta tiene como objetivo el diseño, desarrollo e implementación de un sistema IoT para la gestión inteligente de riego. Este proyecto es una combinación de una parte hardware y otra parte software. En la parte software, se ha diseñado y desarrollado una aplicación web que permite a los usuarios gestionar sus cultivos y dispositivos, ofreciendo una plataforma donde el usuario podrá gestionar y automatizar el riego de sus cultivos. En la parte del hardware, se ha implementado un sistema físico compuesto por un sensor ambiental que recopila datos de temperatura y humedad ambiental y un sensor de humedad del suelo que recopila datos sobre la humedad del suelo, así como de un actuador encargado de ejecutar la bomba de riego cuando sea necesario. La activación de dicho actuador se realiza en función de ciertas condiciones de riego y de las decisiones tomadas por el usuario a través de la interfaz web. En conjunto, se ha desarrollado una solución que facilita la gestión remota, eficiente y personalizada del riego, adaptable tanto a cultivos de grandes explotaciones agrícolas como a huertos urbanos o jardines particulares.

El sistema ofrece tres modos de operación: manual, programado y automático, permitiendo al usuario la gestión inteligente de riego. La aplicación web facilita la gestión de dispositivos IoT, la gestión de cultivos y el monitoreo en tiempo real de las lectura de los sensores.

En definitiva, el trabajo intenta dar una solución para la gestión inteligente del riego de cultivos y ha sido denominado como Vivantia. El sistema desarrollado pretende contribuir a la modernización de las prácticas agrícolas a pequeña y gran escala mediante la gestión inteligente de riego. Gestionar inteligentemente un riego implica que en base a unas condiciones ambientales y del suelo determinadas, capturadas por los sensores, se tomen decisiones de activación/desactivación de riego en tiempo real, existiendo una base de datos que indica las condiciones ideales para cada cultivo. Una de las principales características de este sistema es que puede operar de forma remota y durante tiempo prolongados, siendo innecesaria la intervención del humano en el riego.

Título: Diseño y Desarrollo de un Sistema IoT para la gestión inteligente de Riego

Keywords: Internet de las Cosas, agricultura inteligente, automatización del riego, sensores ambientales, eficiencia de riego, monitoreo remoto, Edge computing, Fog Computing, Cloud computing.

Abstract

This Final Degree Project presents the design, development, and implementation of an IoT system for intelligent irrigation management. The solution combines hardware and software. On the software side, a web application was designed and developed to let users manage their crops and devices, providing a platform to supervise and automate irrigation. On the hardware side, the system integrates an environmental sensor that collects air temperature and humidity and a soil–moisture sensor that measures soil water content, together with an actuator that operates the irrigation pump when required. The actuator is triggered according to defined irrigation conditions and user decisions taken through the web interface. Overall, the system enables remote, efficient, and customizable irrigation management, suitable for large agricultural holdings as well as urban gardens and private yards.

The system supports three operating modes—manual, scheduled, and automatic—allowing intelligent irrigation control. The web application streamlines IoT device and crop management and offers real-time monitoring of sensor readings.

Named *Vivantia*, the project aims to modernize agricultural practices at both small and large scales through intelligent irrigation. Decisions to start/stop irrigation are made in real time based on environmental and soil conditions captured by sensors, with a database that stores ideal thresholds per crop. A key feature of the system is its ability to operate remotely for extended periods, minimizing the need for human intervention.

Title: Design and Development of an IoT System for Intelligent Irrigation Management

Keywords: Internet of Things, smart agriculture, irrigation automation, environmental sensors, irrigation efficiency, remote monitoring, Edge computing, Fog computing, Cloud computing.

Índice general

Resumen **I**

Abstract **III**

Índice de figuras **XI**

Índice de tablas **XIX**

Lista de Acrónimos **XXIII**

1. Introducción **1**

2. Objetivos **5**
2.1. Objetivos técnicos 5
2.1.1. Objectivos de Arquitectura y aplicación web 7
2.1.2. Objectivos funcionales 8
2.2. Objetivos de seguridad 9

3. Antecedentes **11**
3.1. Antecentes técnicos 11
3.1.1. The Things Network 11
3.1.2. LoRaWAN y LoRa 12
3.1.3. MQTT 13
3.1.4. Micro válvula RS (RS Pump) 14
3.1.5. Arduino MKR WAN 1310 15
3.1.6. Sensor de temperatura y humedad ambiental 16
3.1.7. Sensor de humedad capacitivo 17
3.1.8. FIWARE 18
3.1.9. Puertas de enlace LoRaWAN 19
3.1.10. Relé HUAREW 5V 20
3.1.11. Edge Computing 21

3.1.12. Fog Computing . . . 23
3.1.13. Cloud Computing . . . 23
3.2. Iniciativas previas . . . 24
3.2.1. Sistema de Riego BRESSER Smart Garden Smart Home . . . 25
3.2.2. Programador de riego Solem LoRa de 9V, 4 estaciones de riego 25
3.2.3. Comparativa de Vivantia frente a otras soluciones de riego inteligente . . . 26

4. Análisis del problema **29**
4.1. Definición detallada del problema . . . 29
4.1.1. Definición del problema real . . . 29
4.1.2. Definición del problema técnico . . . 30
4.2. Contexto operativo del problema . . . 36
4.2.1. Entorno . . . 36
4.2.2. Ciclo de mantenimiento . . . 39
4.2.3. Competencia . . . 39
4.2.4. Aspecto externo . . . 39
4.2.5. Estandarización . . . 40
4.2.6. Calidad y fiabilidad . . . 40
4.2.7. Programación temporal . . . 41
4.2.8. Pruebas . . . 41
4.2.9. Seguridad . . . 42
4.3. Recursos . . . 42
4.3.1. Recursos Humanos . . . 42
4.3.2. Recursos Hardware y Software . . . 43
4.4. Especificación de requisitos . . . 44
4.4.1. Requisitos funcionales . . . 45
4.4.2. Requisitos no funcionales . . . 48
4.5. Casos de uso . . . 49
4.5.1. Sistema Completo . . . 51
4.5.2. Especificación de los casos de uso . . . 58

5. Diseño **99**
5.1. Restricciones . . . 99
5.1.1. Factores Dato . . . 100
5.1.2. Factores estratégicos . . . 101
5.2. Sistema completo . . . 113
5.3. Diagramas de actividades . . . 118
5.3.1. Envío de datos de sensores y postprocesamiento (Uplink) . . . 118
5.3.2. Riego Manual . . . 119

5.3.3. Riego Automático . . . 121
5.3.4. Riego programado . . . 122
5.3.5. Registro . . . 123
5.3.6. Confirmar cuenta . . . 124
5.3.7. Iniciar Sesión . . . 125
5.3.8. Emitir downlink TTN(ON/OFF) . . . 125
5.3.9. Pausar/Reanudar riego . . . 126
5.4. Diagramas de Secuencia . . . 127
5.4.1. Login del usuario . . . 127
5.4.2. Captura y almacenamiento de lecturas de sensores . . . 128
5.4.3. Emitir comando ON/OFF desde la web para control de riego . 129
5.4.4. Configurar y ejecutar riego en modo programado . . . 131
5.4.5. Activación por condiciones en el modo automático . . . 131
5.4.6. Registro del usuario . . . 132
5.4.7. Confirmación de la cuenta . . . 133
5.5. Diseño del Módulo 1: Edge Computing - Captación de datos y acción sobre el actuador . . . 134
5.5.1. Componentes hardware . . . 135
5.5.2. Proceso de operación . . . 136
5.5.3. Formato de datos . . . 138
5.6. Diseño del módulo 2: Fog Computing - Recolección y preprocesamiento 139
5.6.1. Arquitectura general . . . 140
5.6.2. Proceso de decodificación . . . 141
5.6.3. Integración webhook . . . 143
5.7. Diseño del Módulo 3.1: Cloud Computing - Recepción y postprocesamiento . . . 144
5.7.1. Arquitectura REST API . . . 146
5.7.2. Patrón de diseño - MVC . . . 153
5.7.3. Estructura del servidor web . . . 153
5.8. Diseño del Módulo 3.2: Cloud Computing - Almacenamiento de datos 154
5.8.1. Modelo conceptual entidad-relación . . . 154
5.8.2. Modelo entidad-relación . . . 159
5.9. Diseño del módulo 4: Interfaz web - Visualización . . . 166
5.9.1. Diagrama de navegación . . . 167
5.9.2. Bocetos de las páginas . . . 167
5.9.3. Paleta de colores . . . 175

6. Desarrollo e implementación **177**
6.1. Desarrollo del Módulo 1: Edge computing - Captación de datos y acción sobre el actuador . . . 177

6.1.1. OTAA . . . 179
6.1.2. Uplinks . . . 180
6.1.3. Downlink . . . 182
6.2. Desarrollo del Módulo 2: Fog computing - Recolección y preprocesamiento . . . 182
6.2.1. Configurar payload formatter de uplink . . . 183
6.2.2. Configurar payload formatter de downlink . . . 184
6.2.3. Configuración de webhook HTTP . . . 184
6.2.4. Generar API key para downlink . . . 186
6.3. Desarrollo del Módulo 3.1: Cloud computing - Recepción y postprocesamiento . . . 188
6.3.1. Arquitectura del servidor web . . . 188
6.3.2. Estructura de directorios del servidor web . . . 191
6.3.3. Servicios Auxiliares . . . 194
6.3.4. Implementación de la funcionalidad uplink . . . 196
6.3.5. Implementación de la funcionalidad downlink . . . 197
6.4. Desarrollo del Módulo 3.2: Cloud Computing - Almacenamiento de datos . . . 199
6.5. Desarrollo del Módulo 4: Interfaz web - Visualización . . . 203
6.5.1. Arquitectura y tecnologías del cliente . . . 204
6.5.2. Principios de diseño adoptados . . . 204
6.5.3. Estilización . . . 206
6.6. Pruebas . . . 207
6.6.1. Pruebas Unitarias . . . 207
6.6.2. Pruebas de Integración . . . 249

7. Conclusión **261**
7.1. Futuras mejoras . . . 262

Bibliografía **263**

A. Manual de Usuario **269**
A.1. Introducción . . . 269
A.2. Requisitos del sistema . . . 269
A.2.1. Requisitos Técnicos . . . 269
A.2.2. Requisitos Finales . . . 272
A.3. Instalación y configuración para el usuario técnico . . . 274
A.3.1. Módulo 1: Edge Computing - Captación de datos y acción sobre el actuador . . . 274
A.3.2. Módulo 2: Fog Computing - Recolección y preprocesamiento . 276
A.3.3. Módulo 3.1: Cloud Computing - Recepción y postprocesamiento 286

A.3.4. Módulo 3.2: Cloud Computing - Almacenamiento de datos . . 291
A.3.5. Módulo 4: Interfaz Web - Visualización 295
A.4. Desinstalación Usuario Técnico 298
A.4.1. Módulo 1: Edge Computing - Captación de datos y acción sobre el actuador 298
A.4.2. Módulo 2: Fog Computing - Recolección y preprocesamiento . 298
A.4.3. Módulo 3.1: Cloud Computing - Recepción y postprocesamiento 300
A.4.4. Módulo 3.2: Cloud Computing - Almacenamiento de datos . . 301
A.4.5. Módulo 4: Interfaz Web - Visualización 301
A.5. Guía Usuario Final 301
A.5.1. Preparación y verificación del sistema 301
A.6. Uso del módulo 1 para la gestión de riego y captación de datos de los sensores 302
A.6.1. Estado inicial del módulo 1 302
A.6.2. Realización de mediciones de temperatura y humedad con los sensores 303
A.7. Uso de la interfaz web 304
A.7.1. Acceder a demo del sistema (Visitante) 304
A.7.2. Acceso a gestión de usuarios (Administrador) 305
A.7.3. Acceso a gestión de cultivos (Administrador) 305
A.7.4. Acceso a gestión de dispositivos (Administrador) 306
A.7.5. Acceso a logs del sistema (Administrador) 306
A.7.6. Acceso y autenticación 307
A.7.7. Gestión de dispositivos 311
A.7.8. Gestión de cultivos 312
A.7.9. Modos de riego 313
A.7.10. Gestión de alertas 316
A.8. Resolución de Problemas Comunes 317
A.8.1. Problemas con el módulo 1 317
A.8.2. Problemas con la interfaz web 318

B. Manual de código **319**
B.1. Repositorio github del servidor web 319
B.1.1. Estructura de directorios del backend 320
B.2. Repositorio github del cliente o frontend 320
B.2.1. Estructura de directorios del frontend 321
B.3. Repositorio github del microcontrolador 322

Índice de figuras

1.1. Logo de Vivantia 3

3.1. Infraestructura LoRaWAN 12
3.2. Jerarquía de capas de la arquitectura LoRaWAN 13
3.3. Micropump - RS PRO 14
3.4. Arduino MKR WAN 1310 15
3.5. DHT11 16
3.6. Capacitive Soil Moisture Sensor v1.2 18
3.7. Capacitive Soil Moisture Sensor v1.2 19
3.8. Relé HUAREW 5V 21
3.9. Representación Edge Computing 22
3.10. Arquitectura distribuida Edge-Fog-Cloud Computing 22
3.11. Fog Computing 23
3.12. Cloud Computing 24
3.13. Sistema de Riego Inteligente BRESSER 25
3.14. Programador de riego Solem LoRA 26

4.1. Diagrama de sistema completo 33
4.2. Casos de uso del sistema completo 52
4.3. Casos de uso del Microcontrolador 52
4.4. Casos de uso del sistema completo 53
4.5. Casos de uso del Visitante 53
4.6. Casos de uso del Administrador 54
4.7. Casos de uso del Usuario Verificado - Dashboard y Gestión de Alertas 55
4.8. Casos de uso del Usuario Verificado - Autenticación y perfil 55
4.9. Casos de uso del Usuario Verificado - Dispositivos 56
4.10. Casos de uso del Usuario Verificado - Riego 56

5.1. Representación RX1 Delay y RX1 Delay en Clase A 102
5.2. Situaciones posibles para los mensajes downlink en Clase A 103
5.3. Implementación de Clase C 105

5.4. Comparación Wifi vs 4G/5G vs LoRa . . . 108
5.5. Diagrama de la arquitectura del sistema . . . 114
5.6. Diagrama de componentes del sistema completo . . . 115
5.7. Diagrama de flujo de uplink . . . 116
5.8. Diagrama de flujo de downlink . . . 117
5.9. Diagrama de actividad de envió de datos de sensores . . . 118
5.10. Diagrama de actividad de envió de datos de sensores . . . 119
5.11. Diagrama de actividad del riego manual . . . 120
5.12. Diagrama de actividad del riego automático . . . 122
5.13. Diagrama de actividad del riego programado . . . 123
5.14. Diagrama de actividad de registro . . . 124
5.15. Diagrama de actividad de confirmar cuenta . . . 124
5.16. Diagrama de actividad de login . . . 125
5.17. Diagrama de actividad de emitir downlink . . . 126
5.18. Diagrama de actividad de pausar/reanudar riego . . . 126
5.19. Diagrama de secuencia de login . . . 128
5.20. Diagrama de secuencia de captura y almacenamiento de lecturas de sensores . . . 129
5.21. Diagrama de secuencia de emitir comando ON/OFF desde la web para control de riego . . . 130
5.22. Diagrama de secuencia de configurar y ejecutar riego en modo programado . . . 131
5.23. Diagrama de secuencia de activación por condiciones en el modo automático . . . 132
5.24. Diagrama de secuencia de registro . . . 133
5.25. Diagrama de secuencia de confirmación de cuenta . . . 134
5.26. Esquema eléctrico del circuito - Realizado en fritzing . . . 135
5.27. Diagrama de flujo del código del dispositivo IoT . . . 137
5.28. Arquitectura General del Módulo 2: Recolección y Preprocesamiento . 141
5.29. Arquitectura General del módulo 3 - Cloud Computing . . . 145
5.30. Modelo-vista-controlador . . . 153
5.31. Estructura de directorios del servidor web del sistema Vivantia IoT . 154
5.32. Modelo conceptual entidad-relación . . . 155
5.33. Modelo entidad-relación . . . 159
5.34. Diagrama de navegación . . . 167
5.35. Layout principal del sistema . . . 168
5.36. Boceto de la dashboard . . . 169
5.37. Boceto de la página de los cultivos . . . 170
5.38. Boceto de la página de dispositivos . . . 171
5.39. Boceto de la página de modos de riego . . . 172

5.40. Bocetos de los modos de riego . . . 172
5.41. Boceto de la página de Alertas del usuario y Logs del administrador . 173
5.42. bocetos de las páginas de gestión del administrador . . . 174
5.43. Boceto de la página de perfil de usuario . . . 175

6.1. Módulo 1 implementado y listo . . . 178
6.2. Añadir formato y dominio del servidor web en el webhook . . . 185
6.3. Filtros configurados en el webhook . . . 186
6.4. Configuración del endpoint HTTP . . . 186
6.5. Configuración de la API Key . . . 187
6.6. Arquitectura cliente-servidor . . . 188
6.7. Esquema de la base de datos del sistema . . . 200
6.8. Estructura de directorios del cliente del sistema completo . . . 205
6.9. Correcto funcionamiento de los sensores . . . 207
6.10. Caso de prueba PU-M1-01 . . . 207
6.11. Caso de prueba PU-M1-02 . . . 209
6.12. Correcto funcionamiento del envío de uplink . . . 210
6.13. Caso de prueba PU-M1-03 . . . 210
6.14. Correcto funcionamiento de recibdo de downlink . . . 211
6.15. Caso de prueba PU-M1-04 . . . 211
6.16. Relé energizado . . . 211
6.17. Caso de prueba PU-M1-04 - Relé . . . 211
6.18. Correcta recepción y decodificación de los datos de los sensores . . . 212
6.19. Caso de prueba PU-M2-01 . . . 212
6.20. Correcto funcionamiento del webhook HTTP . . . 213
6.21. Caso de prueba PU-M2-02 . . . 213
6.22. Correcto funcionamiento del downlink . . . 214
6.23. Caso de prueba PU-M2-03 . . . 214
6.24. Caso de prueba PU-M3.1-01 . . . 217
6.25. Caso de prueba PU-M3.1-02 . . . 217
6.26. Caso de prueba PU-M3.1-03 . . . 217
6.27. Caso de prueba PU-M3.1-04 . . . 218
6.28. Caso de prueba PU-M3.1-05 . . . 218
6.29. Caso de prueba PU-M3.1-06 . . . 218
6.30. Caso de prueba PU-M3.1-07 . . . 219
6.31. Caso de prueba PU-M3.1-08 . . . 219
6.32. Caso de prueba PU-M3.1-09 . . . 219
6.33. Caso de prueba PU-M3.1-10 . . . 220
6.34. Caso de prueba PU-M3.1-11 . . . 221
6.35. Caso de prueba PU-M3.1-12 . . . 221

6.36. Caso de prueba PU-M3.1-13 221
6.37. Caso de prueba PU-M3.1-14 222
6.38. Caso de prueba PU-M3.1-15 222
6.39. Listado de dispositivos mostrado con éxito - Código de estado 200 . . 224
6.40. Caso de prueba PU-M3.1-16 224
6.41. Caso de prueba PU-M3.1-17 224
6.42. Caso de prueba PU-M3.1-18 224
6.43. Caso de prueba PU-M3.1-19 225
6.44. Caso de prueba PU-M3.1-20 225
6.45. Caso de prueba PU-M3.1-20 225
6.46. Dispositivo activado con éxito - Código de estado 200 226
6.47. Caso de prueba PU-M3.1-21 226
6.48. Caso de prueba PU-M3.1-22 226
6.49. Caso de prueba PU-M3.1-23 226
6.50. Caso de prueba PU-M3.1-24 227
6.51. Todos los cultivos listados con éxito - Código de estado 200 228
6.52. Caso de prueba PU-M3.1-25 228
6.53. Caso de prueba PU-M3.1-26 228
6.54. Caso de prueba PU-M3.1-27 228
6.55. Caso de prueba PU-M3.1-28 229
6.56. Caso de prueba PU-M3.1-29 229
6.57. Caso de prueba PU-M3.1-30 229
6.58. Caso de prueba PU-M3.1-31 229
6.59. Caso de prueba PU-M3.1-32 230
6.60. Obtener todas las categorías de los cultivos - Código de estado 200 . . 230
6.61. Caso de prueba PU-M3.1-33 230
6.62. Lecturas de sensores obtenidas exitosamente - Código de estado 200 . 231
6.63. Caso de prueba PU-M3.1-34 231
6.64. Caso de prueba PU-M3.1-35 231
6.65. Caso de prueba PU-M3.1-36 231
6.66. Caso de prueba PU-M3.1-39 232
6.67. Alertas listadas con éxito - Código de estado 200 233
6.68. Caso de prueba PU-M3.1-38 233
6.69. Caso de prueba PU-M3.1-39 233
6.70. Caso de prueba PU-M3.1-40 233
6.71. Caso de prueba PU-M3.1-41 234
6.72. Alertas verificadas con éxito - Código de estado 200 234
6.73. Caso de prueba PU-M3.1-42 234
6.74. Caso de prueba PU-M3.1-43 234
6.75. Todas las alertas del sistema listadas con éxito - Código de estado 200 234

6.76. Caso de prueba PU-M3.1-44 234
6.77. Configuración creada con éxito - Código de estado 200 236
6.78. Caso de prueba PU-M3.1-45 236
6.79. Caso de prueba PU-M3.1-46 236
6.80. Configuración creada con éxito - Código de estado 200 236
6.81. Caso de prueba PU-M3.1-47 236
6.82. Riego activado correctamente - Código de estado 200 237
6.83. Caso de prueba PU-M3.1-48 237
6.84. Riego desactivado correctamente - Código de estado 200 237
6.85. Caso de prueba PU-M3.1-49 237
6.86. Riego desactivado correctamente - Código de estado 200 237
6.87. Caso de prueba PU-M3.1-50 237
6.88. Riego pausado correctamente - Código de estado 200 238
6.89. Caso de prueba PU-M3.1-51 238
6.90. Riego reanudado correctamente - Código de estado 200 238
6.91. Caso de prueba PU-M3.1-52 238
6.92. Riego completado correctamente - Código de estado 200 238
6.93. Caso de prueba PU-M3.1-53 238
6.94. Correcta implementación de la tabla users 240
6.95. Caso de prueba PU-M3.2-01 240
6.96. Correcta implementación de la tabla crops 241
6.97. Caso de prueba PU-M3.2-02 241
6.98. Correcta implementación de la tabla devices 242
6.99. Caso de prueba PU-M3.2-03 242
6.100. Correcta implementación de la tabla irrigation_configs 243
6.101. Caso de prueba PU-M3.2-04 243
6.102. Correcta implementación de la tabla programmed_settings 244
6.103. Caso de prueba PU-M3.2-05 244
6.104. Correcta implementación de la tabla automatic_settings 245
6.105. Caso de prueba PU-M3.2-06 245
6.106. Correcta implementación de la tabla pump_activations 246
6.107. Caso de prueba PU-M3.2-07 246
6.108. Correcta implementación de la tabla sensor_readings 247
6.109. Caso de prueba PU-M3.2-08 247
6.110. Correcta implementación de la tabla sql_alerts 248
6.111. Caso de prueba PU-M3.2-09 248
6.112. Caso de prueba PU-M1/M2-01 251
6.113. Caso de prueba PU-M2/M3-01 253
6.114. Caso de prueba PU-M2/M3-02 254

6.115 Preprocesamiento y guardado en la base de datos de los datos de los sensores . . . 256
6.116 Caso de prueba PU-M3.1/M3.2-01 . . . 256
6.117 Caso de prueba PU-M3/M4-01 . . . 257
6.118 Caso de prueba PU-M3/M4-02 . . . 259

A.1. Visual Studio Code Download . . . 275
A.2. Preparación del entorno en VScode para la programación del microcontrolador . . . 275
A.3. Descargar código del microcontrolador . . . 276
A.4. Registro en TTN . . . 277
A.5. Login y acceso al servidor de red de TTN . . . 278
A.6. Crear una aplicación . . . 279
A.7. Dar de alta un dispositivo IoT . . . 280
A.8. Registrar gateway en TTN . . . 281
A.9. Etiqueta de información específica del gateway . . . 281
A.10. Botón de reset . . . 282
A.11. Botón Setup del gateway . . . 282
A.12. Pantalla configuración Wi-Fi - Minihub . . . 283
A.13. Configurar payload formatters Uplinks/Downlinks . . . 284
A.14. Dar de alta un webhook . . . 285
A.15. Descarga del código del servidor web . . . 287
A.16. Proceso de descarga de Node.js . . . 287
A.17. Instalar dependencias del servidor web . . . 288
A.18. Despliegue del servidor web en localhost . . . 288
A.19. Proceso de registro en ngrok . . . 289
A.20. Asignar dominio temporal al servidor web . . . 289
A.21. Asignación del dominio temporal . . . 290
A.22. Proceso de descarga de PostgreSQL . . . 291
A.23. Pantalla de inicio de PgAdmin . . . 292
A.24. Proceso de creación del server en PostgreSQL . . . 292
A.25. Crear base de datos en el server . . . 293
A.26. Proceso de creación del server en PostgreSQL . . . 293
A.27. Crear base de datos en el server . . . 294
A.28. Crear base de datos en el server . . . 294
A.29. Descargar del código del frontend . . . 296
A.30. Instalar dependencias del servidor web . . . 296
A.31. Frontend ejecutado . . . 297
A.32. Pantalla renderizada en el frontend . . . 297
A.33. Desinstalar dispositivos dados de alta . . . 298

A.34.Desinstalarel gateway de TTN 299
A.35.Desinstalar payloads formatters 299
A.36.Desinstalar webhook 300
A.37.Desinstalar cuenta de usuario 300
A.38.Descargar el script para la eliminación de la base de datos y sus tablas301
A.39.Demo del sistema 304
A.40.Gestión de usuarios (Administrador) 305
A.41.Gestión de cultivos (Administrador) 306
A.42.Gestión de dispositivos (Administrador) 306
A.43.Logs del sistema (Administrador) 307
A.44.Panel de autenticación 307
A.45.Panel de autenticación - Acceso al registro 308
A.46.Panel de autenticación - Formulario de registro 308
A.47.Panel de autenticación - Revise su email 309
A.48.Email de verificación de cuenta 309
A.49.Proceso de login 310
A.50.Dashboard 311
A.51.Formulario para el registro de un dispositivo en el sistema 311
A.52.Filtrar Dispositivos y Dispositivos Registrados 312
A.53.Operaciones sobre los dispositivos registrados 312
A.54.Filtrar y añadir cultivos 313
A.55.Operaciones sobre cultivos registrados 313
A.56.Modos de riego 314
A.57.Modo de riego Manual 314
A.58.Modo de riego programado 315
A.59.Modo de riego automático 316
A.60.Alertas 316

B.1. Estructura de directorios del servidor web 320
B.2. Estructura de directorios del cliente del sistema completo 321

Índice de tablas

3.1. Comparativa de Vivantia (TFG) frente a otras soluciones de riego inteligente . . . 27

4.1. Planificación temporal del proyecto . . . 41
4.2. Requisitos Funcionales . . . 48
4.3. Requisitos no funcionales . . . 49
4.4. Casos de uso del sistema completo . . . 51
4.5. Especificación del Caso de Uso: Recepción de datos . . . 58
4.6. Especificación del Caso de Uso: Enviar datos (uplink) . . . 59
4.7. Especificación del Caso de Uso: Recibir datos (downlink) . . . 60
4.8. Especificación del Caso de Uso: Ejecutar riego . . . 61
4.9. Especificación del Caso de Uso: Recolectar Uplink . . . 62
4.10. Especificación del Caso de Uso: Preprocesar (decode/encode) . . . 63
4.11. Especificación del Caso de Uso: Reenviar a Backend . . . 64
4.12. Especificación del Caso de Uso: Recepción downlink . . . 65
4.13. Especificación del Caso de Uso: Recepción downlink . . . 66
4.14. Especificación del Caso de Uso: Recepción downlink . . . 67
4.15. Especificación del Caso de Uso: Recepción downlink . . . 68
4.16. Especificación del Caso de Uso: Almacenar datos . . . 69
4.17. Especificación del Caso de Uso: Login . . . 70
4.18. Especificación del Caso de Uso: Registro . . . 71
4.19. Especificación del Caso de Uso: Acceder Dashboard RT . . . 72
4.20. Especificación del Caso de Uso: Iniciar riego . . . 73
4.21. Especificación del Caso de Uso: Cancelar riego . . . 74
4.22. Especificación del Caso de Uso: Confirmar cuenta . . . 75
4.23. Especificación del Caso de Uso: Restablecer contraseña . . . 76
4.24. Especificación del Caso de Uso: Entrar en modo demo . . . 76
4.25. Especificación del Caso de Uso: Crear usuario . . . 77
4.26. Especificación del Caso de Uso: Listar usuarios . . . 78
4.27. Especificación del Caso de Uso: Ver usuario . . . 78

4.28. Especificación del Caso de Uso: Editar usuario 79
4.29. Especificación del Caso de Uso: Eliminar usuario 80
4.30. Especificación del Caso de Uso: Cerrar sesión 81
4.31. Especificación del Caso de Uso: Listar cultivos 81
4.32. Especificación del Caso de Uso: Crear cultivo 82
4.33. Especificación del Caso de Uso: Editar cultivo 82
4.34. Especificación del Caso de Uso: Eliminar cultivo 83
4.35. Especificación del Caso de Uso: Seleccionar/Deseleccionar cultivo . . 84
4.36. Especificación del Caso de Uso: Crear dispositivo 84
4.37. Especificación del Caso de Uso: Editar dispositivo 85
4.38. Especificación del Caso de Uso: Eliminar dispositivo 85
4.39. Especificación del Caso de Uso: Activar/Desactivar comunicación . . 86
4.40. Especificación del Caso de Uso: Listar dispositivos 87
4.41. Especificación del Caso de Uso: Marcar alerta resuelta 87
4.42. Especificación del Caso de Uso: Historial completo de alertas 88
4.43. Especificación del Caso de Uso: Ver perfil 89
4.44. Especificación del Caso de Uso: Seleccionar modo de riego 90
4.45. Especificación del Caso de Uso: Configurar modo manual 91
4.46. Especificación del Caso de Uso: Configurar modo programado 92
4.47. Especificación del Caso de Uso: Configurar modo automático 93
4.48. Especificación del Caso de Uso: Definir programación 94
4.49. Especificación del Caso de Uso: Definir umbrales (Auto) 95
4.50. Especificación del Caso de Uso: Pausar/Reanudar riego 96
4.51. Especificación del Caso de Uso: Cancelar programación completa . . . 97
4.52. Matriz de trazabilidad casos de uso - requisitos funcionales 98

5.1. Tabla API REST - Autenticación 147
5.2. Tabla API REST - Usuarios 147
5.3. Tabla API REST - Dispositivos 148
5.4. Tabla API REST - Cultivos 148
5.5. Tabla API REST - Lecturas de Sensores 149
5.6. Tabla API REST - Configuración de riego 150
5.7. Tabla API REST - Activaciones de bomba 151
5.8. Tabla API REST - Alertas 152
5.9. Tabla API REST - Alertas 152
5.10. Tabla Usuario Verificado (`users`) 160
5.11. Tabla Crops (`crops`) 161
5.12. Tabla Dispositivo (`devices`) 162
5.13. Tabla Sensor Reading (`sensor_readings`) 162
5.14. Tabla Irrigation Config (`irrigation_configs`) 163

5.15. Tabla Programmed Setting (`programmed_settings`) . . . 163
5.16. Tabla Automatic Settings (`automatic_settings`) . . . 164
5.17. Tabla Pump Activation (`pump_activations`) . . . 164
5.18. Tabla Alerta (`alerts`) . . . 165
5.19. Tipos Enumerados del Sistema . . . 165
5.20. Subtipos de Alertas (`alert_subtype`) . . . 166

6.1. Caso de prueba PU-M1-01: Verificación de inicialización de sensores . 207
6.2. Caso de prueba PU-M1-02: Verificación del proceso de autenticación OTAA . . . 208
6.3. Caso de prueba PU-M1-03: Verificación de la transmisión de datos de los sensores hacia TTN . . . 209
6.4. Caso de prueba PU-M1-04: Verificación del control de la bomba de riego mediante las órdenes de control provenientes de TTN . . . 210
6.5. Caso de prueba PU-M2-01: Verificación de la correcta decodificación del payload en bytes de los datos de temperatura y humedad ambiental, y humedad del suelo a formato JSON. Así como comprobar la correcta recepción de datos desde el dispositivo IoT. . . . 212
6.6. Caso de prueba PU-M2-02: Verificación del funcionamiento del webhook HTTP . . . 213
6.7. Caso de prueba PU-M2-03: Verificación de la recepción y envío de donwlinks hacia el servidor web . . . 214
6.8. Pruebas Unitarias - APIs de Autenticación . . . 216
6.9. Pruebas Unitarias - APIs de Gestión de Usuarios . . . 220
6.10. Pruebas Unitarias - APIs de Gestión de Dispositivos . . . 223
6.11. Pruebas Unitarias - APIs de Gestión de Cultivos . . . 227
6.12. Pruebas Unitarias - APIs de Datos de Sensores . . . 230
6.13. Pruebas Unitarias - Endpoints de API de Alertas . . . 232
6.14. Pruebas Unitarias - APIs de Control de Riego (Endpoints Principales) 235
6.15. Caso de prueba PU-M3.2-01: Verificación de la correcta implementación de la tabla users . . . 239
6.16. Caso de prueba PU-M3.2-02: Verificación de la correcta implementación de la tabla crops . . . 240
6.17. Caso de prueba PU-M3.2-03: Verificación de la correcta implementación de la tabla devices . . . 242
6.18. Caso de prueba PU-M3.2-04: Verificación de la correcta implementación de la tabla irrigation_configs . . . 243
6.19. Caso de prueba PU-M3.2-05: Verificación de la correcta implementación de la tabla programmed_settings . . . 244

6.20. Caso de prueba PU-M3.2-06: Verificación de la correcta implementación de la tabla automatic_settings . . . 245
6.21. Caso de prueba PU-M3.2-07: Verificación de la correcta implementación de la tabla pump_activations . . . 246
6.22. Caso de prueba PU-M3.2-08: Verificación de la correcta implementación de la tabla sensor_readings . . . 247
6.23. Caso de prueba PU-M3.1-09: Verificación de la correcta implementación de la tabla alerts . . . 248
6.24. Caso de prueba PU-M1/M2-01: Verificación de la correcta integración entre el Módulo 1 y Módulo 2 . . . 250
6.25. Caso de prueba PU-M2/M3-01: Verificación de la correcta integración entre el Módulo 1 y Módulo 2 - Uplink . . . 252
6.26. Caso de prueba PU-M2/M3-02: Verificación de la correcta integración entre el Módulo 1 y Módulo 2 - downlink . . . 254
6.27. Caso de prueba PU-M3.1/M3.2-01: Verificación de la correcta integración entre el Módulo 3.1 y Módulo 3.2 . . . 255
6.28. Caso de prueba PU-M3/M4-01: Verificación de la correcta integración entre el Módulo 3 y módulo 4 - Uplink . . . 257
6.29. Caso de prueba PU-M3/M4-02: Verificación de la correcta integración entre el Módulo 3 y módulo 4 - Downlink . . . 258

A.1. Problemas comunes del módulo 1 . . . 317
A.2. Problemas comunes del módulo 4 . . . 318

Acrónimos

API Application Programming Interface. 7, 19, 32, 34, 35, 37, 38, 40, 42, 65, 67, 73, 74, 109, 110, 112–114, 129, 131, 140, 143, 145–152, 154, 162, 183, 186–188, 190, 192, 193, 197–199, 204, 206, 215, 216, 220, 222, 223, 227, 230, 232, 235, 250, 254, 255, 257–259, 270, 271, 273, 285, 297, 318, VII, VIII, XIII, XX, XXI

DOM Document Object Model. 112

HTML HyperText Markup Language. 111, 192, 194

HTTP Hypertext Transfer Protocol. 7, 8, 31–33, 40, 48, 62–64, 66, 109, 110, 112, 114, 128, 140, 141, 143, 146, 183, 184, 186, 190, 191, 196, 199, 206, 212, 213, 215, 251, 252, 286, 288, 290, VIII, XIII, XXI

HTTPS Hypertext Transfer Protocol Secure. 42

IDE Integrated Development Environment. 101, 270–272, 274

IoS Internet of Services. 1

IoT Internet of Things. 1–3, 5–8, 11, 13–15, 18, 19, 21–24, 30, 31, 33–39, 42, 45, 46, 61, 62, 65, 67, 69, 73, 74, 84–87, 100–105, 107–112, 114, 117, 128, 129, 131, 137, 139–143, 145, 154, 156, 157, 161, 166, 170, 179, 181, 182, 184–186, 189–193, 196–198, 205, 208, 212–214, 247, 250, 251, 261, 272, 278–281, 283, 301–303, 310–312, 315, 318, XII, XVI, XXI

JSON JavaScript Object Notation. 37, 63, 64, 66, 108, 140–143, 146, 183, 184, 186, 193, 198, 212, 251, 252, 254, 285, 286, XXI

JWT JSON Web Tokens. 9, 36, 40, 42, 45, 48, 68, 81, 127, 190, 192–195, 271, 290

LLM Large Language Model. 262

LoRa Long Range. 2, 12, 14, 15, 19, 25, 31, 33, 39, 106–108, 113, 137, 180–182, 208, 209, 270, V, XII

LoRaWAN Long Range Wide Area Network. 6, 7, 11–15, 19, 24, 31–33, 35–42, 44, 59, 62, 100–103, 106, 107, 128, 135–137, 140, 156, 181, 187, 209, 261, 270, 278, 281, 302, 303, V, XI

MVC Model-View-Controller. 40, 145, 153, 188, VII

OTAA Over-The-Air Activation. 7, 8, 33, 34, 36, 42, 59, 62, 128, 137, 179–182, 208, 209, 283, VIII, XXI

PCB Printed Circuit Board. 262

SPA Single-Page Application. 32, 37, 111, 166, 168, 204

TTN The Things Network. 2, 7, 8, 11, 31–36, 38, 39, 42, 46–49, 51, 53, 59–67, 73, 74, 84, 100–102, 105, 108, 113–115, 117, 125, 128, 129, 131, 135, 136, 139–145, 152, 154, 156, 161, 162, 179–187, 190–192, 196–199, 208–210, 212–214, 223, 224, 250, 251, 254–259, 261, 273, 274, 276–279, 281, 283, 284, 286, 288, 298–300, 311, 318, VII, XVI, XVII, XXI

UML Unified Modeling Languague. 49

VPD Vapor Pressure Deficit. 262

WiFi Wireless Fidelity. 11–13, 25, 107, 108, 272, 274, 281, 282, 298, 302

Capítulo 1

Introducción

En los últimos años, ha ganado terreno una tecnología cuyo propósito es convertir los dispositivos electrónicos que usamos a diario en equipos "inteligentes", llamada *Internet of Things* [1]. La evolución del Internet of Things (IoT) a escala mundial está dando lugar a unan nueva manera de proveer servicios digitales que es Internet of Services (IoS) [2]. El objetivo de Internet of Things es lograr que estos dispositivos puedan comunicarse entre ellos y con los usuarios, ayudando a mejorar las tareas cotidianas del ser humano así como también mejorar procesos industriales, agrónomos y un largo etcétera. Todo esto a un consumo de energía relativamente bajo. En esencia, se trata de crear una red global de dispositivos conectados que recogen y transmiten datos en tiempo real para resolver problemas del día a día. Es por esto que el tamaño del mercado de internet de las cosas sigue creciendo constantemente, para 2024 se estimó que este fuera de USD 1,17 billones y se espera que alcance los USD 2,27 billones para 2029, creciendo a una CAGR (Tasa de crecimiento anual compuesta) del 15,12 % durante el período de pronóstico (2024-2029) [3].

Por su parte, el IoS surge de la necesidad de transformar en servicios digitales la información generada por IoT y además que estos sean accesibles a través de Internet. Mientras que IoT se centra en la capa física, en la que se da lugar la conexión entre dispositivos IoT, IoS se centra más en la capa de servicios. La capa de servicios tiene por objetivo ofrecer servicios a partir de los datos generados por los dispositivos físicos conectados a la red IoT, como por ejemplo dashboards para visualizar el histórico de lecturas de sensores de los dispositivos IoT, entre otros. Por tanto, IoS se constituye como una nueva manera de ofrecer servicios a partir tecnología IoT.

Un ejemplo de éxito de países que han adoptado Internet of Things como parte de sus economías es Singapur, donde esta tecnología mejora la eficiencia de los servicios públicos y la calidad de vida de sus ciudadanos. Por ejemplo, se han ins-

talado cientos de miles de sensores en diversas regiones del país que monitorizan de forma permanente el consumo energético, la generación de residuos y el uso del agua potable y residual, en tiempo real. Todos estos dispositivos proporcionan a los proveedores de servicios públicos, residentes, al gobierno, a las industrias y a las organizaciones civiles, información exacta sobre el uso de los recursos y servicios. De esta forma es posible identificar los focos de desperdicio, disminuir costos, hacer más eficiente la prestación de servicios, distribuir mejor los recursos, y sobre todo, crear conciencia en la población. Pero no solo Singapur, también empresas como Jonh Deere, empresa dedicada a la fabricación de maquinaria agrícola, está utilizando Internet of Things para conectar máquinas y sistemas a una infraestructura digital, lo que permite la recolección y análisis de datos en tiempo real, mejorando la gestión y sostenibilidad de la agricultura [4] [5].

En el caso de la agricultura, la gestión inteligente del agua se ha convertido en uno de los mayores desafíos a nivel global, sobre todo en zonas proclives a sequías o con pocos recursos hídricos. Se estima que solo el 2.5 % del agua terrestre es potable; y de esa parte, menos del 1 % se encuentra en ríos, lagos o acuíferos accesibles [6]. La agricultura usa más del 70 % de las extracciones mundiales de agua dulce, según el Banco Mundial [7] y la ineficiencia de riegos por inundación o aspersión tradicionales hace que hasta un 60 % se pierda por evaporación o escorrentía en climas cálidos. Especialmente en España, por ejemplo, se están imponiendo restricciones de consumo de agua en varias comunidades autónomas, como Cataluña y Andalucía, debido a la escasez. Se estima que alrededor del 79,1 % del recurso hídrico nacional se destina a la agricultura y la ganadería [8]. Ante esta situación, la adopción de tecnologías IoT se presenta como una alternativa innovadora para disminuir el consumo de agua sin sacrificar la productividad de los cultivos.

En este proyecto se plantea desarrollar un sistema de gestión de riego inteligente basado en IoT, capaz de monitorizar en tiempo real la humedad del suelo, humedad y temperatura ambiental y controlar de forma remota cuándo y en qué condiciones se va a activar o desactivar el riego de un cultivo. Para ello va a ser necesario emplear un sensor de humedad del suelo y un sensor de humedad y temperatura ambiental, mientras que el microcontrolador o dispositivo IoT se encargará de gestionar la captación de datos y comunicación a través de un gateway enlazado a The Things Network (TTN), y una micro válvula RS encargada de suministrar el agua. Todos estos elementos se integrarán en un sistema IoT que permitirá almacenar y procesar la información en la nube, así como generar órdenes de riego automático basadas en umbrales o requisitos específicos de cada cultivo.

Aparte del hardware y la comunicación inalámbrica mediante Long Range (LoRa), el proyecto incluye el desarrollo e implementación de una aplicación web que

proporciona al usuario todas las funcionalidades necesarias para gestionar el riego de sus cultivos. Así como asociar, en lo que denominamos la parte "inteligente" del sistema, una base de datos donde se especifican las necesidades ambientales y de riego de cada uno de los cultivos definidos en el sistema. Dicha aplicación permitirá al usuario registrar cultivos y dispositivos IoT, definir ejecuciones de riego y consultar en una dashboard en tiempo real los datos de los sensores. El sistema completo ha sido llamado Vivantia y el logo denominativo de la aplicación se puede ver en la Figura 1.1.

Figura 1.1: Logo de Vivantia

Con este sistema no sólo se pretende ahorrar recursos hídricos, sino que también busca contribuir a la sostenibilidad del sector agrícola, mostrando hasta dónde puede llegar la tecnología IoT aplicada al campo.

Capítulo 2

Objetivos

El principal objetivo del proyecto es desarrollar un sistema de regadío automático basado en IoT que optimice el uso de agua en cultivos o jardines, empleando un microcontrolador, conectividad inalámbrica de bajo consumo, y un servicio en la nube para el almacenamiento y la toma de decisiones, con el fin de monitorizar la humedad del suelo y controlar de manera remota la cantidad de agua suministrada por una micro válvula

2.1. Objetivos técnicos

1. **OB-01. Estudio de los fundamentos de redes IoT, así como el estudio del funcionamiento de sensores**

   a) Estudio de tecnologías relacionadas con el IoT, incluyendo microcontroladores (Arduino, ESP32) y sistemas de comunicaciones.

   b) Estudio y análisis de un sensor de humedad capacitivo en diferentes tipos de suelo.

2. **OB-02. Diseño de la arquitectura distribuida del sistema:**

   a) Definir la arquitectura que va a integrar el sensor de humedad capacitivo, el sensor de temperatura y humedad relativa, la micro válvula RS y las comunicaciones. Arquitectura Edge Computing.

   b) Definir la arquitectura del procesamiento en la niebla para retención de información y envío a sistemas de almacenamiento. Arquitectura FoG Computing.

c) Diseño de la base de datos, mecanismos de integración de información y visualización. Desarrollo de scripts para la interacción con la base de datos. Arquitectura Cloud Computing.

d) Describir el flujo de datos desde el sensor, pasando por el microcontrolador, hasta la nube. Especificando claramente la funcionalidad de cada plataforma de procesamiento.

3. **OB-03. Desarrollo del firmware para sistemas microcontroladores:**

   a) Programar el microcontrolador para la lectura del sensor de humedad y la activación de la micro válvula RS bajo ciertas condiciones.

   b) Configurar la transmisión de datos hacia la nube, así como la recepción de posibles órdenes de riego provenientes de la aplicación web.

4. **OB-04. Desarrollo de la aplicación web:**

   a) Diseño de una interfaz amigable e intuitiva, que permita mostrar en tiempo real la humedad del suelo y permita configurar parámetros de riego, elegir tipo de cultivo a regar etc...

   b) Diseño e implementación de la conectividad y la sincronización de datos con la plataforma en la nube con el objetivo de que puedan ser tratados y visualizados en la aplicación web.

5. **OB-05. Implementación de comunicación del dispositivo IoT Long Range Wide Area Network (LoRaWAN) Clase A:**

   a) Configurar dispositivo Arduino MKR WAN 1310 como nodo LoRaWAN Clase A con el objetivo de cumplir con las exigencias de bajo consumo energético y permitir el funcionamiento de baterías durante períodos prolongados

   b) Implementar protocolo de comunicación bidireccional Clase A que permita la recepción de downlinks únicamente después de enviar uplinks, maximizando la eficiencia energética

6. **OB-06. Implementación del sistema completo:**

   a) Integrar hardware y software en un prototipo funcional.

   b) Configurar la red IoT para garantizar la conectividad del sistema.

7. **OB-07. Validación y pruebas:**

   a) Creación y realización de pruebas para evaluar la fiabilidad y la precisión del sensor de humedad capacitivo y la eficacia del riego.
   b) Reconfigurar y ajustar el sistema en función de los resultados obtenidos.

8. **OB-08. Documentación y guía del usuario:**
   a) Creación de manuales técnicos
   b) Elaborar un manual de usuario para facilitar la comprensión y uso del sistema.

2.1.1. Objectivos de Arquitectura y aplicación web

1. **OB-01. Definición de la arquitectura distribuida**
   a) **Edge Computing:** Captación de datos del Arduino MKR WAN 1310 con sensores DHT11 y capacitive soil moisture. Además del accionamiento del actuador para activar/desactivar el riego
   b) **Fog Computing:** Gateway LoRaWAN + TTN como capa intermedia de recolección y preprocesamiento
   c) **Cloud Computing:** servidor web Node.js + PostgresSQL para el almacenamiento, posprocesamiento, visualización de datos y lógica de negocio

2. **OB-01. Definición de la comunicación bidireccional**
   a) Definición de los uplink para la transmisión de datos de sensores vía LoRaWAN Clase A hacia el webhook Hypertext Transfer Protocol (HTTP) del servidor de red TTN
   b) Definición de los downlinks para el envío de ordenes de control de la válvula RS a través de la Application Programming Interface (API) de TTN
   c) Implementación de autenticación Over-The-Air Activation (OTAA) para seguridad de la comunicación desde el dispositivo IoT hasta el servidor de red de TTN
   d) Configuración del endpoint HTTP del Webhook HTTP para el reenvío de datos obtenidos de los sensores hacia el servidor web

3. **OB-01. Implementación de una aplicación web escalable y mantenible**
   a) Definición de una arquitectura modular basada en componentes Vue reutilizables

b) Definición de la apis a partir de los principios RESTful

c) Implementación de autenticación OTAA para seguridad de la comunicación desde el dispositivo IoT hasta el servidor de red de TTN

d) Configuración del endpoint HTTP del Webhook HTTP para el reenvío de datos obtenidos de los sensores hacia el servidor web

2.1.2. Objectivos funcionales

1. **OB-01. Implementación de un sistema de gestión de usuarios y roles**

   a) Implementar sistema de autenticación con verificación de email obligatoria

   b) Controlar acceso diferenciado según roles y permisos específicos

   c) Implementar sistema de recuperación de contraseñas y gestión de cuentas

2. **OB-02. Implementación de un sistema de gestión de cultivos**

   a) Permitir registro, visualización, eliminación, edición y selección de cultivos para riego

   b) Configurar umbrales de humedad y temperatura específicos por tipo de cultivo

3. **OB-03. Implementación de un sistema de gestión de dispositivos IoT**

   a) Registro, configuración, edición, eliminación y asignar denegación o confirmación del grabado en la base de datos de las lecturas de los sensores enviadas desde los dispositivos IoT registrados

   b) Integración con TTN para comunicación bidireccional

4. **OB-04. Implementación de los modos de riego**

   a) **Modo Manual:** Control manual de la válvula con una duración de riego personalizable

   b) **Modo programado:** Control de riego mediante horarios específicos

   c) **Modo automático:** Control de riego automático en base a los umbrales de temperatura y humedad

2.2. Objetivos de seguridad

1. **OB-01. Implementación de medidas de seguridad**

   a) Autenticación JSON Web Tokens (JWT) con tokens seguros

   b) Cifrado de contraseñas con bcrypt

   c) Protección de rutas mediante middleware para los distintos roles.

Capítulo 3

Antecedentes

3.1. Antecentes técnicos

3.1.1. The Things Network

The Things Network (TTN) es un sistema colaborativo global de Internet de las Cosas que crea redes, dispositivos y soluciones utilizando tecnología LoRaWAN. TTN está propulsado por The Things Stack, un servidor de red de LoRaWAN. Este servidor es el componente crítico para cualquier solución LoRaWAN.

Como podemos ver en la *Figura 3.1*, la red IoT está compuesta por un conjunto de sensores, en nuestro caso se compondrá de 2 sensores conectados a un microcontrolador Arduino MKR 1310 que a su vez tendrá conexión con un Gateway que permitirá enrutar los paquetes de datos hacia TTN. Las gateways o puertas de enlace son el puente entre dispositivos y The Things Network. Los dispositivos IoT usan redes de baja potencia como LoRaWAN para conectarse al Gateway, mientras que la Gateway usa redes de un alto ancho de banda como Wireless Fidelity (WiFi) o Ethernet para conectarse al The Things Network. Todas las Gateways de cada dispositivo IoT recibirá los mensajes de dichos dispositivos IoT y los enviará a The Things Network.

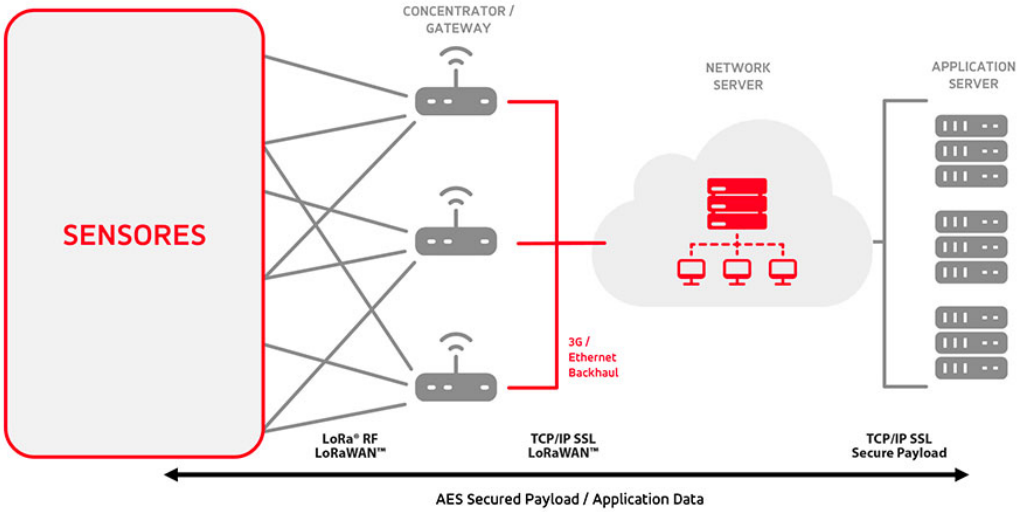


Figura 3.1: Infraestructura LoRaWAN

3.1.2. LoRaWAN y LoRa

LoRaWAN [6] es un protocolo de red que usa la tecnología LoRa, se caracteriza por tener un bajo consumo, ser bidireccional y para transmisión de un bajo volumen de datos y grandes distancias. A diferencia de otras tecnologías de comunicación inalámbrica, permite el despliegue en un área extensa de un gran conjunto de dispositivos alimentados por baterías de una manera sencilla y viable económicamente. Este tipo de comunicación es llamada LPWAN (Low Power Wide Area Network). Podemos ver la arquitectura de LoRaWAN en la Figura 3.2.

LoRa [9] es una tecnología inalámbrica (al igual que WiFi, Bluetooth, LTE, SigFox o Zigbee) que emplea un tipo de modulación en radiofrecuencia patentado por Semtech, una importante empresa fabricante de chips de radio. La tecnología de modulación se denomina Chirp Spread Spectrum (o CSS) y se emplea en comunicaciones militares y espaciales desde hace décadas. LoRa comprende la capa física y permite una comunicación de largo alcance de 10 a 20 km. Cuenta con una baja transferencia de datos, permite la conexión punto a punto y trabaja con unas frecuencias de 868 Mhz en Europa, 915 Mhz en América y 433 Mhz en Asia.

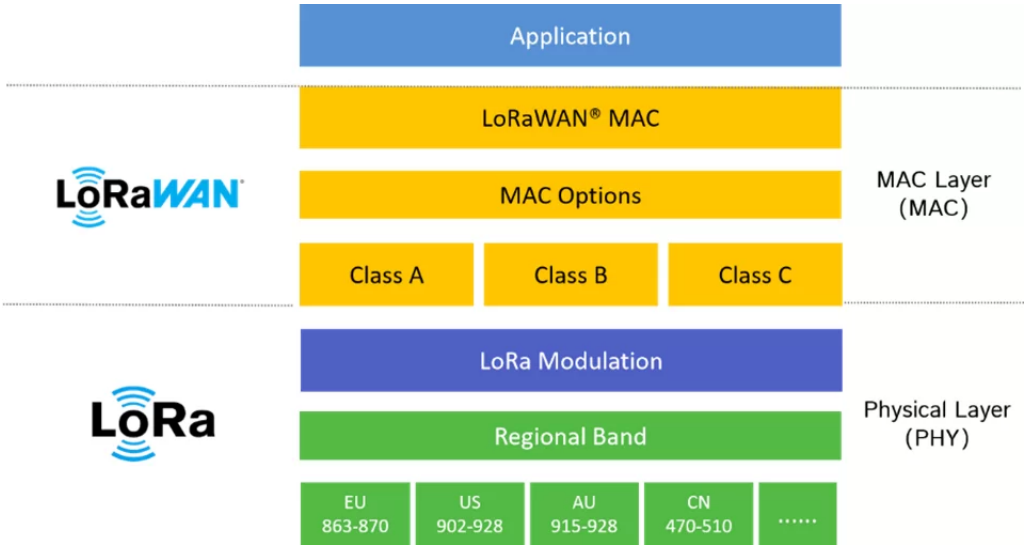


Figura 3.2: Jerarquía de capas de la arquitectura LoRaWAN

3.1.3. MQTT

A parte de LoRaWAN, existen otros protocolos de comunicación para la transmisión de mensajes por la red IoT. Uno de los más famosos junto con LoRaWAN es MQTT.

MQTT es un protocolo de mensajería basado en estándares, o un conjunto de reglas, que se utiliza para la comunicación de un equipo a otro. Los sensores inteligentes, los dispositivos portátiles y otros dispositivos de Internet de las cosas (IoT) generalmente tienen que transmitir y recibir datos a través de una red con recursos restringidos y un ancho de banda limitado. Estos dispositivos IoT utilizan MQTT para la transmisión de datos, ya que resulta fácil de implementar y puede comunicar datos IoT de manera eficiente.MQTT admite la mensajería entre dispositivos a la nube y la nube al dispositivo. Destaca por ser ligero, escalable, fiable y seguro.

Este protocolo se basa en el modelo publicador-suscriptor, que consiste en un esquema de comunicación en el que los emisores de mensajes (publicadores) envían datos a un canal o "tópico", y los receptores (suscriptores) reciben automáticamente esos mensajes si están suscritos a ese canal, sin que exista una conexión directa entre ambos.

A pesar de sus ventajas, este protocolo de comunicación no se puede aplicar a nuestro proyecto puesto que se trata de un protoclo que transmite mensajes entre dispositivos a través de redes IP. Por lo que es ideal para redes WiFi, Ethernet, 4G,

etc...

Pero pierde su eficacia cuando los dispositivos IoT tienen que enviar información desde un lugar remoto como es por ejemplo el campo. Donde no existe conexión directa entre una red wifi o 4G con el dispositivo final. Por ello es necesario utilizar un protocolo de comunicación que permita enviar mensajes en pequeñas cantidades de datos y a largas distancias, y aquí es donde entra en juego el protocolo LoRaWAN. A difrencia de MQTT que opera en la capa de aplicación, LoRaWAN opera a nivel de capa de red y enlace en un sistema IoT y está construido encima de LoRa, que es la tecnología física (modulación de radio). LoRa es la tecnología física mientras que LoRaWAN es el protocolo de red que define cómo los dispositivos usan LoRa para comunicarse con gateways y sensores. Esta arquitectura de red de comunicación inalámbrica permite como se ha dicho anteriormente mandar mensajes a grandes distancias. Es por ello que para el proyecto te utilizará LoRaWAN.

3.1.4. Micro válvula RS (RS Pump)

La micro válvula RS [10] es el componente más característico y el más importante del sistema de riego automático. Puesto que ejecutará el riego al recibir la señal de control. Esta micro válvula RS es eficiente, pequeña y liviana. Su alimentación está en el rango de 3V - 6V Podemos verla en la Figura 3.3. Por otro lado, destacamos las siguientes características:

- Funcionamiento: Al ser activada, bombea agua desde un depósito o contenedor hasta la zona de riego.
- Riego: Es adecuada para riegos a pequeña escala, en un principio está pensada para regar una superficie aproximadamente de 2 metros cuadrados a un caudal de 0.4l/min.

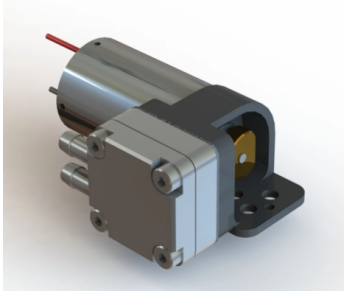


Figura 3.3: Micropump - RS PRO

3.1.5. Arduino MKR WAN 1310

El Arduino MKR WAN 1310 es un microcontrolador diseñado específicamente para aplicaciones de bajo consumo y comunicación LoRaWAN. Combina la facilidad de uso de la plataforma Arduino con la conectividad necesaria para integrarse en redes IoT de largo alcance. Algunas de las características son:

- Conectividad LoRaWAN: Incorpora un módulo LoRa que se ajusta a bandas de frecuencias específicas. Esto facilita la transmisión de datos a grandes distancias con consumo energético reducido.
- Alimentación y bajo consumo: Dispone de funcionalidades de "sleep mode" que permite optimizar el consumo eléctrico, lo cual es esencial en entornos donde se requieren soluciones autónomas y de larga duración

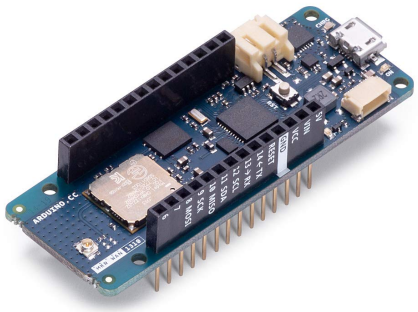


Figura 3.4: Arduino MKR WAN 1310

3.1.5.1. Especificaciones técnicas

- **Microcontrolador:** Microchip SAMD21 Cortex-M0+ de 32 bits, 48 MHz.
- **Memoria:** 256 KB de Flash y 32 KB de SRAM.
- **Módulo LoRa:** Murata CMWX1ZZABZ (basado en Semtech SX1276).
- **Voltaje de operación:** 3,3 V
- **Conectividad:** LoRaWAN Clase A, frecuencias EU868 / US915 (según región).
- **Alimentación**
  - USB-C (5 V)

- Li-Po de 3,7 V (con cargador integrado y conector JST)
- Pin VIN (5 V máx.)

- Interfaces: UART, SPI, I²C, PWM, entradas analógicas, salidas digitales.

3.1.5.2. Conexiones

- **Pines E/S:**
  - 8 digitales (12 si se usan funciones compartidas)
  - 7 analógicas (incluye un pin DAC para salida analógica real)
- **Puerto USB-C:** Programación y alimentación.
- **Conector Li-Po:** Permite alimentación por batería con recarga automática al conectar USB.

3.1.6. Sensor de temperatura y humedad ambiental

La función de este sensor es básica, y es medir temperatura y humedad relativa en ambientes interiores y exteriores. Se trata de un sensor digital de bajo coste con calibración interna de fábrica [11].

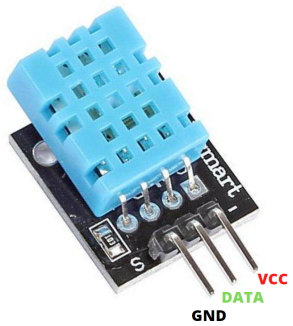


Figura 3.5: DHT11

3.1.6.1. Especificaciones técnicas

- **Voltaje de operación:** 3,3 V a 5,5 V DC (compatible con la mayoría de microcontroladores: Arduino, ESP8266, ESP32, Raspberry Pi, etc.).
- **Rango de temperatura:** 0 °C a 50 °C, con precisión aproximada de ±2 °C.

- **Rango de humedad:** 20 % a 90 % HR, con precisión aproximada de ±5 % HR.
- **Frecuencia de muestreo:** hasta 1 lectura cada 1 segundo (1 Hz).
- **Salida:** Digital mediante protocolo de comunicación propietario de un solo hilo (single-wire serial).
- **Consumo de corriente:** aprox. 2,5 mA durante medición.

3.1.6.2. Conexiones

El módulo DHT11 suele tener 3 o 4 pines, en este caso nuestro sensor está compuesto de 3 pines:

- **VCC:**Alimentación (3,3 V o 5 V)
- **DATA:**Señal digital de salida
- **GND:** → Tierra

3.1.7. Sensor de humedad capacitivo

Los sensores de humedad capacitivos [12] son dispositivos electrónicos que se utilizan para medir los niveles de humedad relativa en el aire o en materiales. Aprovechan la variación de la capacitancia eléctrica de un material sensible a la humedad en función de la humedad ambiental para determinar el nivel de humedad. Cabe destacar que utiliza detección capacitiva en lugar de resistiva, lo que minimiza la corrosión y prolonga su vida útil. Está fabricado a partir de un material resistente anticorrosivo para mantener una precisión a lo largo del tiempo

Haciendo mención anteriormente de la precisión de este sensor. La precisión puede variar según el modelo de sensor específico, la calidad de construcción y las condiciones de uso, pero generalmente son confiables para muchas aplicaciones. Algunas de sus ventajas son su precisión, bajo costo, bajo consumo de energía y rápida respuesta a los cambios de humedad.

El que utilizaremos para este proyecto puede verse en la *Figura 4.10*:

Figura 3.6: Capacitive Soil Moisture Sensor v1.2

3.1.7.1. Especificaciones técnicas

- **Voltaje de operación:** 3,3 V y 5,5 V DC, compatible con microcontroladores como Arduino, ESP8266, NodeMCU, PIC y Raspberry Pi (con ADC)
- **Salida analógica (AOUT)**
  - Rango típico de voltaje entre 0V (suelo muy húmedo) y aprox. 5V (suelo seco).
  - En algunos modelos, el rango de salida es entre 0V y 3V
- **Consumo eléctrico:** Aproximadamente 5mA, ideal para sistemas de bajo consumo.
- **Dimensiones y peso:** Aproximadamente 98 x 23mm, con un peso cercano a los 15g.

3.1.7.2. Conexiones y uso básico

- **Pinout (3 pines)**
  - **VCC:** alimentación (3,3–5,5V)
  - **GND:** Masa
  - **AOUT:** Salida analógica proporcional a la humedad del suelo

3.1.8. FIWARE

Es una plataforma intermedia de código abierto para el desarrollo de aplicaciones globales para el IoT, cubriendo la necesidad de manejar el contexto de la información [13].

Esta apoyada por la Unión Europea, proporcionado especificaciones de API y sin licencia ni derechos de autor, donde la implicación de los usuarios y desarrolladores es crítica para la plataforma y su estandarización.

3.1.9. Puertas de enlace LoRaWAN

Las puertas de enlace son el dispositivo que actúa de intermediario de conexión entre aparatos o dispositivos, y también posibilita compartir recursos entre ellos. El uso de este dispositivo permite crear una red local que le permite a los dispositivos IoT enviar y recibir información hacia y desde internet, utilizando como puente una puerta de enlace. Hay 2 tipos de puerta de enlace, la que están diseñadas para interiores que cubren una pequeña área de comunicación y las exteriores que cubren una gran área de comunicación entre dispositivos IoT e internet. En nuestro caso utilizaremos el primer caso.

3.1.9.1. The Things Indoor Gateway (TTIG)

Como se ha comentado anteriormente, los Gateways LoRaWAN son dispositivos esenciales para conectar dispositivos finales IoT con la red central en arquitecturas IoT que trabajen con la modulación LoRa a través de internet. El Gateway que se utilizará para el proyecto será un TTIG, diseñado para aplicaciones de interior, este gateway se caracteriza por su fácil configuración, soporte para múltiples canales y compatibilidad con The Things Network. Es adecuado para oficinas, hogares y áreas cerradas donde la cobertura de red se limita a espacios reducidos. Cabe destacar que este tipo de gateway soporta implementaciones de clase A y clase C Se utilizará este gateway por su coste económico que ronda los 100 EUR. Mientras que un gateway para exteriores puede costar mínimo 600 EUR.

Figura 3.7: Capacitive Soil Moisture Sensor v1.2

3.1.10. Relé HUAREW 5V

El módulo de relé de 1 canal HUAREW 5V [14] es un dispositivo que permite controlar circuitos o componentes eléctricos de alta potencia con una señal de baja potencia, como por ejemplo una señal de un pin digital como la de un microcontrolador. No se trata de un componente digital sino de un componente mecánico que consta de una bobina y un conjunto de contactos. Cuando se aplica una corriente a la bobina, como por ejemplo 3.3 V de un pin digital, se crea un campo magnético que atrae una pieza metálica móvil. Este movimiento acciona los contactos, abriendo o cerrando el circuito. A esto se le conoce como activación breve puesto que el relé se energiza momentáneamente ante la respuesta de una señal eléctrica de activación. Este será el actuador de la bomba de riego.

El aspecto más importante a destacar es que este módulo relé tiene un aislamiento de optoacoplador, que protege al circuito de control de cualquier pico de voltaje o interferencias del componente de alta potencia. La protección se hace a través de diodos que impiden picos de voltaje, puesto que su propósito es retener estos picos.

Por tanto, Un optoacoplador es un componente electrónico que transfiere señales eléctricas entre dos circuitos aislados mediante la luz. Este aislamiento galvánico evita que la corriente del relé retorne al microcontrolador, lo que podría dañarlo. El módulo funciona con una fuente de alimentación de 5V (DC+ y DC-) para su circuito de control.

3.1.10.1. Terminales del Módulo

El módulo de relé tiene un total de 6 bornas (o terminales) distribuidas en 2 grupos, uno a cada lado.

3.1.10.1.1. Bornas de salida Estas tres bornas son los contactos del relé, que se conectan al circuito que se desea controlar:

- **NO (Normalmente Abierto):** En su estado normal (relé desactivado), este terminal está abierto, sin conexión a COM. Cuando el relé se activa, se cierra, conectándose a COM.
- **COM (Común):**Este terminal es el punto de conexión común. Es el polo de la línea que se va a conectar al circuito. La energía se suministra a través de este terminal.

- **NC (Normalmente Cerrado):** En su estado normal (relé desactivado), este terminal está conectado a COM. Cuando el relé se activa, se abre, interrumpiendo la conexión a COM.

3.1.10.1.2. Bornas de entrada Estas tres bornas son las conexiones al microcontrolador para energizar el relé:

- **IN (Entrada de Señal):** Este es el pin de control. Una señal de 5V o un pulso corto a este pin activa el relé, cerrando la conexión entre COM y NO.
- **DC- (GND):** Este terminal se conecta al GND (tierra) del microcontrolador.
- **DC+ (VCC):** Este terminal se conecta a la fuente de alimentación de 5V del microcontrolador.

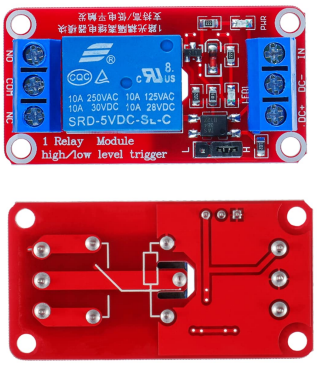


Figura 3.8: Relé HUAREW 5V

3.1.11. Edge Computing

Edge computing o computación de borde [15] es un marco informático distribuido que acerca las aplicaciones empresariales a fuentes de datos, como dispositivos IoT o servidores periféricos locales. Esta proximidad a los datos en su origen puede reportar grandes beneficios empresariales, como la obtención más rápida de conocimiento, mejores tiempos de respuesta y mayor disponibilidad de ancho de banda.

En el pasado, la regla general era enviar todos los datos generados por los dispositivos a un centro de datos centralizado o a la nube lo que provocaba problemas de

ancho de banda y latencia. El Edge Computing ofrece una alternativa más eficiente; los datos se procesan y analizan más cerca del punto donde se crean. Debido a que los datos no atraviesan una red hasta la nube o un centro de datos para ser procesados, la latencia se reduce. La computación de borde puede ser realizada por dispositivos conectados directamente a sensores, por routers o pasarelas que transmiten datos, o por pequeñas instalaciones de servidores que se despliegan in situ en un armario o reciento.

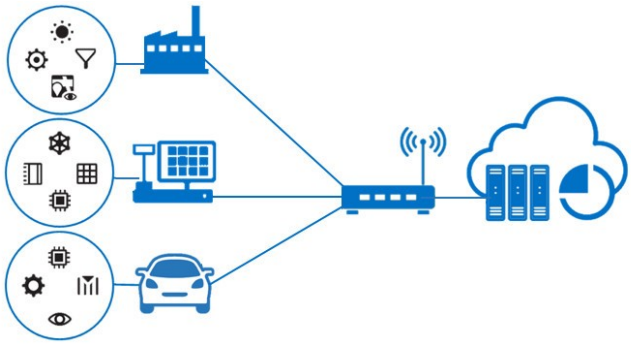


Figura 3.9: Representación Edge Computing

La capa de Edge Computing será la base para construir la arquitectura distribuida Edge-Fog-Cloud Computing, a partir de la cual será formado nuestro sistema IoT.

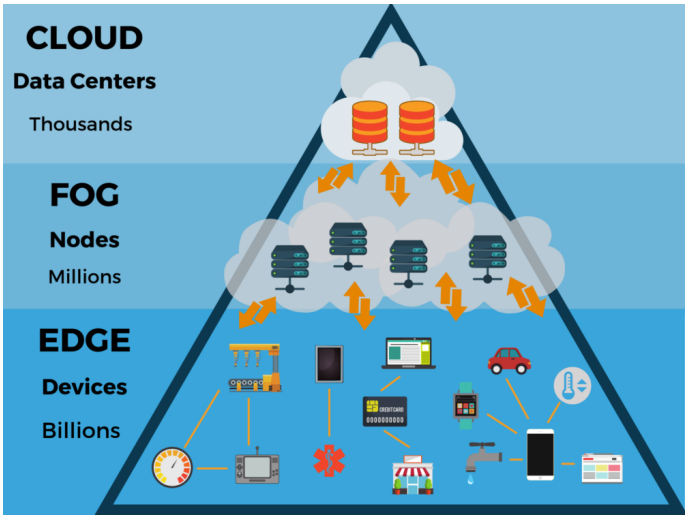


Figura 3.10: Arquitectura distribuida Edge-Fog-Cloud Computing

3.1.12. Fog Computing

El concepto de Fog Computing [16] se refiere a una estructura de red descentralizada en la que los recursos, incluyendo datos y aplicaciones, se sitúan en algún lugar lógico entre el servidor cloud y la fuente que genera los datos. De manera que actúa como especie de intermediario entre los dispositivos IoT y el servidor web. En lugar de enviar toda la información directamente a la nube, esta arquitectura introduce una capa intermedia conocida como "la niebla" compuesta por nodos locales conocidos como routers o gateways que procesan y filtran los datos en tiempo real.

Este enfoque es especialmente útil en entornos donde la latencia debe ser mínima y la respuesta inmediata, como en vehículos autónomos, ciudades inteligentes o sistemas industriales conectados. Al reducir la distancia que recorren los datos, se mejora la eficiencia, se optimiza el uso del ancho de banda y se refuerza la seguridad al mantener los datos sensibles más cerca de su origen. Es necesario destacar que el Fog Computing no reemplaza directamente a la nube, sino que la complementa. Mientras que la nube sigue siendo esencial para el almacenamiento a largo plazo y el análisis masivo de datos, la niebla permite una gestión más ágil y localizada de la información.

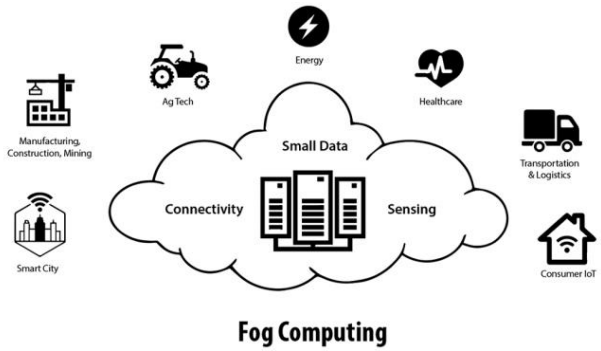


Figura 3.11: Fog Computing

3.1.13. Cloud Computing

La última capa de la arquitectura distribuida que adoptará nuestro sistema, será la de Cloud Computing [17]. Consiste básicamente en el acceso bajo demanda a recursos informáticos (servidores físicos o virtuales, almacenamiento de datos, capacidades de red, herramientas de desarrollo de aplicaciones, software y demás) a través de Internet con precios de pago por uso.

Cuando se accede a servicios en la nube básicamente se accede a servidores re-

motos ubicados en centros de datos. Mediante un pago se le proporciona al usuario almacenamiento para desplegar su aplicación, base de datos o cualquier otro aplicativo que el usuario desee almacenar. Normalmente a este almacenamiento se accede bajo demanda, es decir, conforme más almacenamiento requiera el usuario más debe de pagar por este servicio cloud. El cloud Computing es fundamental en nuestra vida diaria, ya sea para acceder a una aplicación en la nube como Google Gmail, ver una película en Netflix o en el caso de IoT disponer de almacenamiento para almacenar un gran volumen de datos provenientes de sensores distribuidos en un espacio o región. Con el cloud computing, se obtiene la potencia de cálculo o el almacenamiento que necesita el usuario, sin tener que poseer ni gestionar el hardware físico por si mismo.

Figura 3.12: Cloud Computing

3.2. Iniciativas previas

A pesar de la utilidad de las redes IoT, especialmente en aquellas basadas en LoRaWAN. No existen muchas soluciones de gestión de riego inteligente basados en esta tecnología, al ser demasiado reciente en el mercado. A continuación en esta sección se comentarán cuales son las soluciones en las que se ha basado el sistema Vivantia.

3.2.1. Sistema de Riego BRESSER Smart Garden Smart Home

El primer sistema de riego que tomamos como referencia es el BRESSER Smart Garden Smart Hombe [18]. Este sistema de riego inteligente permite controlar el riego a través de su propia aplicación llamada *HomGar*, *Amazon Alexa* [19] y *Google Assistant* [20]. Además permite ajustar un temporizador de agua que se ajusta al reaccionar a la medición actual del sensor de suelo. También cuenta con sensores de presión atmosférica y de tempera y humedad ambiental. Por otro lado es capaz de ofrecer estadísticas de consumo de agua diario y mensual y realizar predicciones de tiempo con probabilidad de lluvia. Sin embargo, el protocolo de comunicación que utiliza para controlar el riego es mediante WiFi por lo que solo se podría activar el riego para un cultivo que esté dentro del rango de la señal WiFi, que es de corto alcance.

Figura 3.13: Sistema de Riego Inteligente BRESSER

3.2.2. Programador de riego Solem LoRa de 9V, 4 estaciones de riego

El programador de riego Solem LoRa de 9V [21] se trata de un programador de riego el cual cual funciona bajo tecnología bluetooth [22] y LoRa. Esta pensando para instalaciones de riego de largo alcance y se configura a través de una app propia llamada *My Solem App*. Funciona con baterías y no cuenta con ningún sensor integrado para mediciones de temperatura y humedad ambiental y humedad del

suelo, simplemente se programa el riego desde la app que provee el fabricante.

Figura 3.14: Programador de riego Solem LoRA

3.2.3. Comparativa de Vivantia frente a otras soluciones de riego inteligente

Tabla 3.1: Comparativa de Vivantia (TFG) frente a otras soluciones de riego inteligente

Criterio	**Vivantia (TFG)**	**Sistema de riego BRESSER**	**Programador de riego Solem**
Conectividad principal	LoRaWAN Clase A (TTN) con gateway	Wi-Fi 2.4 GHz (LAN/Internet)	LoRaWAN Clase A/C con gateway y Bluetooth
Cobertura típica en campo	Cientos de metros o km (con gateway)	Decenas de metros; sensible a obstáculos	Cientos de metros o km (con gateway)
Modos de riego	Manual, Programado y Automático	Manual y Programado por voz; Automático (humedad del suelo)	Programado
Gestión remota sin Wi-Fi local	Sí (LoRaWAN)	No, requiere Wi-Fi/Internet local	Sí (LoRaWAN)
Consumo energético del nodo	Muy bajo (batería)	Muy bajo (batería)	Muy bajo (batería)
Arquitectura	Edge–Fog–Cloud	App + cloud del fabricante (entorno cerrado)	App + cloud del fabricante (entorno cerrado)
Base de datos asociada a los cultivos	Sí	No	No
Interfaz de usuario	Interfaz Web	App móvil del fabricante	App móvil del fabricante
Dependencia de energía de red	No (operación con batería)	(operación con batería)	No (operación con batería)
Coste hardware aproximado	Medio (aprox 200€)	Medio (159,00€)	Alto (378,81€)
Predicciones/meteo integrada	Integrable (mediante API'S)	Sí	No
Sistema de alertas	Sí	Sí	No
Orientación de uso	Agricultura/huertos con baja infraestructura	Hogar/jardín con Wi-Fi	Agricultura a gran escala

Capítulo 4

Análisis del problema

4.1. Definición detallada del problema

4.1.1. Definición del problema real

Anteriormente se habló sobre los problemas de sequía, bastante comunes en España. No solo las grandes explotaciones agrarias, sino que también pequeños agricultores enfrentan un desafío relacionado con la gestión eficiente del agua en sus sistemas de riego. La agricultura tradicional se basa en métodos manuales y poco precisos para determinar cuándo y cuánto regar, lo que conlleva una serie de problemas significativos que afectan tanto la productividad como la sostenibilidad de los cultivos.

Tanto los grandes agricultores como pequeños agricultores domésticos no disponen de una manera rápida y eficaz que permita gestionar de manera personalizada y automatizada el riego de sus cultivos en función de las condiciones ambientales reales. En muchos casos, la complejidad de las variables ambientales como la temperatura, humedad del suelo y humedad ambiental es tan elevada que se hace imposible atender a todas las necesidades de riego de manera óptima.

Son muchos los problemas que se presentan al implementar un sistema de riego tradicional, aparte del uso ineficiente del agua, podemos destacar:

1. Los agricultores deben estar físicamente presentes para activar y desactivar los sistemas de riego.

2. No hay capacidad de respuesta automática a cambios en las condiciones ambientales y de suelo.

3. Carencia de un sistema automatizado que proporcione datos en tiempo real sobre las condiciones del suelo y ambiente.
4. Los agricultores no pueden tomar decisiones sobre el control de riego sin datos precisos.
5. Riego excesivo que desperdicia agua y puede dañar plantas.
6. Aumento de costos por el uso excesivo de agua y energía.

El problema requiere de una solución tecnológica, una solución que integre tecnología IoT, automatización y gestión inteligente del riego para transformar la agricultura tradicional en una práctica más eficiente, sostenible y rentable. Esta solución debe de aportar:

1. **Automatización completa** del proceso de riego basada en datos reales
2. **Monitoreo en tiempo real** de las condiciones ambientales y del suelo
3. **Accesibilidad** de los agricultores que no tienen mucha experiencia manejando sistemas informáticos.

La implementación de un sistema de riego automatizado no solo serviría para solucionar los actuales problemas relacionados con la gestión del agua, sino que, además, establecería un punto de partida para una agricultura más ecológica y con tecnología de punta, lo que permitiría encarar los futuros retos de la seguridad en el suministro de alimentos y la protección de los bienes naturales.

4.1.2. Definición del problema técnico

Para la definición e identificación precisa del problema, es necesario definirlo desde el punto de vista técnico. En esta sección explicaremos detalladamente el enfoque escogido para dar una solución técnica al problema del cual partimos. Para ello debemos comprender su naturaleza y sus particularidades específicas.

La aplicación web implementará un sistema completo de gestión de usuarios con tres roles bien diferenciados: visitante, usuario y administrador, cada uno con acceso a diferentes funcionalidades del sistema. Se requerirá de registro e inicio de sesión con verificación obligatoria de correo electrónico, lo que garantizará que solo usuarios verificados y administradores del sistema puedan acceder a las funcionalidades de la aplicación web.

Uno de los principales retos para implementar un sistema de gestión inteligente del riego agrícola será la obtención precisa y continua de datos ambientales y del

suelo que permitan tomar decisiones inteligentes sobre cuándo y cuánto regar los cultivos. El otro gran reto que se presenta es el cómo se hará para enviar una señal eléctrica al actuador para que active el riego. Para abordar este desafío, el sistema que se desarrollará implementará una arquitectura distribuida basada en los principios de Edge computing[15], Fog computing[16] y Cloud computing[17], que permitirá captar, procesar, transmitir y almacenar datos en cada una de sus capas de forma eficiente y escalable.

En la capa de Edge computing[15], el sistema empleará sensores especializados como el DHT11[11] para medir temperatura y humedad ambiental, y el capacitive soil moisture v1.2 [12] para medir la humedad del suelo. Estos sensores estarán conectados a un microcontrolador Arduino MKR WAN 1310 [23], que actuará como nodo inteligente o dispositivo IoT en el campo, jardín, huerto o cualquier espacio donde se de una explotación agrícola. El microcontrolador no solo se encargará de leer las lecturas de los sensores, sino también de realizar un preprocesamiento básico, empaquetado y codificación de datos, así como el encargado de mandar la señal eléctrica al actuador para que active o desactive el riego. Gracias al uso del protocolo de comunicación LoRaWAN bajo la tecnología de comunicación inalámbrica LoRa[9], estos dispositivos podrán transmitir datos a largas distancias y con un consumo energético muy bajo, lo que los hará ideales para el despliegue en entornos rurales sin acceso a red eléctrica continua. Además, su tamaño compacto y autonomía permitirá instalar el dispositivo directamente en el terreno de cultivo sin restricciones de ubicación. Siempre y cuando esté en el rango de señal del gateway.

Una vez recogidos y preparados los datos en el nodo, comenzará la fase de Fog Computing [16], representada por los Gateways LoRaWAN [24] que se desplegarán en el entorno y estarán encargados de recibir las transmisiones LoRa. Estos dispositivos actuarán como punto de enlace entre el nodo y el servidor de red de TTN [25] que será la plataforma IoT que estaremos utilizando para gestionar la comunicación LoRaWAN del dispostivo IoT de nuestro proyecto. A través del gateway, los datos se enviarán a TTN, una plataforma cloud pública que actuará como servidor de red LoRaWAN. TTN verificará la validez del mensaje a nivel de protocolo (estructura LoRaWAN, integridad y origen del dispositivo) y, si todo es correcto, reenviará el contenido del mensaje al servidor web del sistema a través de un endpoint HTTP que se definirá en el Webhook de TTN.

En la capa de Cloud Computing [17], los datos serán gestionados por un servidor web que se desarrollará con Node.js y Express, el cual actuará como receptor de los mensajes enviados desde TTN. Este servidor validará que los datos provengan de un dispositivo autorizado, analizará que su estructura y contenido sean correctos, y en caso afirmativo, los transformará al modelo interno y los almacenará en una base

de datos PostgreSQL [26] para su consulta posterior.

Hay que destacar que existirán 2 direcciones de comunicación. Hasta el momento, se ha hablado de un tipo de comunicación llamada uplink [27] en la que activamente el microcontrolador enviará información a través de un Gateway LoRaWAN [24] al servidor de red de TTN con el objetivo de enviarlo al endpoint HTTP [28] definido previamente en el Webhook HTTP, para finalmente reenviarlo al servidor web del sistema donde se procederá a postprocesar y almacenar los datos. Pero existirá otra dirección de comunicación que es downlink [29] que será inversa del uplink [27], en la que desde el servidor web del sistema se realizará una consulta a la API de TTN [30] para crear una orden de control de riego que se encolará en el servidor de red, y que posteriormente se desencolará tras recibir un uplink, para luego pasar a través del Gateway LoRaWAN [24] hasta llegar al microcontrolador, con el fin de ejecutar acciones remotas como la activación o desactivación de una bomba de riego, incluso permitirá la posibilidad de enviar información desde el servidor web del sistema al microcontrolador.

La información almacenada podrá ser consultada desde una aplicación web de tipo Single-Page Application (SPA) [31] híbrida que se desarrollará con Nuxt.js [32] y Vue.js [33]. Esta aplicación permitirá a los usuarios verificados monitorear en tiempo real las condiciones de su cultivo, configurar dispositivos, seleccionar y gestionar cultivos, definir umbrales y modos de riego, así como recibir alertas y notificaciones sobre el estado del sistema. La interfaz estará diseñada para ser intuitiva, adaptable a dispositivos móviles y computadores, y ser accesible desde cualquier navegador moderno.

La arquitectura completa del sistema se estructurará en 4 módulos distintos, en función de las 3 capas comentadas anteriormente. El primer módulo cubrirá la capa Edge computing, encargada de la captación de datos y acción sobre el actuador. El segundo módulo será el encargado de la recolección y preprocesamiento de información. El tercer módulo estará dividido en dos submódulos, uno para la recepción y postprocesamiento y el otro para el almacenamiento de datos. Por último, tenemos el módulo cuatro encargado de la visualización de datos.

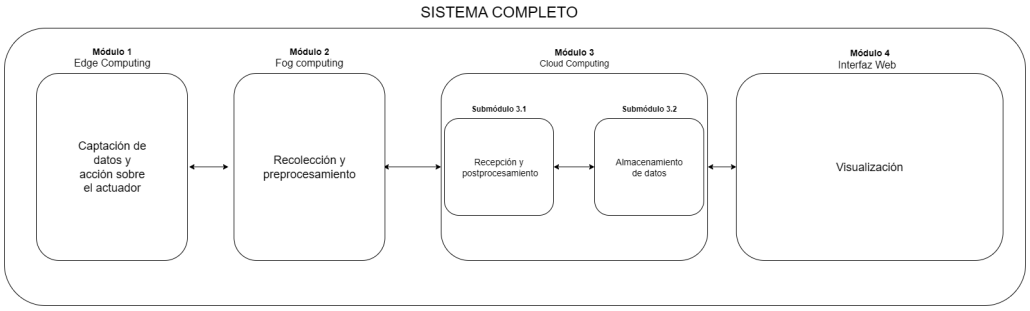


Figura 4.1: Diagrama de sistema completo

El módulo 1 (Edge Computing) estará formado por sensores físicos: el sensor DHT11 para la medición de temperatura y humedad ambiental, y el sensor capacitive soil moisture v1.2 para la medición de humedad del suelo. Estos sensores estarán conectados al microcontrolador Arduino MKR WAN 1310, que actuará como el cerebro del sistema Edge, donde se realizará la captación continua de datos a un intervalo de tiempo determinado, procesamiento de las lecturas de los sensores, accionar el actuador mediante la recepción de ordenes de riego y el envío de datos mediante comunicación inalámbrica LoRa utilizando el protocolo de comunicación LoRaWAN.

El Módulo 2 (Fog computing) incluirá el Gateway LoRaWAN y el servidor de red TTN. Este gateway funcionarán como un puente de comunicación bidireccional entre los dispositivos IoT del módulo 1 y el servidor de red de TTN, transmitiendo información en ambas direcciones.

El gateway recibirá las transmisiones LoRaWAN de datos del dispositivo IoT y las reenviarán a través de internet hacia TTN. El servidor de red de TTN recibirá estos datos y los reenviará automáticamente al servidor web de nuestra aplicación a través de un en endpoint del webhook HTTP configurado previamente. En sentido contrario, el servidor de red de TTN enviará ordenes de riego hacia el dispositivo IoT a través del gateway. En la capa Fog computing no se interpretará el contenido del mensaje, pero sí se garantizará la autenticidad del dispositivo mediante el protocolo de autenticación OTAA [34] y se validará la estructura del protocolo LoRaWAN, asegurando la integridad de la transmisión.

La autenticación OTAA funcionará mediante un proceso de activación en dos fases: durante la activación inicial, el dispositivo IoT enviará una solicitud de unión (Join Request) que incluirá un identificador único del dispositivo (DevEUI), un identificador de aplicación (AppEUI) y un número aleatorio (DevNonce). El servidor de red TTN responderá con un mensaje de aceptación (Join Accept) que contendrá las

claves de sesión derivadas (AppSKey y NwkSKey) necesarias para el cifrado y descifrado de mensajes posteriores. Una vez completada la activación OTAA, todos los mensajes uplink y downlink se cifrarán utilizando estas claves de sesión, garantizando la confidencialidad y autenticidad de la comunicación. Este mecanismo permitirá que los dispositivos se autentiquen de forma segura sin necesidad de almacenar claves precompartidas, facilitando la gestión de credenciales y la seguridad de la red.

El módulo tres (Cloud computing) quedará dividido en 2 submódulos como hemos comentado anteriormente. El primer submódulo estará compuesto por el servidor web, este será el responsable de recibir los datos provenientes de TTN a través del endpoint definido en el webhook http y postprocesarlos para ser almacenados y visualizados posteriormente en la aplicación web. A través del servidor web también se enviará la orden de activar o desactivar el riego, haciendo una consulta hacia la API de ttn. Cuando los datos han sido procesados y verificado que el formato es correcto para su almacenamiento, pasamos al submódulo de almacenamiento de datos. Como su nombre indica, se almacenarán los datos provenientes de los sensores para ser tratados y visualizados en la interfaz web, que será el cuarto y último módulo.

En cuanto a la funcionalidad del módulo 4 (Interfaz web) estará compuesto por 3 tipos de roles: Visitantes, usuarios registrados y administrador.

Los visitantes del sistema tienen acceso restringido a las funcionalidades del sistema. Sólo se les tiene permitido acceder a un modo demo, en este modo, el visitante sólo puede acceder a la dashboard principal con datos simulados. El resto de páginas y funcionalidades quedan totalmente inaccesibles en este modo.

Por otro lado, los usuarios autenticados y verificados dispondrán de acceso completo a la gestión inteligente de riego dentro de la plataforma. Podrán gestionar sus cultivos, configurando parámetros como nombre, categoría, umbrales de temperatura y humedad ambiental y humedad del suelo, temporada y días de crecimiento estimados. Solo se permitirá un cultivo activo por usuario para riego y un único dispositivo activo. En lo referente a los dispositivos IoT, los usuarios podrán registrar nuevos dispositivos IoT, asignarles credenciales generadas desde TTN y además permitir activar o desactivar la comunicación del dispositivo IoT con el almacén de datos. Entre otras funcionalidades básicas.

En cuanto a la automatización y gestión del riego, la aplicación pone a disposición tres modos de operación al usuario: manual, programado y automático. El modo manual permitirá activar o desactivar la bomba de riego con una duración configurada, pudiendo pausar, reanudar o cancelar el riego en curso en cualquier momento. El modo programado permitirá establecer eventos de riego en fechas y horas específicas,

definir frecuencias (una sola vez, diarias o días específicos de la semana) y configurar la duración del riego. El modo automático, por su parte, evaluará en tiempo real las condiciones ambientales y del suelo recibidas por los sensores, y activará el riego si se incumple el umbral de humedad del suelo definido por el usuario para las lecturas entrantes del sensor de humedad del suelo. Por otro lado, lo valores de temperatura y humedad ambiental tendrán como objetivo servir de referencia al usuario, para valorar si el ambiente es propicio para un cambio en las condiciones del suelo.

Cómo apoyo visual para aumentar la accesibilidad, la aplicación contará con numerosas guías en forma de widgets reactivos que mostrarán datos como temperatura y humedad ambientales y humedad de suelo actuales, estado de la bomba (activa/inactiva), tiempo restante de riego, última fecha de riego y próxima ejecución programada, cultivo actualmente seleccionado, dispositivo actualmente seleccionado, etc... Además, el sistema generará gráficas en tiempo real a partir de los datos enviados por los sensores lo que permitirá al usuario analizar condiciones ambientales y de suelo. Todo ello se complementará con un sistema de alertas bien definido, que informará sobre eventos disparados en el sistema, cambios en configuración de los dispositivos, alertas ambientales, entre otros eventos.

El rol de administrador tendrá acceso a funcionalidades dedicadas a la gestión total de usuarios, cultivos, y dispositivos registrados en el sistema. Desde su panel de administración podrá visualizar, crear, editar, eliminar cultivos, cuentas de usuario y dispositivos. Por otro lado el administrador contará con logs para comprobar el estado del sistema en todo momento y llevar a cabo decisiones en el sistema.

En cuanto a la comunicación bidireccional con los dispositivos IoT, el sistema manejará tanto uplinks [27] como downlinks [29]. Los uplinks [27] corresponderán a los mensajes enviados por los microcontroladores cada vez que se obtengan nuevas lecturas de sensores, los cuales serán enviados a través del Gateway LoRaWAN a TTN y reenviados al servidor web mediante el webhook. Una vez validados, se almacenarán en la base de datos y quedarán disponibles para visualización inmediata. Los downlinks [29] permitirán que la aplicación envíe ordenes de control de riego desde el frontend hasta los dispositivos IoT. Cuando un usuario ejecute una acción, como activar la bomba manualmente, se generará un paquete de datos codificado en base64 [35] que será enviado por el servidor a la API de TTN [30]. TTN codificará a bytes este paquete de datos, lo encolará y lo entregará al dispositivo en la próxima ventana de recepción, justo después de que este realice un nuevo uplink [27]. El microcontrolador detectará la llegada del downlink [29], interpretará la orden recibida y ejecutará la acción correspondiente, como encender o apagar la bomba, modificar un parámetro de configuración o responder con un nuevo uplink [27] de confirmación.

El sistema, principalmente la web, estará protegida mediante un sistema de autenticación y autorización robusto, que incluirá verificación de email obligatoria para usuarios registrados en la aplicación web, uso de tokens JWT [36] para la gestión segura de sesiones y middlewares de protección de rutas para usuarios. Al igual que la comunicación entre el dispositivo IoT y el servidor de red TTN está protegida bajo validación OTAA. Además, el modo demo se diseñará para ofrecer un acceso previo a la aplicación web sin comprometer la seguridad ni la integridad de los datos del sistema real, ya que serán simulados, lo que le permite al usuario ver una vista previa de lo que se encontrará en la aplicación cuando se haya registrado y forme parte del sistema.

4.2. Contexto operativo del problema

En esta sección se analizará el contexto en el que operará el sistema. Para el correcto desarrollo e implementación del sistema a constituir, es fundamental establecer y comprender los diferentes aspectos del entorno operativo, vida esperada, ciclo de mantenimiento, competencia, aspecto externo, estandarización, pruebas y seguridad.

4.2.1. Entorno

A continuación se detalla los distintos entornos en los que el proyecto será analizado, desarrollado e implementado. Dividiéndolo en 3 entornos:

4.2.1.1. Entorno Software

- El código del microcontrolador del primer módulo se programará en el lenguaje C++ [37], utilizando el plugin PlatformIO [38] en Visual Studio Code [39] para el desarrollo del código del microcontrolador Arduino WAN 1310 [23]. Que se encargará de obtener los datos de los sensores, codificarlos e emitir información. No solo eso sino que también recibirá ordenes de control riego provenientes del servidor web a través del servidor de red de TTN con el objetivo de accionar el actuador para activar o desactivar el riego. Para ello se emplearán las librerías MKRWAN [40] para la comunicación LoRaWAN y las librerías específicas para los sensores DHT11 y capacitive soil moisture v1.2.

- El segundo módulo es una plataforma cloud en internet ya creada por lo que solo hará falta registrar el dispositivo, registrar el gateway, configurar el webhook y poner todo en marcha. Lo único que hará falta programar será el preprocesamiento de datos que se encargará de la decodificación de información en

formato JavaScript Object Notation (JSON) proveniente de los sensores. Así como también programar el decodificador de las ordenes de control que vienen del servidor web. Para programar los decodificadores se utilizará el lenguaje Javascript [41]

- El tercer módulo que estará compuesto por el servidor web y el sistema de almacenamiento de datos. Se desarrollará utilizando Node.js [42], que es un entorno de ejecución Javascript para crear aplicaciones en el lado del servidor. Haciendo uso también del framework Express.js [43] para la definición de las API's de la aplicación. Para el almacenamiento de datos se ha hecho uso del sistema de gestión de bases de datos PostgreSQL [26].

- El cuarto módulo lo constituye la interfaz web. La interfaz web se desarrollará bajo el framework de frontend, Vue.js [33]. Este framework permite el desarrollo de interfaces de usuario interactivas y aplicaciones web de una sola página (SPA [31]). Se destaca por su facilidad de uso y su arquitectura basada en componentes, permitiendo crear una interfaz de usuario fácilmente escalable. Para facilitar aún más el desarrollo de la interfaz de usuario se ha hecho uso del metaframework de Vue.js llamado Nuxt.js [32]. Un metaframework permite extender las funcionalidades de un framework y proveer mecanismos de automatización de ciertas tareas. Por ejemplo, el uso de Nuxt.js simplifica enormemente el manejo de rutas, que de otra manera habría que configurarlas en un fichero aparte, siendo tedioso cuando el proyecto es extenso y tenemos muchas páginas distintas. Además simplifica el uso de middleware para la protección de rutas y provee de mecanismos que simplifican la llamadas a las API's del servidor web desde el frontend. Por último para darle estilo a la aplicación se ha utilizado el framework Tailwindcss [44] y estilos personalizados utilizando CSS Vanilla [45].

4.2.1.2. Entorno Hardware

Se destaca:

- El primer módulo requerirá del microcontrolador Arduino MKR WAN 1310, sensores DHT11 para temperatura y humedad ambiental, sensor capacitive soil moisture v1.2 para humedad del suelo, y un actuador para el control de la bomba de riego. El microcontrolador incluirá un módulo LoRaWAN integrado que permitirá la comunicación inalámbrica de largo alcance con el gateway.

- El segundo módulo requerirá de un gateway LoRaWAN que actuará como punto de acceso inalámbrico, recibiendo las transmisiones de los dispositivos Edge y proporcionando conectividad a la red IoT LoRaWAN. El gateway deberá ser

compatible con el protocolo LoRaWAN y estar configurado para comunicarse con The Things Network. Por otro lado The things Network es una plataforma cloud que cuenta con sus propios servidores dedicados.

- El tercer módulo requerirá almacenar en cloud tanto el servidor web de la aplicación como el sistema de almacenamiento de datos. Al igual que el frontend.

4.2.1.3. Entorno de Programación

El código para el primer módulo se desarrollará utilizando PlatformIO como entorno de desarrollo integrado en Visual Studio Code, con el lenguaje Arduino C++ y las librerías específicas para comunicación LoRaWAN y sensores.

Para el módulo 3, se utilizará Visual Studio Code como editor principal, con extensiones específicas para Node.js y JavaScript. Se empleará npm como gestor de paquetes y se utilizarán herramientas de desarrollo como nodemon para el reinicio automático del servidor durante el desarrollo cada vez que se efectúe un cambio. La configuración de TTN se realizará a través de su interfaz web y API's REST [46].

El desarrollo de la Interfaz Web se realizará en Visual Studio Code con extensiones para Vue.js y Nuxt.js. Se utilizará también npm [47] para la gestión de dependencias.

4.2.1.4. Entorno de Usuario

La interfaz web será donde el usuario ejecute al funcionalidades del sistema, ya que la parte del hardware esta totalmente automatizada. Obviamente con previa identificación y autenticación en el sistema. La aplicación web proporcionará una interfaz intuitiva y responsiva que permitirá a los agricultores monitorizar sus cultivos en tiempo real a través de una dashboard, configurar diferentes modos de riego (manual, programado y automático), gestionar sus dispositivos IoT y cultivos, además de recibir alertas sobre eventos del sistema. La interfaz web estará diseñada para ser accesible sin necesidad de conocimientos técnicos específicos, permitiendo a los usuarios gestionar eficientemente el riego de sus cultivos.

4.2.1.5. Vida esperada

La aplicación web podrá ser utilizada indefinidamente siempre que sea compatible con versiones posteriores de los navegadores web modernos y siempre y cuando no se descubra alguna vulnerabilidad en un paquete npm que implique actualizarlo. Por otro lado los servicios de TTN estarán disponibles mientras que la plataforma siga activa.

El sistema estará desarrollado de manera modular y escalable, permitiendo que la actualización de componentes individuales (incluir nuevas funcionalidades) no requiera una reestructuración completa del código. Dependiendo de estos factores, se estima una vida útil operativa de entre 1 a 2 años.

4.2.2. Ciclo de mantenimiento

El mantenimiento del sistema no será necesario de forma periódica una vez se libere la versión definitiva, no obstante, se requerirá intervención en los siguientes casos:

- Corrección de errores detectados en el funcionamiento del sistema
- Actualización de dependencias de seguridad en el servidor web y frontend.
- Integración con nuevas funcionalidades o sensores adicionales
- Actualización de la interfaz de usuario para mejorar la experiencia del usuario

El sistema estará diseñado de tal manera que un cambio en el servidor web no tenga repercusiones mayores sobre la operatividad del sistema. Permitiendo que el resto de funcionalidades sigan funcionando perfectamente.

4.2.3. Competencia

Como competencia del software existen diversas soluciones de riego inteligente y plataformas IoT agrícolas que ofrecen sistemas de gestión de cultivos con funcionalidades similares, no obstante, este proyecto estará desarrollado para un propósito muy específico de automatización de riego basado en sensores conectados a un Arduino MKR WAN 1310 con conectividad LoRa, diferenciándose por la integración con TTN y el protocolo de comunicación a largo alcance LoRaWAN, el enfoque específico en la gestión de riego automático con tres modos de operación, la arquitectura distribuida Edge-Fog-Cloud bien definida, el sistema de roles con funcionalidades específicas para visitantes, usuarios y administradores, y la interfaz web híbrida desarrollada con Nuxt.js y Vue.js.

4.2.4. Aspecto externo

El aspecto externo de la aplicación será fundamental para el uso efectivo del sistema. La interfaz de usuario estará diseñada siguiendo principios de diseño moderno y accesibilidad, utilizando una paleta de colores que impliquen reducir el daño a la

vista lo máximo posible ante usos prolongados de la aplicación, cuenta con una gran variedad de iconos que servirán como guía para navegar por todas las funcionalidades del sistema por parte del usuario. El diseño será completamente responsivo, adaptándose correctamente a dispositivos móviles, tablets y ordenadores de escritorio, con una usabilidad que permita a los agricultores usar la aplicación sin necesidad de formación técnica previa. La aplicación web contará con un dashboard principal que constará de gráficas que se irán trazando con los datos ambientales y de suelo provenientes de los sensores. El modo demo permitirá a los visitantes explorar la funcionalidad del sistema de manera parcial y con datos simulados.

4.2.5. Estandarización

En el proyecto se aplicará una serie estándares que permitirán crear un sistema uniforme y lo más simple posible. Por un lado el proyecto contará con el seguimiento de prácticas RESTful para el diseño de endpoints HTTP en la API REST. En relación al diseño de la aplicación se ha seguido el patrón de diseño Model-View-Controller (MVC) con el objetivo de aplicar un diseño ya estandarizado y fácilmente mantenible y escalable. Por otro lado, se ha llevado a cabo la implementación de JWT [36] para autenticación y HTTP para comunicación en el entorno de desarrollo. Otro aspecto a destacar es el seguimiento de las mejores prácticas de normalización y optimización de la base de datos, así como el desarrollo con Vue.js y Nuxt.js ddel frontend siguiendo las guías oficiales de estilo mientras que para el lado del servidor (servidor web) se ha desarrollado el código en el entorno Node.js siguiendo las convenciones de Express.js y las mejores prácticas de JavaScript ES6+. Por otro lado el protocolo de comunicación LoRaWAN es un protocolo ya estandarizado y bien especificado.

4.2.6. Calidad y fiabilidad

Este es uno de los apartados más importantes de nuestro proyecto puesto que una falta de calidad y fiabilidad en el sistema no solo repercutirá en el usuario final de la aplicación sino que también repercutirá en la escalabilidad de la aplicación y en la manera en la que responderá a fallos imprevistos que puedan afectar a la disponibilidad del sistema. Para mejorar la calidad se llevará a cabo una serie de buenas prácticas, como por ejemplo seguir una estructura modularizada y desacoplada por módulos. Además se seguirán una serie estándares de programación limpio con el fin de alcanzar un alto grado de mantenibilidad.

Respecto a la fiabilidad esta va a depender de la disponibilidad del servidor, en nuestro caso, el sistema soportará múltiples usuarios accediendo de forma simultanea a la web por lo que será altamente fiable.

4.2.7. Programación temporal

De acuerdo con el Reglamento de Trabajo Fin de Grado de la Escuela Politécnica Superior de Córdoba (EPSC), la valoración del TFG en créditos es de 12, esto siendo equivalente a unas 300 horas de trabajo. Por esto mismo, la programación en la realización del proyecto se ha hecho en base a esta exigencia de tiempo. Sabiendo esto, las 300 horas se han distribuido en las siguientes fases (Tabla 1):

Tabla 4.1: Planificación temporal del proyecto

Fase	**Tiempo (Horas)**
Preparación del entorno y aprendizaje	15
Análisis de requisitos	25
Diseño del sistema	40
Desarrollo e implementación	130
Prueba	45
Documentación	45
Total	**300**

4.2.8. Pruebas

- **Pruebas unitarias:** Este tipo de pruebas se realizarán con el fin de descubrir casos de uso que puedan dar lugar a fallos inesperados o no controlados por la aplicación, dando lugar a correcciones en base a los resultados obtenidos.
- **Pruebas de integración:** Este tipo de pruebas verifican la conectividad entre los diferentes módulos del sistema. En nuestro caso, el sistema seguirá una arquitectura distribuida formada por los módulos principales, Edge, Fog, Cloud Computing e Interfaz web. Estas pruebas permitirán asegurar que estos módulos funcionan de manera coordinada. Se validará que los datos fluyan correctamente desde los sensores del módulo Edge, a través del Gateway LoRaWAN hasta llegar al servidor de red del módulo Fog, para luego ser reenviados al servidor web, y finalmente alcanzar su destino en la base de datos. También se hará pruebas de integración sobre el recorrido inverso, es decir, desde la interfaz web hasta el microcontrolador.

4.2.9. Seguridad

En lo que concierne a nuestra aplicación, la seguridad es un factor clave. Por ello se implementarán medidas de seguridad a distintos niveles.

A nivel de comunicación IoT, nuestro sistema utilizará autenticación OTAA para LoRaWAN, que asegurará que únicamente dispositivos previamente autorizados podrán conectarse a la red. Estas credenciales de acceso se grabarán directamente en el código del microcontrolador del Arduino MKR WAN 1310. Los datos transmitidos entre el microcontrolador Arduino y los gateways LoRaWAN se cifran utilizando claves de sesión que se generan dinámicamente durante el proceso de activación, proporcionando una capa adicional de seguridad. La comunicación con TTN se realiza exclusivamente a través de conexiones Hypertext Transfer Protocol Secure (HTTPS) [48], garantizando que toda la información transmitida esté protegida contra interceptaciones no autorizadas.

A nivel de la aplicación web, se implementará un sistema de autenticación basado en JWT que validará la identidad de los usuarios y gestionará sus sesiones de forma segura. Las contraseñas de los usuarios se protegerán utilizando el algoritmo bcrypt [49], que convertirá las contraseñas en códigos hash irreversibles, protegiendo la información sensible incluso en el caso de que la base de datos sea comprometida. El sistema también implementará un control de acceso basado en roles que asigna permisos específicos a cada tipo de usuario, asegurando que cada persona tenga acceso únicamente a las funcionalidades que le corresponden según su rol.Todas las rutas de la API REST estarán protegidas mediante middleware de autenticación que verificará la validez del token JWT en cada petición, bloqueando automáticamente cualquier intento de acceso por parte de usuarios no autorizados a los endpoints del sistema.

Por último, el dispositivo físico estará protegido mediante encapsulamiento en una carcasa resistente a condiciones ambientales adversas.

4.3. Recursos

En este capítulo se expondrán cuáles son los recursos que serán utilizados a lo largo del proyecto para su desarrollo, se dividirá en recursos humanos y recursos materiales, tanto hardware como software.

4.3.1. Recursos Humanos

Los recursos humanos disponibles para este proyecto son:

Autor

- Nombre: Miguel Ángel Corredor Ferrer
- Titulación: Grado en Ingeniería Informática
- Mención: Ingeniería de Computadores
- Correo electrónico: i12cofem@uco.es

Directores

- Ezequiel Herruzo Gómez

  Departamento de Ingeniería Electrónica y de Computadores

  Escuela Politécnica Superior de Córdoba - Universidad de Córdoba

- Rafael M. Navarro Cerrillo

  Departamento de Ingeniería Agroforestal

  Escuela Técnica Superior Ingeniería Agronómica y de Montes - Universidad de Córdoba

4.3.2. Recursos Hardware y Software

A continuación, se describirán los recursos tanto hardware, como software.

4.3.2.1. Recursos Hardware

1. Ordenador portátil:
   a) CPU: Intel Core i7 - 10875H.
   b) RAM: 32 GB.
   c) GPU: NVIDIA GeForce RTX 2070 Mobile / Max-Q.
2. Sensor de humedad del suelo: Capacitive Soil Moisture Sensor v1.2 [12].
3. Sensor de temperatura y humedad ambiental: DHT11 [11]
4. Microcontrolador Arduino MKR WAN 1310 [23].

5. D260BLX-V Brushless Micropump Range [10].
6. Gemtek / MiniHub v1.5 Wifi Indoor LoRaWAN Gateway [24].
7. HUAREW 5V El aislamiento del optoacoplador del módulo de relé de 1 canal activación [14].
8. Caja de conexión estanca IP55 170x220x85 mm [50]
9. 2 tuberías de 2 mm de diámetro.
10. Protoboard

4.3.2.2. Recursos Software

1. Sistema operativo del equipo personal: Microsoft Windows 10 Home [51].
2. Entornos de desarrollo: Visual Studio Code [39] con plugin PlatformIO [38].
3. Lenguajes de programación:
   - Aplicación web: Javascript [41].
   - Programación del microcontrolador. C/C++ [37].
4. Frameworks:
   a) Framework Frontend: Vue.js [33] + Nuxt.js + Tailwindcss [44]
   b) Framework Backend: Node.js [42] + Express.js [43]
5. Ecosistema gestor de la red: The Things Network [25].
6. Editor de documentación: LaTeX con Overleaf [52].
7. Sistema gestor de base de datos: PostgreSQL [26].

4.4. Especificación de requisitos

En esta sección se listarán con detalle los requisitos que componen la funcionalidad principal de la aplicación. Para considerar que el proyecto ha sido finalizado con éxito se deben de cumplir con todos y cada uno de los requisitos. Se ha dividido los requisitos en 3 grupos: funcionales, no funcionales y de información.

Los requisitos funcionales son son un conjunto de especificaciones detalladas que describen lo que un sistema, aplicación o producto debe hacer para cumplir con

sus objetivos. Estas especificaciones son esenciales para traducir las necesidades del usuario en funciones concretas que guían el diseño, desarrollo e implementación del sistema [53].

Los requisitos no funcionales son criterios que definen cómo debe comportarse un sistema en lugar de describir sus funciones específicas. Estos se centran en aspectos como la seguridad, la usabilidad, el rendimiento, la escalabilidad y la disponibilidad del sistema. Además, estos requisitos son cruciales para el éxito de un proyecto, ya que ayudan a establecer expectativas claras sobre el rendimiento y la calidad del software [54].

4.4.1. Requisitos funcionales

ID	Descripción
RF1	El sistema deberá permitir a los visitantes registrarse proporcionando email, contraseña y nombre completo.
RF2	El sistema deberá enviar un email de verificación automático al completar el registro.
RF3	El sistema deberá permitir a los usuarios confirmar su cuenta mediante un enlace en el email de verificación.
RF4	El sistema deberá permitir a los usuarios verificados iniciar sesión con email y contraseña.
RF5	El sistema deberá generar y gestionar tokens JWT para mantener sesiones activas.
RF6	El sistema deberá permitir a los usuarios solicitar recuperación de contraseña mediante email.
RF7	El sistema deberá permitir a los usuarios restablecer su contraseña.
RF8	El sistema deberá permitir a los usuarios editar su perfil (nombre y contraseña).
RF9	El sistema deberá permitir a los usuarios eliminar su propia cuenta.
RF10	El sistema deberá proporcionar un modo demo para visitantes sin registro.
RF11	El sistema deberá permitir a los usuarios activar/desactivar la comunicación del dispositivo IoT con la base de datos.

ID	Descripción (cont.)
RF12	El sistema deberá permitir activar únicamente un dispositivo por usuario simultáneamente.
RF13	El sistema deberá permitir a los usuarios agregar dispositivos IoT con credenciales TTN, además de editarlos y eliminarlos.
RF14	El sistema deberá permitir a los usuarios crear cultivos especificando nombre, descripción, categoría, días de crecimiento, temporada de cosecha, foto de cultivo y configurando los parámetros de cultivo como humedad mínima del suelo, humedad máxima del suelo, humedad mínima del ambiente, humedad máxima del ambiente y temperatura máxima.
RF15	El sistema deberá permitir a los usuarios seleccionar únicamente un cultivo activo a la vez.
RF16	El sistema deberá permitir a los usuarios visualizar y editar la información de sus cultivos, y además poder eliminarlos.
RF17	El sistema deberá verificar automáticamente si las condiciones del suelo están en rango óptimo para el cultivo.
RF18	El sistema deberá permitir activar únicamente un modo de riego a la vez.
RF19	El sistema deberá permitir seleccionar entre los 3 modos de riego: Manual, Programado y Automático.
RF20	El sistema deberá permitir configurar la duración del riego en el modo manual y programado.
RF21	El sistema deberá permitir pausar y reanudar el riego.
RF22	El sistema deberá permitir cancelar el riego en cualquier momento.
RF23	El sistema deberá permitir cancelar la configuración para modo automático y programado en cualquier momento.
RF24	El sistema deberá permitir configurar umbrales para activación automática.
RF25	El sistema deberá activar el riego automáticamente cuando las condiciones excedan los umbrales.
RF26	El sistema deberá permitir programar riegos con fecha, hora específicas y definir frecuencia de riego.

ID	Descripción (cont.)
RF27	El sistema deberá generar alertas al iniciar y finalizar riegos programados.
RF28	El sistema deberá recibir datos de sensores de temperatura y humedad cada aproximadamente 2 minutos vía webhook TTN.
RF29	El sistema deberá validar que los datos provienen de dispositivos autorizados.
RF30	El sistema deberá permitir al usuario acceder a los últimos 20 datos de sensores en tiempo real en el dashboard.
RF31	El sistema deberá simular datos de sensores en modo demo.
RF32	El sistema deberá permitir a los usuarios enviar ordenes de activación/desactivación de la bomba de riego vía downlink TTN.
RF33	El sistema deberá generar alertas por acciones de usuario o cambios de estado en la aplicación.
RF34	El sistema deberá permitir marcar alertas como resueltas.
RF35	El sistema deberá mostrar historial completo de alertas por usuario.
RF36	El sistema deberá permitir al administrador crear, editar y eliminar usuarios.
RF37	El sistema deberá permitir al administrador crear, editar y eliminar cultivos.
RF38	El sistema deberá permitir al administrador crear, editar y eliminar dispositivos.
RF39	El sistema deberá permitir la captación de datos de temperatura y humedad ambiental y del suelo a través de los sensores
RF40	El sistema deberá transmitir los datos capturados a TTN
RF41	El sistema debe almacenar los datos recibidos después de verificar su validez y formato.
RF42	El sistema deberá permitir al usuario verificado y administrador cerrar sesión
RF43	El servidor de TTN deberá recibir datos de uplink y downlink, y preprocesarlos.
RF44	El sistema deberá permitir recibir los downlinks y procesarlos para activar el riego

ID	Descripción (cont.)
RF45	El sistema deberá permitir al usuario verificado listar los cultivos y dispositivos disponibles
RF46	El sistema deberá permitir al administrador listar los usuarios, cultivos y dispositivos disponibles

Tabla 4.2: Requisitos Funcionales

4.4.2. Requisitos no funcionales

ID	Descripción
RNF1	El sistema deberá almacenar las contraseñas de los usuarios usando bcrypt con un factor de coste mínimo de 12.
RNF2	La actualización de datos de sensores en el dashboard deberá realizarse cada aproximadamente 2 min debido a que las condiciones climatológicas no cambian de un segundo a otro.
RNF3	Los tokens JWT deberán tener una expiración de 30d y ser invalidados al cerrar sesión..
RNF4	El lenguaje de programación utilizado será javascript junto con c++ para la programación del microcontrolador.
RNF5	El sistema deberá de proveer un sistema de protección de rutas para evitar que usuarios puedan acceder a funcionalidades para las cuales su rol no está definido.
RNF6	Toda la comunicación con The Things Network deberá realizarse mediante HTTPS y claves de acceso encriptadas.
RNF7	El acceso al panel de administración deberá estar restringido únicamente a usuarios con rol administrador verificado.
RNF8	Los fallos de comunicación con TTN no deberán afectar la funcionalidad del sistema web.
RNF9	La interfaz de usuario deberá ser responsive y adaptarse a dispositivos móviles, tablets y PC.
RNF10	El sistema deberá proporcionar retroalimentación visual inmediata tras la interacción del usuario.

ID	Descripción (cont.)
RNF11	Los formularios deberán incluir validación en tiempo real con mensajes de error claros y comprensibles.
RNF12	El modo demo deberá simular datos realistas para permitir al usuario visitante previsualizar una aproximación al sistema.
RNF13	La arquitectura deberá facilitar la integración de nuevos tipos de sensores sin modificaciones mayores.

Tabla 4.3: Requisitos no funcionales

4.5. Casos de uso

Para la descripción funcional del sistema, usaremos diagramas de caso de uso construidos a partir del estándar Unified Modeling Languague (UML) [55]. Debido a la envergadura del proyecto, a continuación, se ha facilitado una tabla para comprender los casos de uso que describen la funcionalidad del sistema.

ID	Nombre del caso de uso
CU-01	Recepción de datos
CU-02	Enviar datos (uplink)
CU-03	Recibir datos (Downlink)
CU-04	Ejecutar riego
CU-05	Recolectar uplink
CU-06	Preprocesar (decode/encode)
CU-07	Reenviar a Backend
CU-08	Recepción downlink
CU-09	Recepción y postproceso
CU-10	Emitir downlink TTN
CU-11	Autenticar
CU-12	Almacenar datos
CU-13	Login
CU-14	Registro
CU-15	Acceder Dashboard RT

ID	Nombre del caso de uso
CU-16	Iniciar riego
CU-17	Cancelar riego
CU-18	Confirmar cuenta
CU-19	Restablecer contraseña
CU-20	Entrar en modo demo
CU-21	Crear usuario
CU-22	Listar usuarios
CU-23	Ver usuario
CU-24	Editar usuario
CU-25	Eliminar usuario
CU-26	Cerrar sesión
CU-27	Listar cultivos
CU-28	Crear cultivo
CU-29	Editar cultivo
CU-30	Eliminar cultivo
CU-31	Seleccionar/Deseleccionar cultivo
CU-32	Crear dispositivo
CU-33	Editar dispositivo
CU-34	Eliminar dispositivo
CU-35	Activar/Desactivar comunicación
CU-36	Listar dispositivos
CU-37	Marcar alerta resuelta
CU-38	Ver Historial completo de alertas
CU-39	Ver perfil
CU-40	Seleccionar modo de riego
CU-41	Configurar modo manual
CU-42	Configurar modo programado
CU-43	Configurar modo automático
CU-44	Definir programación

ID	Nombre del caso de uso
CU-45	Definir umbrales
CU-46	Pausar/Reanudar riego
CU-47	Cancelar programación completa

Tabla 4.4: Casos de uso del sistema completo

Lo primero que haremos, será realizar un diagrama de casos de uso general, en el cual englobaremos las funcionalidades generales del sistema, de una manera simplificada e intuitiva. Para mejorar la legibilidad se ha decidido dividir en 5 subgrupos los diagramas de caso de uso, en función de los actores que intervienen. El primer grupo de casos de uso pertenece al actor microcontrolador, el segundo grupo de casos de uso pertenece al actor TTN, los 3 grupos restantes pertenece a los usuarios verificados de la aplicación web.

4.5.1. Sistema Completo

Este diagrama representa una perspectiva general de los casos de uso que describen la funcionalidad del sistema completo. Vemos como está dividido en 4 módulos. El primer módulo representa la capa de Edge Computing donde se da la captación de datos y el accionamiento del actuador para la ejecución del riego, se representan los casos de uso necesarios para describir la funcionalidad que implica al actor microcontrolador. En el segundo módulo, se representará la capa de Fog Computing y en la que participará el actor TTN que también ejecuta sus propios casos de uso imprescindibles para la funcionalidad total del sistema. En el tercer módulo y cuarto módulo el principal actor es el propio usuario verificado de la aplicación, en este diagrama no se han representado todas las funcionalidades ya que quedaría visualmente ilegible, por lo que, en secciones inferiores se han descritos todos los casos de uso, en diferentes diagramas de casos de uso, para los distintos usuarios de la aplicación web.

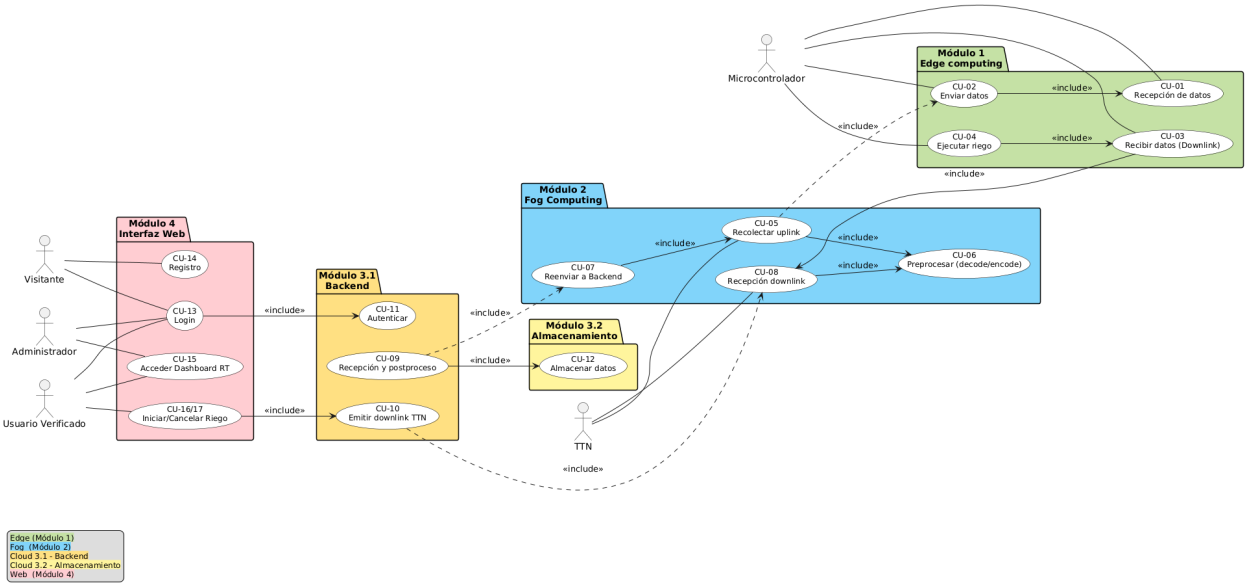


Figura 4.2: Casos de uso del sistema completo

4.5.1.1. Diagrama de casos de uso del microcontrolador

En este apartado se expondrá el diagrama de casos de uso del módulo 1, cuyo actor principal es el microcontrolador.

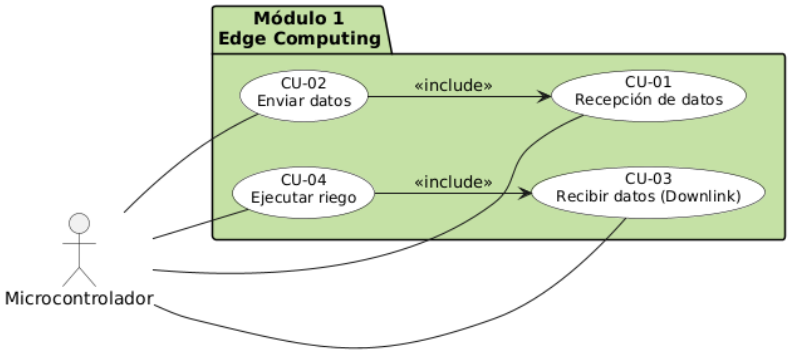


Figura 4.3: Casos de uso del Microcontrolador

4.5.1.2. Diagrama de casos de uso de TTN

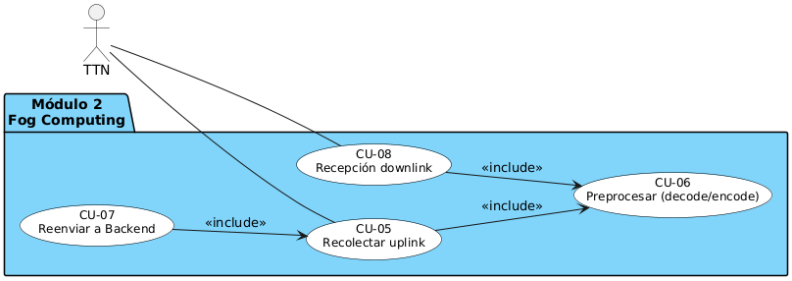


Figura 4.4: Casos de uso del sistema completo

4.5.1.3. Diagrama de casos de uso de Visitante

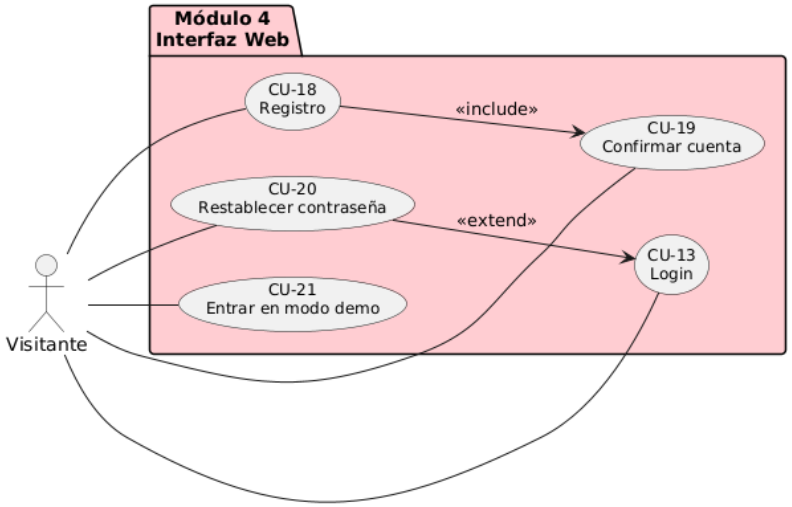


Figura 4.5: Casos de uso del Visitante

4.5.1.4. Diagrama de Casos de uso de Administrador

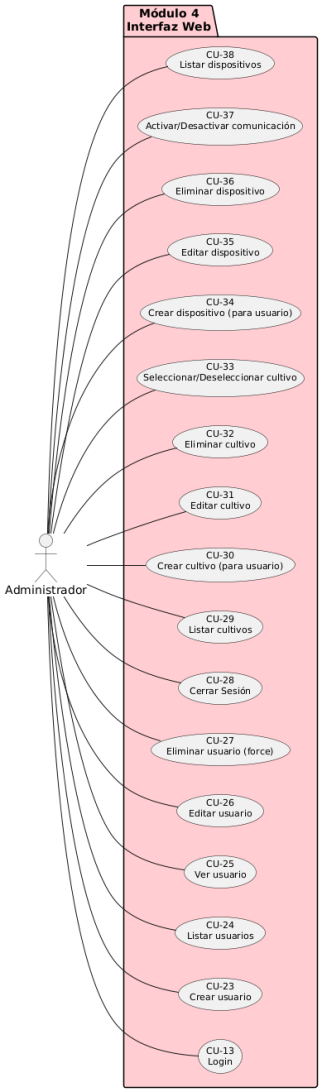


Figura 4.6: Casos de uso del Administrador

4.5.1.5. Casos de uso de Usuario Verificado

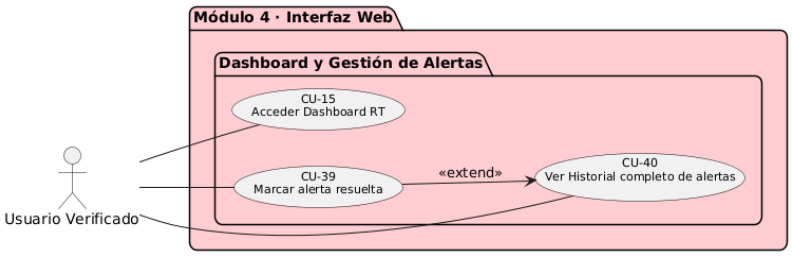


Figura 4.7: Casos de uso del Usuario Verificado - Dashboard y Gestión de Alertas

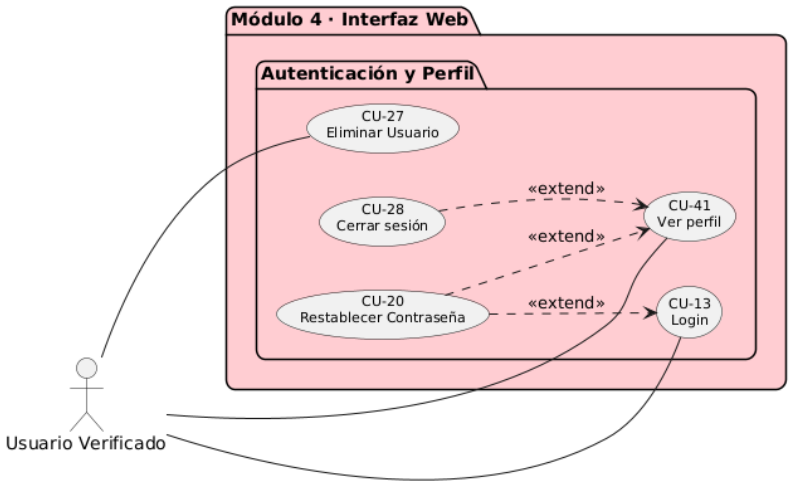


Figura 4.8: Casos de uso del Usuario Verificado - Autenticación y perfil

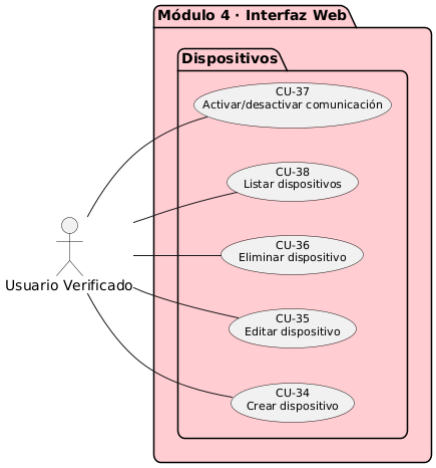


Figura 4.9: Casos de uso del Usuario Verificado - Dispositivos

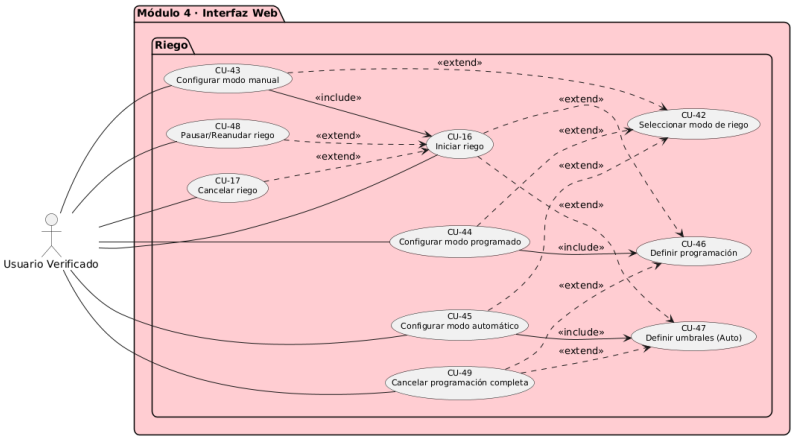


Figura 4.10: Casos de uso del Usuario Verificado - Riego

Una vez expuestos los diagramas de caso de uso que describen la funcionalidad de nuestra aplicación, vamos a a especificar todos y cada uno de ellos

4.5.2. Especificación de los casos de uso

Tabla 4.5: Especificación del Caso de Uso: Recepción de datos

Identificador	CU-01
Nombre	Recepción de datos
Descripción	El microcontrolador obtiene datos de temperatura y humedad del sensor DHT11 y de humedad del suelo del sensor capacitivo, aplicando conversión y validación para garantizar que los valores sean coherentes y utilizables.
Actor principal	Microcontrolador Arduino MKR WAN 1310
Actor secundario	–
Precondiciones	1. Los sensores deben estar conectados físicamente al microcontrolador. 2. El microcontrolador debe estar alimentado.
Flujo principal	1. El microcontrolador solicita la lectura de temperatura y humedad relativa al sensor DHT11. 2. Obtiene el valor de humedad del suelo desde el sensor capacitivo. 3. Convierte la lectura de humedad a un porcentaje, ajustándola al rango establecido. 4. Limita los valores de humedad al rango permitido (0–100 %). 5. Verifica que la temperatura y humedad relativa sea un valor válido y no un error de lectura. 6. Prepara los valores para su transmisión, adaptándolos al formato definido.
Postcondiciones	1. Los valores de temperatura y humedad están listos para su transmisión. 2. Los datos han sido validados y cumplen con el formato esperado. 3. Los valores quedan disponibles para revisión o depuración local.
Flujo alternativo	▪ Si el sensor DHT11 devuelve un error, el sistema lo registra y omite la transmisión. ▪ Si el valor de humedad está fuera del rango de calibración, el sistema lo corrige automáticamente. ▪ Si ambos sensores fallan, el sistema reinicia el ciclo de lectura.

Tabla 4.6: Especificación del Caso de Uso: Enviar datos (uplink)

Identificador	CU-02
Nombre	Enviar datos (uplink)
Descripción	El microcontrolador envía los datos de temperatura y humedad al gateway LoRaWAN, encapsulados en un formato predefinido, a intervalos regulares.
Actor principal	Microcontrolador Arduino MKR WAN 1310
Actor secundario	–
Precondiciones	1. El dispositivo debe estar registrado y conectado a la red TTN mediante OTAA. 2. Los datos deben haber sido leídos y validados previamente. 3. La configuración de comunicación debe corresponder a la región y parámetros establecidos.
Flujo principal	1. Include(*Recepción de datos 4.5*) 2. Los datos se empaquetan siguiendo el formato de transmisión definido. 3. El microcontrolador envía el paquete al gateway mediante la conexión LoRaWAN. 4. Se registra el intento de envío para control y depuración. 5. El sistema espera un tiempo predefinido antes de la próxima transmisión, aproximadamente 2 minutos.
Postcondiciones	1. Los datos se han transmitido al gateway. 2. Se registra si el envío ha sido exitoso o ha fallado. 3. El microcontrolador queda en espera del próximo ciclo de transmisión.
Flujo alternativo	▪ Si ocurre un error en el envío, se registra y se intentará en el siguiente ciclo. ▪ Si la conexión OTAA se pierde, el sistema requerirá reinicio manual para reconexión.

Tabla 4.7: Especificación del Caso de Uso: Recibir datos (downlink)

Identificador	CU-03
Nombre	Recibir datos (downlink)
Descripción	El microcontrolador recibe instrucciones desde TTN en las ventanas de recepción posteriores a un envío (uplink), interpretando la orden de riego y preparandose para apagar o encender la bomba de riego.
Actor principal	Microcontrolador Arduino MKR WAN 1310
Actor secundario	–
Precondiciones	1. Debe haberse enviado un mensaje uplink para abrir las ventanas de recepción. 2. Debe existir un mensaje downlink en la cola de TTN para este dispositivo.
Flujo principal	1. El microcontrolador verifica periódicamente si hay datos entrantes. 2. El microcontrolador envía un uplink vacío periódico. 3. Si existe un mensaje, lo lee y obtiene el valor de la orden. 4. El sistema interpreta la orden según la codificación establecida (por ejemplo, encender o apagar el actuador). 5. Include(**Recibir datos 4.7**)
Postcondiciones	1. El comando ha sido interpretado y queda listo para ejecución. 2. Se registra el estado resultante para control y depuración.
Flujo alternativo	▪ Si no hay una orden disponible, el sistema continúa su operación normal. ▪ Si hay error en la lectura, se ignora la orden y se mantiene el estado actual.

Tabla 4.8: Especificación del Caso de Uso: Ejecutar riego

Identificador	CU-04
Nombre	Ejecutar riego
Descripción	El microcontrolador activa o desactiva la bomba de agua en función de la orden de riego recibida por downlink y envía una confirmación al servidor de red de TTN.
Actor principal	Microcontrolador Arduino MKR WAN 1310
Actor secundario	–
Precondiciones	1. Debe haberse recibido un comando válido para control de la bomba. 2. El dispositivo IoT debe estar conectado y operativo.
Flujo principal	1. Include(*Recibir datos 4.7*) 2. El microcontrolador cambia el estado del actuador a encendido o apagado.
Postcondiciones	1. El actuador adopta el estado solicitado.

Tabla 4.9: Especificación del Caso de Uso: Recolectar Uplink

Identificador	CU-05
Nombre	Recolectar uplink
Descripción	El servidor TTN recibe y procesa los datos uplink enviados desde el dispositivo LoRaWAN a través del gateway.
Actor principal	TTN
Actor secundario	–
Precondiciones	1. Debe haberse recibido un orden válida para control de la bomba. 2. El dispositivo IoT debe estar conectado y operativo.
Flujo principal	1. Include(*Enviar datos 4.6*) 2. TTN recibe la señal LoRaWAN del gateway. 3. Se verifica la autenticidad del dispositivo mediante autenticación OTAA 4. Se valida la integridad del mensaje 5. Include(*Preprocesar decode/encode 4.10*). 6. Se extrae metadata de transmisión (RSSI, SNR, timestamp) 7. Se prepara el mensaje para webhook
Postcondiciones	1. El mensaje está verificado y decodificado 2. Los datos están listos para ser reenviados al endpoint HTTP configurado en el webhook HTTP 3. Se registra la recepción en los logs de TTN

Tabla 4.10: Especificación del Caso de Uso: Preprocesar (decode/encode)

Identificador	CU-06
Nombre	Preprocesar (decode/encode)
Descripción	TTN decodifica payloads uplink y codifica payloads downlink según los formatos definidos para la aplicación.
Actor principal	TTN
Actor secundario	–
Precondiciones	1. Debe existir un payload formatter válido configurado en TTN para preprocesar 2. El formato de datos debe ser coherente
Flujo principal	1. TTN identifica el tipo de mensaje (uplink/downlink) 2. Para uplink: decodifica datos de sensores desde en formato JSON desde base hexadecimal 3. Para downlink: codifica comandos a formato hexadecimal 4. Include(*Preprocesar decode/encode 4.10*). 5. Se valida la estructura resultante 6. Se prepara el mensaje para webhook
Postcondiciones	1. El mensaje está verificado y decodificado 2. Los datos están listos para ser reenviados al endpoint HTTP configurado en el webhook HTTP 3. Se registra la recepción en los logs de TTN

Tabla 4.11: Especificación del Caso de Uso: Reenviar a Backend

Identificador	CU-07
Nombre	Reenviar a Backend
Descripción	TTN envía los datos procesados al backend de la aplicación mediante webhook HTTP configurado.
Actor principal	TTN
Actor secundario	–
Precondiciones	1. El backend debe estar disponible y un dominio dns asignado. 2. El webhook debe estar configurado correctamente 3. Los datos deben estar decodificados y validados
Flujo principal	1. Include(*Preprocesar decode/encode 4.10*). 2. Se incluyen datos en el payload JSON en base a los filtros configurados en el webhook TTN 3. Se realiza petición HTTP POST al endpoint configurado en el webhook desde el backend. 4. Se espera confirmación del backend 5. Se registra el envío exitoso
Postcondiciones	1. Los datos han llegado al backend correctamente 2. Se confirma la recepción por parte del servidor

Tabla 4.12: Especificación del Caso de Uso: Recepción downlink

Identificador	CU-08
Nombre	Recepción downlink
Descripción	TTN recibe comandos downlink desde el backend y los encola para envío al dispositivo en la próxima ventana de transmisión.
Actor principal	TTN
Actor secundario	–
Precondiciones	1. El backend debe de hacer la consulta a la API TTN con las credenciales correctas. 2. El dispositivo IoT objetivo debe estar operativo y conectado 3. Los datos deben estar decodificados y validados
Flujo principal	1. TTN recibe petición POST con comando downlink desde API TTN consultada desde el backend 2. Se verifica que el dispositivo esté activo 3. Include(*Preprocesar decode/encode 4.10*). 4. Se encola el mensaje hasta que se abran las próximas ventanas de recibo RX1 y RX2
Postcondiciones	1. TTN espera próximo uplink 2. El comando está encolado para envío
Flujo alternativo	▪ Si la autenticación falla, se rechaza la petición ▪ Si el dispositivo está inactivo, se notifica error

Tabla 4.13: Especificación del Caso de Uso: Recepción downlink

Identificador	CU-09
Nombre	Recepción y postproceso
Descripción	El sistema consulta el endpoint HTTP definido en el webhook de TTN con datos de sensores y los procesa para almacenamiento y análisis automático.
Actor principal	Usuario Verificado
Actor secundario	–
Precondiciones	1. El endpoint HTTP del webhook debe estar disponible 2. TTN debe tener configurado el webhook correctamente
Flujo principal	1. Include(*Reenviar Backend 4.11*) 2. Se valida la estructura del payload JSON 3. Se comprueba que el uplink recibido proviene del puerto 2 4. Se extrae información del payload, sobre datos del dispositivo y de los sensores 5. El sistema comprueba que el dispositivo desde provienen los datos de los sensores está registrado en la base de datos y está activo 6. Include(*Almacenar datos 4.16*)
Postcondiciones	1. Los datos están en almacenamiento
Flujo alternativo	▪ Si el dispositivo no tiene activa la comunicación, no guarda la información en la base de datos

Tabla 4.14: Especificación del Caso de Uso: Recepción downlink

Identificador	CU-10
Nombre	Emitir downlink TTN
Descripción	El sistema envía ordenes de control de riego al dispositivo IoT a través de la API de TTN para activar/desactivar el sistema de riego.
Actor principal	Usuario Verificado
Actor secundario	–
Precondiciones	1. El usuario debe estar autenticado 2. Debe existir un dispositivo activo asociado 3. En almacenamiento deben existir las credenciales válidas de TTN para ese dispositivo. 4. Debe de crearse una API KEY en TTN.
Flujo principal	1. El backend recibe orden de control desde la interfaz web 2. Se valida la autenticación del usuario 3. Se verifica que el dispositivo esté activo 4. Se prepara payload con comando (ON/OFF) 5. Se codifica la orden de control en base64 6. Se realiza petición POST a API TTN 7. Include(*Recepción downlink 4.12*)
Postcondiciones	1. Los datos están en almacenamiento
Flujo alternativo	▪ Si falla autenticación, se rechaza comando

Tabla 4.15: Especificación del Caso de Uso: Recepción downlink

Identificador	CU-11
Nombre	Autenticar
Descripción	El sistema valida credenciales de usuarios y gestiona tokens JWT para mantener sesiones seguras en el sistema.
Actor principal	Todos los usuarios
Actor secundario	–
Precondiciones	1. El usuario debe existir en la base de datos
Flujo principal	1. El backend recibe credenciales (email/password). 2. Se consulta usuario en la base de datos. 3. Se verifica hash de contraseña con bcrypt. 4. Se valida que la cuenta esté verificada. 5. Se genera token JWT con expiración. 6. Se retorna token al cliente.
Postcondiciones	1. Se ha generado token JWT válido. 2. La sesión está activa.
Flujo alternativo	▪ Si las credenciales son incorrectas, se retorna error 401.

Tabla 4.16: Especificación del Caso de Uso: Almacenar datos

Identificador	CU-12
Nombre	Almacenar datos
Descripción	El sistema persiste en PostgreSQL todos los datos de sensores, configuraciones y eventos del sistema IoT.
Actor principal	Usuario Verificado
Actor secundario	-
Precondiciones	1. Debe existir conectividad con PostgreSQL
Flujo principal	1. Se inicia transacción en base de datos 2. Se inserta lectura en tabla la SensorReading 3. Se verifica integridad referencial 4. Se confirma transacción 5. Se indexan datos para consultas futuras
Postcondiciones	1. Los datos están disponibles para consultas
Flujo alternativo	-

Tabla 4.17: Especificación del Caso de Uso: Login

Identificador	CU-13
Nombre	Login
Descripción	El usuario inicia sesión en el sistema proporcionando sus credenciales para acceder a las funcionalidades de la aplicación.
Actor principal	Todos los usuarios
Actor secundario	–
Precondiciones	1. El usuario debe tener una cuenta registrada y verificada 2. Las credenciales deben ser válidas
Flujo principal	1. El usuario accede a la página de login 2. Introduce email y contraseña 3. Se envían credenciales al backend 4. Include(*Autenticar 4.15*) **Extension Point: *Restablecer contraseña 4.23***
Postcondiciones	1. El usuario está autenticado en el sistema 2. Se establece sesión activa con expiración de token
Flujo alternativo	▪ Si las credenciales son incorrectas, se muestra error

Tabla 4.18: Especificación del Caso de Uso: Registro

Identificador	CU-14
Nombre	Registro
Descripción	Un visitante se registra en el sistema proporcionando sus datos personales para crear una nueva cuenta de usuario.
Actor principal	Visitante
Actor secundario	-
Precondiciones	1. El visitante no debe tener cuenta existente con ese email 2. Los datos proporcionados deben ser válidos
Flujo principal	1. El visitante accede al formulario de registro 2. El visitante completa campos: nombre, email, contraseña 3. El sistema verifica que el email no esté registrado 4. El sistema hashea la contraseña con bcrypt 5. El sistema crea usuario con estado "no verificado" 6. El sistema envía email de verificación automático
Postcondiciones	1. El usuario está registrado pero no verificado 2. Se ha enviado email de verificación 3. El usuario debe verificar para activar cuenta
Flujo alternativo	▪ Si el email ya existe, se notifica error ▪ Si falla envío de email, se reintenta ▪ Si hay campos inválidos, se solicita corrección

Tabla 4.19: Especificación del Caso de Uso: Acceder Dashboard RT

Identificador	CU-15
Nombre	Acceder Dashboard RT
Descripción	El usuario autenticado accede al dashboard principal que muestra información en tiempo real de sus dispositivos y cultivos.
Actor principal	Usuario Verificado
Actor secundario	–
Precondiciones	1. Debe de haber un dispositivo seleccionado.
Flujo principal	1. Se obtienen lecturas recientes de sensores. 2. Se carga información del cultivo y dispositivo seleccionado si existe. 3. Se inicializa actualización automática cada aproximadamente 2 minutos. 4. Se muestran gráficas interactivas con tendencias.
Postcondiciones	-
Flujo alternativo	-

Tabla 4.20: Especificación del Caso de Uso: Iniciar riego

Identificador	CU-16
Nombre	Iniciar riego
Descripción	El usuario inicia el proceso de riego según el modo configurado (manual, programado o automático).
Actor principal	Usuario Verificado
Actor secundario	–
Precondiciones	1. El usuario debe estar autenticado 2. Debe tener dispositivo activo configurado 3. Debe haber seleccionado un modo de riego
Flujo principal	1. El usuario configura parámetros del riego seleccionado 2. Confirma inicio del riego 3. El sistema comprueba que no exista un riego ya activo para el usuario activo 4. El sistema envía comando de inicio ON mediante un downlink al dispositivo IoT a través de API TTN 5. Se registra inicio de riego en base de datos **Extension point:** ***Cancelar Riego 4.21*** **Extension point:** ***Pausar/Reanudar Riego 4.50***
Postcondiciones	1. El riego está activo según modo configurado
Flujo alternativo	▪ Si ya hay riego activo, se notifica conflicto

Tabla 4.21: Especificación del Caso de Uso: Cancelar riego

Identificador	CU-17
Nombre	Cancelar riego
Descripción	El usuario cancela el riego activo en cualquier momento, independientemente del modo utilizado.
Actor principal	Usuario Verificado
Actor secundario	–
Precondiciones	1. El usuario debe estar autenticado 2. Debe haber un riego activo en progreso
Flujo principal	1. Se solicita confirmación de cancelación 2. El sistema envía comando de parada OFF mediante un downlink al dispositivo IoT a través de API TTN 3. Se actualiza estado a "cancelado" en base de datos
Postcondiciones	-
Flujo alternativo	▪ Si no hay riego activo, se notifica al usuario

Tabla 4.22: Especificación del Caso de Uso: Confirmar cuenta

Identificador	CU-18
Nombre	Confirmar cuenta
Descripción	El usuario confirma su cuenta mediante un enlace único enviado por email para activar su acceso al sistema.
Actor principal	Visitante
Actor secundario	–
Precondiciones	1. El usuario debe haber completado el registro. 2. Debe existir un token de verificación válido. 3. El enlace no debe haber expirado.
Flujo principal	1. El usuario recibe email con enlace de verificación. 2. El usuario hace clic en el enlace de confirmación. 3. El sistema valida el token de verificación. 4. El sistema actualiza estado del usuario a “verificado”. 5. El sistema elimina el token de verificación en la base de datos. 6. El sistema redirige a página de login con mensaje de éxito.
Postcondiciones	1. La cuenta está verificada y activa 2. El usuario puede iniciar sesión normalmente
Flujo alternativo	▪ Si la cuenta ya está verificada, se redirige a login

Tabla 4.23: Especificación del Caso de Uso: Restablecer contraseña

Identificador	CU-19
Nombre	Restablecer contraseña
Descripción	El usuario solicita y completa el proceso de restablecimiento de contraseña cuando la ha olvidado.
Actor principal	Visitante, Usuario Verificado
Actor secundario	-
Precondiciones	1. La cuenta debe estar verificada
Flujo principal	1. El usuario solicita restablecimiento desde página de login. 2. Introduce su email registrado. 3. El sistema verifica que el email existe. 4. El sistema genera token único de restablecimiento. 5. El sistema envía email con enlace temporal. 6. El usuario accede al enlace. 7. El usuario introduce nueva contraseña. 8. El sistema hashea y actualiza la contraseña.
Postcondiciones	1. El token de restablecimiento se invalida. 2. El usuario puede acceder con nueva contraseña.
Flujo alternativo	▪ Si el email no existe, se muestra mensaje genérico.

Tabla 4.24: Especificación del Caso de Uso: Entrar en modo demo

Identificador	CU-20
Nombre	Entrar en modo demo
Descripción	Un visitante accede al modo demo del sistema para explorar funcionalidades sin necesidad de registro.
Actor principal	Visitante
Actor secundario	–
Precondiciones	-
Flujo principal	1. El visitante selecciona la opción “Modo Demo”. 2. El sistema inicializa datos simulados. 3. El sistema carga interfaz con datos ficticios.
Postcondiciones	-
Flujo alternativo	-

Tabla 4.25: Especificación del Caso de Uso: Crear usuario

Identificador	CU-21
Nombre	Crear usuario
Descripción	El administrador crea nuevas cuentas de usuario directamente desde el panel administrativo sin verificación por email requerida.
Actor principal	Administrador
Actor secundario	
Precondiciones	1. El usuario debe tener rol de administrador. 2. El email no debe estar registrado previamente.
Flujo principal	1. El administrador accede al panel de usuarios. 2. El administrador selecciona “Crear nuevo usuario”. 3. El administrador completa formulario con datos requeridos.
Postcondiciones	1. El nuevo usuario está creado en el sistema.
Flujo alternativo	▪ Si el email ya existe, se notifica error. ▪ Si faltan campos obligatorios, se solicita completar.

Tabla 4.26: Especificación del Caso de Uso: Listar usuarios

Identificador	CU-22
Nombre	Listar usuarios
Descripción	El administrador visualiza todos los usuarios registrados en el sistema con opciones de filtrado y búsqueda.
Actor principal	Administrador
Actor secundario	–
Precondiciones	1. El usuario debe tener rol de administrador. 2. Debe haber usuarios registrados en el sistema.
Flujo principal	1. El administrador accede a gestión de usuarios. 2. El sistema carga listado completo de usuarios desde el almacén de datos. 3. El sistema muestra datos básicos de cada usuario.
Postcondiciones	-
Flujo alternativo	▪ Si no hay usuarios, se muestra mensaje informativo.

Tabla 4.27: Especificación del Caso de Uso: Ver usuario

Identificador	CU-23
Nombre	Ver usuario
Descripción	El administrador visualiza información detallada de un usuario específico incluyendo dispositivos y cultivos asociados.
Actor principal	Administrador
Actor secundario	–
Precondiciones	1. El usuario debe tener rol de administrador. 2. Debe existir el usuario a consultar.
Flujo principal	1. El sistema carga datos completos del usuario.
Postcondiciones	1. Se muestra información completa del usuario.
Flujo alternativo	▪ Si hay datos faltantes, se muestran campos vacíos.

Tabla 4.28: Especificación del Caso de Uso: Editar usuario

Identificador	CU-24
Nombre	Editar usuario
Descripción	El administrador modifica información de un usuario existente.
Actor principal	Administrador
Actor secundario	–
Precondiciones	1. El usuario debe tener rol de administrador. 2. Debe existir el usuario a editar.
Flujo principal	1. El administrador accede a edición del usuario. 2. El sistema carga datos actuales en formulario. 3. El administrador modifica campos necesarios. 4. El sistema valida cambios antes de guardar. 5. El sistema actualiza información en base de datos.
Postcondiciones	1. Los datos del usuario están actualizados.
Flujo alternativo	▪ Si el email se cambia a uno existente, se rechaza.

Tabla 4.29: Especificación del Caso de Uso: Eliminar usuario

Identificador	CU-25
Nombre	Eliminar usuario
Descripción	El usuario elimina su propia cuenta (Usuario) o el administrador elimina cualquier cuenta (Admin) permanentemente.
Actor principal	Usuario Verificado, Administrador
Actor secundario	–
Precondiciones	1. Debe existir el usuario a eliminar.
Flujo principal	1. Si es usuario: accede a su perfil y selecciona eliminar cuenta. 2. Si es admin: selecciona usuario a eliminar en el listado. 3. El sistema muestra advertencia sobre eliminación permanente. 4. El sistema solicita confirmación explícita. 5. El sistema elimina dispositivos y cultivos asociados. 6. El sistema elimina cuenta y todos los registros.
Postcondiciones	1. El usuario y todos sus datos están eliminados.
Flujo alternativo	▪ Si se cancela confirmación, se aborta eliminación. ▪ Si hay error en eliminación, se revierten cambios.

Tabla 4.30: Especificación del Caso de Uso: Cerrar sesión

Identificador	CU-26
Nombre	Cerrar sesión
Descripción	El usuario termina su sesión activa en el sistema y retorna a la página de inicio.
Actor principal	Usuario Verificado, Administrador
Actor secundario	–
Precondiciones	1. Debe tener sesión activa. 2. No debe haber operaciones críticas en progreso.
Flujo principal	1. El usuario selecciona opción “Cerrar sesión”. 2. El sistema invalida token JWT actual. 3. El sistema elimina el token del usuario de localStorage. 4. Se confirma cierre exitoso.
Postcondiciones	1. La sesión está completamente cerrada 2. No hay tokens válidos en el cliente
Flujo alternativo	-

Tabla 4.31: Especificación del Caso de Uso: Listar cultivos

Identificador	CU-27
Nombre	Listar cultivos
Descripción	El usuario visualiza cultivos (Usuario: solo los suyos, Admin: todos los del sistema).
Actor principal	Usuario Verificado, Administrador
Actor secundario	–
Precondiciones	1. Debe haber cultivos registrados en el sistema.
Flujo principal	1. El usuario accede a gestión de cultivos. 2. Se cargan cultivos según rol (propios/todos). 3. Se muestra información de cada cultivo. 4. Se proporcionan acciones según permisos.
Postcondiciones	-
Flujo alternativo	▪ Si no hay cultivos, se muestra mensaje informativo.

Tabla 4.32: Especificación del Caso de Uso: Crear cultivo

Identificador	CU-28
Nombre	Crear cultivo
Descripción	El usuario crea un nuevo cultivo con parámetros específicos.
Actor principal	Usuario Verificado, Administrador
Actor secundario	–
Precondiciones	1. Los datos proporcionados deben ser válidos. 2. El usuario puede tener múltiples cultivos, pero solo uno seleccionado.
Flujo principal	1. El usuario accede a gestión de cultivos. 2. El usuario selecciona "Crear cultivo". 3. El usuario completa información: nombre, descripción, categoría, etc. 4. El sistema almacena el cultivo en almacenamiento.
Postcondiciones	-
Flujo alternativo	▪ Si faltan datos obligatorios, se solicita completar

Tabla 4.33: Especificación del Caso de Uso: Editar cultivo

Identificador	CU-29
Nombre	Editar cultivo
Descripción	El usuario modifica información y parámetros de un cultivo.
Actor principal	Usuario Verificado, Administrador
Actor secundario	–
Precondiciones	1. Debe existir el cultivo a editar.
Flujo principal	1. El usuario selecciona cultivo a editar. 2. El sistema carga datos actuales en formulario. 3. El usuario modifica información necesaria. 4. Guarda modificaciones.
Postcondiciones	1. El cultivo tiene información actualizada
Flujo alternativo	-

Tabla 4.34: Especificación del Caso de Uso: Eliminar cultivo

Identificador	CU-30
Nombre	Eliminar cultivo
Descripción	El usuario elimina permanentemente un cultivo y todas sus configuraciones asociadas.
Actor principal	Usuario Verificado, Administrador
Actor secundario	–
Precondiciones	1. Debe existir el cultivo a eliminar 2. No debe haber riego activo en el cultivo
Flujo principal	1. El usuario selecciona cultivo a eliminar 2. El sistema verifica que no hay riego activo 3. El sistema solicita confirmación de eliminación 4. El sistema elimina configuraciones de riego asociadas 5. El sistema elimina registro del cultivo 6. El sistema notifica eliminación exitosa
Postcondiciones	-
Flujo alternativo	▪ Si se cancela confirmación, se aborta proceso

Tabla 4.35: Especificación del Caso de Uso: Seleccionar/Deseleccionar cultivo

Identificador	CU-31
Nombre	Seleccionar/Deseleccionar cultivo
Descripción	El usuario activa o desactiva un cultivo específico como su cultivo activo.
Actor principal	Usuario Verificado, Administrador
Actor secundario	–
Precondiciones	1. Debe existir el cultivo objetivo 2. El cultivo debe pertenecer al usuario
Flujo principal	1. El usuario accede a gestión de cultivos 2. El usuario selecciona cultivo específico 3. El usuario cambia estado de selección del cultivo 4. El sistema actualiza configuración del usuario 5. El sistema confirma actualización
Postcondiciones	1. Solo un cultivo puede estar activo
Flujo alternativo	▪ Si hay riego activo, se solicita confirmación

Tabla 4.36: Especificación del Caso de Uso: Crear dispositivo

Identificador	CU-32
Nombre	Crear dispositivo
Descripción	El usuario registra un nuevo dispositivo IoT con credenciales TTN válidas.
Actor principal	Usuario Verificado, Administrador
Actor secundario	–
Precondiciones	1. Las credenciales TTN deben ser válidas
Flujo principal	1. El usuario accede a gestión de dispositivos 2. El usuario selecciona “Crear dispositivo” 3. El usuario introduce credenciales TTN (DevEUI, AppEUI, etc.) 4. Registra dispositivo en el sistema
Postcondiciones	-
Flujo alternativo	-

Tabla 4.37: Especificación del Caso de Uso: Editar dispositivo

Identificador	CU-33
Nombre	Editar dispositivo
Descripción	El usuario modifica configuración y credenciales de su dispositivo IoT.
Actor principal	Usuario Verificado, Administrador
Actor secundario	–
Precondiciones	1. Debe existir el dispositivo a editar
Flujo principal	1. El usuario selecciona dispositivo a editar 2. El sistema carga los datos actuales del dispositivo 3. El usuario modifica datos necesarios 4. Actualiza registro del dispositivo
Postcondiciones	-
Flujo alternativo	▪ Si las nuevas credenciales fallan, se revierten ▪ Si hay dispositivo activo, se valida impacto ▪ Si hay error, se mantiene configuración anterior

Tabla 4.38: Especificación del Caso de Uso: Eliminar dispositivo

Identificador	CU-34
Nombre	Eliminar dispositivo
Descripción	El usuario elimina permanentemente su dispositivo IoT y todos sus datos asociados.
Actor principal	Usuario Verificado, Administrador
Actor secundario	–
Precondiciones	1. Debe existir el dispositivo a eliminar 2. No debe haber riego activo con el dispositivo
Flujo principal	1. El usuario selecciona dispositivo a eliminar 2. El sistema solicita confirmación de eliminación 3. Se elimina registro del sistema
Postcondiciones	-
Flujo alternativo	-

Tabla 4.39: Especificación del Caso de Uso: Activar/Desactivar comunicación

Identificador	CU-35
Nombre	Activar/Desactivar comunicación
Descripción	El usuario habilita o deshabilita la comunicación de su dispositivo IoT con la base de datos.
Actor principal	Usuario Verificado, Administrador
Actor secundario	–
Precondiciones	1. El usuario debe estar autenticado 2. Debe existir el dispositivo objetivo 3. El dispositivo debe estar registrado
Flujo principal	1. El usuario accede a listado de dispositivos. 2. Selecciona dispositivo específico. 3. Cambia estado de comunicación (activo/inactivo). 4. Se actualiza configuración del dispositivo en la base de datos. 5. El sistema comprueba que el dispositivo desde donde proceden las lecturas de los sensores existe en la base de datos. 6. El sistema comprueba que el dispositivo tenga la comunicación activa. 7. El sistema guarda los datos de los sensores en la base de datos 8. Se notifica nuevo estado
Postcondiciones	1. Los datos se procesan según nuevo estado
Flujo alternativo	▪ Si hay el dispositivo no está registrado en la base de datos, la información de los sensores no se almacena. ▪ Si la comunicación no está activa, la información en la base de datos no se almacena

Tabla 4.40: Especificación del Caso de Uso: Listar dispositivos

Identificador	CU-36
Nombre	Listar dispositivos
Descripción	El usuario visualiza sus dispositivos IoT (Usuario: solo los suyos, Admin: todos los del sistema).
Actor principal	Usuario Verificado, Administrador
Actor secundario	–
Precondiciones	1. Debe haber dispositivos registrados
Flujo principal	1. El usuario accede a gestión de dispositivos 2. Se cargan dispositivos según rol (propios/todos) 3. Se muestra información de cada dispositivo 4. Se proporcionan acciones según permisos
Postcondiciones	-
Flujo alternativo	▪ Si no hay dispositivos, se muestra mensaje informativo

Tabla 4.41: Especificación del Caso de Uso: Marcar alerta resuelta

Identificador	CU-37
Nombre	Marcar alerta resuelta
Descripción	El usuario marca una alerta específica como resuelta para indicar que ha tomado las acciones necesarias.
Actor principal	Usuario Verificado
Actor secundario	–
Precondiciones	1. Debe existir al menos una alerta activa 2. La alerta debe pertenecer al usuario
Flujo principal	1. El usuario accede a la lista de alertas 2. El usuario identifica la alerta a resolver 3. Hace clic en botón “Marcar como resuelta” 4. El sistema actualiza estado de la alerta
Postcondiciones	-
Flujo alternativo	-

Tabla 4.42: Especificación del Caso de Uso: Historial completo de alertas

Identificador	CU-38
Nombre	Ver historial completo de alertas
Descripción	El usuario visualiza el historial completo de todas las alertas generadas, tanto resueltas como pendientes.
Actor principal	Usuario Verificado
Actor secundario	–
Precondiciones	1. Debe haber alertas registradas en el sistema
Flujo principal	1. El usuario selecciona vista "Historial completo" 2. El sistema carga todas las alertas del usuario 3. Se muestran ordenadas por fecha descendente **Extension Point:** ***Marcar alerta resuelta 4.41***
Postcondiciones	-
Flujo alternativo	▪ Si no hay alertas, se muestra mensaje informativo

Tabla 4.43: Especificación del Caso de Uso: Ver perfil

Identificador	CU-39
Nombre	Ver perfil
Descripción	El usuario visualiza y gestiona su información personal incluyendo opciones para editar datos y eliminar cuenta.
Actor principal	Usuario Verificado
Actor secundario	–
Precondiciones	1. El usuario debe estar autenticado 2. Debe tener una cuenta verificada 3. Debe tener acceso a la sección de perfil
Flujo principal	1. El usuario accede a su perfil desde el menú 2. Se cargan datos personales actuales 3. Se muestra información de cuenta y configuraciones 4. Se proporcionan opciones para editar nombre 5. Se incluye opción para cambiar contraseña 6. Se ofrece opción para eliminar cuenta **Extension Point:** ***Cerrar Sesión 4.30*** **Extension Point:** ***Restablecer contraseña 4.23***
Postcondiciones	-
Flujo alternativo	-

Tabla 4.44: Especificación del Caso de Uso: Seleccionar modo de riego

Identificador	CU-40
Nombre	Seleccionar modo de riego
Descripción	El usuario elige entre los tres modos disponibles de riego: manual, programado o automático.
Actor principal	Usuario Verificado
Actor secundario	–
Precondiciones	1. Debe tener dispositivo y cultivo configurados. 2. No debe haber riego activo.
Flujo principal	1. El usuario accede a configuración de riego. 2. El sistema muestra los tres modos disponibles. 3. El usuario selecciona el modo deseado. 4. Se activa la interfaz específica del modo. **Extension Point:** ***Configurar Modo Manual 4.45*** **Extension Point:** ***Configurar Modo Programado 4.46*** **Extension Point:** ***Configurar Modo Automático 4.47***
Postcondiciones	1. El sistema queda listo para la configuración detallada del modo elegido.
Flujo alternativo	▪ Si hay riego activo, se debe cancelar primero (CU-17).

Tabla 4.45: Especificación del Caso de Uso: Configurar modo manual

Identificador	CU-41
Nombre	Configurar modo manual
Descripción	El usuario configura riego manual especificando duración y confirma el inicio inmediato del riego.
Actor principal	Usuario Verificado
Actor secundario	–
Precondiciones	1. El usuario debe haber seleccionado modo manual. 2. El dispositivo debe estar disponible. 3. No debe haber otro riego activo.
Flujo principal	1. Se muestra la interfaz de configuración manual. 2. El usuario especifica la duración del riego en minutos. 3. El usuario confirma el inicio inmediato. 4. Include(*Iniciar riego 4.20*) 5. Se inicia el riego con la duración especificada.
Postcondiciones	1. El riego manual está activo. 2. Se registra la configuración utilizada. 3. El sistema bloquea el acceso al resto de modos mientras dure el riego.
Flujo alternativo	▪ Si la duración es inválida, se solicita corrección. ▪ Si el dispositivo falla, se muestra error. ▪ Si el usuario cancela antes de iniciar, se regresa a la selección de modo.

Tabla 4.46: Especificación del Caso de Uso: Configurar modo programado

Identificador	CU-42
Nombre	Configurar modo programado
Descripción	El usuario programa riegos automáticos definiendo horarios y duración para ejecución posterior.
Actor principal	Usuario Verificado
Actor secundario	–
Precondiciones	1. El usuario debe haber seleccionado modo programado. 2. El sistema debe permitir programación. 3. Debe tener dispositivo configurado.
Flujo principal	1. Se muestra la interfaz de programación. 2. Include(*Definir programación 4.48*) 3. Se guarda y activa la programación.
Postcondiciones	1. La programación está almacenada y activa. 2. El sistema ejecutará riegos automáticamente en los horarios definidos.
Flujo alternativo	▪ Si hay conflictos horarios, se notifica para corrección. ▪ Al llegar la fecha/hora de un evento programado, se dispara *Iniciar riego 4.20* ▪ El usuario puede *Cancelar programación completa 4.51* posteriormente desde la configuración de programación.

Tabla 4.47: Especificación del Caso de Uso: Configurar modo automático

Identificador	CU-43
Nombre	Configurar modo automático
Descripción	El usuario define umbrales de humedad/temperatura para que el sistema active riego automáticamente según condiciones.
Actor principal	Usuario Verificado
Actor secundario	–
Precondiciones	1. El usuario debe haber seleccionado modo automático. 2. Debe tener cultivo con parámetros definidos. 3. Los sensores deben estar operativos.
Flujo principal	1. Se cargan parámetros del cultivo seleccionado como base. 2. Include (*Definir umbrales 4.49*) 3. El sistema guarda la configuración del modo automático.
Postcondiciones	1. El modo automático queda configurado y activo. 2. El sistema evaluará continuamente los umbrales para decidir el inicio de riego.
Flujo alternativo	▪ Cuando se cumplan los umbrales, se dispara *Iniciar riego 4.20* ▪ El usuario puede *Cancelar programación completa 4.51* posteriormente desde la configuración de umbrales.

Tabla 4.48: Especificación del Caso de Uso: Definir programación

Identificador	CU-44
Nombre	Definir programación
Descripción	El usuario establece horarios, días y duraciones para el modo programado.
Actor principal	Usuario Verificado
Actor secundario	–
Precondiciones	1. Estar en configuración de modo programado.
Flujo principal	1. El usuario selecciona fecha y hora de inicio de riego. 2. Especifica la duración del riego. 3. Selecciona la frecuencia de riego. 4. Se valida y almacena la programación completa. **Extension Point:** ***Cancelar programación completa 4.51***
Postcondiciones	1. Programación guardada y activa. 2. Quedan programados uno o más eventos futuros de riego.
Flujo alternativo	▪ Si se cancela durante la edición, no se guarda la programación.

Tabla 4.49: Especificación del Caso de Uso: Definir umbrales (Auto)

Identificador	CU-45
Nombre	Definir umbrales (Auto)
Descripción	El usuario establece los valores límite (humedad del suelo, humedad relativa y temperatura) que activarán el riego en modo automático.
Actor principal	Usuario Verificado
Actor secundario	–
Precondiciones	1. Estar en configuración de modo automático. 2. Existir un cultivo seleccionado con parámetros base. 3. Sensores operativos.
Flujo principal	1. Se muestran los umbrales actuales del cultivo. 2. El usuario ajusta umbral mínimo y máximo de humedad del suelo. 3. El usuario ajusta umbral mínimo y máximo de humedad relativa. 4. El usuario ajusta el umbral de temperatura máxima. 5. Se valida y guardan los umbrales personalizados. **Extension Point:** ***Cancelar programación completa 4.51***
Postcondiciones	1. Umbrales personalizados almacenados. 2. El sistema usará estos valores para decidir el inicio de riego.
Flujo alternativo	▪ Si no se especifica duración, se aplica un valor por defecto definido por el sistema. ▪ Si hay error de validación, se mantienen los umbrales previos.

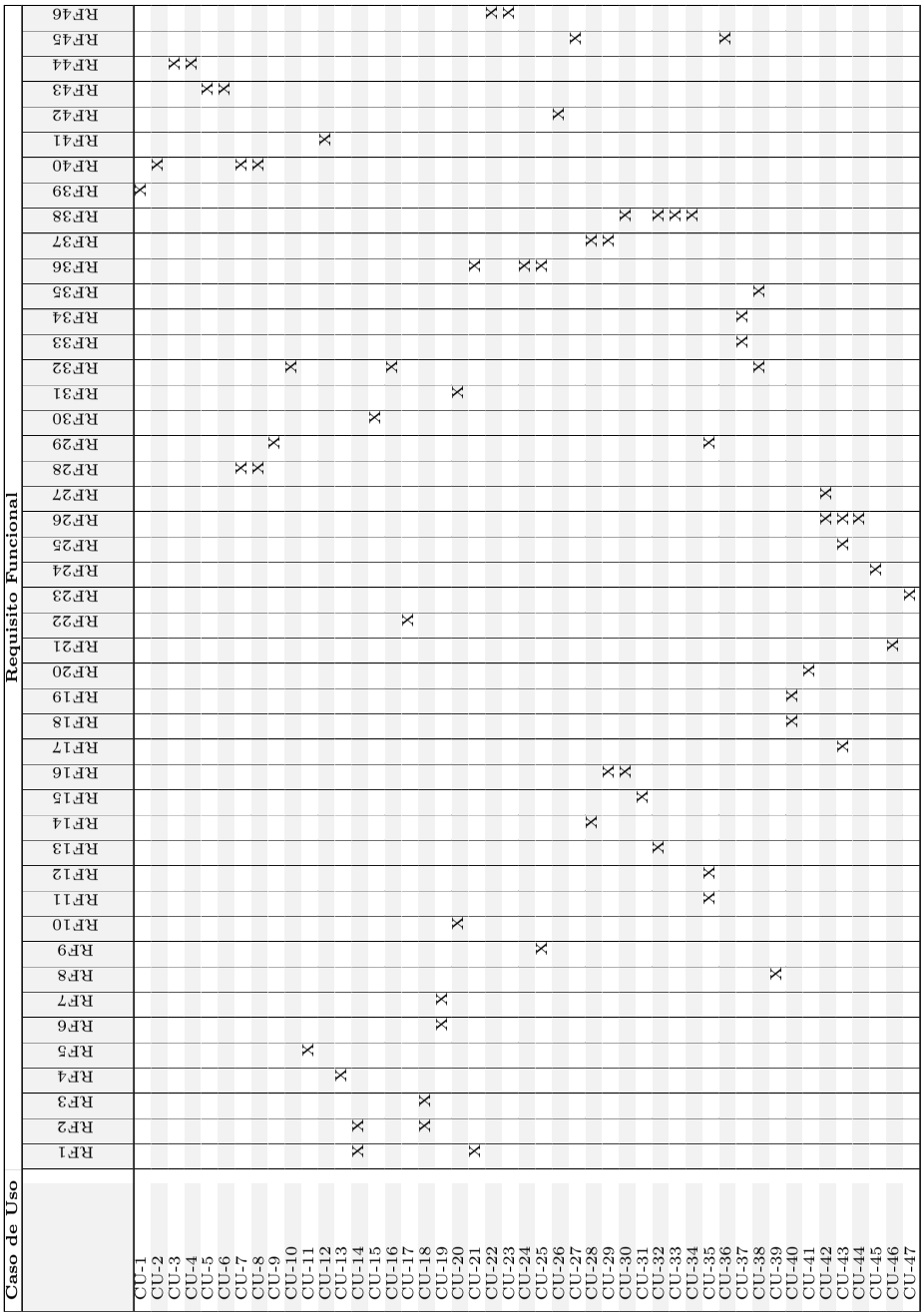
Tabla 4.50: Especificación del Caso de Uso: Pausar/Reanudar riego

Identificador	CU-46
Nombre	Pausar/Reanudar riego
Descripción	El usuario pausa temporalmente un riego activo y puede reanudarlo posteriormente sin perder la configuración.
Actor principal	Usuario Verificado
Actor secundario	–
Precondiciones	1. Debe haber un riego activo en progreso.
Flujo principal	1. El usuario accede a los controles del riego activo. 2. El usuario selecciona “Pausar” riego. 3. Se envía un downlink de pausa al dispositivo. 4. Se registra el timestamp de pausa. 5. El usuario selecciona “Reanudar” riego. 6. Se envía un downlink de reanudación y se continúa el riego.
Postcondiciones	1. El riego queda pausado o reanudado según la acción. 2. Se mantiene la configuración original. 3. Se registra el histórico de pausas/reanudaciones.
Flujo alternativo	▪ Si falla el envío del downlink, se notifica el error y se permite reintentar.

Tabla 4.51: Especificación del Caso de Uso: Cancelar programación completa

Identificador	CU-47
Nombre	Cancelar programación completa
Descripción	El usuario anula la programación activa del modo programado y/o desactiva la lógica de arranque del modo automático (umbrales), eliminando eventos futuros y deshabilitando disparos automáticos.
Actor principal	Usuario Verificado
Actor secundario	–
Precondiciones	1. Existir programación guardada y/o umbrales activos. 2. No es necesario que haya riego activo.
Flujo principal	1. El usuario accede a la gestión de programación/umbrales. 2. El usuario selecciona el ámbito a cancelar: Programado, Automático o ambos. 3. El sistema elimina eventos programados y/o desactiva el disparo por umbrales. 4. Se registra la cancelación y se actualiza el estado en la base de datos.
Postcondiciones	1. No quedan eventos futuros programados ni disparos automáticos por umbral. 2. El usuario puede seleccionar y configurar de nuevo cualquier modo. 3. Si hubiera un riego corriendo, permanece activo hasta su fin o hasta que se ejecute.
Flujo alternativo	▪ Si no hay nada que cancelar, el sistema informa que no existen elementos activos.

Tabla 4.52: Matriz de trazabilidad casos de uso - requisitos funcionales



Capítulo 5

Diseño

En este capítulo se aborda la fase de diseño, posterior a la fase de análisis en la que se identificaron y definieron los requisitos del sistema con el objetivo de establecer y aclarar sus funcionalidades principales. Un diseño bien orientado constituye la base de una futura implementación efectiva, ya que permite garantizar el cumplimiento de los requisitos establecidos en el análisis.

Para desarrollar esta fase se emplearán diversos diagramas que faciliten la comprensión de la estructura y funcionamiento del sistema. En primer lugar, se presentará el diseño global mediante un diagrama de arquitectura, seguido de un diagrama de componentes que identificará los componentes que conforman el sistema completo.

Como recurso visual, se incluirán dos diagramas de flujo que muestre la interacción entre los distintos componentes para las 2 direcciones de comunicación tanto uplink como downlink, con el fin de facilitar la comprensión de ambos procesos de comunicación. Posteriormente, se representarán diagramas de actividades y diagramas de secuencia de los casos de uso principales de la aplicación con el objetivo de describir el diseño de cada uno de los módulos.

5.1. Restricciones

En este capitulo vamos a tratar una de las cuestiones más importantes de nuestro proyecto que son las restricciones, las restricciones nos permiten definir el dominio del problema con el objetivo de evitar desviarnos de la resolución del problema real. Además el establecimiento de restricciones nos ayuda a marcar pautas de cómo se desarrollará el proyecto en cuestión. Diferenciaremos dos grupos de factores que condicionarán el desarrollo de estas restricciones sobre el dominio del problema:

- **Factores dato:** Son aquellos intrínsecos al problema, no podrán ser modificados y condicionarán inevitablemente el proyecto
- **Factores estratégicos:** Son aquellos sobre los que tendremos capacidad de decisión pero serán estos los que afectarán sobre el diseño final del proyecto. Por lo que se propondrán distintas soluciones a seguir y se someterán bajo un análisis para evaluar cuál será la mejor solución para nuestro proyecto.

5.1.1. Factores Dato

Estos factores inherentes son:

- Se requiere de conexión a internet constante tanto para la aplicación web como para el Gateway LoRaWAN, ya que este necesita enrutar los paquetes de datos a través internet hasta el servidor de red de TTN.
- La aplicación debe gestionar únicamente un cultivo por usuario, por el motivo de que el hardware está pensado para regar solo un tipo de cultivo ya que tenemos solamente un única válvula RS por cada dispositivo montado.
- La aplicación sólo puede habilitar un dispositivo IoT por cada usuario.
- El proyecto ha de ser finalizado antes del 10 de septiembre de 2025, habiendo superado un mínimo de 300 horas estipuladas para el desarrollo del Trabajo de Fin de Grado.
- El proyecto ha de ser desarrollado únicamente por el autor, junto con el apoyo y la guía de los directores del proyecto.
- La solución debe de ser un sistema que cualquier usuario final pueda usar fácilmente
- El sistema de obtención de datos debe de consumir poco y hacerse portable
- El presupuesto disponible es limitado, por lo que se prescindirá del uso de servicios de pago y la compra de hardware adicional.
- El contenido, estructura y presentación tanto de la implementación técnica como de la memoria explicativa deberán conformarse según los criterios y directrices formales vigentes en la normativa de la Escuela Politécnica Superior de Córdoba.

- La características físicas del dispositivo deben contemplar la movilidad como requisito esencial, incorporando soluciones de eficiencia energética que maximicen su autonomía durante las sesiones de captura de datos y accionamiento del actuador para la activación del riego.
- El sistema hardware está pensando para que pueda funcionar de manera remota a un rango de 200 metros desde el Gateway, ya que el rango de acción del Gateway es 200 metros desde su ubicación. Si se requiere aumentar el rango de acción se deberá de obtener un Gateway para exteriores.

5.1.2. Factores estratégicos

Estos factores influirán sobre el diseño final:

5.1.2.1. Entorno de desarrollo

Entre todas las opciones disponibles se ha optado por Visual Studio Code [39] como Integrated Development Environment (IDE) para el desarrollo del proyecto tanto a nivel de frontend como de backend (servidor web). La programación del microcontrolador se va a llevar a cabo en el mismo entorno de desarrollo pero con la diferencia de que se va a utilizar el plugin de PlatformIO [38] ya integrado en Visual Studio Code. Este plugin proporciona un entorno de desarrollo unificado y de múltiples librerías para la programación de microcontroladores. Siempre y cuando el microcontrolador a programar este entre los modelos de microcontrolador soportados por el el plugin. Además permite la compilación y flasheo del código del microcontrolador.

5.1.2.2. Módulo 1: Edge computing

5.1.2.2.1. Tipo de dispositivo LoRaWAN

Este factor es de gran importancia en nuestra aplicación puesto que va a definir como se va a comunicar el dispositivo IoT con el servidor de red TTN a través del protocolo de comunicación inalámbrico LoRaWAN.

La especificación de LoRaWAN define 3 tipos de dispositivos: Clase A, Clase B y Clase C [56]. Todas las clases de dispositivos tienen soporte de comunicación bidireccional pero dependiendo de una clase u otra tendremos ciertas ventajas y desventajas. Vamos a ver en que consisten cada una de las clases y cual implementaremos nosotros en nuestro dispositivo

En primer lugar vamos a describir la implementación de la clase A.

Una condición clave para que a un dispositivo IoT se le pueda implementar la clase A, es que este debe ser compatible con la clase A. En nuestro caso el microcontrolador Arduino MKR WAN 1310 soporta exclusivamente la Clase A. Otra condición que se tiene que dar es que el Gateway LoRaWAN soporte implementación de clase A. El gateway utilizado en el proyecto soporta clase A y clase C.

Un dispositivo clase A puede enviar un mensaje uplink en cualquier momento. Una vez completada la transmisión del uplink, el dispositivo IoT abre dos ventanas de recepción cortas para ver si el servidor de red TTN quiere respondedle con el propósito de recibir mensajes downlinks de la red. Estas ventanas de escucha reciben el nombre de:

- RX1 (Ventana de recepción 1)
- RX2 (Ventana de recepción 2)

Existe un retardo entre el final de la transmisión del uplink y el inicio de cada ventana de transmisión al que se le da el nombre de:

- RX1 Delay (Retardo de apertura de la ventana de recepción 1)
- RX1 Delay (Retardo de apertura de la ventana de recepción 2)

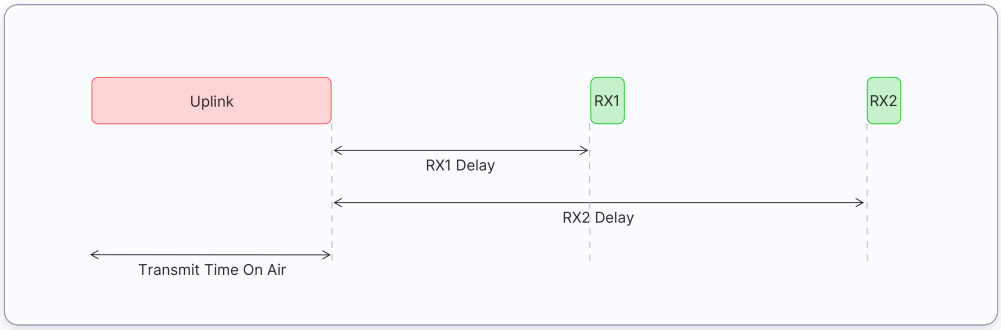


Figura 5.1: Representación RX1 Delay y RX1 Delay en Clase A

Cuando el dispositivo IoT abre las ventanas de escucha, si quiere responder debe hacerlo dentro de esas dos ventanas. Si no lo hace, el mensaje se pierde. Si lo hace, el servidor tendrá que esperar al próximo uplink para volver a intentarlo. En resumen, existen 3 casos posibles.

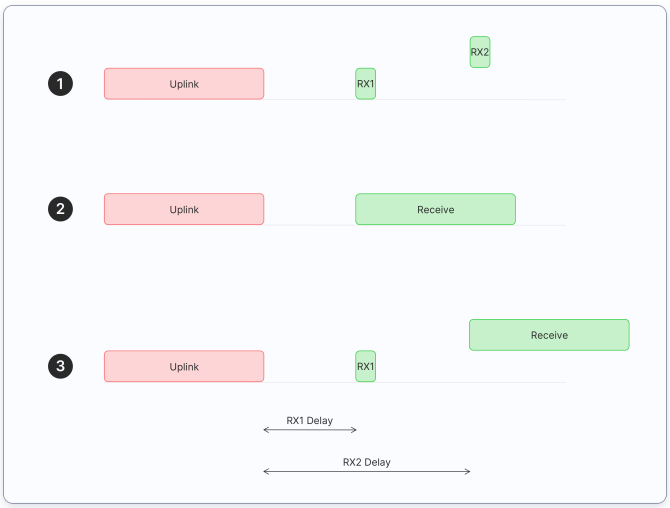


Figura 5.2: Situaciones posibles para los mensajes downlink en Clase A

- El dispositivo IoT abre ambas ventanas de recepción, pero no recibe ningún mensaje downlink durante ninguna de ellas.
- El dispositivo IoT recibe un downlink durante la primera ventana de recepción y, por lo tanto, no abre la segunda ventana de recepción.
- El dispositivo IoT abre la primera ventana de recepción, pero no recibe ningún downlink. Por lo tanto, abre la segunda ventana de recepción y recibe un downlink durante la segunda ventana de recepción.

La clase A se implementa cuando se requiere que el consumo energético sea muy bajo. Por lo tanto, pueden funcionar perfectamente con batería. Esto se debe a que el dispositivo IoT entra en modo suspensión y suelen tener largos intervalos entre uplinks. Además los dispositivos de clase A tienen una alta latencia downlink por el hecho de que requieren enviar un uplink para recibir un downlink

En segundo lugar, describiremos la implementación de la clase B.

Este tipo de clase no podría implementarse en nuestro dispositivo IoT puesto que no soporta la clase B ni tampoco el gatway LoRaWAN soporta clase B. Por lo que queda descartado del análisis de decisión.

Los dispositivos clase B amplían las capacidades de clase A abriendo ventanas de recepción periódicas llamadas ping slots, sincronizadas mediante beacons enviados por la red a intervalos regulares. Estos beacons permiten a los dispositivos ajustar su reloj interno y así el servidor puede saber con precisión cuándo enviar mensajes

de bajada a un dispositivo o grupo de dispositivos. El tiempo entre dos beacons se llama período de beacon.

Los dispositivos clase B tienen menor latencia en los mensajes de bajada que los de clase A gracias a sus ping slots periódicos, aunque su latencia sigue siendo mayor que en clase C. Suelen funcionar con batería, pero su vida útil es más corta que en clase A debido al mayor tiempo en modo activo. Por su latencia moderada, son adecuados para aplicaciones que requieren actuación con criticidad media, como los medidores inteligentes de servicios públicos.

En tercer y último lugar describiremos la implementación de la clase C para dar lugar a conclusión.

Este tipo de clase no podría implementarse en nuestro dispositivo IoT puesto que no soporta la clase C, a no ser que se utilice un módulo específicamente para la implementación de clase C, este tipo de módulos son bastante costosos. Por otro lado, el gateway usado en el proyecto si soporta implementación de clase C.

Los dispositivos de clase C extienden las capacidades de clase A al mantener abiertas las ventas de recepción de manera constante a no ser que se transmita un uplink. Por lo que los dispositivos de clase C pueden recibir mensajes downlink en cualquier momento a una latencia muy baja. Los dispositivos de clase C abren 2 ventanas de recepción, RX1 Y RX2, similares a las de la clase A. Sin embargo, con la peculiaridad de que la ventana de recepción RX2 queda en escucha permanentemente hasta la siguiente transmisión ascendente.

Después de que el dispositivo envía un uplink, se abre una ventana de recepción RX2 corta, seguida de una ventana de recepción RX1 corta, y luego se abre la ventana de recepción RX2 continua. Esta ventana de recepción RX2 permanece abierta hasta que se programa el siguiente uplink. Los uplinks se envían cuando no hay ningún downlink en curso. Por lo que se podría decir que los downlink tienen prioridad sobre los uplinks

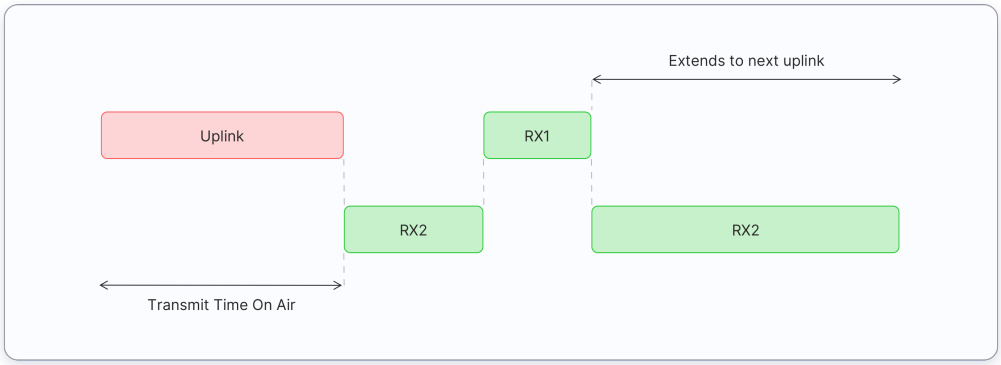


Figura 5.3: Implementación de Clase C

En conclusión, quedando la clase B descartada completamente, hay 2 posibilidades: Implementación de Clase A o Clase C. Debido a que el control de la válvula debe ser inmediato, ya que se espera que el apagado y encendido de la válvula tiene que ser instantáneo. Lo más recomendable sería implementar Clase C, pero surgen varios problemas. Implementar clase C implicaría que el dispositivo tendría que estar conectado continuamente a la red eléctrica debido al elevado consumo energético de la clase C. Por otro lado los costes se elevarían puesto que se tendría que comprar un módulo específicamente para este tipo de implementación.

La solución que se pretende dar, es la implementación de Clase A con características de clase C. Es decir, si reducimos el tiempo de envío de uplinks, reduciremos el tiempo de recepción de los downlink. Logrando una respuesta casi inmediata con baja latencia. Esto tiene un problema es que van a existir 2 tipos uplinks que se van a enviar desde el microcontrolador. Un tipo uplinks va a ser el que va a enviar los datos de temperatura y humedad codificados a TTN que se van a enviar a una frecuencia de 2 minutos por cada uplink, mientras que el otro va a ser uplinks vacíos que se van a enviar a una frecuencia de 2 segundos por cada uplink.

Esto se hace debido a que si utilizamos el primer tipo de uplinks para transmitir los downlinks al dispositivo IoT, si encendemos la válvula desde la interfaz web en el peor de los casos tardaría 2 minutos.

Si utilizamos el segundo tipo de uplinks para transmitir downlinks al dispositivo IoT, si encendemos la válvula desde la interfaz web en el peor de los casos tardaría 2-15 segundos.

Evidentemente este mecanismo obliga al dispositivo a tener un consumo relativamente alto de energía pero comparado con la Clase C, es mucho menor. Permitiendo

utilizar batería y poder relocalizar el dispositivo a zonas donde no exista suministro eléctrico. Además para reducir aún más el consumo, los uplinks vacíos se envían 0 bytes consumiendo prácticamente nada de energía.

5.1.2.2.2. Sensores, actuador y microcontrolador

Para elegir el hardware a usar en el proyecto, se tendrán en cuenta los requisitos de calidad que hay que cumplir. Estos requisitos son fundamentales para que el resultado sea satisfactorio y todo encaje y funcione como se espera.

El módulo 1, encargado de la captación y el accionamiento del actuador para la activación del riego se espera que el consumo sea lo más reducido posible, permitiendo que una simple batería sea capaz de alimentarlo durante semanas o incluso meses o en el mejor de los casos años. También se desea que el sistema sea lo más compacto posible, permitiendo su portabilidad.

Por estos motivos, se ha decidido usar el sensor de humedad relativa y temperatura, DHT11. Y el sensor de humedad del suelo, Capacitive Soil Moisture Sensor v1.2. Se ha escogido estratégicamente estos sensores por su facilidad de uso y por la información ambiental que aportan para tomar decisiones sobre la activación del riego. Por otro lado, como actuador que se encargará de activar/desactivar el riego, se ha decidido usar una D260BLX-V Brushless Micropump Range por su facilidad de uso y gran potencia, permitiendo extraer agua a un caudal de 0.4l/min. Es necesario puntualizar que para que la bomba de agua se encienda y se apague se necesita de un mecanismo que haga switch entre apagado/encendido cuando el microcontrolador reciba la orden de activar/desactivar riego. Para ello se utilizará un relé que actuará como interruptor para la activación y desactivación del riego. El funcionamiento del relé es explicado en la sección 3.1.10.1.2.

Para que el módulo 1 pueda captar los datos para transmitirlos y a su vez recibir información, se ha decidido usar el Arduino MKR WAN 1310 por su reducido tamaño, bajo consumo y porque para este proyecto es suficiente con un microcontrolador. Además, al incluir un módulo de comunicación mediante LoRa y LoRaWAN, no se necesita incluir ningún módulo adicional a no ser que se requiera implementar en el dispositivo clase B ó clase C.

5.1.2.2.3. Mecanismo de activación de riego

Al plantear el sistema, se llegó a la conclusión de que los pines digitales que proveía el Arduino MKR WAN 1310 no podían proporcionar el suficiente voltaje

como para activar/desactivar la bomba de riego mediante una orden de control, por el motivo de que los pines digitales sólo aportan 3.3 V no siendo lo suficiente para los 3.3V-6.0V que pide la bomba de riego. Lo única solución sería conectando VCC de la bomba de riego al Vbus de 5V del Arduino MKR WAN 1310, pero estaría constantemente encendida, lo cuál obviamente no es viable para el proyecto. Así que la solución sería utilizar un interruptor que abriera paso a los 5 V del bus de Arduino hacia la bomba de riego y por otro lado cerrara el paso a los 5 V apagando la bomba de riego. Todo esto controlado a través de un pin digital.

Para ello se optó por un relé de 1 canal con optoacoplador, como el HUAREW 5V. En lugar de conectar directamente la bomba de riego al Arduino MKR WAN 1310, lo que podría dañar el microcontrolador, el relé actúa como un intermediario seguro. Su bobina de baja potencia se energiza con la señal del Arduino (3.3V en este caso), y sus contactos internos (NO, COM, NC) permiten controlar un circuito externo dejando paso o no a una fuente de alimentación independiente y de mayor voltaje/corriente.

Resumiendo, el relé que se utilizará permitirá actuar como un interruptor en el que ante una señal eléctrica proveniente de un pin digital, este interruptor podrá abrirse o cerrarse dependiendo de la orden de riego ejecutada por el usuario. Los 3.3 V de los pines digitales permiten energizar la bobina del relé haciendo que abra el paso de los 5V hacia la bomba de riego o por el contrario cerrar el paso. Cumpliendo con la funcionalidad de activación/desactivación de riego.

5.1.2.2.4. Sistema de comunicación

Para poder transmitir información desde el dispositivo IoT hacia el servidor web de mi aplicación, desde una localización remota donde ni la tecnología WiFi ni 5G llega. Se necesita de una tecnología de comunicación a larga distancia, y no solo esto, sino que también sea de bajo consumo. Para ello se ha optado por utilizar la tecnología de comunicación inalámbrica LoRa y el protocolo de comunicación subyacente, LoRaWAN.

LoRa y LoRaWAN proporciona un mayor rango alcance de comunicación y penetración que WiFi, a consta de reducir la velocidad de transferencia y ancho de banda, pues la frecuencia de LoRa y LoRaWAN es menor pero su ancho de onda es mayor. En la figura *Figura 5.4* se puede ver una comparación entre las distintas tecnologías de comunicación que existen en el mercado.

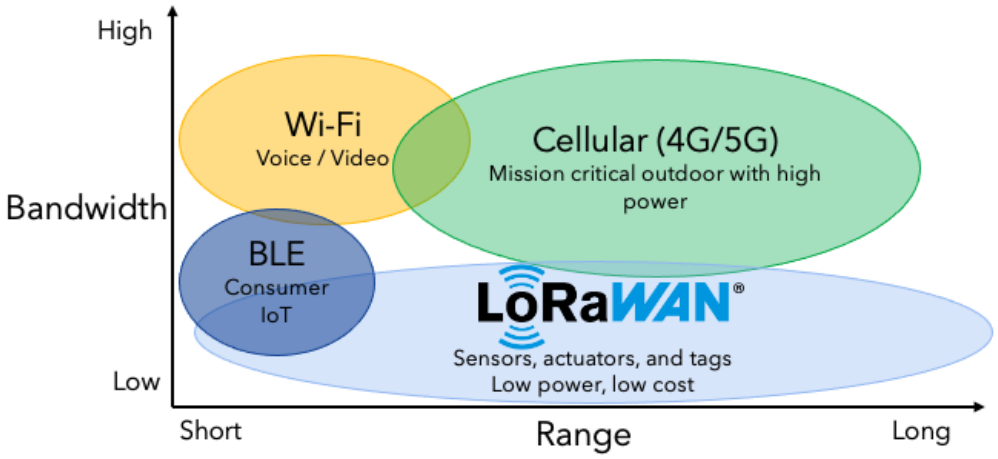


Figura 5.4: Comparación Wifi vs 4G/5G vs LoRa

Otra particularidad de esta tecnología, es que no requiere de muchos puntos de acceso, puesto que un gateway puede ofrecer rangos de alcance de varios kilómetros, abaratando los costes. Por el hecho de que nuestro dispositivo IoT va a estar localizado en zonas donde la señal WiFi y la cobertura móvil no llega, la característica de largo alcance de la tecnología LoRa permite solucionar los problemas de comunicación.

Para la comunicación LoRa de nuestro proyecto, se ha utilizado un microcontrolador Arduino MKR WAN 1310 con un módulo LoRa interno y el gateway The Things Indoor Gateway, que permite que el acceso a Internet a través de WiFi, con el objetivo de enrutar los paquetes de información hacia la plataforma TTN.

5.1.2.3. Módulo 2: Fog computing

La información transmitida por el dispositivo IoT, después de ser filtrada por el gateway hacia Internet. Es necesario de una plataforma que recolecte la información, la decodifique y la redirija al servidor web. Para ello se ha optado por una plataforma cloud de red abierta que permite redirigir la información capturada por los sensores al servidor de la aplicación, esta plataforma es The Things Network (TTN). Además de recolectar información y decodificarla en formato JSON, también permite codificar información proveniente del servidor de la aplicación y transmitirla hacia el dispositivo IoT. De esta manera se logra una comunicación bidireccional entre el dispositivo IoT y el servidor.

The Things Network es usado por multitud de empresas, lo que respalda su elección. Entre las distintos planes que ofrece, hay uno gratuito que es el plan sandbox

gratuito destinado a pruebas a pequeña escala como nuestro proyecto, por lo que elegiremos este plan.

5.1.2.4. Módulo 3: Cloud Computing

La elección de un framework para el backend es incluso más importante que la elección de un framework de frontend puesto que una adecuada elección permitirá un manejo sencillo de datos en tiempo real, comunicación con API's externas y una correctamente manipulación del Webhook HTTP. Tras un análisis exhaustivo se ha llegado a la conclusión de que la mejor solución es utilizar Node.js [42] como runtime de JavaScript + Express.js [43] como framework del servidor web.

5.1.2.4.1. Node.js como Runtime de Javascript

Node.js se ha seleccionado como plataforma principal para el servidor web por su capacidad de ejecutar JavaScript en el servidor, lo que permite utilizar el mismo lenguaje tanto en el frontend como en el servidor web. Esta unificación simplifica significativamente el desarrollo, ya que no es necesario aprender diferentes lenguajes de programación para trabajar en ambas partes de la aplicación.

La arquitectura asíncrona de Node.js es especialmente útil para aplicaciones IoT como la del proyecto. Cuando el sistema reciba datos de sensores desde The Things Network, Node.js podrá procesar múltiples peticiones simultáneamente sin bloquear el servidor. Esto significa que mientras se está guardando un dato de temperatura en la base de datos, el servidor podrá seguir recibiendo y procesando nuevos datos de humedad sin interrupciones.

Otro beneficio de utilizar Node.js es por el ecosistema de gestión de paquetes npm. Este es un gestor de paquetes eficiente y fácil de usar, que tiene disponible de una gran cantidad de librerías.

5.1.2.4.2. Express.js como Framework Web

Express.js es un framework web minimalista que se ejecuta sobre Node.js y proporciona las herramientas necesarias para crear APIs REST siguiendo los principios REST de manera sencilla. Su simplicidad es especialmente valiosa para nuestro proyecto IoT, donde necesitamos crear endpoints específicos para las diferentes funcionalidades que conforman el sistema completo.

La gestión de rutas en Express.js es intuitiva y permitirá organizar el código de manera modular. Las rutas se organizarán en archivos separados por funcionalidad

(por ejemplo, authRoutes.js, deviceRoutes.js, cropRoutes.js), donde cada archivo de rutas tendrá controladores específicos asociados. Cuando se haga una petición HTTP a un endpoint específico, Express ejecutará el controlador correspondiente. Esta arquitectura de separación de responsabilidades mejora significativamente la escalabilidad y mantenibilidad del proyecto, ya que permite modificar la lógica de negocio sin afectar la estructura de URLs y viceversa.

El sistema de middleware de Express.js será fundamental para la seguridad de nuestra aplicación. Antes de que una petición llegue a un controlador, puede pasar por varios middleware que verificarán la autenticación del usuario. Por ejemplo, cuando un usuario intente acceder a la configuración de riego, un middleware verificará automáticamente que esté autenticado y tenga los permisos necesarios para llevar a cabo esta funcionalidad.

La capacidad de manejar diferentes tipos de peticiones (GET, POST, PUT, DELETE) permitirá crear API's completas para gestionar todos los aspectos funcionales del sistema IoT. Los usuarios pueden visualizar datos en tiempo real de sensores (GET), crear nuevas configuraciones de riego (POST), actualizar información de dispositivos (PUT), o eliminar cultivos (DELETE), etc...

5.1.2.4.3. Base de datos

Entre las diversas opciones que había disponibles, se ha optado por PostgreSQL [26], no sólo por su sistema de gestión de base de datos robusto sino también por su capacidad para manejar las complejas relaciones de datos que requiere nuestro proyecto. El hecho de que no se haya optado por una base de datos no relacional se basa principalmente en la gran cantidad de relaciones que existen entre las diferentes entidades del sistema. Por ejemplo, los usuarios registran dispositivos IoT que generan lecturas de sensores, estos usuarios también tienen disponibles cultivos, de los cuales, un cultivo será seleccionado para aplicarle una configuración de riego específica. A su vez estas configuraciones generan activaciones de la válvula RS y alertas del sistema. Este conjunto de relaciones complejas son el motivo por que el que una base de datos no relacional sería ineficiente en el proyecto y aumentaría la complejidad innecesariamente. Otro motivo de el por qué se ha escogido PostgreSQL, es por la familiaridad con este sistema de gestión de base de datos y por su interfaz simple.

Módulo 4: Interfaz Web

En un principio se plantearon varios alternativas para el desarrollo del frontend, hasta que se llegó a una conclusión final basada en la simplicidad de desarrollo y la integración de herramientas modernas.

5.1.2.4.4. Tipo de aplicación web

La aplicación web se desarrollará como una SPA [31]. Para explicar el por qué se optó por la arquitectura SPA para diseñar nuestra aplicación, necesitamos primer saber como funcionan este tipo de páginas. Una SPA se define como un tipo de aplicación web que carga una única página HyperText Markup Language (HTML) y luego actualiza dinámicamente el contenido sin recargar la página completa.

A diferencia de las aplicaciones web tradicionales que requieren múltiples páginas HTML separadas, una SPA mantiene toda la funcionalidad en una sola página que se actualiza mediante JavaScript. Este tipo de paginas es ideal cuando se tienen dashboards en un aplicación web, Puesto que de otra manera, afectaría gravemente a la experiencia de usuario al tener que recargar la pagina cada vez que llegue un dato de los sensores. Aparte de la inmediatez que proporciona este tipo de aplicaciones web, permiten reducir la carga del servidor al minimizar las peticiones de pantalla completa.

Por último, es híbrida puesto que, aunque la mayoría de funcionalidades se renderizan en el lado de cliente, Cliente-Side Rendering (CSR). Hay otras funcionalidades que resultaría más eficiente al renderizarlas en el lado del servidor, Server-Side Rendering (SSR). Cómo por ejemplo:

- **Páginas de autenticación:** Las paginas estáticas de login, registro, verificación de email, se renderizan en el servidor con el fin de mejorar el SEO y la velocidad de carga inicial. Hay que tener en cuenta que ya el SEO de nuestra web se ve bastante penalizado por el hecho de tratarse una SPA donde la mayoría de funcionalidades se cargan del lado del cliente. Por lo que sería conveniente añadirle una mejora de SEO para que los usuarios puedan encontrar fácilmente la aplicación web en los motores de búsqueda.
- **Middleware de autenticación:** Este debe de cargarse en el servidor ya que es el que intercepta las rutas y redirije según el estado de autenticación del usuario.

5.1.2.4.5. Vue.js como Framework

Vue.js se ha seleccionado como framework principal por su enfoque progresivo y su simplicidad inherente. A diferencia de otros frameworks como React o Angular, Vue.js destaca por su facilidad y una sintaxis intuitiva .

La arquitectura basada en componentes de Vue.js servirá de gran ayuda para nuestro proyecto IoT, ya que permite crear componentes reutilizables para elementos

como tarjetas de sensores, gráficos de temperatura y humedad, y controles de riego. Esta modularidad facilita el mantenimiento y escalabilidad del código y permite que páginas totalmentes diferentes de la aplicación compartan elementos comunes, reduciendo el código.

La reactividad bidireccional o también llamado two-binding de Vue.js es crucial para una aplicación web para IoT donde los datos de sensores se actualizan constantemente. Cuando los valores de temperatura y humedad cambian, la interfaz se actualiza automáticamente sin necesidad de manipulación manual del Document Object Model (DOM), lo cual complicaría mucho el código.

5.1.2.4.6. Nuxt.js como MetaFramework

Para facilitar aún más el desarrollo de la interfaz web utilizando Vue.js se ha optado por utilizar el metaframework, Nuxt.js. Un metaframework permite extender las funcionalidades de un framework, además de proporcionar herramientas listas para usar.

Uno de los problemas de Vue.js es el manejo de rutas, que se hace tedioso cuando en un proyecto de gran envergadura como el nuestro existen multitud de estas. Nuxt.js viene a simplificar el manejo de rutas, con un sistema de archivos basado en rutas. Permitiendo manejar la navegación entre diferentes páginas de manera automática. Eliminando la necesidad de configurar manualmente cada ruta en un fichero.

Otro hecho a destacar es que las llamadas a APIs se simplifican significativamente con Nuxt.js. El framework proporciona composables como useFetch y $fetch que manejan automáticamente el estado de carga, errores y caché de las peticiones HTTP. Evitando utilizar librerías externas como axios para el manejo de peticiones HTTP. Esto es crucial para una aplicación web basada en IoT que necesita hacer continuamente llamadas a APIs para obtener datos de sensores, configuraciones de riego, entre otras funcionalidades.

5.1.2.4.7. Pinia como gestor global de estados

Elegir un gestor de estados global es crucial cuando se trabaja con aplicaciones web reactivas, es por ello, que se utilizará la biblioteca de gestión de estado global Pinia, pese a que por defecto Nuxt.js utiliza Vuex como gestor de estados de globales. Se utilizará Pinia por su facilidad de uso. A diferencia de Vuex, Pinia ofrece una API más intuitiva.

5.1.2.4.8. Tailwind CSS

Tailwind CSS es un framework de estilos, cuyo enfoque se centra en reducir la complejidad que supondría diseñar el estilo de una página web con reglas de CSS. Este framework está constituido por clases que engloban reglas de css específicas, de manera que proporciona las clases necesarias para crear componentes modernos sin la complejidad de desarrollar un css personalizado.

5.2. Sistema completo

La Figura 5.5 representa la arquitectura general del sistema, organizada en cuatro módulos: User Layer, Cloud Computing, Fog Computing y Edge Computing.

En primer lugar, podemos ver el módulo Edge Computing donde se encuentran el microcontrolador con conectividad LoRa y los sensores físicos (DHT11 para temperatura y humedad, y sensor capacitivo de humedad de suelo), así como la válvula controlada por un actuador (relé) para ejecutar el riego. Estos dispositivos se comunicarán con el gateway, que actuará como un puente de enlace y nodo del módulo Fog Computing, donde el servidor de red de TTN enviará datos a través de un webhook que se configurará en esta plataforma hacia al servidor web. Además de recibir mensajes a través de una API que expone TTN y que se consumirá en el servidor web.

En este mismo módulo se integrará la API y el webhook de TTN para la comunicación bidireccional con el módulo Cloud Computing, donde el servidor web postprocesará y recibirá la información, interactuará con la base de datos PostgreSQL y expondrá la lógica de negocio. Finalmente, en la User Layer, el usuario interactuará con la interfaz web para acceder a la dashboard, gestionar cultivos y dispositivos y gestionar el sistema de riego inteligente de forma manual, programada o automática.

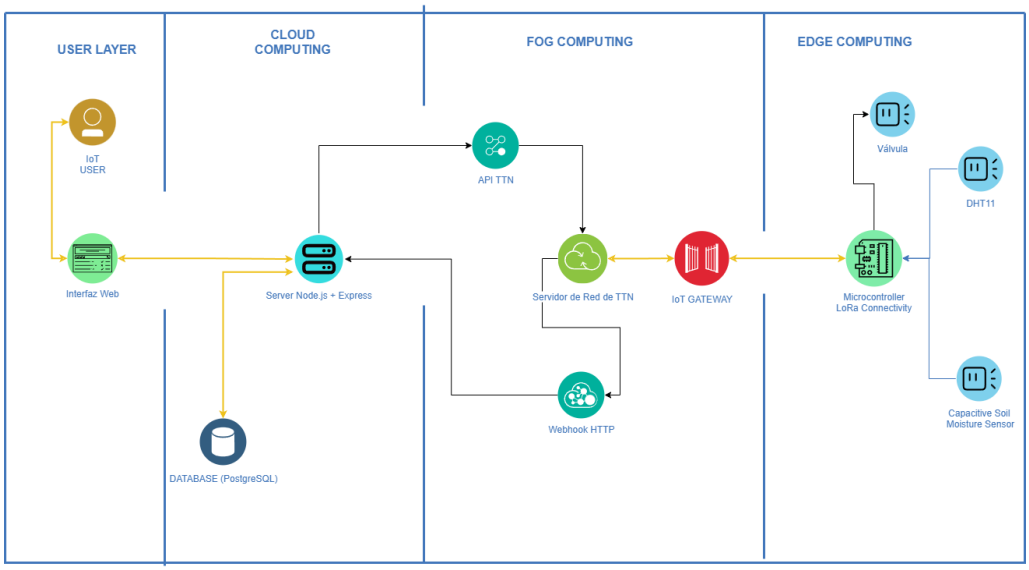


Figura 5.5: Diagrama de la arquitectura del sistema

La Figura 5.6 detalla cómo se interconectarán los componentes principales del sistema y el papel que desempeña cada uno. El dispositivo con sensores estará constituido por microcontrolador Arduino MKR WAN 1310 y los sensores DHT11 y de humedad de suelo capacitivo, encargados de la lectura de datos, así como del control del actuador que va a activar/desactivar el riego, este módulo enviará uplinks periódicos con lecturas y uplinks vacíos para reducir la latencia de los downlinks con las ordenes de control de riego, los cuales será recibidos por el microcontrolador.

La información que será filtrada por el gateway, es recibida por el servidor de red de TTN cuya función será recolectar y preprocesar mensajes y exponer un webhook HTTP y API hacia la capa Cloud Computing. Esta capa está formada por el servidor web y el almacén de datos. El servidor web, a través del endpoint HTTP configurado en el webhook, recibe los datos enviados y filtrados por TTN para luego transformarlos al modelo interno de la aplicación y persistirlos en la base de datos implementada en PostgreSQL. Adicionalmente, generará y enviará ordenes de control de riego, haciendo uso de la API de TTN, hacia el servidor de red de TTN. Con el fin de ser enviadas de nuevo hacia el dispositivo IoT.

Finalmente, la Interfaz web consumirá la API en dos direcciones: En primer lugar, obtendrá información para su visualización en tiempo real (gráficas, indicadores) y enviará datos para procesar (registro de cultivos y dispositivos, cambios de configuración, activaciones o desactivaciones de riego), permitiendo al usuario gestionar el riego de forma inteligente y remota.

Dividir el sistema en módulos conlleva múltiples ventajas entre las cuales podemos destacar: facilita la escalibilidad del sistema, ya que cada componente puede evolucionar independientemente sin tener que afectar a otros módulos o sustituirse sin afectar al resto. También mejora la mantenibilidad, permitiendo aislar y corregir errores de forma más rápida. Por último, el bajo acoplamiento entre módulos incrementa la tolerancia a fallos, pues una incidencia en un módulo no comprometer el funcionamiento del sistema completo.

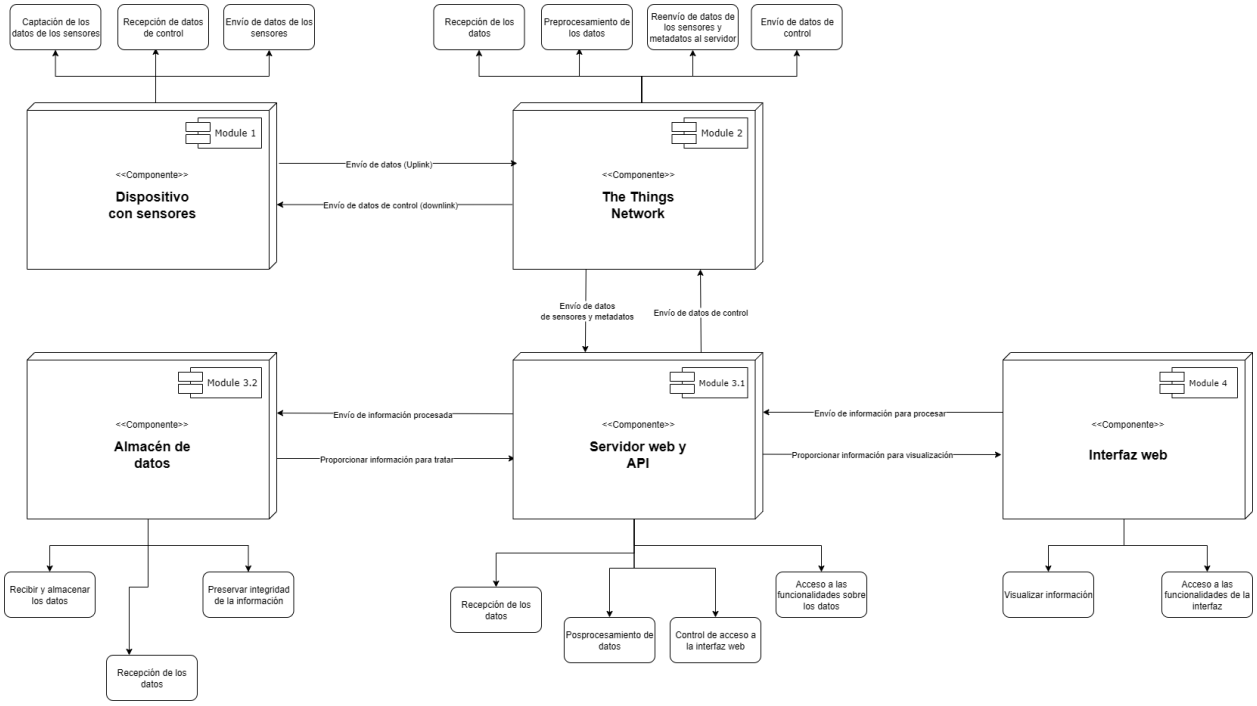


Figura 5.6: Diagrama de componentes del sistema completo

Estos dos diagramas de flujo (uplink 5.7 y downlink 5.8) constituirán los 2 tipos de comunicación que definirán el comportamiento del sistema completo.

El flujo de uplink describe la captura de las lecturas de los sensores, validación y envío hasta su decodificación en TTN, reenvío por webhook, almacenamiento y visualización.

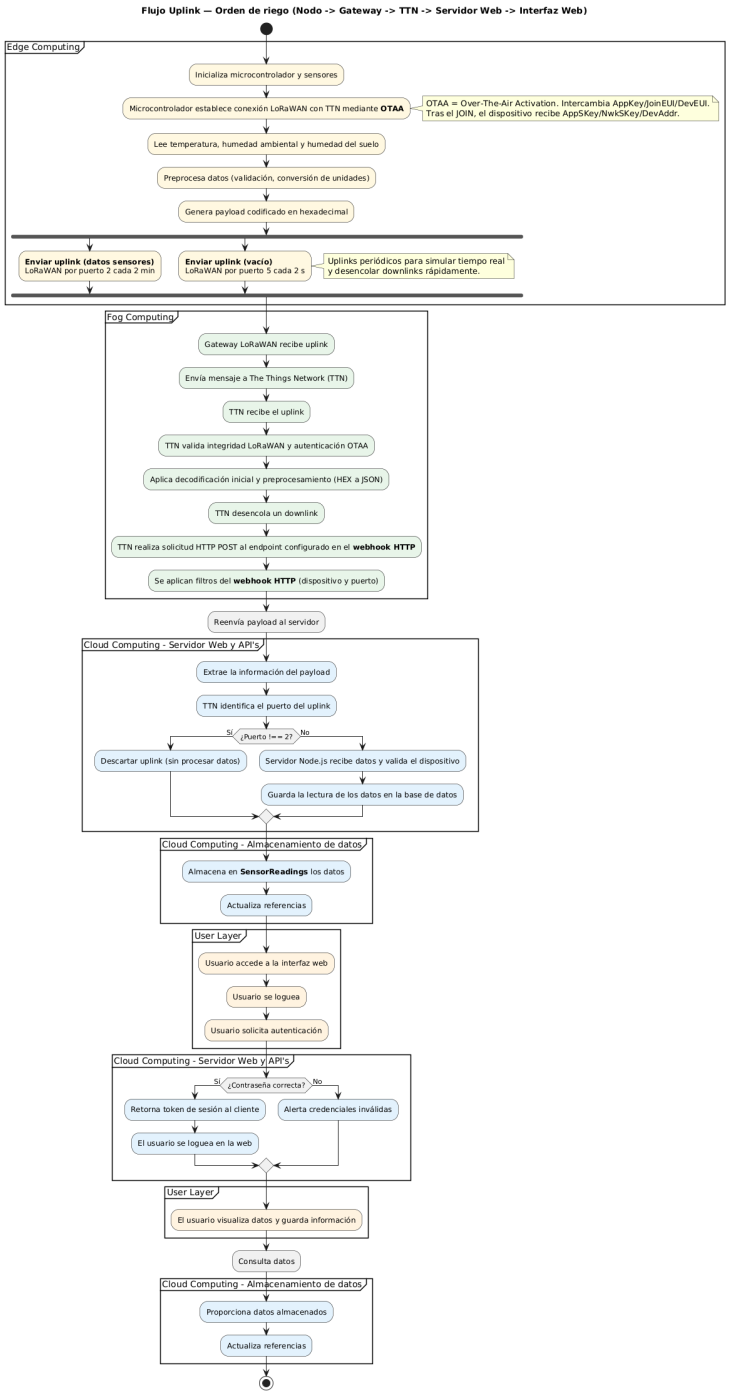


Figura 5.7: Diagrama de flujo de uplink

El flujo de downlink refleja el camino inverso desde una orden de riego iniciada

en la interfaz web, para ser procesada por el servidor web y encolada en TTN, hasta finalmente su entrega al dispositivo IoT y posterior ejecución del riego. En conjunto, ambos diagramas representan el comportamiento completo del sistema y las secuencias de los datos entre módulos.

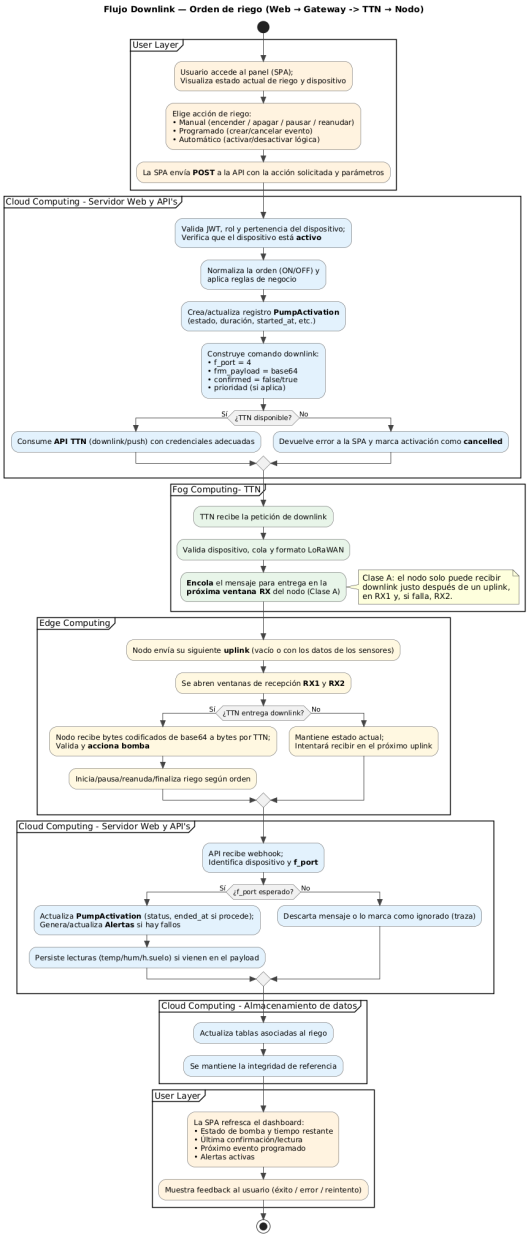


Figura 5.8: Diagrama de flujo de downlink

5.3. Diagramas de actividades

En la fase de diseño los diagramas de actividad nos sirven para describir procesos de negocio o flujos de trabajo relacionados con el software que se desarrolla. Nos van a servir de gran ayuda para describir el comportamiento de los casos de uso más importantes de la aplicación. Evidentemente no se analizarán todos los casos de uso, sólo se especificaran los diagramas de actividades sobre los casos de uso más importantes.

5.3.1. Envío de datos de sensores y postprocesamiento (Uplink)

Los casos de uso que describe este diagrama de actividad son los siguientes:
CU-05 4.9, CU-06 4.10, CU-07 4.11, CU-09 4.13, CU-12 4.16 y CU-35 4.39

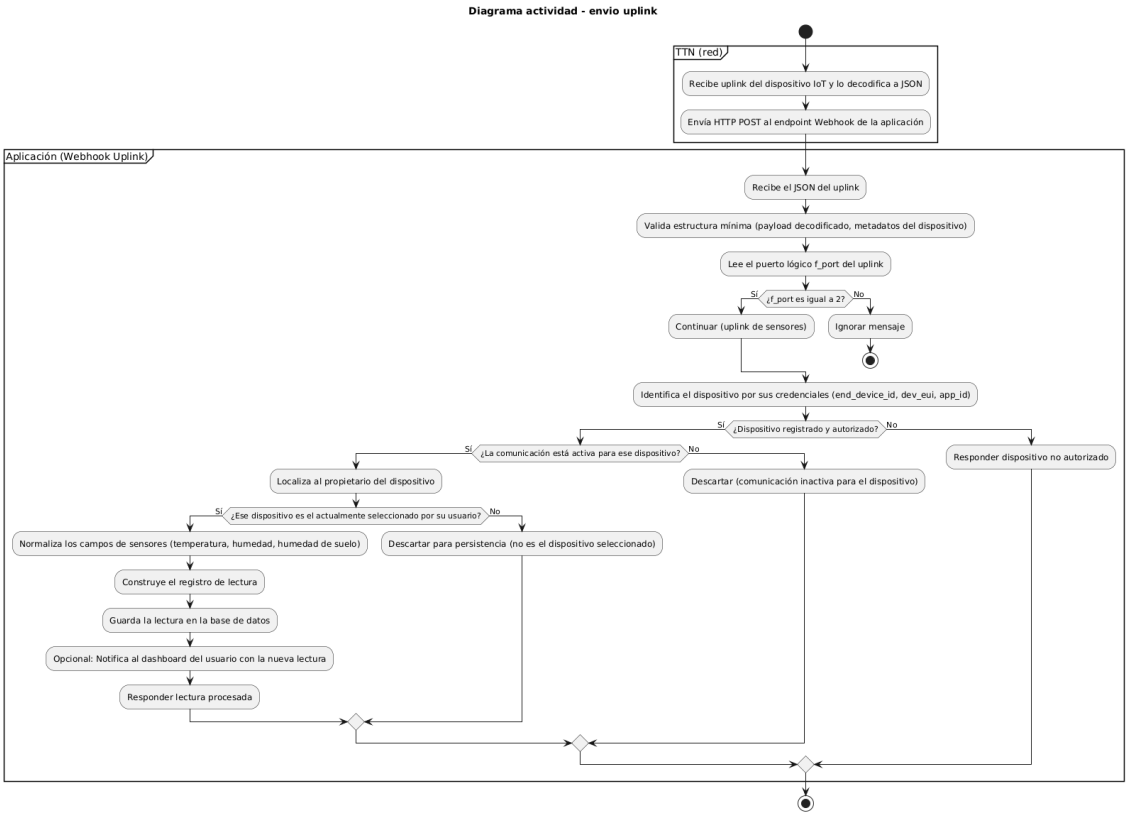


Figura 5.9: Diagrama de actividad de envió de datos de sensores

5.3.1.1. Activar/Desactivar comunicación del dispositivo

Los casos de uso que describe este diagrama de actividad son los siguientes: CU-35 4.39. Este diagrama de actividad tiene un propósito muy específico que es describir como se activar/desactiva comunicación dispositivo. Es decir lo que hace es autorizar o no que los datos que provienen de los sensores se guarden en la base de datos o no.

Figura 5.10: Diagrama de actividad de envió de datos de sensores

5.3.2. Riego Manual

Los casos de uso que describe este diagrama de actividad son los siguientes: CU-41 4.45 CU-16 4.20, CU-10 4.14, CU-17 4.21 y CU-46 4.50

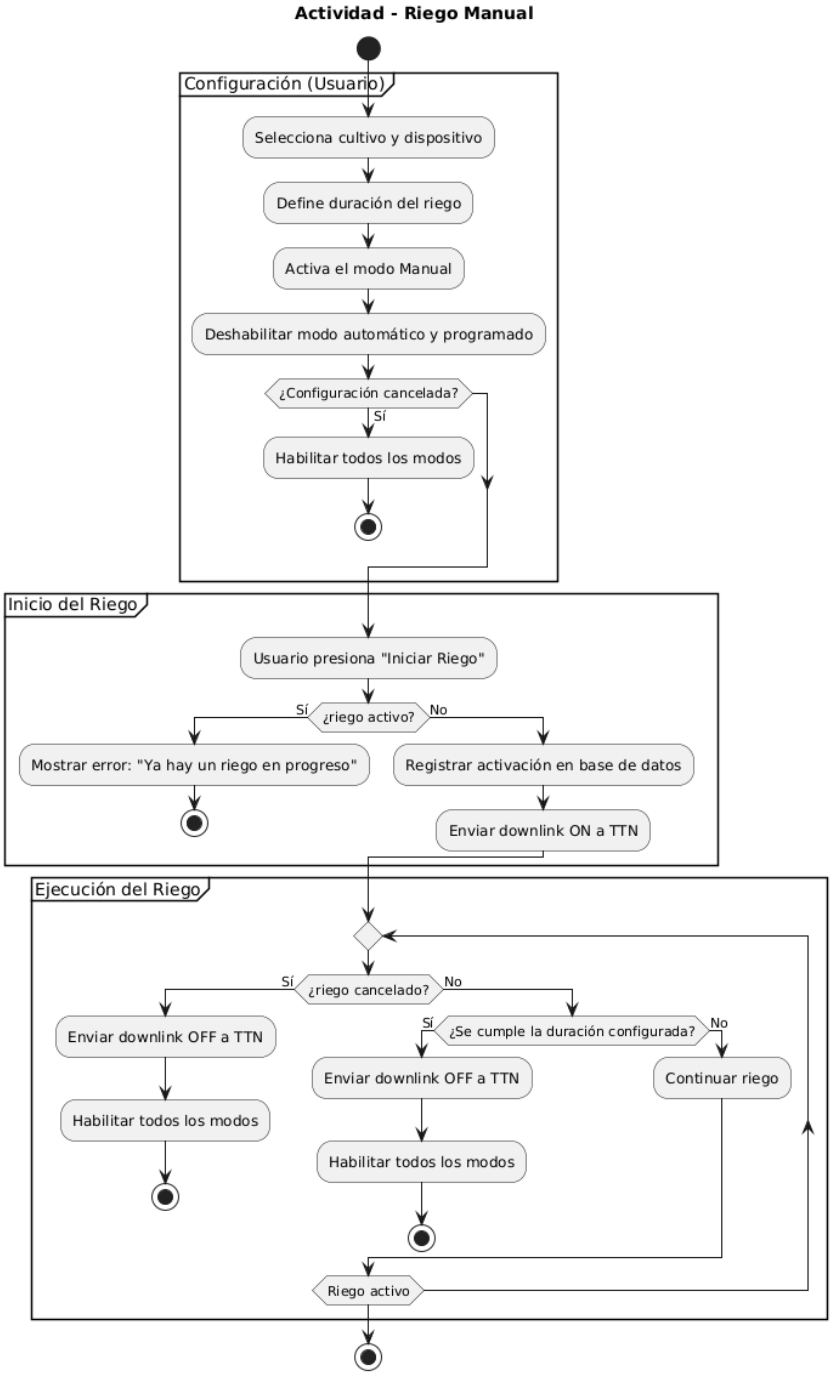


Figura 5.11: Diagrama de actividad del riego manual

5.3.3. Riego Automático

Los casos de uso que describe este diagrama de actividad son los siguientes: CU-43 4.47, CU-45 4.49, CU-16 4.20, CU-47 4.51 y CU-10 4.14. En este diagrama es necesario aclarar aclarar las condición principal que se debe de cumplir para que el riego se efectúe o por contrario se pare:

- Para que el riego se efectúe, Se debe de cumplir la siguiente condición
  - Humedad suelo $\leq$ mínimo
- Para que el riego se pare, se debe de cumplir una de estas condiciones
  - Humedad suelo en rango óptimo
  - Humedad >máximo

Tanto la temperatura ambiental como la humedad ambiental, son valores informativos y ayudará al usuario a determinar cuando se puede aproximar un riego, puesto que, las variaciones de temperatura y humedad ambiental tienen repercusión sobre la humedad del suelo.

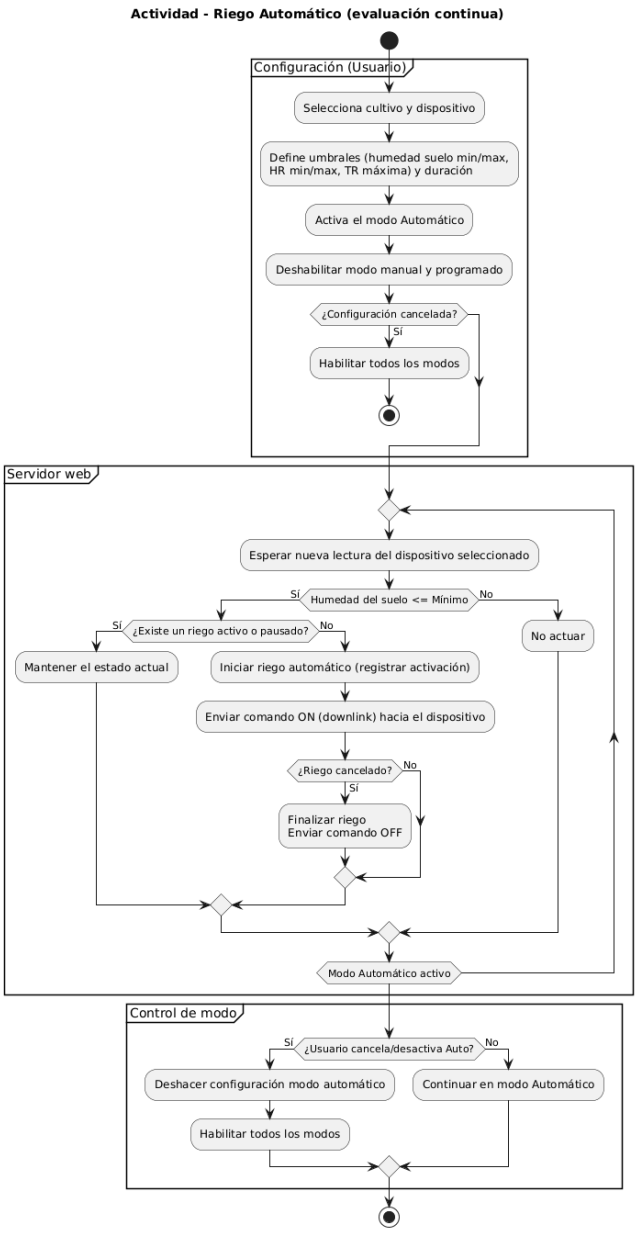


Figura 5.12: Diagrama de actividad del riego automático

5.3.4. Riego programado

Los casos de uso que describe este diagrama de actividad son los siguientes: CU-42 4.46, CU-44 4.48, CU-16 4.20, CU-17 4.21, CU-47 4.51 y CU-10 4.14

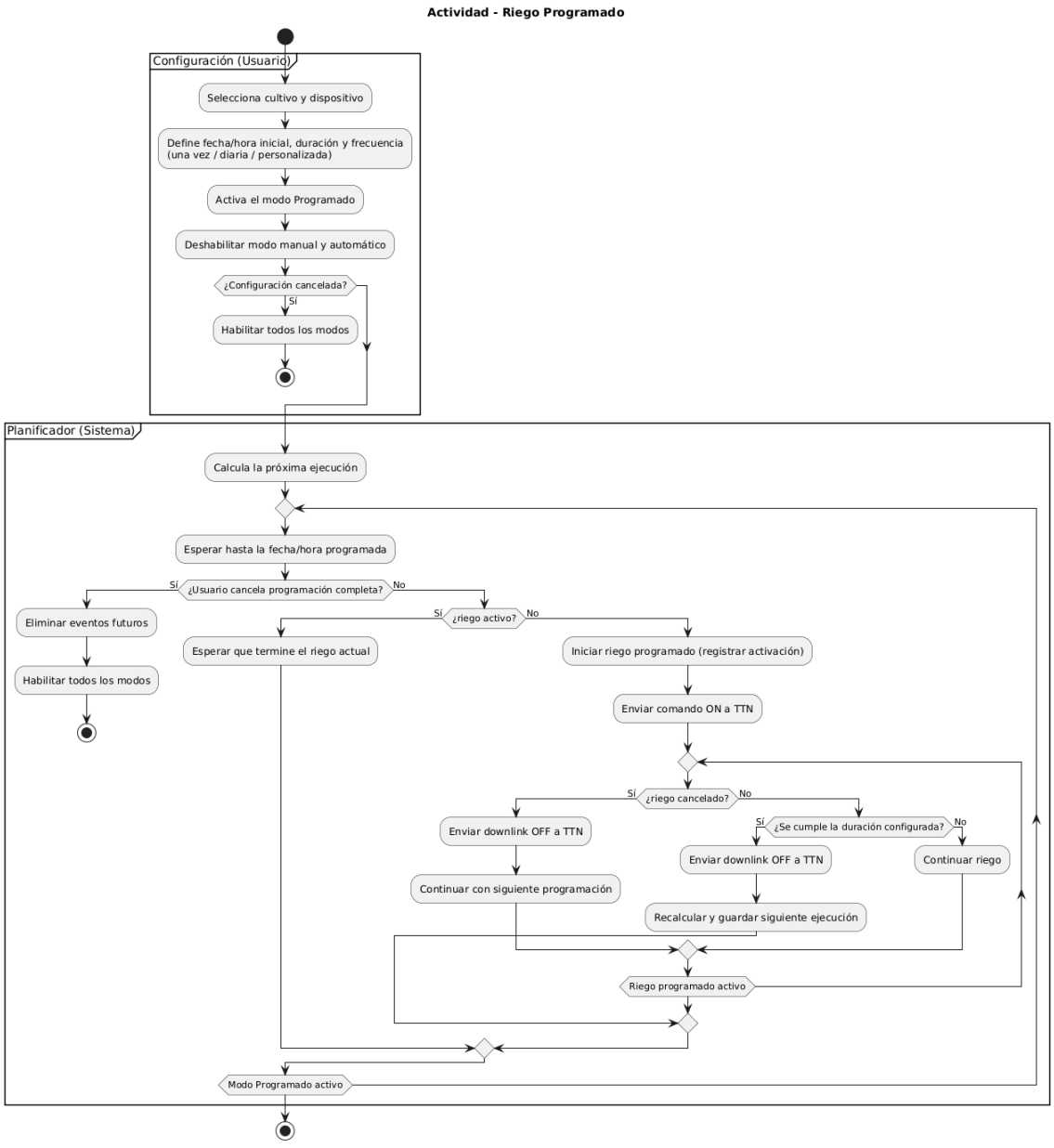


Figura 5.13: Diagrama de actividad del riego programado

5.3.5. Registro

Los casos de uso que describe este diagrama de actividad son los siguientes: CU-14 4.18

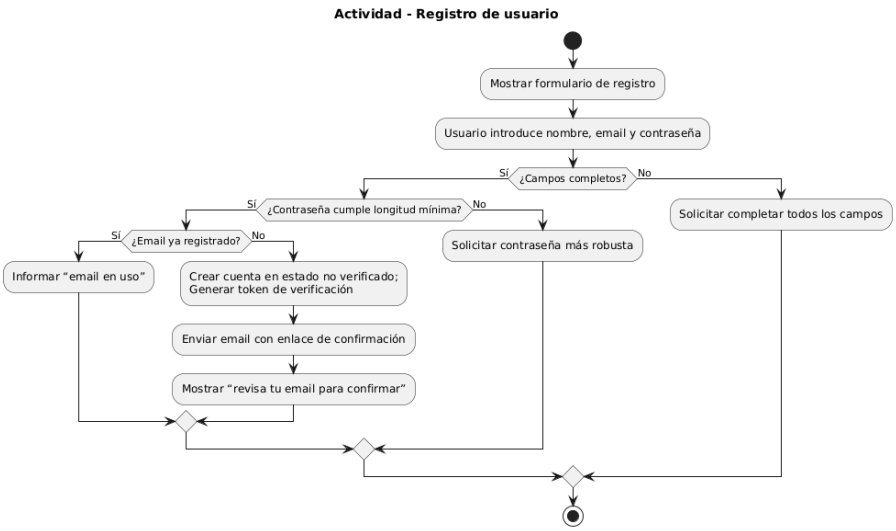


Figura 5.14: Diagrama de actividad de registro

5.3.6. Confirmar cuenta

Los casos de uso que describe este diagrama de actividad son los siguientes: CU-18 4.22

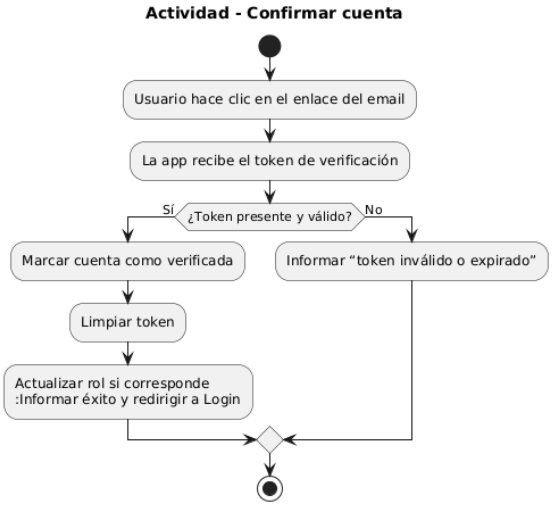


Figura 5.15: Diagrama de actividad de confirmar cuenta

5.3.7. Iniciar Sesión

Los casos de uso que describe este diagrama de actividad son los siguientes: CU-13 4.17, CU-11 4.15

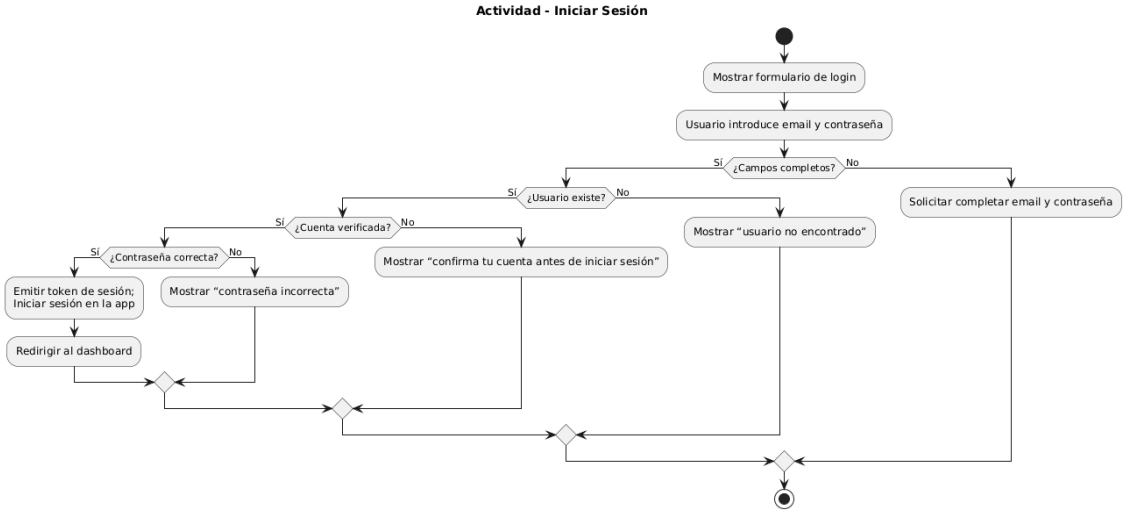


Figura 5.16: Diagrama de actividad de login

5.3.8. Emitir downlink TTN(ON/OFF)

Los casos de uso que describe este diagrama de actividad son los siguientes: CU-10 4.14, CU-16 4.20, CU-174.21 y CU-464.50

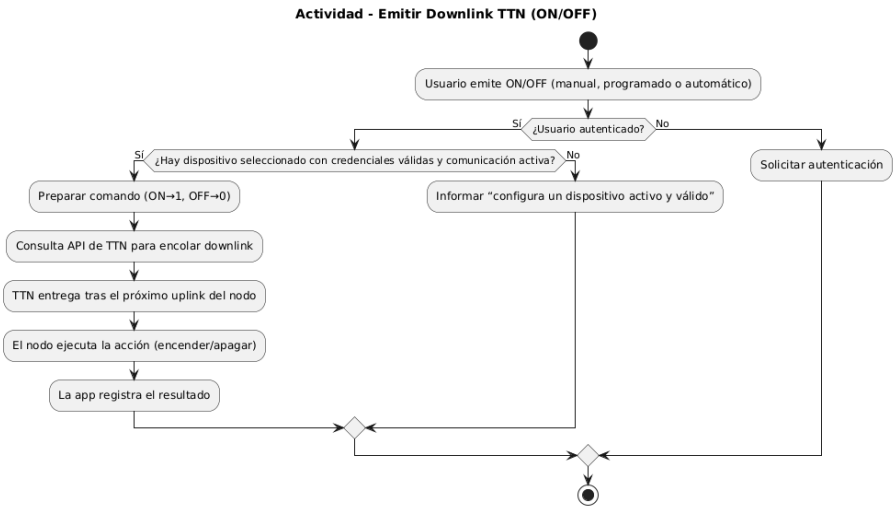


Figura 5.17: Diagrama de actividad de emitir downlink

5.3.9. Pausar/Reanudar riego

Los casos de uso que describe este diagrama de actividad son los siguientes: CU-46 4.50 y CU-10 4.12

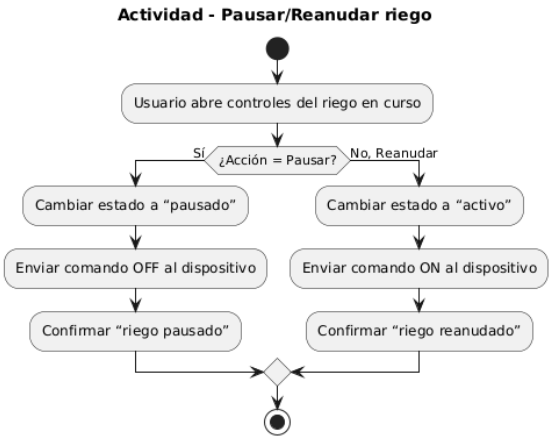


Figura 5.18: Diagrama de actividad de pausar/reanudar riego

5.4. Diagramas de Secuencia

Los diagramas de secuencia [57] es un tipo de diagrama de interacción que describe cómo y en qué orden, un grupo de objetos funcionan en conjunto. Aplicados a nuestro proyecto nos permitirán representar cómo se comunican los componentes y los objetos que constituyen estos componentes. Este tipo de diagramas están ligados estrechamente a los casos de uso, por lo que nos basaremos en estos para trazarlos. Debido a la gran cantidad de funcionalidades y casos de uso que que se han descrito anteriormente, sólo se realizarán los más importantes.

5.4.1. Login del usuario

El diagrama de secuencia mostrado en Figura 5.19 ilustra el flujo de interacción durante el inicio de sesión en la aplicación web. Muestra cómo el usuario introduce sus credenciales en el panel de autenticación , que luego se envían mediante una llamada fetch al servidor web a través de la ruta /auth/login. El servidor web consulta la base de datos PostgreSQL para verificar la existencia del usuario y validar su contraseña y estado de verificación. Si las credenciales son correctas y la cuenta está verificada, se genera un token JWT que se devuelve al cliente para almacenarse en el localStorage junto con datos básicos del usuario, permitiendo su acceso al Dashboard. En caso contrario, se retorna un mensaje de error que se muestra en la interfaz. Este diagrama de secuencia contempla los casos de uso CU-13 4.17 y CU-14 4.15.

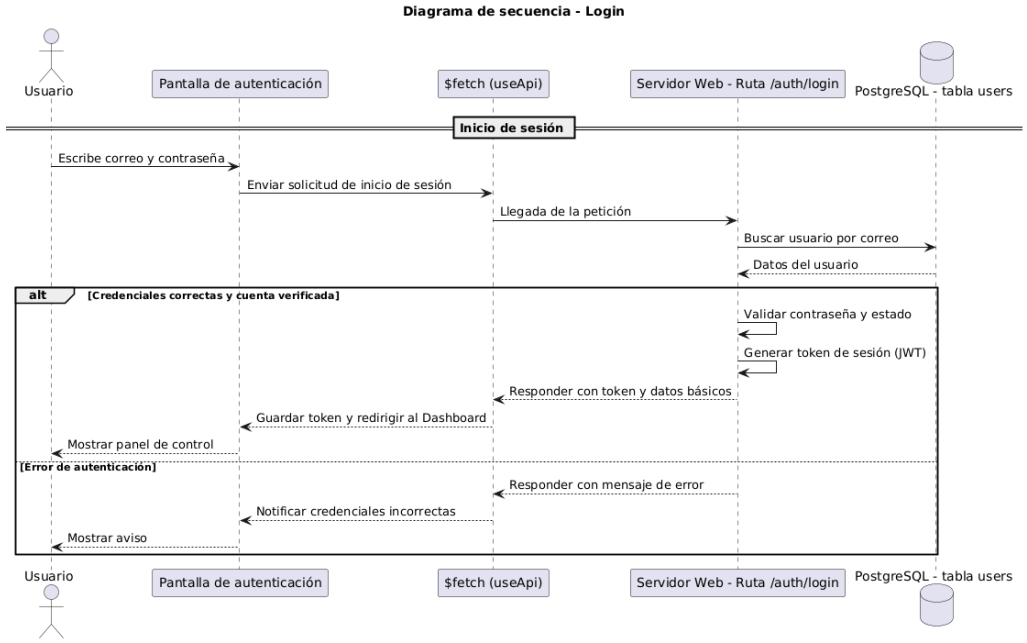


Figura 5.19: Diagrama de secuencia de login

5.4.2. Captura y almacenamiento de lecturas de sensores

Este diagrama de secuencia describe el flujo completo de envío y almacenamiento de lecturas de sensores en el sistema IoT. El proceso inicia en el dispositivo IoT, que captura los valores de temperatura y humedad ambiental y humedad del suelo, los empaqueta en 6 bytes y los transmite mediante LoRaWAN al gateway.

El gateway reenvía el paquete al servidor de TTN, que autentica el dispositivo vía OTAA, decodifica el payload y realiza una solicitud HTTP POST al endpoint HTTP configurado previamente en el webhook de TTN para luego reenviarlo al servidor web. El servidor valida la estructura de los datos y comprueba que el dispositivo está autorizado y con la comunicación activa. Si es así, almacena la lectura en la base de datos PostgreSQL y que traza el nuevo dato a la gráfica en tiempo real. En caso contrario, los datos son ignorados. Los casos de uso implicados son CU-01 4.5, CU-02 4.6, CU-05 4.9, CU-06 4.10, CU-07 4.11, CU-09 4.13 y CU-12 4.16.

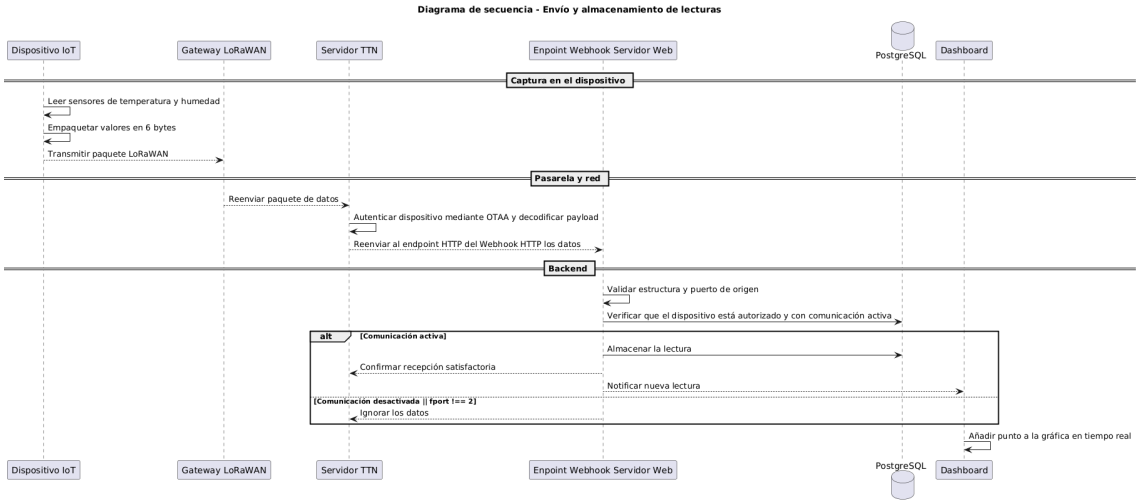


Figura 5.20: Diagrama de secuencia de captura y almacenamiento de lecturas de sensores

5.4.3. Emitir comando ON/OFF desde la web para control de riego

Este diagrama de secuencia muestra el proceso completo de control de la bomba de riego mediante ordenes ON/OFF. Inicia con las acciones del usuario en la interfaz web activar, pausar, reanudar o cancelar el riego. Cada orden se envía al servidor web, que valida el dispositivo y consume la API que expone TTN para encolar los datos de control mediante un downlink. El servidor TTN entrega la orden de riego al dispositivo IoT en la siguiente ventana de recepción tras un uplink. El dispositivo interpreta el byte recibido (0x01 para encender, 0x00 para apagar), ejecuta la acción sobre la bomba y envía un uplink de confirmación. También se contempla el apagado automático al incumplirse las condiciones de riego y la cancelación manual por parte del usuario. Los casos de uso implicados son CU-16/17 4.20 y 4.21, CU-10 4.14, CU-08 4.12, CU-03 4.7 y CU-04 4.8

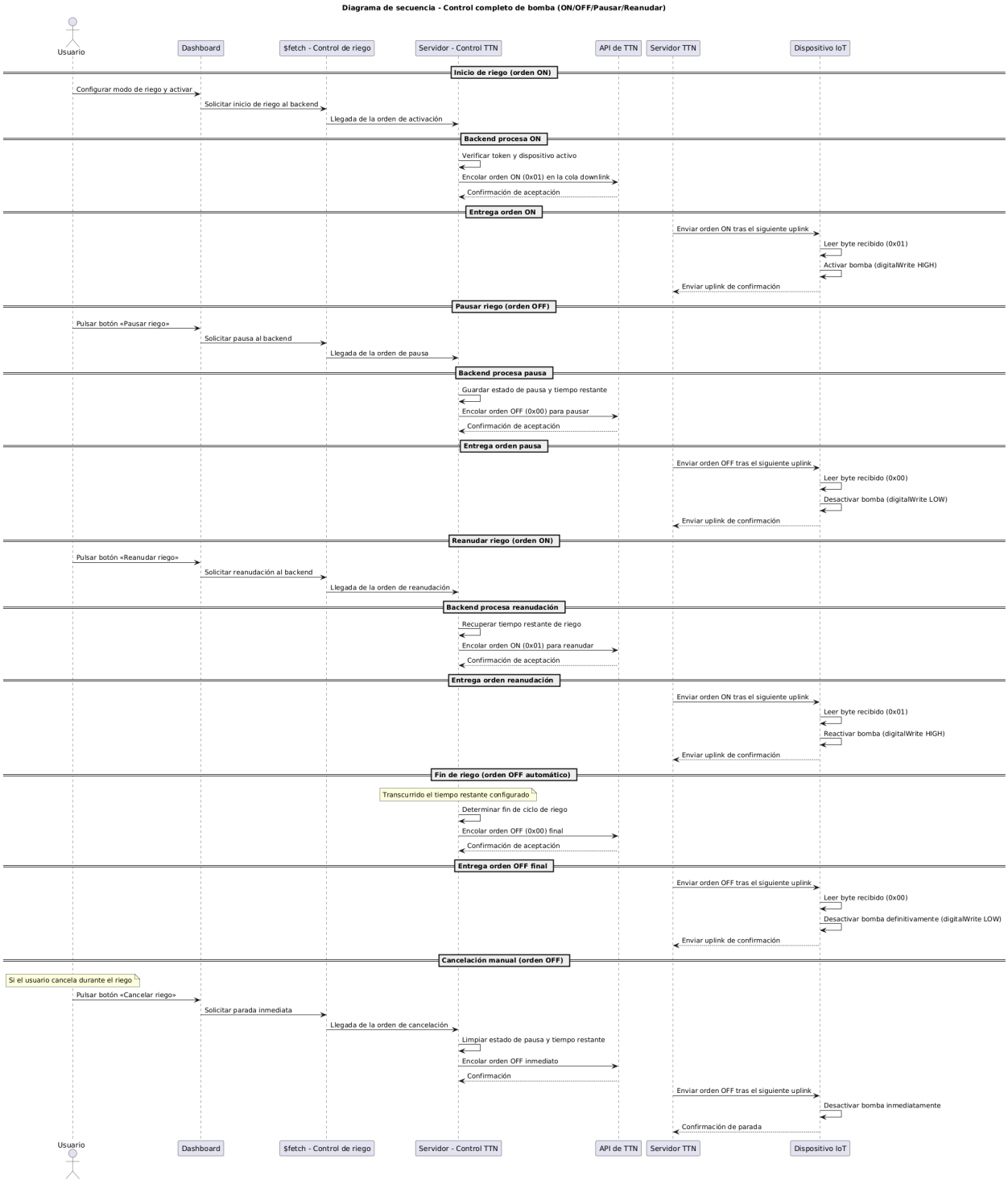


Figura 5.21: Diagrama de secuencia de emitir comando ON/OFF desde la web para control de riego

5.4.4. Configurar y ejecutar riego en modo programado

Este diagrama de secuencia representa el flujo para configurar y ejecutar un riego programado. El usuario define fecha, hora y duración desde la interfaz web, que envía la configuración al servidor web para registrarla en la base de datos. De forma periódica, el servidor revisa los eventos pendientes y, al llegar el momento programado, envía una orden ON al dispositivo IoT a través de la API de TTN para activar la bomba. Transcurrido el tiempo configurado, se envía un comando OFF para detener el riego, registrando en la base de datos tanto el inicio como el fin del riego. Los casos de uso implicados son CU-42 4.46, CU-44 4.48 y CU-16 4.20

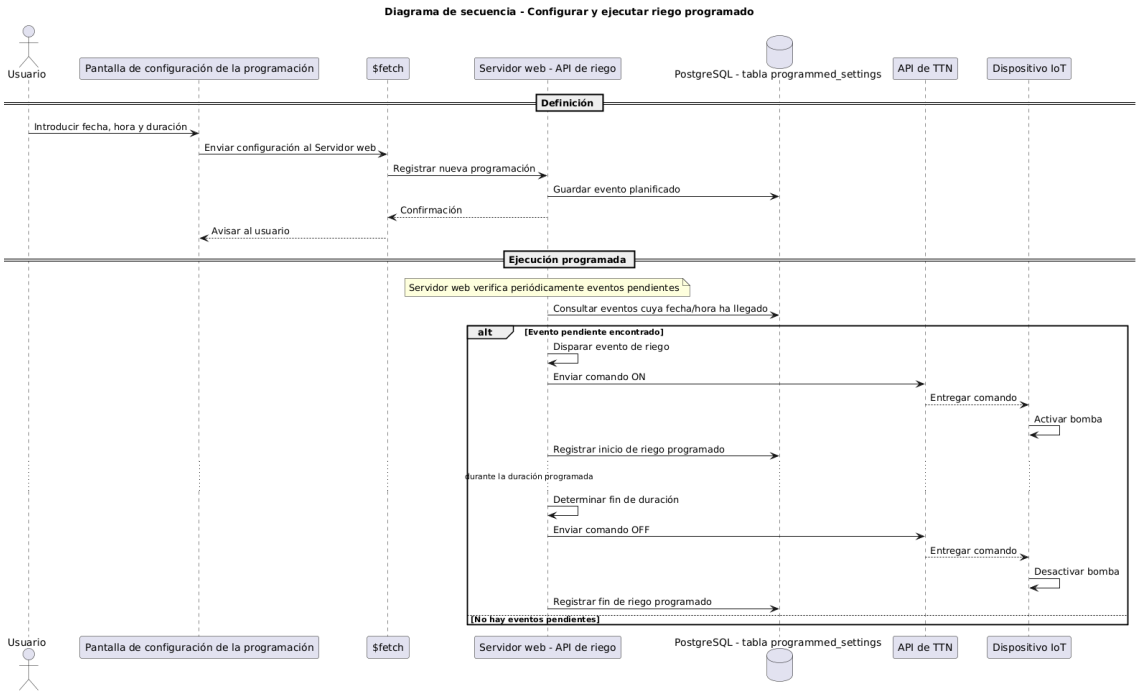


Figura 5.22: Diagrama de secuencia de configurar y ejecutar riego en modo programado

5.4.5. Activación por condiciones en el modo automático

Este diagrama de secuencia describe el funcionamiento del riego automático basado en umbrales. Cada vez que se almacena una nueva lectura de sensores, el proceso de evaluación consulta en la base de datos los valores registrados y los umbrales configurados por el usuario. Si detecta condiciones fuera de rango, envía un comando ON al dispositivo IoT mediante la API de TTN para activar la bomba, registrando tanto la alerta como el inicio del riego automático. Si se incumplen las condiciones

de riego se envía un comando OFF para detener la bomba de riego y registra el fin de riego. Los casos de uso implicado son CU-43 4.47 y CU-45 4.49

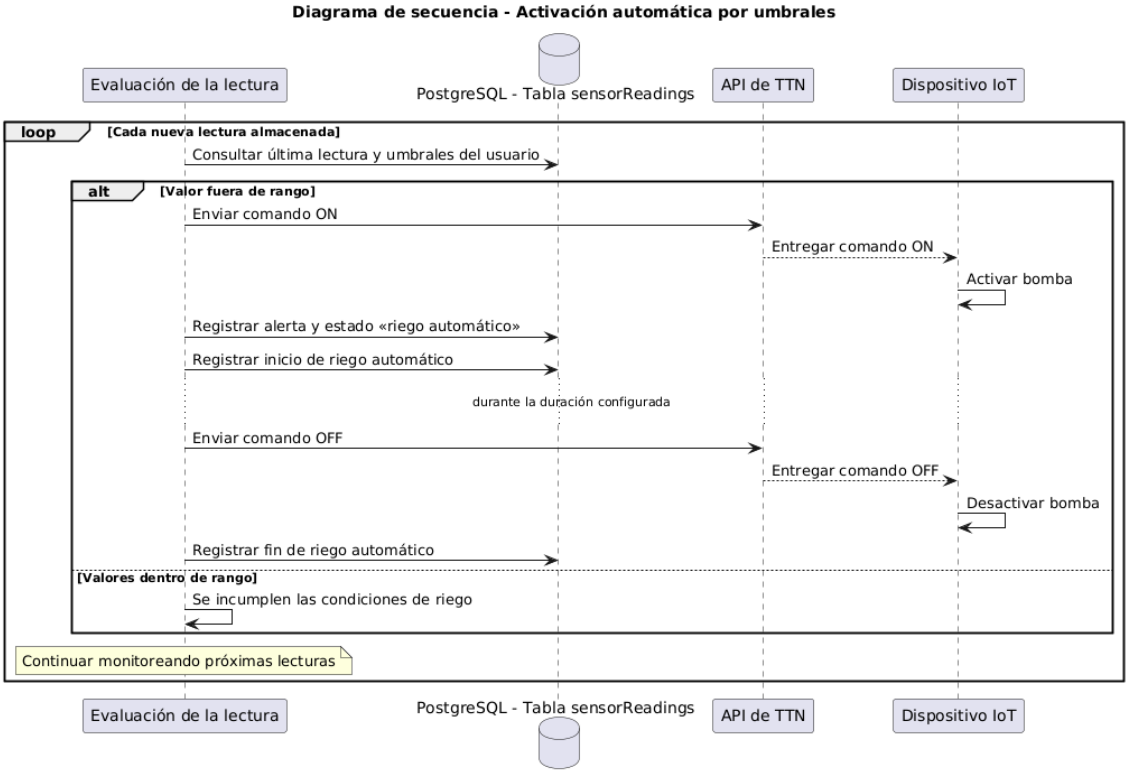


Figura 5.23: Diagrama de secuencia de activación por condiciones en el modo automático

5.4.6. Registro del usuario

Este diagrama de secuencia muestra el proceso de registro de un visitante, desde que completa el formulario de registro hasta que el servidor valida los datos, guarda el usuario como no verificado, genera un token y envía un correo de confirmación. Los casos de uso que implica son CU-14 4.18.

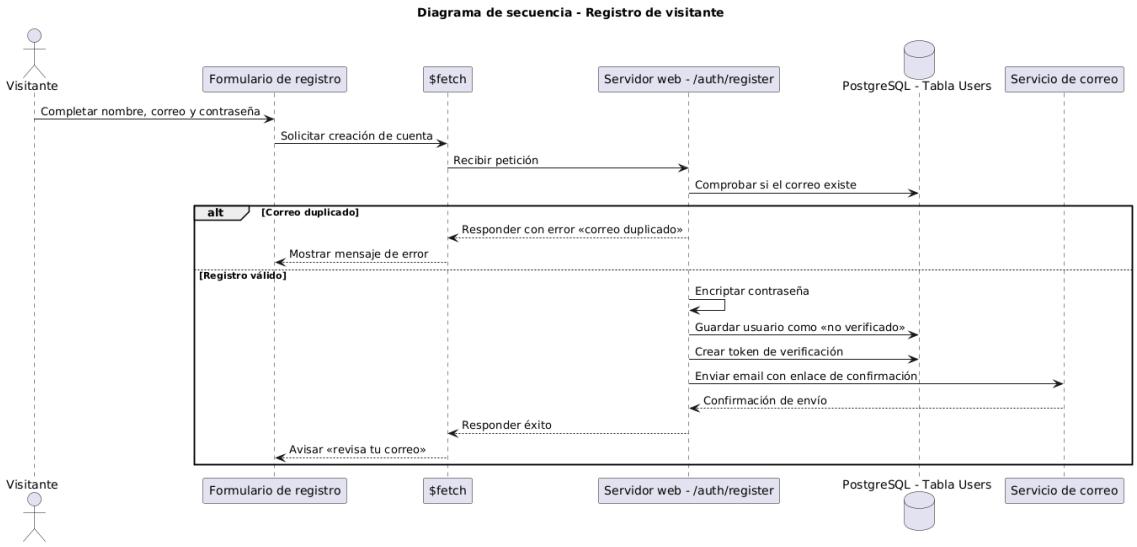


Figura 5.24: Diagrama de secuencia de registro

5.4.7. Confirmación de la cuenta

Este diagrama de secuencia describe el proceso de confirmación de cuenta mediante un enlace de verificación enviado al correo electrónico. El usuario accede a la URL con el token, que el servidor valida consultando la base de datos. Si el token es válido, se marca la cuenta como verificada y se elimina el token, redirigiendo al usuario al login con un mensaje de éxito. Si el token es inválido o ha expirado, se devuelve un error con la causa del fallo. Los casos de uso implicados son CU-18 4.22

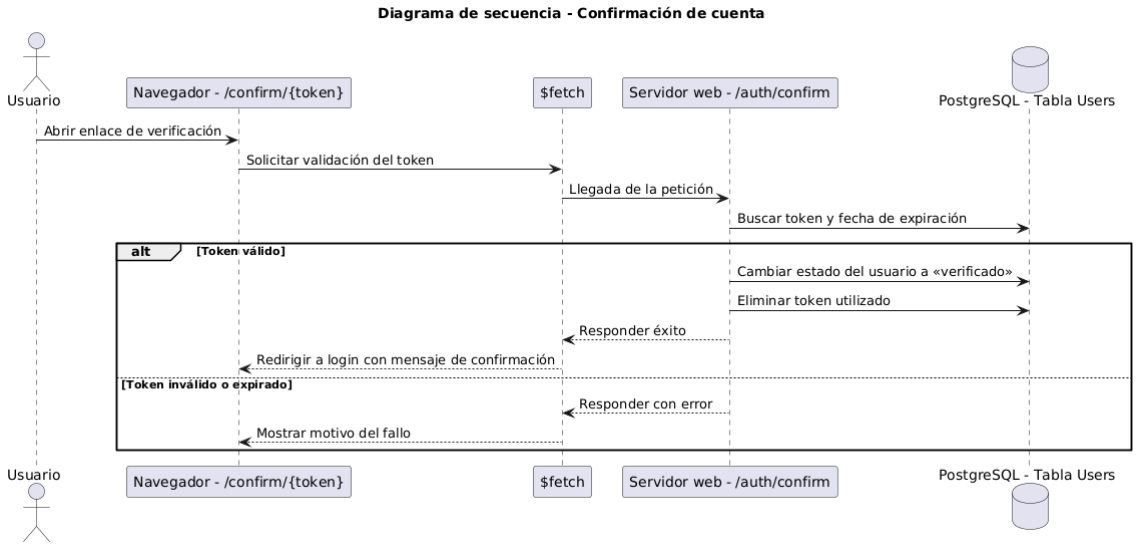


Figura 5.25: Diagrama de secuencia de confirmación de cuenta

5.5. Diseño del Módulo 1: Edge Computing - Captación de datos y acción sobre el actuador

El módulo 1 está constituido por el microcontrolador, encargado de capturar los datos de los sensores y de accionar el riego a través de la válvula o bomba de riego. El diseño de este sistema se enfoca en la portabilidad y facilidad de uso.

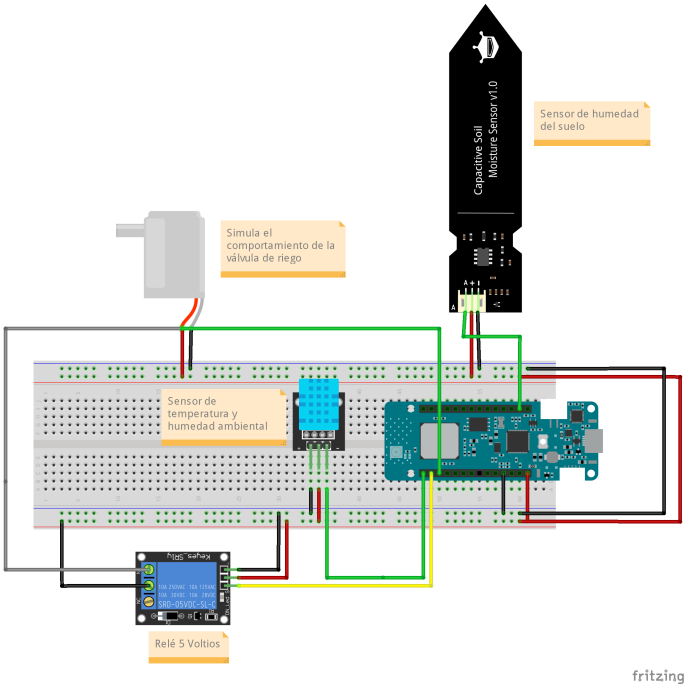


Figura 5.26: Esquema eléctrico del circuito - Realizado en fritzing

5.5.1. Componentes hardware

- **Placa Arduino MKR WAN 1310 con módulo LoRA:** Es el microcontrolador central que coordina la operación del sistema completo en la capa Edge o como se dice, es el nodo frontera. El nodo frontera es el dispositivo que está ubicado mas cerca de la fuente de datos y que actúa como punto de entrada antes de que los datos sean enviados a un centro de datos centralizado o a la nube. Por otro lado, permite la comunicación con TTN a través de LoRaWAN.

- **HUAREW 5V El aislamiento del optoacoplador del módulo de relé de 1 canal activación:** Un relé es un interruptor controlado eléctricamente. Su función es permitir que un circuito de baja potencia pueda controlar un dispositivo que funciona con una tensión o corriente mucho mayor, como es en nuestro caso la bomba de riego [14]. En resumen, permite abrir y cerrar el riego.

- **Sensor DHT11:** Este sensor es el encargado de medir la temperatura y humedad ambiental. Está compuesto por 3 pines, 1 pin de transmisión de datos a la izquierda (DATA), 1 pin de alimentación de 3.5V - 5.5V (VCC) en el medio y 1 pin de conexión a tierra (GND).

- **Capacitive Soil Moisture Sensor v1.2:** Este sensor es el encargado de medir la humedad en el suelo. Está compuesto por 3 pines, 1 pin a la izquierda para la salida análogica proporcional a la humedad del suelo (AOUT), 1 pin en el medio para la alimentación de 3.3V-5.5V (VCC), 1 pin a la derecha para conexión a tierra (GND). Algo importante a destacar es que este sensor necesita calibración. Para calibrar el sensor se tienen que leer los valores en aire (suelo seco) y luego sumergido en agua (suelo muy húmedo) para mapear correctamente el rango de salida y obtener lecturas precisas.

- **D260BLX-V Brushless Micropump Range:** Este es el componente principal de nuestro sistema puesto que es el encargado de extraer el agua desde un contenedor para expulsarla en un recipiente. Se trata de una bomba de agua no sumergible que permite extraer hasta 0.4l/min de agua. Haciéndola ideal para agricultura a pequeña escala. En el esquema eléctrico, representado en la Figura 5.26 se representa una bomba de riego sumergible puesto que no había otro tipo de bomba de agua disponible en la aplicación, pero el concepto es el mismo.

- **MiniHub v1.5 Wifi Indoor LoRaWAN Gateway:** En este apartado podríamos poner al gateway como un componente hardware del módulo 1. Pero realmente actúa como un puente entre el microcontrolador y TTN. Permitiendo el paso de mensajes entre estos 2 componentes. A través de la capa fog computing.

5.5.2. Proceso de operación

El diagrama de flujo de la Figura 5.27 describe el funcionamiento principal del firmware del microcontrolador encargado de gestionar la captación de datos y la comunicación con la red LoRaWAN mediante la implementación de clase A.

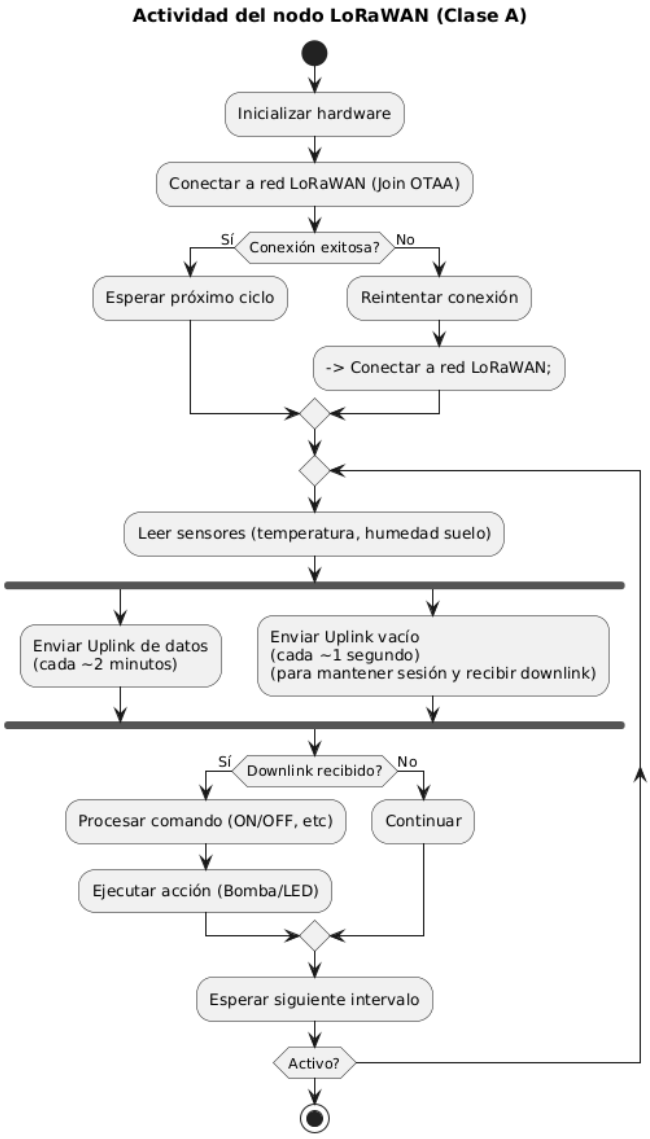


Figura 5.27: Diagrama de flujo del código del dispositivo IoT

El ciclo comienza con la inicialización del hardware, incluyendo sensores (temperatura y humedad relativa y humedad del suelo) y el módulo de comunicación LoRa. A continuación, el dispositivo intenta conectarse a la red LoRaWAN mediante el procedimiento de autenticación OTAA. Si la conexión falla, se inicia un proceso de reconexión hasta que sea exitosa.

Una vez conectado, el dispositivo entra en un bucle que repite las siguientes

acciones:

1. **Lectura de sensores:** se obtienen los valores de temperatura y humedad ambiental y humedad del suelo.
2. **Envío de uplinks:**
   - Un uplink de datos cada 2 minutos, para emitir las medidas de los sensores.
   - Un uplink vacío cada 1 segundo, cuyo propósito es permitir la recepción de downlinks de manera casi inmediata.
3. **Recepción de downlinks:** si se recibe una orden desde el servidor web, se procesa y ejecuta (por ejemplo, encendido o apagado de la bomba de riego).
4. Espera el siguiente intervalo antes de repetir el ciclo.

5.5.3. Formato de datos

El programa del microcontrolador envía dos tipos de uplink con diferente estructura de datos para cumplir su función, aparte de los uplink también recibe un downlink:

5.5.3.1. Uplink de datos de sensores

El programa del microcontrolador envía la información de los sensores de temperatura y humedad ambiental y humedad del suelo en un payload binario de 6 bytes. Se ha optado por este formato con el objetivo de minimizar el tamaño del paquete y, por tanto, reducir el tiempo de transmisión y el consumo energético.

La estructura del **payload** es la siguiente:

- **Bytes 0-1:** Temperatura ×10 (entero con signo, 16 bits, big-endian)
- **Bytes 2-3:** Humedad ambiental ×10 (entero con signo, 16 bits, big-endian)
- **Bytes 4-5:** Humedad del suelo ×10 (entero con signo, 16 bits, big-endian)

La conversión y empaquetado se realiza de la siguiente forma:

```
// Convierte los valores a enteros * 10
int16_t temp_int = (int16_t)(temperature * 10);
int16_t hum_int = (int16_t)(humidity * 10);
int16_t soil_int = (int16_t)(soilMoisturePercent * 10);
```

```
// Empaquetado en buffer de transmisión (big-endian)
payload[0] = (uint8_t)(temp_int >> 8);
payload[1] = (uint8_t)(temp_int & 0xFF);
payload[2] = (uint8_t)(hum_int >> 8);
payload[3] = (uint8_t)(hum_int & 0xFF);
payload[4] = (uint8_t)(soil_int >> 8);
payload[5] = (uint8_t)(soil_int & 0xFF);
```

Al limitar el tamaño a 6 bytes, se optimiza el tiempo de aire y se reduce el consumo de batería

5.5.3.2. Uplink Vacío (heartbeat)

Este uplink tiene la única finalidad de abrir las ventanas RX1/RX2 más rapidamente permitiendo un comportamiento casi en tiempo real y permitir la entrega en cualquier momento. Se envía con un solo byte con ningún contenido.

La estructura del **payload** es la siguiente:

- **Byte 0:** Código de control (ejemplo: 0x00 para paquete vacío sin acción, 0x01 para confirmar recepción o indicar estado ON).

Este formato reduce el tamaño al mínimo posible, asegurando un tiempo de aire muy bajo y un consumo mínimo de energía.

5.5.3.3. Downlink con datos de control de riego

Es el comando enviado desde el servidor de red de TTN hacia el dispositivo IoT para actuar sobre la bomba.

La estructura del **payload** es la siguiente:

- **Byte 0** del contenido
  - 0x01 → Encender bomba
  - 0x00 → Apagar bomba

5.6. Diseño del módulo 2: Fog Computing - Recolección y preprocesamiento

El módulo 2 se encarga de gestionar la comunicación intermedia entre los dispositivos IoT y el servidor de red de TTN. Su función principal es recibir los datos

ambientales (temperatura y humedad) y datos de humedad del suelo enviados desde el microcontrolador a través de la red LoRaWAN, así como las ordenes de control para accionar el actuador de la bomba procedentes del sevidor de red de TTN, previamente enviadas desde el servidor web.

En esta etapa se lleva a cabo el preprocesamiento de la información, que incluye la decodificación de los bytes generados por el dispositivo final en formato JSON, con el fin de que sean interpretables por el servidor web, el cual tiene un endpoint expresamente definido para recibir información desde TTN, a su vez este endpoint HTTP está configurado en el webhook de TTN. TTN aplicará el método HTTP POST a este endpoint, reenviando los datos de los sensores al servidor web. Tras este proceso, el servidor recibe un mensaje en formato JSON para su posterior normalización en el modelo interno del sistema y, por ende, su almacenamiento y visualización en la dashboard.

Del mismo modo, el módulo 2 se encarga de codificar de base64 a bytes la orden enviada desde el servidor web. De esta forma, se transforman en un formato compatible con el protocolo LoRaWAN antes de ser enviadas desde TTN hacia el dispositivo IoT en la próxima ventana de recepción. Hay que destacar que los downlinks no se hacen a través del webhook, sino que se hacen a a través de la API de TTN, al consumir la API permite al usuario encolar downlinks con datos de control de riego. Con la finalidad de ser desencolados posteriormente al recibir un uplink, para luego mandar esa información al dispositivo IoT al abrirse las ventanas de recepción.

En conjunto, este módulo asegura una comunicación bidireccional consistente entre el dispositivo IoT y el servidor web.

5.6.1. Arquitectura general

- **Servidor de red de TTN:** Recibe los mensajes a través del protocolo LoRaWAN del gateway conectado a TTN. Por otro lado también recibe datos de control de riego desde el servidor web.
- **Proceso de codificación/decodificación** como hemos dicho anteriormente se encarga de decodificar a formato JSON los datos provenientes del dispositivo final y codificar los datos de control de riego que llegan desde el servidor web. Esto se hace a través de un payload formatter definido en TTN.
- **API TTN:** Al consumir la API de TTN por parte del servidor web, las ordenes de riego se encolan en TTN para que en el próximo uplink se abran ventanas de recepción y el mensaje pueda ser desencolado y enviado al dispositivo IoT. En el caso de que el dispositivo IoT esté disponible.El proceso consiste en realizar

una petición HTTP POST al endpoint correspondiente de TTN, enviando un objeto JSON que incluye el comando codificado en base64 (frm_payload), el puerto lógico (f_port) y la prioridad, de forma que TTN almacena el mensaje en cola y lo entrega al dispositivo en la siguiente ventana de recepción disponible tras un uplink.

- **Webhook HTTP:** Los datos se reenvían al endpoint HTTP definido en el webhook HTTP. De forma que TTN realizará una petición HTTP POST al endpoint HTTP configurado, que a su vez este también estará definido en el servidor web.

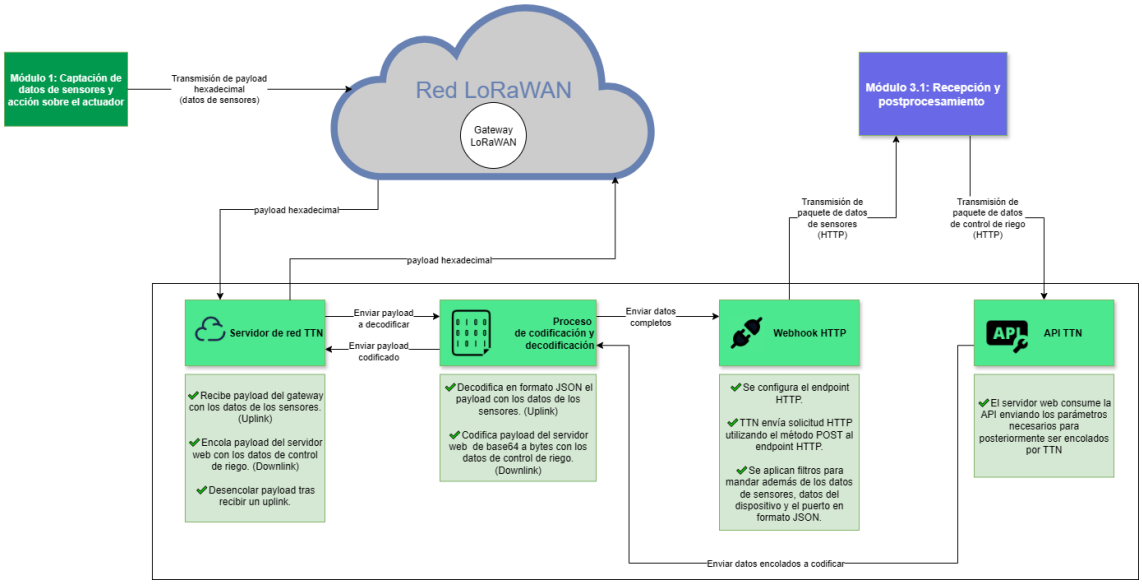


Figura 5.28: Arquitectura General del Módulo 2: Recolección y Preprocesamiento

En la Figura 5.29 se puede una representación visual de cómo trabaja este módulo dentro del sistema completo y que funciones desempeña en el mismo. A continuación pasamos a ver los procesos principales en más detalle.

5.6.2. Proceso de decodificación

Cuando nuestro dispositivo IoT envía un uplink, este llega al servidor de red de TTN en forma de bytes junto con el número de puerto (fPort). Para que el servidor web pueda entender la información que le envía TTN mediante el método POST al endpoint HTTP configurado en el webhook, es necesario hacer uso de una función que permita decodificar la información de formato bytes a JSON. Para ello se hace uso de un Payload Formatter.

Al configurar un Payload Formatter en TTN, en el momento en el que se envía un uplink desde el dispositivo IoT, TTN ejecuta automáticamente la función decodeUplink(input) sobre esos datos antes de mostrarlos en la consola de TTN o reenviarlos a integraciones como el Webhook. En nuestro caso, el formatter interpreta que si el uplink llega por el puerto 3 se trata de un mensaje de control sin datos útiles, devolviendo un JSON con una marca de "ignored" y una advertencia.

Si llega por otro puerto, asume que los dos primeros bytes representan la temperatura ambiental, los dos siguientes la humedad ambiental y dos últimos para la humedad del suelo, combinando cada par en un valor de 24 bits y dividiéndolo entre 10 para obtener el valor real, que se devuelve en un objeto JSON con las claves temperature, humidity y soil_moisture

```
function decodeUplink(input) {
  const bytes = input.bytes;
  const port = input.fPort;

  // Puerto 3 = uplink de control, no almacenar
  if (port === 3) {
    return {
      data: { info: "uplink de control", ignored: true },
      warnings: ["Uplink vacío recibido en puerto 3"]
    };
  }

  // Validación de longitud esperada (6 bytes)
  if (!bytes || bytes.length < 6) {
    return {
      data: {},
      warnings: ["Payload inesperado: se esperaban 6 bytes"],
      errors: ["Longitud de payload insuficiente"]
    };
  }

  // Helper para int16 big-endian
  const toInt16 = (msb, lsb) => {
    let v = (msb << 8) | lsb;
    if (v & 0x8000) v = v - 0x10000; // signo
    return v;
  };

  const temp10 = toInt16(bytes[0], bytes[1]); //  C  * 10
  const hum10  = toInt16(bytes[2], bytes[3]); // %RH * 10
  const soil10 = toInt16(bytes[4], bytes[5]); // % * 10

  const temperature_c      = temp10 / 10.0;
  const humidity_rh        = hum10  / 10.0;
```

```
  const soil_moisture_pct  = soil10 / 10.0;

  return {
    data: {
      temperature_c,
      humidity_rh,
      soil_moisture_pct
    },
  };
}
```

En el caso de los downlinks, no es necesario un formatter para codificar el mensaje, ya que TTN puede realizar internamente la conversión de base64 a bytes. Aunque se ha integrado un formatter cuya finalidad es mostrar un mensaje de apoyo visual, para indicar si la bomba ha sido encendida o apagada cuando se recibe un downlink.

```
var LED_STATES = ['off', 'on'];

function decodeDownlink(input) {
  return {
    data: {
      ledState: LED_STATES[input.bytes[0]]
    }
  }
}
```

5.6.3. Integración webhook

Para transmitir los datos procesados al siguiente módulo, se utiliza un webhook configurado en TTN que envía automáticamente una solicitud HTTP POST al servidor web cuando se recibe un paquete de datos procedente del dispositivo IoT.

Para transmitir los datos procesados al siguiente módulo, se debe de configurar un endpoint HTTP en el webhook. En el servidor web este endpoint es definido y a través de este endpoint, TTN enviará solicitudes HTTP POST al servidor web con la información procedente del dispositivo final. La información se enviará al servidor web en formato JSON y tiene la siguiente estructura. En resumen, básicamente este endpoint es una API que el servidor web consume, y por ende, recibe como respuesta, información del dispositivo final.

```
 "end_device_ids": {
    "device_id": "exampledevice123",
    "application_ids": {
      "application_id": "testprueba005"
    },
    "dev_eui": "A8610A343638860F",
```

```
    "join_eui": "2DCF462904B478D8",
    "dev_addr": "260B54C4"
  },
  "uplink_message": {
    "f_port": 2,
    "decoded_payload": {
      "humidity_rh": 50,
      "soil_moisture_pct": 100,
      "temperature_c": 28.3
    }
  }
}
```

Es necesario destacar que en el webhook se puede filtrar la información que se va a enviar al servidor web. En nuestro caso se han aplicado estos filtros:

- **up.uplink_message_decoded_payload:** Nos permite filtrar los datos de los sensores que llegan dede dispositivo final
- **end_device_ids:** Nos permite filtrar la información del dispositivo desde dónde se envían los datos. La información del dispositivo nos permite compararla con los dispositivos actualmente almacenados en la base de datos, si hay una coincidencia y además la comunicación está activa para ese dispositivo, se guardan los datos de los sensores en la base de datos.
- **up.uplink_message.f_port:** Nos permite filtrar el puerto dónde se ha enviado el uplink. El puerto va a servir para determinar si la información que llega al servidor web desde TTN se va a almacenar o no. Esto es debido a que hay dos tipos de uplink que se van a enviar. El primer tipo, es un uplink con los datos de los sensores que se van a enviar por el primer puerto. El segundo tipo, es un uplink vacío que se envía periódicamente por el tercer puerto para que los downlinks se desencolen más rápido, con el fin de lograr una respuesta más rápida sobre el control del riego. Por lo que a nosotros nos interesa almacenar en la base de datos la información que viene del primer puerto.

5.7. Diseño del Módulo 3.1: Cloud Computing - Recepción y postprocesamiento

El Módulo 3.1 constituye el servidor web del sistema completo, encargado de recibir los datos enviados desde TTN a través de un webhook previamente configurado, procesarlos adecuadamente mediante validación y almacenarlos de forma estructurada para su posterior visualización en el dashboard del sistema. También se encargará de mandar ordenes de control de riego a TTN.

Por otro lado, el servidor web se encargará de recibir payloads de las lecturas del sensor DHT11 (temperatura y humedad) y del sensor de humedad del suelo desde dispositivos IoT, los valida contra credenciales TTN almacenadas en la base de datos para ese dispositivo IoT, y los procesa para activar el sistema de riego en base a unas ciertas condiciones de riego. Además de recibir payloads, también enviará payloads con datos de control de riego a TTN mediante la API de TTN.

Debido a la gran importancia que tiene el servidor web en nuestro proyecto será necesario hacer hincapié en cómo será diseñado y qué patrón de diseño de software se pueden aplicar para que sea lo más escalable y funcional posible.

En la Figura 5.29 podemos ver la arquitectura completa del módulo 3 y como este interactúa con el módulo 2 y con el módulo 4, representado como la interfaz web. Se puede observar que la arquitectura del módulo 3 coincide con el patrón de diseño MVC.

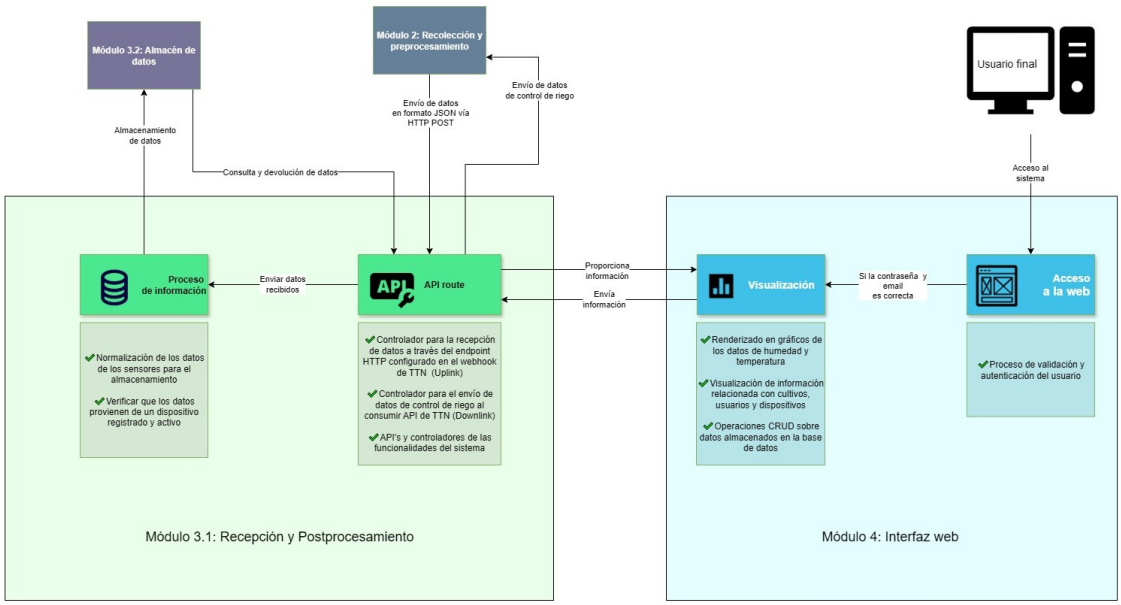


Figura 5.29: Arquitectura General del módulo 3 - Cloud Computing

5.7.1. Arquitectura REST API

Para el diseño de este módulo se ha tenido en cuenta la posterior escalabilidad y por supuesto, la simplicidad del proyecto. Por ello, se ha decidido proveer la funcionalidad del sistema a través de una serie de API'S construidas en base a la arquitectura REST. Las API's están formadas por un conjunto de puntos de acceso o endpoints que permiten interactuar con la funcionalidad del sistema. Para acceder a la funcionalidad del sistema, el cliente deberá realizar una solicitud HTTP usando un método determinado ya sea GET, POST, PUT o DELETE. Cuando el servidor recibe esa petición, ejecuta la lógica necesaria y devuelve una respuesta con el recurso solicitado o el resultado de la acción.

- **Verbos HTTP:** Los métodos HTTP que se implementarán en nuestra aplicación son HTTP GET, POST, PUT, DELETE.
- **Formato de las respuestas:** El servidor web responderá a las solicitudes en formato JSON
- **Códigos de Estado HTTP:** Implementación correcta de códigos de respuesta

A continuación se van a documentar todas las API'S que se han diseñado para cumplir con todos los requisitos de funcionalidad.

Tabla 5.1: Tabla API REST - Autenticación

	Endpoint	**Método**	**Descripción**	**Autenticación**
Autenticación	`/api/auth/register`	POST	Registrar nuevo usuario	No
	`/api/auth/login`	POST	Iniciar sesión	No
	`/api/auth/verify/:token`	GET	Verificar cuenta por email	No
	`/api/auth/forgot-password`	POST	Solicitar recuperación de contraseña	No
	`/api/auth/verify-token/:token`	GET	Verificar token de reset de contraseña	No
	`/api/auth/update-password/:token`	PUT	Actualizar contraseña con token	No
	`/api/auth/change-password/:id`	PUT	Cambiar contraseña (usuario autenticado)	Sí
	`/api/auth/user`	GET	Obtener perfil de usuario autenticado	Sí
	`/api/auth/admin`	GET	Verificar permisos de administrador	Sí
	`/api/auth/logout`	POST	Cerrar sesión	Sí

Tabla 5.2: Tabla API REST - Usuarios

	Endpoint	**Método**	**Descripción**	**Autenticación**
Usuarios	`/api/users`	GET	Obtener todos los usuarios	Sí(Admin)
	`/api/users/:id`	GET	Obtener usuario por ID	Sí
	`/api/users/:id`	PUT	Actualizar usuario	Sí
	`/api/users/:id`	DELETE	Eliminar usuario	Sí

Tabla 5.3: Tabla API REST - Dispositivos

	Endpoint	**Método**	**Descripción**	**Autenticación**
Dispositivos	`/api/devices`	POST	Crear dispositivo	Sí
	`/api/devices`	GET	Obtener todos los dispositivos	Sí
	`/api/devices/:id`	GET	Obtener dispositivo por ID	Sí
	`/api/devices/:id`	PUT	Actualizar dispositivo	Sí
	`/api/devices/:id`	DELETE	Eliminar dispositivo	Sí
	`/api/devices/user/:user_id`	GET	Dispositivos por usuario	Sí
	`/api/devices/:id/activate`	PUT	Activar dispositivo	Sí
	`/api/devices/:id/deactivate`	PUT	Desactivar dispositivo	Sí
	`/api/devices/:id/latest-reading`	GET	Última lectura del dispositivo	Sí

Tabla 5.4: Tabla API REST - Cultivos

	Endpoint	**Método**	**Descripción**	**Autenticación**
Cultivos	`/api/crops`	POST	Crear cultivo	Sí
	`/api/crops`	GET	Obtener todos los dispositivos	No
	`/api/crops/:id`	GET	Obtener cultivo por ID	Sí
	`/api/crops/:id`	PUT	Actualizar cultivo	Sí
	`/api/crops/:id`	DELETE	Eliminar dispositivo	Sí
	`/api/crops/user/:user_id`	GET	Cultivos por usuario	Sí
	`/api/crops/:id/select`	PUT	Seleccionar cultivo activo	Sí
	`/api/crops/categories`	GET	Obtener categorías de cultivos	No

Tabla 5.5: Tabla API REST - Lecturas de Sensores

	Endpoint	**Método**	**Descripción**	**Autenticación**
Lecturas de sensores	`/api/sensor-readings`	POST	Crear lectura de sensor	No
	`/api/sensor-readings/:id`	GET	Obtener lectura por ID	No
	`/api/sensor-readings/device/:device_id`	GET	Lecturas por dispositivo	No
	`/api/sensor-readings/device/:device_id/latest`	GET	Última lectura por dispositivo	No
	`/api/sensor-readings/user/:user_id/active-device/latest`	GET	Última lectura del dispositivo activo	No

Tabla 5.6: Tabla API REST - Configuración de riego

	Endpoint	Método	Descripción	Autenticación
Configuración de riego	`/api/irrigation`	POST	Crear configuración de riego	Sí
	`/api/irrigation/:id`	GET	Obtener configuración por ID	Sí
	`/api/irrigation/:id`	DELETE	Eliminar configuración	Sí
	`/api/irrigation/user/:user_id/active`	GET	Configuraciones activas por usuario	Sí
	`/api/irrigation/user/:user_id/all`	GET	Todas las configuraciones por usuario	Sí
	`/api/irrigation/user/:user_id/type/:mode_type`	GET	Configuraciones por usuario y tipo	Sí
	`/api/irrigation/user/:user_id/last-irrigation`	GET	Obtener fecha del último riego	Sí
	`/api/irrigation/:id/activate`	PUT	Activar configuración	Sí
	`/api/irrigation/:id/deactivate`	PUT	Desactivar configuración	Sí
	`/api/irrigation/:id/update-last-irrigation`	PUT	Actualizar fecha del último riego	Sí
	`/api/irrigation/:id/specific-config`	GET	Obtener configuración específica	Sí
	`/api/irrigation/manual/:id`	PUT	Actualizar configuración manual	Sí
	`/api/irrigation/automatic`	POST	Crear configuración automática	Sí
	`/api/irrigation/programmed`	POST	Crear configuración programada	Sí
	`/api/irrigation/programmed/:irrigation_config_id/cancel`	DELETE	Cancelar configuración programada completa	Sí
	`/api/irrigation/programmed/:irrigation_config_id/cancel-irrigation`	DELETE	Cancelar riego programado específico	Sí
	`/api/irrigation/programmed/:id/next-execution`	PUT	Actualizar próxima ejecución	Sí
	`/api/irrigation/programmed/:id/execution`	PUT	Actualizar ejecución programada	Sí
	`/api/irrigation/pump-activation`	POST	Crear activación de bomba	Sí
	`/api/irrigation/pump-activation/config/:irrigation_config_id/active`	GET	Obtener activación activa de bomba	Sí
	`/api/irrigation/pump-activation/config/:irrigation_config_id/latest`	GET	Obtener última activación de bomba	Sí
	`/api/irrigation/pump-activation/:id/status`	PUT	Actualizar estado de activación de bomba	Sí
	`/api/irrigation/pump-activation/:id/pause`	PUT	Pausar activación de bomba	Sí
	`/api/irrigation/pump-activation/:id/resume`	PUT	Reanudar activación de bomba	Sí
	`/api/irrigation/pump-activation/:id/complete`	PUT	Completar activación de bomba	Sí
	`/api/irrigation/pump-activation/user/:user_id/history`	GET	Obtener historial de activaciones por usuario	Sí

Tabla 5.7: Tabla API REST - Activaciones de bomba

	Endpoint	**Método**	**Descripción**	**Autenticación**
Activaciones de bomba	`/api/irrigation/pump-activations`	POST	Crear activación de bomba	Sí
	`/api/irrigation/pump-activations/user/:user_id`	GET	Activaciones por usuario	Sí
	`/api/irrigation/pump-activations/active`	GET	Activaciones activas	Sí
	`/api/irrigation/pump-activations/:id/pause`	PUT	Pausar activación	Sí
	`/api/irrigation/pump-activations/:id/resume`	PUT	Reanudar activación	Sí
	`/api/irrigation/pump-activations/:id/complete`	PUT	Completar activación	Sí

Tabla 5.8: Tabla API REST - Alertas

	Endpoint	**Método**	**Descripción**	**Autenticación**
Alertas	`/api/alerts/my-alerts`	GET	Obtener alertas del usuario autenticado	Sí
	`/api/alerts/:id`	GET	Obtener alerta por ID	Sí
	`/api/alerts/:id`	DELETE	Eliminar alerta individual	Sí
	`/api/alerts/old`	DELETE	Eliminar todas las alertas del usuario	Sí
	`/api/alerts/user/:user_id`	GET	Alertas por usuario	Sí
	`/api/alerts/user/:user_id/unresolved`	GET	Alertas no resueltas por usuario	Sí
	`/api/alerts/user/:user_id/resolve-all`	PUT	Resolver todas las alertas de un usuario	Sí
	`/api/alerts/user/:user_id/old`	DELETE	Eliminar alertas antiguas de un usuario	Sí
	`/api/alerts/type/:alert_type`	GET	Alertas por tipo	Sí
	`/api/alerts/severity/:severity`	GET	Alertas por severidad	Sí
	`/api/alerts/:id/resolve`	PUT	Resolver alerta	Sí
	`/api/alerts/:id/unresolve`	PUT	Marcar alerta como no resuelta	Sí
	`/api/alerts/resolve-all`	PUT	Resolver todas las alertas del usuario	Sí
	`/api/alerts/stats/type`	GET	Estadísticas de alertas por tipo	Sí
	`/api/alerts/stats/severity`	GET	Estadísticas de alertas por severidad	Sí
	`/api/alerts/admin/all`	GET	Obtener todas las alertas del sistema	Sí
	`/api/alerts/admin/all`	DELETE	Eliminar todas las alertas del sistema	Sí
	`/api/alerts/user-registered`	POST	Crear alerta de registro	Sí
	`/api/alerts/device/api-key-copied`	POST	Crear alerta de clave API copiada	Sí

Tabla 5.9: Tabla API REST - Alertas

	Endpoint	**Método**	**Descripción**	**Autenticación**
TTN Integration	`/api/ttn/uplink`	POST	Webhook TTN uplink	No
	`/api/ttn/led/on`	POST	Encender LED via downlink	No
	`/api/ttn/led/off`	POST	Apagar LED via downlink	No

5.7.2. Patrón de diseño - MVC

En cuanto a la arquitectura del proyecto, se ha optado por el patrón de diseño MVC [58] que se encarga de separar la lógica de negocio de la interfaz de usuario y es el más utilizado en aplicaciones web. Facilita la funcionalidad, mantenibilidad y escalabilidad del sistema. Mediante MVC el proyecto se divide en tres niveles de abstracción:

- **Modelo:** Es la lógica de negocio. Es decir las clases y métodos que se comunican directamente con la base de datos
- **Vista:** Es la encargada de mostrar la información al usuario, con de forma gráfica y legible.
- **Controlador:** El intermediario entre la vista y el modelo, se encarga de controlar las interacciones del usuario en la vista, pide los datos al modelo y los devuelve de nuevo a la vista para que esta los muestre al usuario. Es decir las llamadas a clases y métodos, y los datos recibidos de formularios.

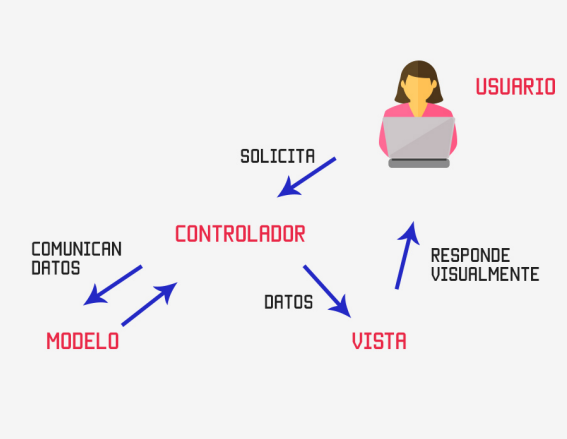


Figura 5.30: Modelo-vista-controlador

Este patrón de diseño nos indica las pautas a seguir para estructurar nuestro servidor web de forma correcta. Como veremos en el siguiente apartado.

5.7.3. Estructura del servidor web

A partir del patrón de diseño arquitectónico modelo-vista-controlador, podemos diseñar la estructura de directorios de nuestro servidor web. Esta estructura debe

ser clara, puesto que una estructura de directorios desorganizada puede afectar gravemente al rendimiento y a la escalabilidad del proyecto. En la siguiente Figura se puede observar la estructura seguida

```
server/
├── config/ ..... Configuraciones del sistema
├── controllers/ ..... Lógica de control y endpoints
├── models/ ..... Modelos de datos y entidades
├── routes/ ..... Definición de rutas REST
├── middleware/ ..... Autenticación y validación
├── services/ ..... Servicios de negocio especializados
├── emails/ ..... Servicios de notificación por email
├── utils/ ..... Utilidades y funciones auxiliares
└── index.js ..... Punto de entrada de la aplicación
```

Figura 5.31: Estructura de directorios del servidor web del sistema Vivantia IoT

En resumen, la estructura propuesta establece una base modular, mantenible y escalable para el servidor web. Esta organización servirá como guía durante el desarrollo: en la fase de implementación se irán incorporando los endpoints y controladores definitivos, los middlewares de seguridad y validación, los servicios de negocio necesarios, así como la integración con API's externas como por ejemplo, API de TTN. En consecuencia, este apartado describe el diseño de la estructura del servidor web. En la fase de implementación, se comenzará a integrar las funcionalidades.

5.8. Diseño del Módulo 3.2: Cloud Computing - Almacenamiento de datos

El módulo 3.2 constituye la base de datos del proyecto, está implementado sobre un sistema de gestión de bases de datos relacional ampliamente conocido y utilizado como es PostgreSQL.

5.8.1. Modelo conceptual entidad-relación

El modelo conceptual entidad-relación de la base datos, representa las entidades principales y sus interrelaciones para gestionar usuarios, dispositivos IoT, cultivos, configuraciones de riego y el monitoreo ambiental. El modelo conceptual entidad-relación representa un nivel más alto de abstracción que el diseño del modelo entidad

relación, nos servirá para tener un mayor entendimiento sobre como estarán estructurados los datos del sistema. En este diagrama no se detallará con precisión cuales serán los atributos de cada entidad, estos se describirán en el modelo entidad-relación con más exactitud. A continuación pasaremos a describir cuales son las entidades principales del sistema.

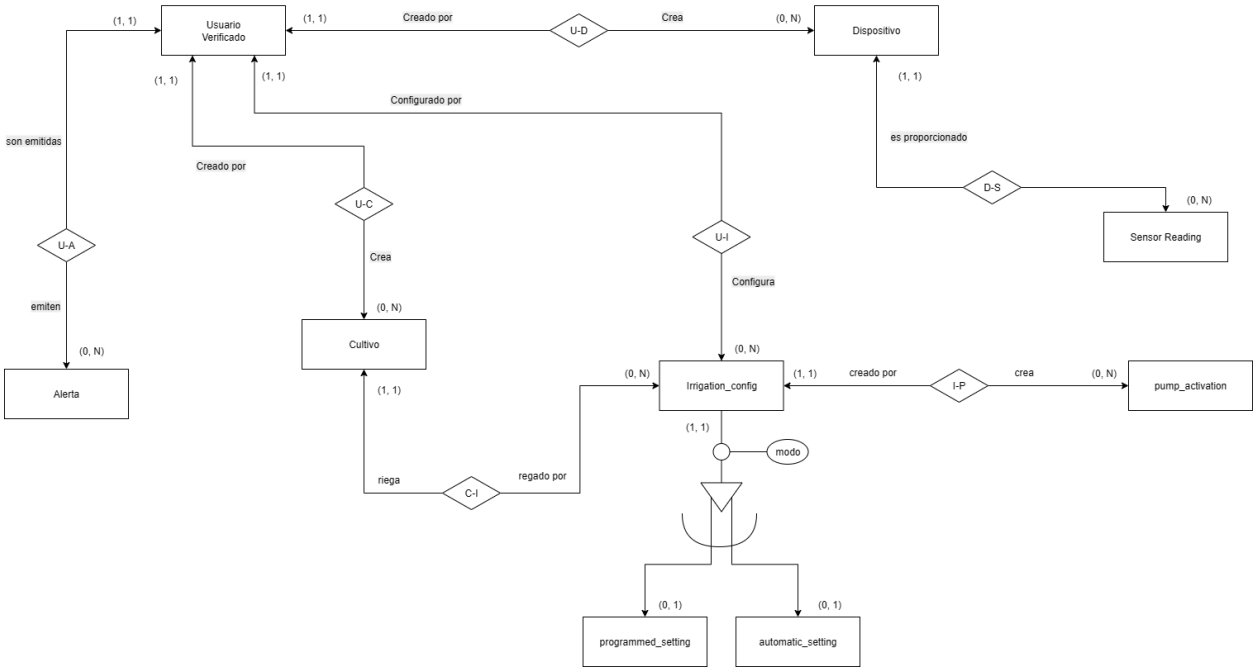


Figura 5.32: Modelo conceptual entidad-relación

5.8.1.1. Entidades principales

5.8.1.1.1. Usuario Verificado

Se trata de la entidad principal que representa a los usuarios registrados y verificados. Sus atributos incluyen información de identificación (email, nombre), credenciales de acceso (contraseña) y estado de verificación. Cada usuario tiene un rol definido que determina sus permisos dentro del sistema (administrador, usuario verificado, o visitante). En este caso no hará falta representar una entidad que sea administrador y otra visitante puesto que únicamente solo puede haber un administrador que gestiona a usuarios, cultivos de los usuarios y dispositivos de los usuarios. Por otro lado visitante no podrá hacer uso de las funcionalidades del sistema por lo

que no puede ni crear, eliminar o modificar datos, únicamente podrá acceder a una demo.

5.8.1.1.2. Dispositivo

Representa los dispositivos IoT físicos que se conectan a la red LoRaWAN. Esta entidad almacena las credenciales específicas de TTN (DevEUI, AppEUI, AppKey), información de configuración de red (región TTN, Application ID), y el estado de comunicación que determina si las lecturas los de sensores deben ser almacenadas en la base datos.

5.8.1.1.3. Cultivo

Entidad que define las características y umbrales de riego de los cultivos que los usuarios desean automatizar el riego. Incluye información descriptiva (nombre, categoría, imagen) y parámetros necesarios para la automatización del riego (umbrales de humedad mínima y máxima relativa y del suelo, temperatura máxima).

5.8.1.1.4. Sensor Reading

Registra las lecturas del sensor ambiental y el sensor de humedad de suelo. Almacena los valores de temperatura, humedad del suelo y humedad ambiental junto con el timestamp de recepción, proporcionando el historial necesario para mostrar las lecturas en tiempo real en la dashboard. Además, en base a la lectura más reciente se tomarán decisiones de riego automatizado.

5.8.1.1.5. Pump Activation

Registra cada activación individual de la bomba de riego, manteniendo un historial completo de todas las operaciones de riego con sus estados, duraciones y resultados.

5.8.1.1.6. Alerta

Sistema de notificaciones que registra eventos significativos del sistema, categorizados por tipo (usuario, ambiental, dispositivo, cultivo, riego) y clasificados por severidad para facilitar la gestión y seguimiento.

5.8.1.1.7. Irrigation Config

La especialización parcial exclusiva implementada en el modelo de base de datos para las configuraciones de riego consiste en que la tabla irrigation_configs actúa como entidad general que contiene los atributos comunes a todos los modos de riego (usuario, cultivo, modo, estado de activación, duración y metadatos temporales), de esta entidad general se especializan dos configuraciones específicas en tablas separadas mediante relaciones uno a uno exclusivas. La especialización es parcial puesto que pueden existir más configuraciones de riego pero en nuestro sistema únicamente utilizaremos tres, y es exclusiva porque cada configuración pertenece exactamente a un modo sin posibilidad de solapamiento. El campo mode_type actúa como discriminador que determina unívocamente la especialización correspondiente, garantizando que las configuraciones programadas y automáticas tengan sus atributos específicos sin interferir entre sí.

- **Configuración Programada** (`programmed`): Esta especialización extiende la configuración base añadiendo atributos relacionados con la programación temporal, permitiendo ajustar la fecha, hora, frecuencia y duración del riego. Posee atributos también para indicar próxima de fecha de riego en caso de que la frecuencia sea para varios días o diariamente.
- **Configuración Automática** (`automatic`): Esta especialización extiende la configuración base añadiendo atributos relacionados con el modo automático, como pueden ser umbrales máximo de temperatura, umbrales máximo y mínimos de humedad ambiental y del suelo. En base a estos atributos se construyen una serie de condiciones de riego que si se cumplen se ejecuta el riego.

5.8.1.2. Relaciones y cardinalidades

5.8.1.2.1. Relaciones de propiedad (1:N)

- **Usuario - Dispositivo:** Un usuario puede crear múltiples dispositivos IoT, pero cada dispositivo pertenece exclusivamente a un usuario.
- **Usuario-Cultivo:** Aunque técnicamente un usuario puede crear múltiples cultivos, existe una restricción de negocio que permite solo un cultivo activo por usuario simultáneamente.
- **Usuario - Alerta** Un usuario puede generar múltiples alertas del sistema, manteniendo un historial personalizado de eventos.

5.8.1.2.2. Relaciones de configuración (1:N)

- **Usuario - Irrigation Config:** Un usuario puede tener múltiples configuraciones de riego (una por cada modo y cultivo), pero cada configuración pertenece a un único usuario.
- **Cultivo - Irrigation Config:** Un cultivo puede tener múltiples configuraciones asociadas (diferentes modos de riego), estableciendo los parámetros base para cada configuración.

5.8.1.2.3. Relaciones de datos operacionales (1:N)

- **Dispositivo - Sensor Reading:** Un dispositivo puede generar múltiples lecturas de sensores a lo largo del tiempo.
- **Irrigation Config - Pump Activation:** Una configuración puede generar múltiples activaciones de bomba, manteniendo el registro histórico de todas las operaciones de riego.

5.8.1.2.4. Relaciones de especialización (1:1)

- **Irrigation Config con programmed/Automatic Settings:** Especialización parcial exclusiva que extienden las configuraciones base con parámetros específicos según el modo seleccionado.

5.8.1.3. Restricciones conceptuales

5.8.1.3.1. Restricciones sobre cultivos y configuraciones de riego

El sistema implementa restricciones para garantizar que solo un cultivo esté activo por usuario y solo una configuración de riego esté activa simultáneamente, evitando conflictos operacionales.

5.8.2. Modelo entidad-relación

El modelo entidad-relación diseñado traduce el diseño conceptual en una estructura de base de datos PostgreSQL para el sistema. El modelo incluye 8 tablas principales relacionadas a través de claves foráneas que garantizan la integridad referencial. Cómo se puede ver en el diagrama las claves foráneas se presentan como **FK** y las claves primarias **PK**.

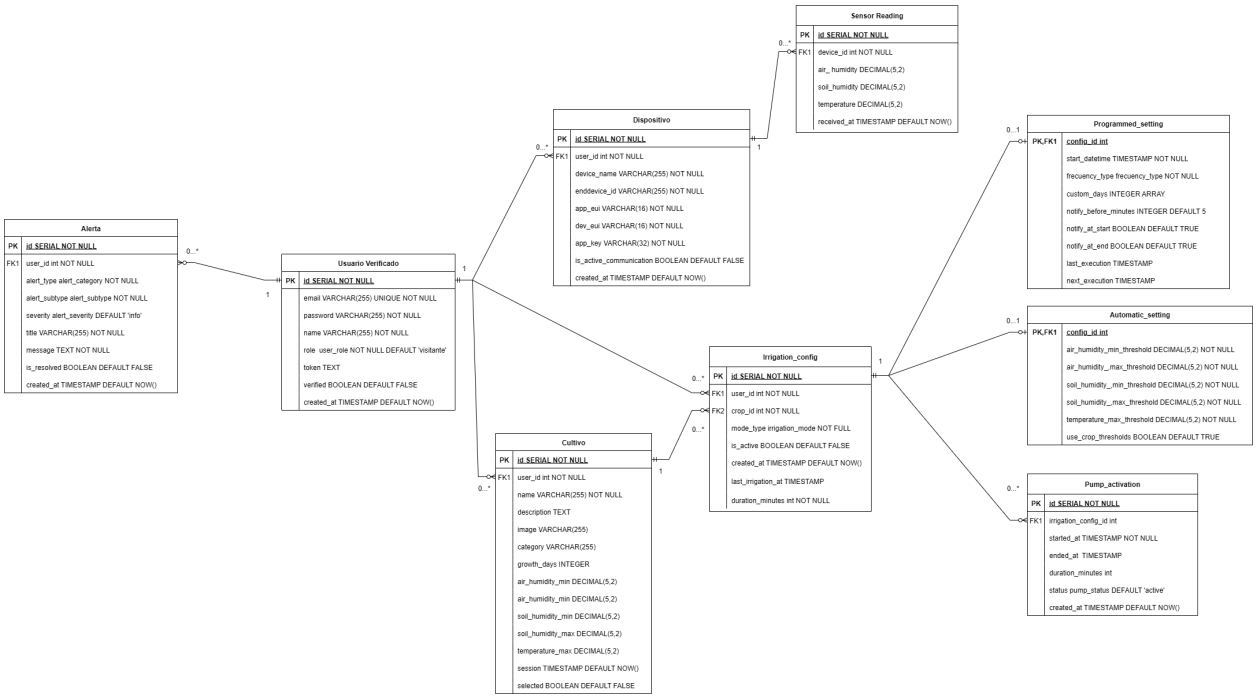


Figura 5.33: Modelo entidad-relación

5.8.2.1. Usuario verificado

Tabla central del sistema que gestiona la autenticación y autorización

Tabla 5.10: Tabla Usuario Verificado (`users`)

Atributo	**Tipo**	**Restricciones**	**Descripción**
`id`	SERIAL	PRIMARY KEY	Identificador único autoincrementable
`email`	VARCHAR(255)	UNIQUE, NOT NULL	Dirección de correo electrónico única
`password`	VARCHAR(255)	NOT NULL	Contraseña hasheada con bcrypt
`name`	VARCHAR(255)	NOT NULL	Nombre completo del usuario
`role`	user_role	NOT NULL, DEFAULT 'visitante'	Rol del usuario en el sistema
`token`	TEXT	NULLABLE	Token de verificación temporal
`verified`	BOOLEAN	DEFAULT FALSE	Estado de verificación de cuenta
`created_at`	TIMESTAMP	DEFAULT NOW()	Fecha y hora de creación

5.8.2.2. Crops

Almacena la información de los cultivos registrados por los usuarios

Tabla 5.11: Tabla Crops (`crops`)

Atributo	Tipo	Restricciones	Descripción
`id`	SERIAL	PRIMARY KEY	Identificador único del cultivo
`user_id`	INTEGER	FK → users(id), CASCADE	Usuario propietario del cultivo
`name`	VARCHAR(255)	NOT NULL	Nombre del cultivo
`description`	TEXT	NULLABLE	Descripción detallada del cultivo
`image`	VARCHAR(255)	NULLABLE	Ruta de la imagen del cultivo
`category`	VARCHAR(255)	NULLABLE	Categoría del cultivo
`growth_days`	INTEGER	NULLABLE	Días de crecimiento del cultivo
`air_humidity_min`	DECIMAL(5,2)	NULLABLE	Humedad del aire mínima requerida (%)
`air_humidity_max`	DECIMAL(5,2)	NULLABLE	Humedad del aire máxima requerida (%)
`soil_humidity_min`	DECIMAL(5,2)	NULLABLE	Humedad del suelo mínima requerida (%)
`soil_humidity_max`	DECIMAL(5,2)	NULLABLE	Humedad del suelo máxima requerida (%)
`temperature_max`	DECIMAL(5,2)	NULLABLE	Temperatura máxima tolerada (°C)
`session`	VARCHAR(255)	NULLABLE	Temporada de cosecha
`selected`	BOOLEAN	DEFAULT FALSE	Indica si es el cultivo activo del usuario
`created_at`	TIMESTAMP	DEFAULT NOW()	Momento de creación del registro

5.8.2.3. Dispositivo

Representa los dispositivos IoT y sus credenciales TTN

Tabla 5.12: Tabla Dispositivo (`devices`)

Atributo	Tipo	Restricciones	Descripción
`id`	SERIAL	PRIMARY KEY	Identificador único del dispositivo
`user_id`	INTEGER	FK → users(id), CASCADE	Propietario del dispositivo
`device_name`	VARCHAR(255)	NOT NULL	Nombre descriptivo del dispositivo
`enddevice_id`	VARCHAR(255)	NOT NULL	ID del dispositivo final en TTN
`app_eui`	VARCHAR(16)	NOT NULL	Application EUI de 8 bytes
`dev_eui`	VARCHAR(16)	NOT NULL	Device EUI único de 8 bytes
`app_key`	VARCHAR(32)	NOT NULL	Application Key de 16 bytes
`ttn_region`	VARCHAR(10)	NULLABLE	Región TTN (ej: .eu1", ús1")
`ttn_app_id`	VARCHAR(100)	NULLABLE	ID de aplicación en TTN
`ttn_access_key`	VARCHAR(255)	NULLABLE	Clave de acceso API TTN
`is_active_communication`	BOOLEAN	DEFAULT FALSE	Estado de comunicación activa
`created_at`	TIMESTAMP	DEFAULT NOW()	Fecha de registro

5.8.2.4. Sensor Reading

Almacena las lecturas ambientales y del suelo capturadas por los sensores

Tabla 5.13: Tabla Sensor Reading (`sensor_readings`)

Atributo	Tipo	Restricciones	Descripción
`id`	SERIAL	PRIMARY KEY	Identificador único de lectura
`device_id`	INTEGER	FK → devices(id), CASCADE	Dispositivo origen de la lectura
`air_humidity`	DECIMAL(5,2)	NULLABLE	Humedad del aire ambiente (0.00-100.00 %)
`soil_humidity`	DECIMAL(5,2)	NULLABLE	Humedad del suelo (0.00-100.00 %)
`temperature`	DECIMAL(5,2)	NULLABLE	Temperatura ambiente (-50.00-100.00°C)
`received_at`	TIMESTAMP	DEFAULT NOW()	Momento de recepción del dato

5.8.2.5. Irrigation Config

Configuraciones base para todos los modos de riego y modo manual definidio implícitamente, puesto que, solamente requiere de un atributo que es duración en minutos y ya se incluye en la configuración base.

Tabla 5.14: Tabla Irrigation Config (`irrigation_configs`)

Atributo	**Tipo**	**Restricciones**	**Descripción**
`id`	SERIAL	PRIMARY KEY	Identificador único de configuración
`user_id`	INTEGER	FK → users(id), CASCADE	Usuario propietario
`crop_id`	INTEGER	FK → crops(id), CASCADE	Cultivo asociado
`mode_type`	irrigation_mode	NOT NULL	Modo de riego seleccionado
`is_active`	BOOLEAN	DEFAULT FALSE	Estado de activación
`duration_minutes`	INTEGER	NOT NULL	Duración base del riego
`last_irrigation_at`	TIMESTAMP	NULLABLE	Último riego ejecutado
`created_at`	TIMESTAMP	DEFAULT NOW()	Fecha de creación
Restricción única: UNIQUE(user_id, crop_id, mode_type)			

Esta tabla tiene una restricción que es UNIQUE(user_id, crop_id, mode_type). Una configuración por usuario, cultivo y modo

5.8.2.6. Programmed Setting

Configuración específica para el modo programado

Tabla 5.15: Tabla Programmed Setting (`programmed_settings`)

Atributo	**Tipo**	**Restricciones**	**Descripción**
`config_id`	INTEGER	PK, FK → irrigation_configs(id), CASCADE	Configuración padre
`start_datetime`	TIMESTAMP	NOT NULL	Fecha y hora de inicio
`frequency_type`	frequency_type	NOT NULL	Tipo de frecuencia
`custom_days`	INTEGER[]	NULLABLE	Días personalizados (array)
`notify_before_minutes`	INTEGER	DEFAULT 5	Minutos de aviso previo
`notify_at_start`	BOOLEAN	DEFAULT TRUE	Notificar al iniciar
`notify_at_end`	BOOLEAN	DEFAULT TRUE	Notificar al finalizar
`last_execution`	TIMESTAMP	NULLABLE	Última ejecución registrada
`next_execution`	TIMESTAMP	NULLABLE	Próxima ejecución calculada

5.8.2.7. Automatic Settings

Configuración específica para el modo automático

Tabla 5.16: Tabla Automatic Settings (`automatic_settings`)

Atributo	Tipo	Restricciones	Descripción
`config_id`	INTEGER	PK, FK → irrigation_configs(id), CASCADE	Configuración padre
`air_humidity_min_threshold`	DECIMAL(5,2)	NOT NULL	Umbral mínimo de humedad del aire
`air_humidity_max_threshold`	DECIMAL(5,2)	NOT NULL	Umbral máximo de humedad del aire
`soil_humidity_min_threshold`	DECIMAL(5,2)	NOT NULL	Umbral mínimo de humedad del suelo
`soil_humidity_max_threshold`	DECIMAL(5,2)	NOT NULL	Umbral máximo de humedad del suelo
`temperature_max_threshold`	DECIMAL(5,2)	NOT NULL	Umbral máximo de temperatura
`use_crop_thresholds`	BOOLEAN	DEFAULT TRUE	Usar umbrales del cultivo como base

5.8.2.8. Pump Activation

Registro histórico de todas las activaciones de riego.

Tabla 5.17: Tabla Pump Activation (`pump_activations`)

Atributo	Tipo	Restricciones	Descripción
`id`	SERIAL	PRIMARY KEY	Identificador único de activación
`irrigation_config_id`	INTEGER	FK → irrigation_configs(id)	Configuración que originó el riego
`started_at`	TIMESTAMP	NOT NULL	Momento de inicio real
`ended_at`	TIMESTAMP	NULLABLE	Momento de finalización
`duration_minutes`	INTEGER	NULLABLE	Duración efectiva realizada
`status`	pump_status	DEFAULT 'active'	Estado actual de la activación
`created_at`	TIMESTAMP	DEFAULT NOW()	Fecha de creación del registro

5.8.2.9. Alerta

Sistema de notificaciones y eventos del sistema.

Tabla 5.18: Tabla Alerta (`alerts`)

Atributo	Tipo	Restricciones	Descripción
`id`	SERIAL	PRIMARY KEY	Identificador único de alerta
`user_id`	INTEGER	FK → users(id), CASCADE	Usuario destinatario
`alert_type`	alert_category	NOT NULL	Categoría principal de la alerta
`alert_subtype`	alert_subtype	NOT NULL	Subtipo específico de evento
`severity`	alert_severity	DEFAULT 'info'	Nivel de importancia
`title`	VARCHAR(255)	NOT NULL	Título descriptivo
`message`	TEXT	NOT NULL	Mensaje detallado
`is_resolved`	BOOLEAN	DEFAULT FALSE	Estado de resolución
`created_at`	TIMESTAMP	DEFAULT NOW()	Momento de generación

5.8.2.10. Tipos enumerados (ENUM)

En este apartado se detallaran los tipos enumerados:

Tabla 5.19: Tipos Enumerados del Sistema

Tipo Enumerado	Valores
`user_role`	`'admin'`, `'usuario'`, `'visitante'`
`irrigation_mode`	`'manual'`, `'automatic'`, `'programmed'`
`frequency_type`	`'once'`, `'daily'`, `'custom'`
`pump_status`	`'active'`, `'completed'`, `'cancelled'`, `'paused'`, `'error'`, `'inactive'`, `'programmed'`
`alert_severity`	`'info'`, `'success'`, `'warning'`, `'error'`
`alert_category`	`'user'`, `'environmental'`, `'device'`, `'crop'`, `'irrigation'`

Tabla 5.20: Subtipos de Alertas (`alert_subtype`)

Categoría	Subtipos
Usuario	`user_registered`, `user_logged_in`, `username_changed`, `password_changed`, `session_closed`
Ambiental	`temperature_max_threshold`, `air_humidity_min_threshold`, `air_humidity_max_threshold`, `soil_humidity_min_threshold`, `soil_humidity_max_threshold`, `system_online`, `system_offline`
Dispositivo	`device_added`, `api_key_copied`, `device_offline`, `device_online`, `device_deleted`, `device_edited`
Cultivo	`crop_selected`, `crop_deselected`, `crop_edited`, `crop_deleted`, `crop_added`
Riego	`manual_started`, `emergency_stop`, `manual_cancelled`, `programmed_saved`, `programmed_reminder`, `programmed_schedule`, `programmed_cancelled`, `automatic_saved`

5.9. Diseño del módulo 4: Interfaz web - Visualización

El diseño de la interfaz web constituye el componente visual del sistema Vivantia, es donde se origina la interacción entre los usuarios y el sistema de automatización de riego. Esta interfaz ha sido conceptualizada siguiendo principios de diseño centrado en el usuario, implementando una arquitectura de SPA híbrida que proporciona una experiencia fluida y responsiva, siendo ideal para aplicaciones IoT.

En los siguientes apartados se diseñará el diagrama de navegación, este será una representación visual que muestra cómo están distribuidas las páginas o elementos de la aplicación web sirve para planificar la estructura y el flujo de navegación. Por otro lado, se crearán una serie bocetos que ilustrarán la disposición estratégica de elementos en cada pantalla, priorizando la usabilidad y la visualización de las lecturas de los sensores en tiempo real, configuraciones de riego, y gestión de dispositivos IoT y cultivos... Todo ello adaptado tanto para usuarios verificados como para administradores del sistema.

5.9.1. Diagrama de navegación

El primero paso que daremos para el diseño de la interfaz web es definir como los usuarios del sistema van a navegar por la aplicación web y de cuáles páginas va a constar. En la Figura 5.34 vemos el diagrama de navegación cuyo objetivo es representar gráficamente el diseño de la distribución de las páginas en la interfaz web.

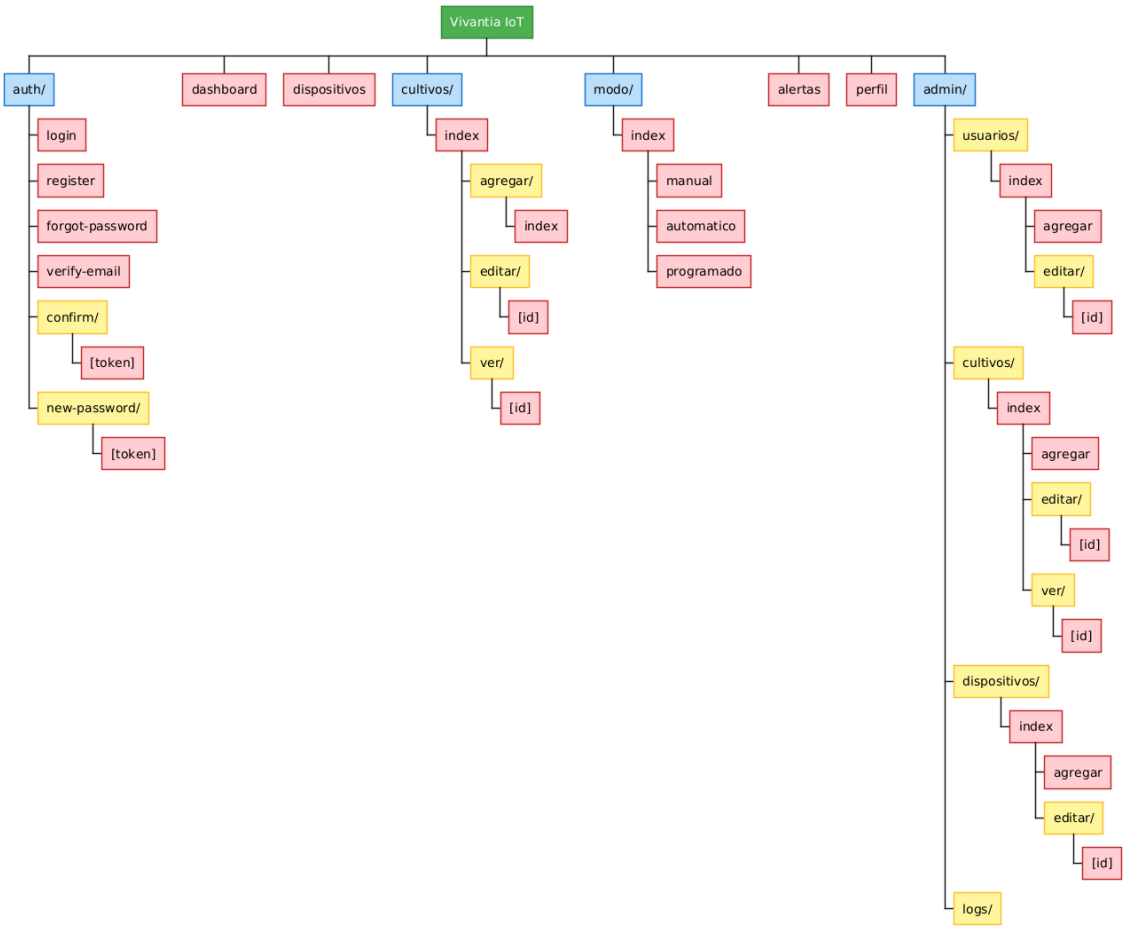


Figura 5.34: Diagrama de navegación

5.9.2. Bocetos de las páginas

En este apartado se presentan los bocetos de las páginas principales que conforman la interfaz web del sistema. El diseño se ha conceptualizado siguiendo principios de diseño, priorizando la usabilidad de la aplicación web.

Los bocetos se han estructurado considerando la arquitectura de SPA implementada, donde la interfaz mantiene elementos de navegación estáticos que proporcionan al usuario la sensación de no estar cambiando y recargando páginas continuamente. Sino que todo está centralizado.

La aplicación implementa un layout modular compuesto por elementos fijos que permanecen visibles durante toda la navegación:

- **Header:** Contiene información adicional del sistema, información del usuario autenticado y un botón para que el usuario pueda ver su perfil y realizar acciones como modificar su información personal, cerrar sesión o borrar cuenta.
- **Sidebar:** Proporciona navegación principal entre las diferentes secciones de la aplicación.
- **Footer:** Incluye información del desarrollador del sistema.
- **Main Panel:** Área dinámica donde se renderiza el contenido específico de cada página.

A continuación en la Figura 5.35 se puede ver el layout principal del sistema:

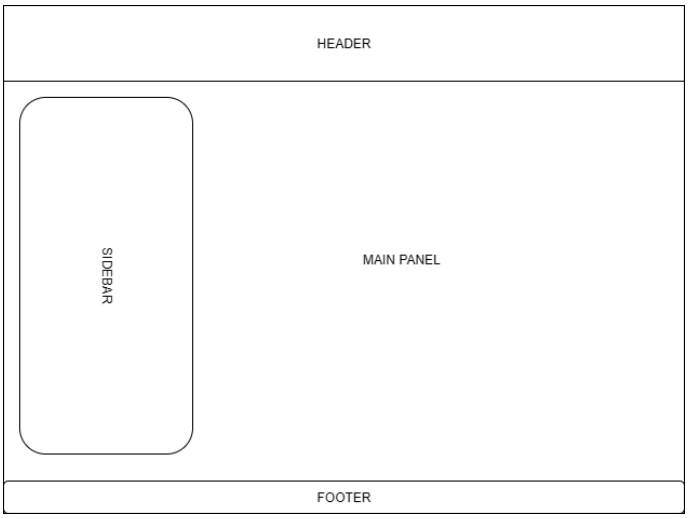


Figura 5.35: Layout principal del sistema

En la sidebar se mostrarán las diferentes secciones de nuestra aplicación, a continuación mostraremos el boceto de todas y cada una de las paginas de la aplicación que se renderizarán en el main panel, por lo que asumiremos que en cada una de estas páginas se mostrará el layout principal anteriormente diseñado.

En la Figura 5.36 vemos el diseño de lo que será la página principal de nuestra aplicación, esta es la dashboard la cual constará de 3 gráficas una para la temperatura ambiental, otra para la humedad ambiental y por último, otra para la humedad del suelo. En estas gráficas se trazarán los datos obtenidos por los sensores. Por otro en la parte inferior con respecto a las gráficas, se distribuirán una serie de elementos visuales que ayudarán a informar al usuario sobre el estado del sistema.

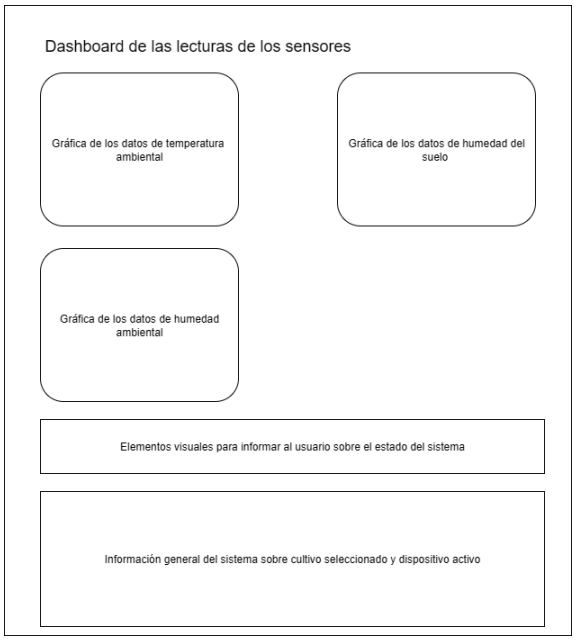


Figura 5.36: Boceto de la dashboard

Para la gestión de cultivos por parte del usuario, se ha diseñado la página que se muestra en la Figura 5.37, que permitirá al usuario añadir cultivos para regar y una lista de cultivos registrados cuyas columnas son los parámetros del cultivo. Cada fila representará cada cultivo registrado por el usuario activo en la aplicación web. Cabe destacar que habrá una fila adicional que sean acciones, que permitirán al usuario ver la información completa del cultivo registrado, editar el cultivo y eliminarlo si ya no es necesario.

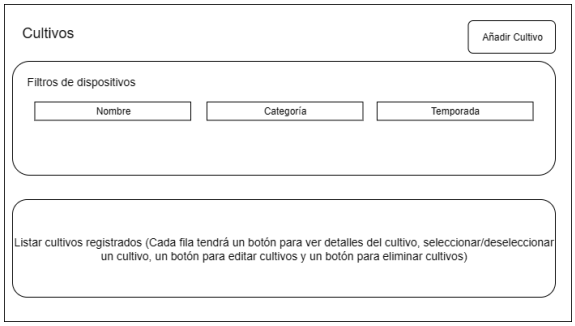


Figura 5.37: Boceto de la página de los cultivos

Para la gestión de dispositivos por parte del usuario, se ha diseñado la página que se muestra en la Figura 5.38, que permitirá al usuario añadir los dispositivos que enviarán los datos de los sensores al servidor web para mostrarlos en la interfaz web y además los dispositivos que recibirán el downlink con los datos de control de riego. Además la página constará de una lista de dispositivos registrados cuyas columnas son los parámetros de configuración dispositivo IoT. Cada fila representará cada dispositivo registrado por el usuario activo en la aplicación web. Cabe destacar que habrá una fila adicional que sean acciones, que permitirán al usuario editar parámetros de configuración del dispositivo y eliminarlo si ya no es necesario. Además de un switch que permitirá activar/desactivar la comunicación con el dispositivo.

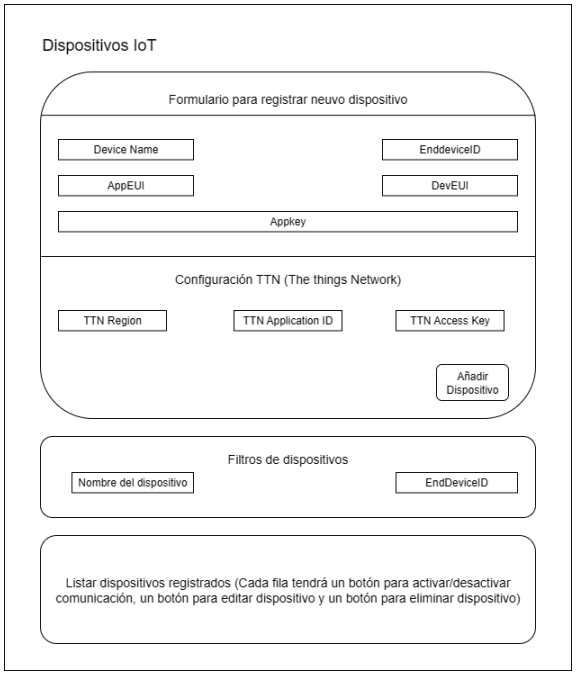


Figura 5.38: Boceto de la página de dispositivos

El diseño de la página, que se muestra en la Figura 5.39 para gestionar los modos de riego es fundamental, puesto que este va a ser la funcionalidad principal de nuestra aplicación. Esta página constará de un serie de paneles informativos en la parte superior, que informarán sobre el modo de riego seleccionado, el dispositivo activo por el usuario, el cultivo actualmente seleccionado, próximo riego programado, estado de la bomba de riego, tiempo restante de riego etc...

En la parte inferior estarán distribuidos los botones de los modos de riego, a partir de los cuales se podrá acceder a cada página de configuración del respectivo modo.

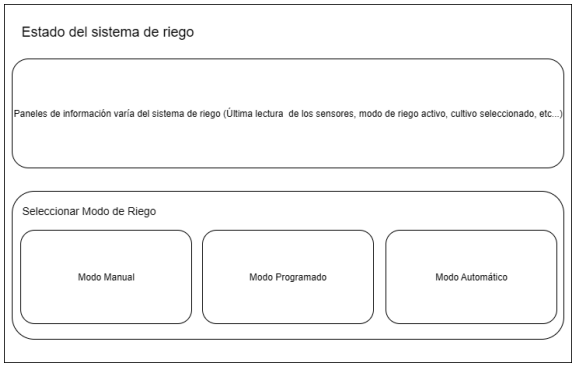
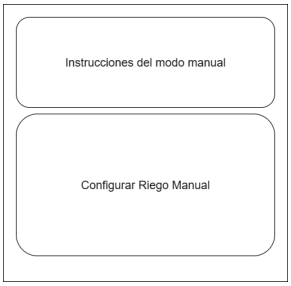
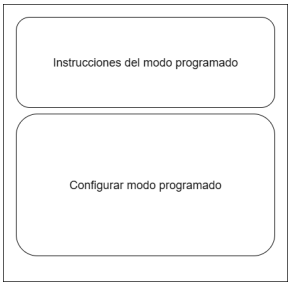


Figura 5.39: Boceto de la página de modos de riego

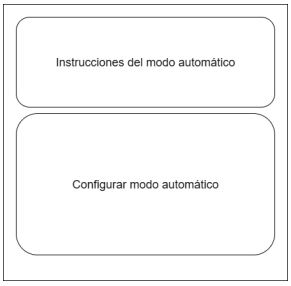
En la Figura 5.40 se puede observar los diseños de las páginas para cada uno de los modos de riego.



(a) Boceto del modo manual



(b) Boceto del modo programado



(c) Boceto del modo automático

Figura 5.40: Bocetos de los modos de riego

Por último tenemos el diseño de la página de Alertas, como vemos en la Figura 5.41. La página de alertas constará de una lista de todos los eventos lanzados por el usuario de la aplicación, también estarán disponibles una serie de filtros para las alertas. El diseño de la página de alertas también coincidirá con el diseño de la página Logs del modo administrador.

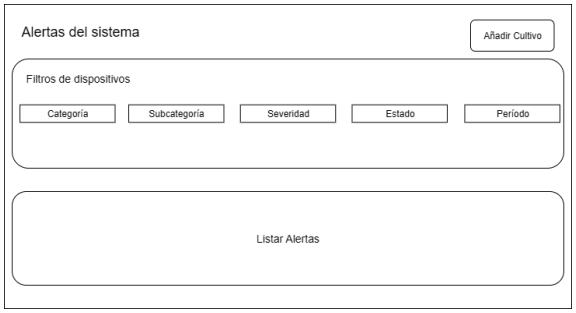
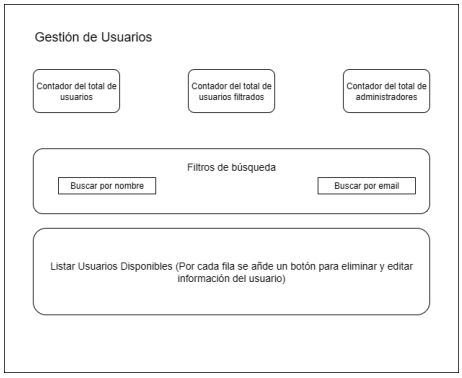
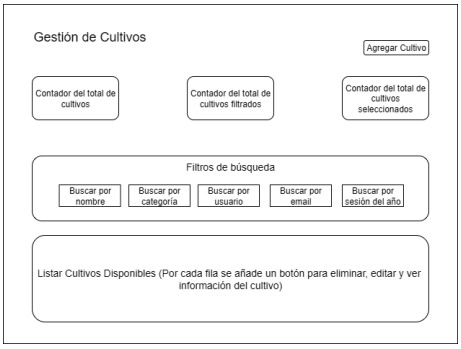


Figura 5.41: Boceto de la página de Alertas del usuario y Logs del administrador

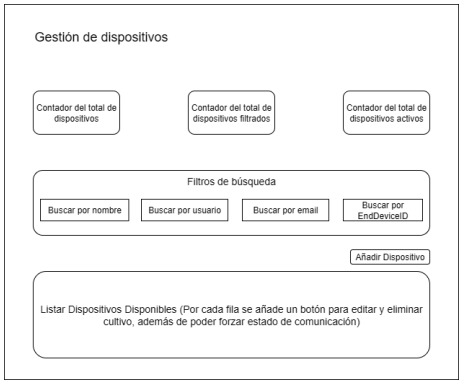
A continuación vamos a diseñar las páginas a las que se podrán acceder en modo administrador. En primer lugar en la Figura 5.42a, se muestra el diseño de la página de gestión de usuarios, cuyo propósito será proporcionar al administrador una interfaz para acceder a todas las funcionalidades relacionadas con el control y gestión de los usuarios de la aplicación. La misma lógica aplicará para las páginas de gestión de cultivos y dispositivos, de la Figura 5.42b y Figura 5.42c.



(a) Boceto de la página de gestión de usuarios



(b) Boceto de la página de gestión de cultivos



(c) Boceto de la página de gestión de dispositivos

Figura 5.42: bocetos de las páginas de gestión del administrador

Por último en la figura 5.43 se muestra el diseño de la página perfil de usuario. Esta página mostrará información del usuario verificado y permitirá acciones como cerrar sesión, cambiar nombre de usuario, cambiar contraseña y eliminar cuenta.

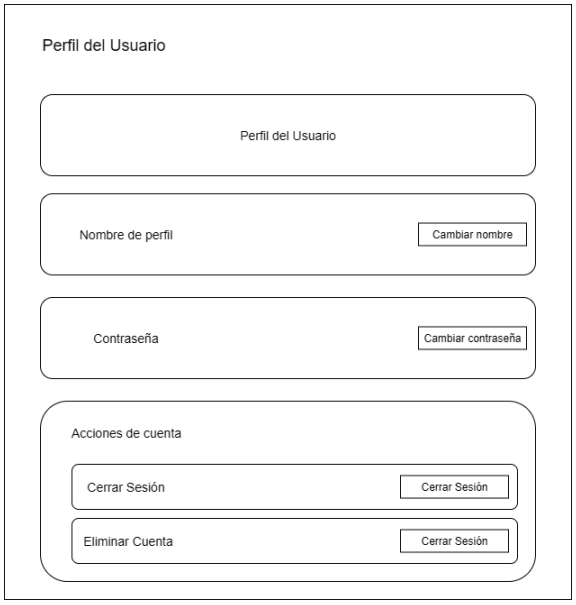


Figura 5.43: Boceto de la página de perfil de usuario

5.9.3. Paleta de colores

La interfaz web implementará una paleta de colores oscura en el tono de sus colores y elegante, con el propósito de reducir el daño ocular que puede producir la luz blanca en exceso y la luz azul. De esta manera se logrará una experiencia de usuario amigable durante períodos prolongados de uso.

5.9.3.1. Colores Base

- **Fondo principal:** Grises oscuros (#1a1a1a, #2d2d2d) que reducen la emisión de luz azul y minimizan el deslumbramiento
- **Sidebar:** Gradiente azul oscuro que va desde #4A5DB8 hasta #2A3B7A, creando profundidad visual y diferenciación clara del área de contenido principal.
- **Elementos de Interfaz:** Grises medios (#3a3a3a) para contraste suave y legibilidad adecuada
- **Texto:** Blancos suaves y grises claros en contraste con los tonos oscuros

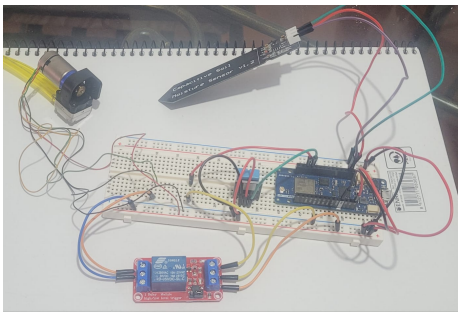
Capítulo 6

Desarrollo e implementación

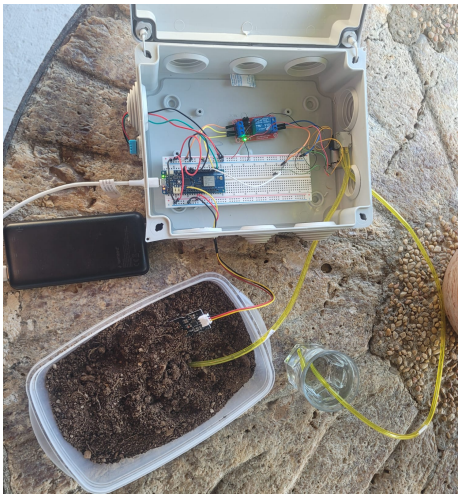
Tras la especificación de requisitos, análisis y diseño. En este capitulo nos adentraremos en la fase de desarrollo e implementación, que culminará finalmente en el sistema completo implementado y desarrollado. Adicionalmente, es importante comentar que el desarrollo del sistema se ha realizado en el orden en el que están enumerados los módulos, siendo esta la forma más óptima y estructurada de llevarlo todo a cabo.

6.1. Desarrollo del Módulo 1: Edge computing - Captación de datos y acción sobre el actuador

Para el desarrollo e implementación de este módulo, me he basado en el esquema eléctrico que se diseñó en la sección 5.5.



(a) Circuito eléctrico implementado



(b) Sistema eléctrico montado completo

Figura 6.1: Módulo 1 implementado y listo

En la Figura 6.1a observamos la implementación del esquema eléctrico. Por otro lado, en la Figura 6.1b observamos el sistema hardware montado completamente en la caja estanca de protección. En la caja estanca para fijar los componentes como el sensor DHT11 se utilizará fijadores de velcro en la parte exterior de la caja, puesto que, el sensor DHT11 dará valores más certeros al aire libre. Además, utilizar fijadores de velcro permite cambiar los sensores de forma sencilla en caso de que alguno falle.

La bomba de riego irá fijada a la caja estanca mediante tornillos y las tuberías para la extracción y expulsión de agua saldrán hacia afuera de la caja, y estarán correctamente etiquetadas para que el usuario sepa donde colocarlas. La tubería con mas longitud servirá para la extracción de agua mientras que la tubería de menor longitud servirá para la expulsión del agua. Por último, el sensor de humedad del suelo saldrá por un orificio de la caja estanca con el fin de poder clavarlo en cualquier suelo de una forma mas sencilla.

Un paso importante para acoplar este módulo al sistema completo es dar de alta al microcontrolador y el gateway en *The things Network* para que podamos transmitir y capturar las mediciones de humedad relativa y del suelo y de temperatura. Este proceso se explica detalladamente en el manual de usuario.

En cuanto al desarrollo del código fuente para programar el microcontrolador, es necesario destacar cómo se ha implementado en el código el envío de uplinks con las lecturas de los sensores y la recepción de downlinks con los datos de control

de riego. Destacando también la necesidad en este proyecto IoT de implementar un uplink periódico con el objetivo de que el microcontrolador responda lo más rápidamente posible cuando emita un downlink desde la interfaz web. No entraremos en profundidad, puesto que para ello ya está el manual de código, dónde se expondrá todo el código fuente.

Para ello será necesario hacer uso de la librería MKRWAN.H , el uso de esta librería lo veremos implementado en el código a continuación.

6.1.1. OTAA

OTAA [34] es el método recomendado de activación del dispositivo IoT en TTN. El dispositivo no trae claves de sesión fijas sino que, en su lugar, arranca con tres claves DevEUI, JoinEUI/AppEUI y AppKey. Tanto JoinEUI/AppEUI como AppKey estarán disponibles en el fichero credenciales.h del repositorio del código del microcontrolador [59]

Funciona de la siguiente manera

1. **Pasos previos:** En la consola TTN registras el dispositivo con su DevEUI, JoinEUI/AppEUI y Appkey. Estos valores se encuentran en el fichero credenciales.h en la carpeta del código fuente del microcontrolador (appEUI, appKey).

2. **Paso 1 (join-request):** Utilizando estas claves, se envía un mensaje de unión llamado Join-request con DevEUI, JoinEUI y un Devnonce, autenticado con el appKey. El DevEUI es el identificador del dispositivo, el JoinEUI/AppEUI identifica la red a la que quiere acceder el dispositivo y el Devonce es un número único de unión. El gateway se encargará de reenviar este join-request hacia TTN.

3. **Paso 2 (join-accept):** TTN recibe el join-request y comprueba que la Appkey es correcta y responde cifrado con la siguiente información

   - **DevAddr:** Esta es la "dirección" del dispositivo dentro de la red
   - **Parámetros de recepción:** Estos parámetros indican cúando y cómo abrir las ventanas de escucha o recepción.

4. **Paso 3 (claves de sesión):** A partir de la Appkey y los Devnonces, el dispositivo IoT y la red TTN calculan unas claves temporales para cifrar y verificar los mensajes. Desde este punto, el nodo puede enviar uplinks cifrados correctamente.

La activación del dispositivo mediante OTAA, se puede ver en el código implementado

```
// Región (EU868, US915, etc.)
_lora_band region = EU868;

// Módem lora del MKR WAN (usa Serial1)
\gls{lora}Modem modem(Serial1);

void setup() {
  Serial.begin(115200);
  while (!Serial) {}

  // 1) Arranca el módem en la banda/región elegida
  if (!modem.begin(region)) {
    Serial.println("Error: No se pudo iniciar el módulo lora");
    while (1) {}
  }

  // (Opcional) ver el DevEUI real del dispositivo para registrarlo
    en ttn
  Serial.print("Device EUI: ");
  Serial.println(modem.deviceEUI());

  // 2) JOIN otaa: usa JoinEUI/AppEUI + AppKey (de tu credenciales.
   h)
  int connected = modem.join\gls{otaa}(appEui, appKey);
  if (!connected) {
    Serial.println("Error en el join \gls{otaa}; revisa
   credenciales, región y cobertura");
    while (1) {}
  }
  Serial.println("  Conectado   a la red!");
}
```

Listing 6.1: Activación del dispositivo mediante OTAA

Como podemos ver en el código, lo primero que se hace es seleccionar una región concreta, es nuestro caso, Europa. En segundo lugar, tendremos que inicializar el módem de LoRa, para posteriormente con las credenciales de activación, ejecutar el método JoinOTAA para activar el dispositivo en la red de TTN.

6.1.2. Uplinks

Para enviar uplinks al servidor de red de TTN a través del gateway, se deben de implementar los siguientes métodos:

```
// ... join otaa hecho antes
```

```
int16_t temp_int = (int16_t)(temperature * 10);
int16_t hum_int = (int16_t)(humidity * 10);
int16_t soil_int = (int16_t)(soilMoisturePercent * 10);
uint8_t payload[6];
payload[0] = (uint8_t)(temp_int >> 8);
payload[1] = (uint8_t)(temp_int & 0xFF);
payload[2] = (uint8_t)(hum_int >> 8);
payload[3] = (uint8_t)(hum_int & 0xFF);
payload[4] = (uint8_t)(soil_int >> 8);
payload[5] = (uint8_t)(soil_int & 0xFF);

// Envío a TTN
modem.beginPacket();
modem.write(payload, 6);
int err = modem.endPacket(false);
```

Listing 6.2: Envío de datos sensores (uplink)

Este fragmento de código se ejecuta después de que el dispositivo ha sido activado en la red LoRaWAN mediante OTAA [34]. Su propósito es preparar y enviar una lectura de datos de los sensores (uplink) desde el dispositivo IoT hacia la red TTN.

Primero, las lecturas de temperatura, humedad ambiental y humedad del suelo se multiplican por 10 y se convierten a enteros de 16 bits (int16_t). Esto se hace para preservar un decimal de precisión sin enviar datos en formato flotante, que ocuparían más espacio y no son óptimos para LoRa.

Luego, se empaquetan los tres valores en un array de 6 bytes, dividiendo cada entero en sus dos bytes correspondientes (alto y bajo) mediante operaciones de desplazamiento (>>8) y enmascarado. Este formato binario compacto permite ahorrar ancho de banda al transmitir los datos. Al tratarse de una comunicación LoRa, mantener este formato de datos no solo permite reducir el ancho de banda sino que también reduce el consumo.

Finalmente, se utiliza el objeto modem para construir y enviar el paquete:

- **beginPacket()** inicia el proceso de envío.
- **write(p,6)** carga los 6 bytes del payload.
- **endPacket(false)** envía el paquete a la red. El parámetro false indica que se trata de un uplink no confirmado, es decir, el nodo no espera una respuesta de confirmación por parte de la red, lo cual reduce el consumo de energía y el tiempo de transmisión.

6.1.3. Downlink

Para recibir downlinks desde el servidor de red de TTN a través del gateway con los datos de control de riego, se deben de implementar los siguientes métodos

```
// ... Después de desencolar downlink, tras enviar uplink periódico
if (modem.available()) {
    uint8_t b = modem.read();
    Serial.print("Downlink recibido: 0x");
    Serial.println(b, HEX);

    digitalWrite(LED_PIN, b == 0x01 ? HIGH : LOW);
    Serial.println(b == 0x01 ? "Bomba ENCENDIDA" : "Bomba APAGADA")
   ;

    delay(500); // (evita conflicto con RX1/RX2)
    sendUplinkStatus(); // Respuesta automática al downlink
}
```

Listing 6.3: Recibir datos de control (Downlink)

Tras un join OTAA exitoso, el dispositivo envía periódicamente un uplink vacío. Por cada uplink que se envía a TTN, el microcontrolador abre ventanas de escucha, con el fin de que, si TTN tiene encolado un downlink para este dispositivo en concreto, lo envíe a través del gateway en una de las 2 ventanas de escucha que se abren. Estos downlinks que recibe el microcontrolador contiene datos de control de riego, concretamente ON/OFF.

Cuando se recibe un downlink (modem.available()), el dispositivo IoT lee su contenido (modem.read()), interpreta el valor recibido y actúa en consecuencia, encendiendo o apagando la bomba de riego. Después, envía un uplink de estado para confirmar a la aplicación que la orden fue recibida y ejecutada.

Este patrón de comunicación permite implementar control remoto en casi en tiempo real sobre dispositivos IoT usando comunicación LoRa con implementación de clase A, combinando el envío periódico de datos (uplinks) con la ejecución de órdenes remotas (downlinks) de forma eficiente.

6.2. Desarrollo del Módulo 2: Fog computing - Recolección y preprocesamiento

El primer paso del desarrollo de este módulo consiste en configurar el entorno TTN. Para configurar el entorno TTN, vamos a realizar los siguientes pasos

1. Configurar payload formatter para decodificar de formato bytes a JSON cuando se recibe un uplink desde el microcontrolador.
2. Configurar payload formatter para decodificar de formato bytes a JSON con el objetivo de hacer legibles en el "Live Data" de TTN las ordenes de riego. Desde el servidor web las ordenes de riego se enviarán en base64 y TTN hará una codificación implícita a formato bytes.
3. Configuración del webhook HTTP.
4. Generar API key para downlink.

6.2.1. Configurar payload formatter de uplink

Cómo se ha comentado anteriormente se necesita de un payload formatter para uplinks, con el propósito de decodificar de formato bytes a JSON. Es necesario decodificarlo a formato JSON puesto que es el formato que va a interpretar el servidor web y es el formato configurado en el webhook. Por lo que cada vez que consulte el endpoint HTTP configurado en el webhook, recibirá la respuesta en formato JSON con los datos de los sensores.

```
function decodeUplink(input) {
  const bytes = input.bytes;
  const port = input.fPort;

  // Puerto 3 = uplink de control, no almacenar
  if (port === 3) {
    return {
      data: {
        info: "uplink de control",
        ignored: true
      },
      warnings: ["Uplink vacío recibido en puerto 3"]
    };
  }

  // Datos reales: temperatura y humedad
  const temp = (bytes[0] << 8) | bytes[1];
  const soil = (bytes[2] << 8) | bytes[3];

  return {
    data: {
      temperature: temp / 10,
      soil_moisture: soil / 10
    }
  };
```

```
}
```

Listing 6.4: Payload formatter de Uplink

Este es el código del payload formatter, el cual es una función de javascript que tiene como propósito interpretar el payload en formato bytes recibido desde el dispositivo IoT.

- **Si el uplink llega por el puerto 3**, lo identifica como un uplink de control (vacío) marca que se ignore y genera un advertencia.
- **Si llega por otro puerto**, interpreta los primeros 2 bytes como la temperatura, los siguientes 2 bytes como humedad ambiental y los últimos 2 bytes como la humedad del suelo.
- Divide ambos valores entre 10 para recuperar el formato original con un decimal y devuelve esos datos listos para usarse en la aplicación en formato JSON

6.2.2. Configurar payload formatter de downlink

En este caso realmente, no haría falta un formatter para codificar de base64, que es el formato en el que llega el dato de control desde el servidor web, a bytes. Ya que se trata de una conversión interna que hace la propia plataforma de TTN. Pero si que se ha creado un formatter para decodificar de bytes a JSON. Esto se hace para descubrir si se el dato de control enviado a sido ON/OFF.

```
var LED_STATES = ['off', 'on'];

function decodeDownlink(input) {
  return {
    data: {
      ledState: LED_STATES[input.bytes[0]]
    }
  }
}
```

Listing 6.5: Payload formatter de Downlink

6.2.3. Configuración de webhook HTTP

Un webhook HTTP sirve para que TTN envíe automáticamente los datos recibidos desde el dispositivo Iot a un endpoint HTTP de nuestro servidor web en cuanto ocurren. El primer paso para configurar el webhook, es elegir el formato de los datos que se van a enviar y añadir el dominio de nuestro servido web. Hay que mencionar que el dominio del servidor web que aparece en la Figura 6.2 es provisional.

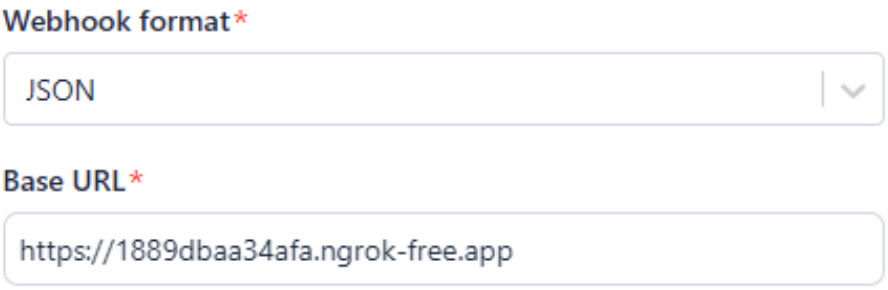


Figura 6.2: Añadir formato y dominio del servidor web en el webhook

Por otro lado, al configurar el webhook, tenemos que elegir que tipos de filtros se van a aplicar con el objetivo de seleccionar qué información se va a mandar al servidor web. En nuestro caso se han escogido 3 tipos de filtro.

- **up.uplink_message_decoded_payload:** Nos permite filtrar los datos de los sensores que llegan desde el dispositivo IoT
- **end_device_ids:** Nos permite filtrar la información del dispositivo desde dónde se envían los datos. La información del dispositivo nos permite compararla con los dispositivos actualmente almacenados en la base de datos, si hay una coincidencia y además la comunicación está activa para ese dispositivo, se guardan los datos de los sensores en la base de datos.
- **up.uplink_message.f_port:** Nos permite filtrar el puerto donde se ha enviado el uplink. El puerto va a servir para determinar si la información que llega al servidor web desde TTN se va a almacenar o no. Esto es debido a que hay dos tipos de uplink que se van a enviar. El primer tipo, es un uplink con los datos de los sensores que se van a enviar por el primer puerto. El segundo tipo, es un uplink vacío que se envía periódicamente por el tercer puerto para que los downlinks se desencolen más rápido, con el fin de lograr una respuesta más rápida sobre el control del riego. Por lo que a nosotros nos interesa almacenar en la base de datos la información que viene del primer puerto.

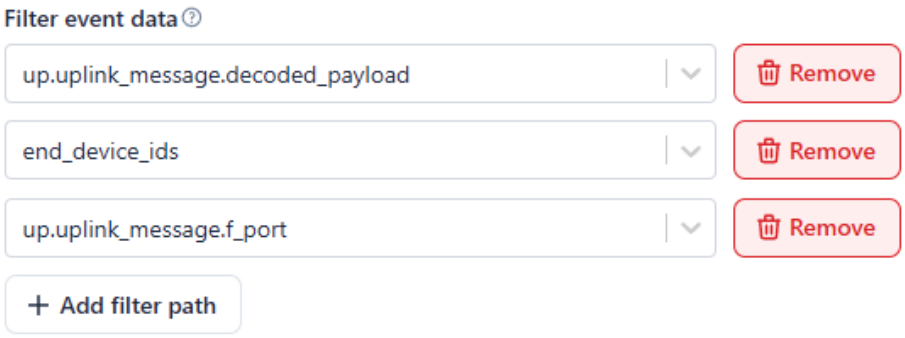


Figura 6.3: Filtros configurados en el webhook

Por último, se configurará en el webhook la ruta o endpoint HTTP mediante el cual TTN enviará solicitudes HTTP POST con los datos filtrados, para que el servidor web los reciba en esa misma ruta definida en un controlador específico. El servidor web recibirá una respuesta en formato JSON, que normalizará y hará las operaciones ya vistas en secciones anteriores.

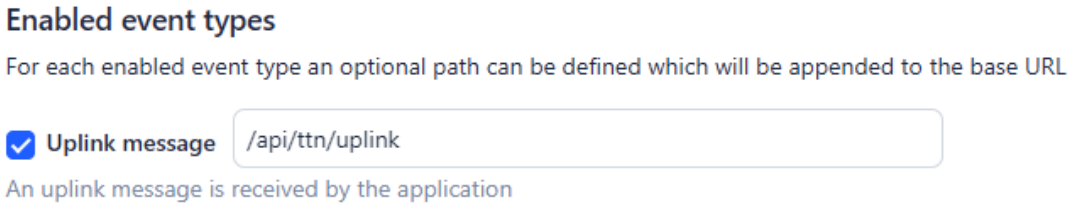


Figura 6.4: Configuración del endpoint HTTP

6.2.4. Generar API key para downlink

Para poder enviar ordenes de control de riego desde el servidor web hacia el dispositivo IoT a través de The Things Network (TTN) al consumir la API de TTN, es necesario contar con una serie de credenciales. En primer lugar, se requieren 4 credenciales, 3 de ellas se se obtienen automáticamente cuando se da de alta un dispositivo en TTN (TTN_REGION, APP_ID, DEV_EUI).

Estas credenciales son únicas para cada dispositivo y son esenciales para establecer una comunicación bidireccional segura. La primera credencial es el TTN_REGION, que sirve para identificar la región del cluster que se utilizará. La segunda credencial, es el DEV_EUI, que representa el identificador único del dispositivo específico en la

red LoRaWAN. La tercera credencial, es el APP_ID, que representa el identificador único de tu aplicación dentro de TTN y , por último, la cuarta credencial que es el APP_KEY, que actúa como la clave de aplicación necesaria para el cifrado de los datos que se transmiten entre el servidor de red de TTN y el dispositivo.

La APP_Key servirá para autorizar el envío de downlinks desde el servidor web hasta TTN. Esta credencial es crucial para la seguridad de la comunicación entre el servidor web y TTN, ya que TTN utiliza este token de acceso para verificar que las peticiones provienen de una fuente autorizada. Sin esta API key válida, TTN rechazará automáticamente todas las peticiones de downlink, independientemente de que los otros parámetros sean correctos. Todos estos parámetros están ya almacenados en la base de datos para su uso.

La creación de la API Key es un paso sencillo, nos tendríamos que dirigir al apartado de API keys de TTN, pulsar "añadir API Key" y proveerle de un nombre, fecha de expiración y asignarle permisos. En nuestro caso le asignaremos todos los derechos.

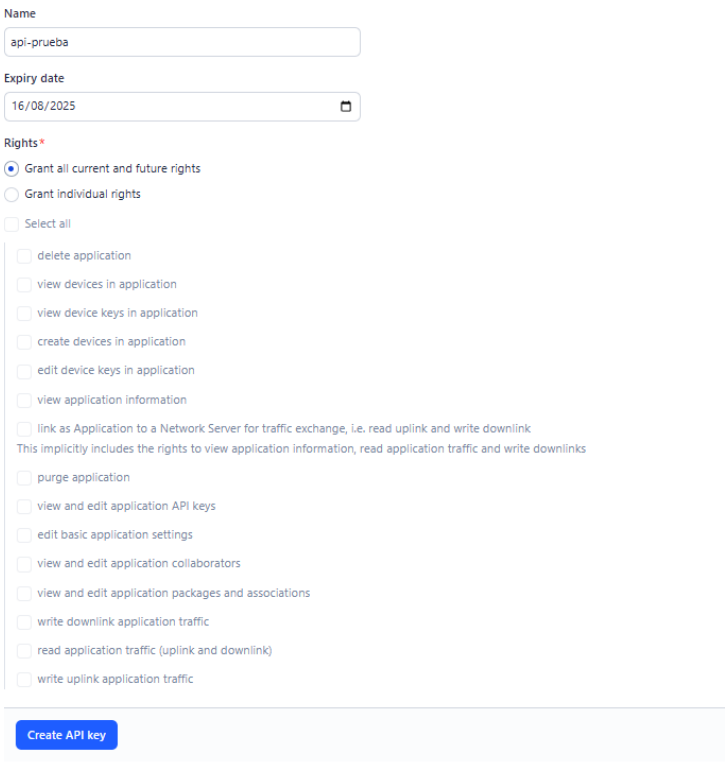


Figura 6.5: Configuración de la API Key

Cuando generemos la API Key, tenemos que tener en cuenta que debemos de copiarla y guardarla en un sitio seguro puesto que después de cerrar la ventana de confirmación la API Key no será más accesible.

6.3. Desarrollo del Módulo 3.1: Cloud computing - Recepción y postprocesamiento

El desarrollo de este módulo se basa en el diseño expuesto en la sección 5.7. No vamos a exponer en esta sección todo el código que se ha implementado para crear las API'S puesto que el manual técnico quedaría excesivamente largo ni tampoco se va a ir explicando el código de cada fichero del servidor web puesto que también quedaría demasiado extenso. Pero sí vamos a comentar como se han implementado a nivel de código las principales API's o funcionalidades de la aplicación. Por otro lado es necesario destacar que en base al diseño expuesto la sección 5.7 se ha decidido implementar una arquitectura cliente-servidor donde el frontend actúa como cliente consumiendo API's REST expuestas por el servidor web. En esta sección nos enfocaremos en la implementación de lado del servidor.

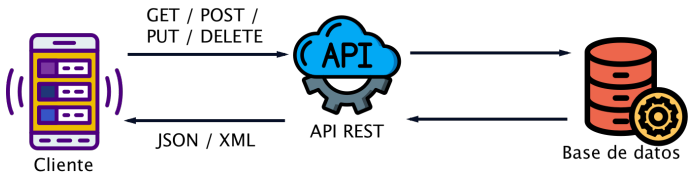


Figura 6.6: Arquitectura cliente-servidor

Se va a explicar también como se ha implementado el patrón de diseño modelo-vista-controlador en el servidor, en qué lenguaje de programación se ha implementado y qué framework se ha utilizado. Así como, listar los ficheros que se han creado en cada uno de los directorios del servidor web y definir para que sirven, además de documentar los servicios implementados.

6.3.1. Arquitectura del servidor web

El servidor web del sistema se ha desarrollado con el entorno de ejecución Node.js y utilizando como framework Express.js, se estructura siguiendo el patrón MVC adaptado a las necesidades de una API REST moderna. La arquitectura se organiza en cuatro componentes principales: modelos, controladores, rutas y servicios auxiliares. Dentro de estos componentes también destacaríamos la vista, pero cómo forma parte del módulo 4, no la mencionaremos en esta sección. Estos componentes

trabajan de forma coordinada para proporcionar las funcionalidades del sistema de gestión de riego inteligente.

6.3.1.1. Modelos (models/)

Los modelos representan las entidades principales del sistema y definen la estructura de datos que se almacena en la base de datos PostgreSQL. Cada modelo encapsula los atributos, métodos y relaciones de su respectiva entidad, proporcionando una abstracción clara entre la lógica de negocio y la persistencia de datos. La implementación de estos modelos se basa en el diseño del modelo entidad-relación presentado en la sección 5.8.2. Los modelos implementados en el sistema son:

- **User.js:** Gestiona la información de usuarios verificados y administradores
- **Crop.js:** Define las características y umbrales de los cultivos gestionados
- **Device.js:** Representa los dispositivos IoT registrados en el sistema
- **IrrigationConfig.js:** Representa las configuraciones de los modos manual, automático y programado
- **SensorReading.js:** Almacena los datos de temperatura y humedad captados por los sensores
- **PumpActivation.js:** Registra las activaciones y cambios de estado del sistema de riego.
- **Alert.js:** Gestiona las notificaciones sobre eventos ocrrudios en el sistema.

6.3.1.2. Controladores (controllers/)

Los controladores contienen toda la lógica de negocio del sistema, actuando como intermediarios entre las rutas y los modelos. Cada controlador se especializa en un dominio específico del sistema y encapsula las operaciones CRUD (Crear, Leer, Actualizar, Eliminar) correspondientes, así como funcionalidades específicas de cada entidad. Los controladores implementan validaciones de datos, manejo de errores y aplicación de reglas de negocio específicas del sistema.

Los controladores implementados en el sistema son:

- **alertController.js:** Controla la gestión de las alertas del sistema. Permitiendo la creación, eliminación y resolución de notificaciones sobre eventos del sistema. Permite crear alertas categorizadas por tipo (usuario, ambiental, dispositivo, cultivo, riego) y subtipo específico.

- **authController.js:** Controla toda la lógica de autenticación y autorización del sistema. Incluye funcionalidades de registro de usuarios, inicio de sesión con generación de tokens JWT, verificación de cuentas mediante email, recuperación y restablecimiento de contraseñas, y gestión de tokens.
- **cropController.js:** Controla la gestión completa de cultivos en el sistema. Permite crear, editar, eliminar y consultar cultivos, así como gestionar la selección/deselección de cultivos activos por usuarios.
- **deviceController.js:** Administra los dispositivos IoT registrados en el sistema. Gestiona las operaciones CRUD sobre dispositivos, configuración de credenciales TTN (The Things Network) y activación/desactivación de la comunicación de un dispositivo específico con la base datos.
- **pumpActivationController.js:** Controla las activaciones de la bomba de riego. Gestiona el estado de riego del sistema (inicio, pausa, reanudación, cancelación, finalización), calcula tiempos restantes de riego, y coordina el envío de comandos downlink al dispositivo IoT.
- **sensorReadingController.js:** Controla las lecturas de los sensores enviadas desde el microcontrolador. Gestiona las lecturas del sensor ambiental (DHT11) y del sensor de humedad (Capacitive Soil Moisture).
- **ttnDownlinkController.js:** Controla el envío de ordenes de riego hacia TTN. Gestiona el envío de ordenes de riego (downlinks) hacia dispositivos IoT a través de la API de TTN.
- **ttnWebhookController.js:** Procesa las peticiones HTTP POST realizadas por TTN hacia el endpoint configurado en el webhook con las lecturas de los sensores. Valida que los datos provengan de dispositivos autorizados y con comunicación activa, filtra uplinks por puerto específico (f_port $\neq$ 2), y coordina el almacenamiento de dichas lecturas.
- **userController.js:** Controla la autenticación y autorización de usuarios en el sistema. Gestiona las operaciones CRUD sobre usuarios, gestión de roles (visitante, usuario verificado, administrador), y funcionalidades específicas para administradores como consulta y modificación de cuentas de usuario.

6.3.1.3. Rutas

Las rutas definen los endpoints de cada una de las API'S REST del sistema y gestionan las peticiones HTTP (GET, POST, PUT, DELETE) dirigiéndolas hacia los controladores apropiados. Cada endpoint corresponde o a una funcionalidad específica. Las API's del sistema de las que parten los endpoints implementados son:

- **/api/auth:** Gestión de autenticación y autorización de usuarios.
- **/api/users:** Operaciones sobre usuarios (CRUD y gestión de perfiles).
- **/api/crops:** Gestión de cultivos y configuración de umbrales
- **/api/devices:** Gestión de dispositivos IoT y su comunicación.
- **/api/irrigation:** Configuración y control de los modos de riego.
- **/api/sensors:** Consulta y almacenamiento de lecturas de sensores.
- **/api/alerts:** Gestión de alertas y notificaciones del sistema.
- **/api/ttn/uplink:** Recepción de peticiones HTTP POST desde el webhook de The Things Network.
- **/api/ttn/downlink:** Envío de ordenes de riego hacia dispositivos IoT pasando por TTN.

6.3.2. Estructura de directorios del servidor web

La implementación del servidor web sigue una arquitectura modular y escalable que separa claramente las responsabilidades de cada componente del sistema. Esta organización facilita el mantenimiento, la escalabilidad y la comprensión del código. En base a estos principios se ha implementado una estructura de directorios robusta.

Además de los directorios principales ya descritos (controllers, routes y models), el servidor incluye varios directorios auxiliares que proporcionan funcionalidades esenciales para el correcto funcionamiento del sistema.

6.3.2.1. Configuración del servidor web (config/)

- **db.js:** Configura la conexión a la base de datos PostgreSQL utilizando un pool de conexiones. Define parámetros de conexión, zona horaria (Europe/Madrid), timeouts y configuración SSL condicional para entornos de producción. Incluye la función conectarDB() que verifica la conectividad durante el arranque del servidor.
- **nodemailer.js:** Proporciona la configuración del transporte de correo electrónico para el envío de emails del sistema. Crea y exporta la función createTransport() que inicializa Nodemailer con las credenciales SMTP configuradas a través de variables de entorno.

6.3.2.2. Plantillas de Emails (emails/)

- **authEmailService.js:** Servicio especializado en el envío de correos electrónicos relacionados con autenticación. Incluye plantillas HTML para verificación de cuenta (sendEmailVerification) y recuperación de contraseñas (sendEmailPasswordReset).

6.3.2.3. Autenticación y protección de rutas (middleware/)

- **authMiddleware.js:** Middleware de autenticación que valida tokens JWT en las rutas protegidas. Extrae el token del header Authorization, verifica su validez, consulta el usuario en base de datos y confirma que la cuenta esté verificada. Añade el objeto usuario a la request para uso posterior en los controladores.
- **passwordMiddleware.js:** Middleware especializado en el cifrado seguro de contraseñas utilizando bcrypt. Incluye dos funciones: hashPassword() para cifrar contraseñas nuevas durante el registro, y hashPasswordIfModified() para actualizar contraseñas solo cuando han sido modificadas.

6.3.2.4. Servicios (services/)

- **deviceValidationService.js:** Este servicio se especializa en proporcionar validación para dispositivos IoT que intentan enviar lecturas de sensores al servidor web. Este servicio actúa como una especie de "guardián", asegurando que solo dispositivos autorizados puedan interactuar con la base de datos.
- **irrigationAlertService.js:** Este servicio se especializa en la gestión de alertas relacionadas con el sistema de riego.
- **ttnService.js:** Este servicio se especializa en la gestión para la comunicación con The Things Network (TTN). Gestiona el envío de ordenes de riego mediante downlink a dispositivos IoT (ON/OFF), validación de credenciales TTN y comunicación con la API de TTN.
- **environmentalAlertService.js:** Este servicio se especializa en la gestión de alertas ambientales basadas en lecturas de sensores. Genera alertas automáticas cuando se exceden umbrales de temperatura máxima, humedad del suelo (valores mínimos y máximos), y humedad ambiental (valores mínimos y máximos).
- **sensorReadingService.js:** Este servicio se especializa en el manejo de las lecturas de los sensores. Incluye funcionalidades como obtener la última lectura del dispositivo activo.

- **deviceAlertService.js:** Este servicio se especializa en la gestión de alertas relacionadas con dispositivos IoT. Maneja notificaciones sobre eventos de dispositivos como agregado, eliminación, edición, activación/desactivación de comunicación, y copia de claves API.
- **authAlertService.js:** Este servicio se especializa en la gestión de alertas relacionadas con la autenticación y gestión de cuentas de usuario. Gestiona notificaciones sobre registro, inicio/cierre de sesión, cambios de contraseña, verificación de cuenta, restablecimiento de credenciales y actualización de información de perfil.
- **cropAlertService.js:** Este servicio se especializa en la gestión de alertas relacionadas con la gestión de cultivos. Maneja notificaciones sobre selección/-deselección de cultivos, agregado, edición y eliminación de cultivos.
- **userService.js:** Este servicio se especializa en la gestión de operaciones CRUD de usuarios. Proporciona métodos para buscar usuarios por email, ID o token de verificación, crear nuevos usuarios, actualizar información de perfil y eliminar cuentas.
- **automaticIrrigationService.js:** Este servicio se especializa en la evaluación continua de las lecturas de los sensores para determinar cuando activar o desactivar el riego según las condiciones ambientales. Este servicio se utilizará únicamente cuando el modo automático este activo.

6.3.2.5. Utilidades extra (utils/)

- **index.js:** Biblioteca de utilidades auxiliares que incluye funciones para manejo de fechas en zona horaria española, generación y validación de tokens JWT, generación de IDs únicos y funciones helper para manejo de errores. Centraliza funcionalidades comunes utilizadas por múltiples controladores.

6.3.2.6. Archivos principales

- **index.js:** Punto de entrada principal del servidor que inicializa la aplicación Express, configura middlewares globales (CORS, parseo JSON), establece la conexión a base de datos, registra todas las rutas del sistema, y arranca el servidor en el puerto configurado. Incluye manejo de errores global y logging colorizado para debugear.
- **package.json:** Archivo del proyecto que define metadatos, dependencias de producción (Express, PostgreSQL, JWT, Nodemailer, bcrypt) y desarrollo

(Nodemon), scripts de ejecución para diferentes entornos (desarrollo, producción, testing), y configuración de módulos ES6.

6.3.3. Servicios Auxiliares

6.3.3.1. Servicio de correo electrónico

El servicio de correo electrónico se ha implementado utilizando Nodemailer como biblioteca principal, configurado para funcionar con Mailtrap durante el desarrollo y pruebas. Mailtrap[60] proporciona un entorno seguro para testear el envío de emails sin riesgo de enviar mensajes a direcciones reales, capturando todos los correos en un inbox virtual para su revisión.

La configuración del transporte de correo está definida en *config/nodemailer.js*

```
import nodemailer from 'nodemailer';
export function createTransport(host, port, user, pass) {
const transporter = nodemailer.createTransport({
    host,
    port,
    auth: {
        user,
        pass
    }
});
return transporter;
}
```

Listing 6.6: Configuración de nodemailer

El servicio de email incluye plantillas HTML para la verificación de cuenta y recuperación de contraseñas, con estilos CSS responsivos. Estas plantillas se pueden encontrar en código en el fichero *emails/authEmailService.js*

6.3.3.2. Servicio de Autenticación JWT

El sistema implementa autenticación basada en JWT [36] para mantener sesiones seguras y sin estado. El servicio JWT incluye funciones para generación, verificación y validación de tokens con expiración configurable, en nuestro caso será de 30 días:

```
import jwt from 'jsonwebtoken';
// Generar ID único para tokens
const uniqueId = () => Date.now().toString(32) + Math.random().
   toString(32).substring(2);
// Generar \gls{jwt}
const generateJWT = (id) => {
```

```
    const token = jwt.sign({ id }, process.env.JWT_SECRET, {
        expiresIn: '30d'
    });
    return token;
};
```

Listing 6.7: Generación del token de sesión

Un aspecto a destacar en el Listing 6.7 anterior es la variable de entorno *JWT_SECRET*, se trata de una clave secreta para generar el token de sesión que se deberá implementar en las variables de entorno, donde ningún usuario pueda verla. Esta clave secreta puede ser cualquier palabra o frase.

El middleware de autenticación valida automáticamente los tokens en las rutas protegidas, extrayendo el token del header Authorization y verificando su validez:

```
const authMiddleware = async (req, res, next) => {
if (req.headers.authorization && req.headers.authorization.
   startsWith('Bearer')) {
try {
    // Extraer el token del header
    const token = req.headers.authorization.split(' ')[1];
    // Verificar el token
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    // Buscar el usuario en la base de datos
    const userData = await findUserById(decoded.id);
    if (!userData) {
        const error = new Error('Usuario no encontrado');
        return res.status(404).json({ msg: error.message });
    }
    // Verificar que la cuenta esté verificada
    if (!user.verified) {
        const error = new Error('Cuenta no verificada');
        return res.status(401).json({ msg: error.message });
    }
    // Agregar usuario sin datos sensibles al request
    req.user = user.getPublicData();
    next();
    } catch (error) {
        const authError = new Error('Token no válido');
        res.status(403).json({ msg: authError.message });
    }
    }
};
```

Listing 6.8: Validación del token de sesión

6.3.4. Implementación de la funcionalidad uplink

El proceso de comunicación uplink es una de las funcionalidades principales de nuestro sistema, se encargará de llevar las capturas de las lecturas de los sensores del dispositivo IoT hasta llegar al módulo 3, donde se almacenarán en el módulo 3.2 (almacén de datos) para su posterior visualización. Pasando por TTN donde se recolectarán los datos, se preprocesarán y se realizará una petición HTTP POST al endpoint HTTP configurado en el webhook para enviar la información al modulo 3.1 (servidor web). La implementación y desarrollo de este proceso queda detallado rigurosamente en los siguientes puntos:

6.3.4.1. Validación del Puerto de Comunicación

En primer lugar, antes de almacenarse la lectura de los sensores en el módulo 3.2 (almacén de datos) se valida el puerto lógico (f_port) del uplink. El sistema únicamente procesa datos de sensores que lleguen por el puerto 2, descartando automáticamente los uplinks vacíos que utilizan otros puertos:

```
const fPort = req.body.uplink_message?.f_port;
// Validar que el uplink viene del puerto 2
if (fPort !== 2) {
    console.log(Uplink ignorado - F_Port ${fPort} no es 2);
    return res.status(200).send('Uplink ignorado - puerto no válido
    ');
}
```

Listing 6.9: Validación del puerto de comunicación

6.3.4.2. Validación de Dispositivo Autorizado

Una vez confirmado el puerto correcto, el sistema valida que el dispositivo IoT que emite la lectura esté registrado en la base de datos y tenga la comunicación activa. Esta validación se realiza mediante el servicio especializado *DeviceValidationService.js*:

```
// Validar dispositivo \gls{ttn}
const deviceValidation = await DeviceValidationService.
    validateTTNDevice(endDeviceIds);
if (!deviceValidation.isValid) {
    console.error('Dispositivo no autorizado:', deviceValidation.
    message);
    return res.status(401).send('Dispositivo no autorizado');
}
const validatedDevice = deviceValidation.device;
console.log('Dispositivo autorizado:', validatedDevice.device_name)
    ;
```

Listing 6.10: Validación del dispositivo

6.3.4.3. Procesamiento y Normalización de Datos

El payload decodificado puede contener diferentes nombres de campos según la configuración del formatter en TTN. Se ha implementado una asignación de desestructuración para extraer los valores de temperatura y humedad independientemente de la nomenclatura utilizada:

```
// Extraer datos según nuevo formato de payload
const { temperature_c, humidity_rh, soil_moisture_pct } = payload;

// Convertir y asegurar números
const finalTemperature = Number(temperature_c) || null;
const finalAirHumidity = Number(humidity_rh) || null;
const finalSoilHumidity = Number(soil_moisture_pct) || null;
```

Listing 6.11: Normalización de datos

6.3.4.4. Almacenamiento en Base de Datos

Finalmente, los datos normalizados se preparan para su inserción en PostgreSQL utilizando el ID del dispositivo validado. El sistema utiliza el método correspondiente del controlador para guardar la lectura de los sensores, el método es createSensorReading.

```
// Preparar datos para guardar en PostgreSQL
const sensorData = {
    device_id: validatedDevice.id,
    air_humidity: finalAirHumidity,
    soil_humidity: finalSoilHumidity,
    temperature: finalTemperature
};
// Usar el servicio para guardar en PostgreSQL directamente
const savedReading = await createSensorReading(sensorData);
```

Listing 6.12: Almacenamiento de datos

6.3.5. Implementación de la funcionalidad downlink

El proceso de downlink permite al usuario, desde la interfaz web, enviar ordenes de control de riego (ON/OFF) a los dispositivos IoT para activar o desactivar la bomba de riego. Este proceso utiliza la API de TTN para encolar ordenes que serán

entregadas al dispositivo IoT en las próximas ventanas de recepción. La implementación y el desarrollo de este proceso queda detallado rigurosamente en los siguientes puntos:

6.3.5.1. Selección del Dispositivo Activo

El sistema identifica automáticamente el dispositivo con comunicación activa asociado al usuario propietario de la configuración de riego:

```
async function getActiveDeviceForConfig(irrigationConfigId) {
const query = SELECT d.* FROM irrigation_configs ic JOIN devices d
   ON d.user_id = ic.user_id WHERE ic.id = $1 AND d.
   is_active_communication = TRUE ORDER BY d.created_at DESC LIMIT
   1 ;
const result = await pool.query(query, [irrigationConfigId]);
return result.rows[0] || null;
}
```

Listing 6.13: Validación y selección de dispositivo activo

6.3.5.2. Construcción del Endpoint de la API TTN

La URL del endpoint se construye dinámicamente utilizando las credenciales almacenadas en la base de datos. La estructura de la API de TTN se constituye de la siguiente manera:

`https://{region}.cloud.thethings.network/api/v3/as/applications/{app_id}/devices/{device_id}/down/push`

```
const ttn_api_URL = https://${ttn_region}.cloud.thethings.network/
   api/v3/as/applications/${ttn_app_id}/devices/${enddevice_id}/
   down/push;
// Normalizar credenciales
ttn_region = String(ttn_region).trim().toLowerCase();
ttn_app_id = String(ttn_app_id).trim();
enddevice_id = String(enddevice_id).trim();
const cleanedKey = String(ttn_access_key).trim().replace(/^Bearer\s
   +/i, '');
```

Listing 6.14: Construcción del endpoint de la api de ttn

6.3.5.3. Estructura del Payload de Downlink

La orden de riego se codifica como un byte único (1 para ON, 0 para OFF) convertido a base64 y formateado en la estructura JSON requerida por la API de TTN:

```
const byte = command === 'ON' ? 1 : 0;
const payload = {
    downlinks: [
        {
            f_port: port, // Puerto lógico (puerto 3)
            frm_payload: Buffer.from([byte]).toString('base64'), //
    AQ== para 0x01
            priority: 'NORMAL', // Prioridad del mensaje
            confirmed: true, // Solicitar confirmación
        },
    ],
};
```

Listing 6.15: Estructura del payload del downlink

6.3.5.4. Envío HTTP a la API de TTN

El sistema implementa compatibilidad dual para el envío HTTP de ordenes de control de riego, utilizando fetch como método preferido:

```
if (typeof fetch === 'function') {
    const response = await fetch(ttn_api_URL, {
        method: 'POST',
        headers: {
            Authorization: Bearer ${cleanedKey},
            'Content-Type': 'application/json',
        },
    body: json.stringify(payload),
    });
    if (!response.ok) {
        throw new Error(TTN api ${response.status}: ${await
   response.text()});
    }
}
```

Listing 6.16: Envío de downlink a ttn usando fetch

6.4. Desarrollo del Módulo 3.2: Cloud Computing - Almacenamiento de datos

Este apartado básicamente consiste en implementar en una base de datos el diseño del modelo entidad-relación propuesto en la sección 5.8.2. Para ello, el sistema gestor de base de datos que se ha escogido es PostgreSQL. En la Figura 6.7 vemos como ha quedado implementada la base de datos.

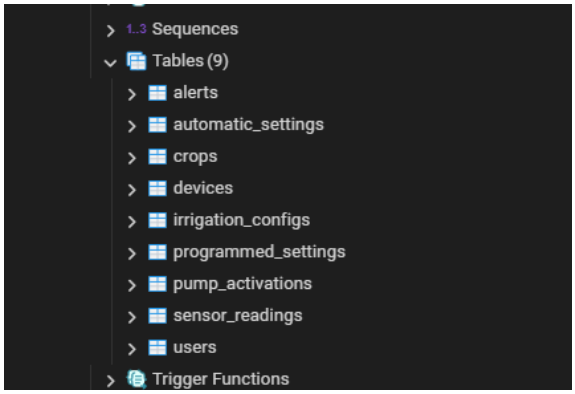


Figura 6.7: Esquema de la base de datos del sistema

Por otro lado, en el listing de a continuación podemos observar el script de SQL desarrollado para generar las tablas y por consecuencia el esquema mostrado en la Figura 6.7

```
CREATE TYPE user_role AS ENUM ('admin', 'usuario', 'visitante');
CREATE TYPE irrigation_mode AS ENUM ('manual', 'automatic', '
    programmed');
CREATE TYPE frequency_type AS ENUM ('once', 'daily', 'custom');
CREATE TYPE pump_status AS ENUM ('active', 'completed', 'cancelled'
    , 'paused', 'error', 'inactive', 'programmed');
CREATE TYPE alert_severity AS ENUM ('info', 'success', 'warning', '
    error');
CREATE TYPE alert_category AS ENUM (
  'user',           -- Acciones del usuario (registro, login,
    cambios)
  'environmental',  -- Condiciones ambientales (temperatura,
    humedad)
  'device',         -- Estado de dispositivos \gls{iot}
  'crop',           -- Gestión de cultivos
  'irrigation'      -- Actividad de riego
);

CREATE TYPE alert_subtype AS ENUM (
  -- User alerts
  'user_registered', 'user_logged_in', 'username_changed', '
    password_changed', 'session_closed',

  -- Environmental alerts
  'temperature_max_threshold', 'air_humidity_min_threshold', '
    air_humidity_max_threshold',
  'soil_humidity_min_threshold', 'soil_humidity_max_threshold', '
    system_online', 'system_offline',
```

```
  -- Device alerts
  'device_added', 'api_key_copied', 'device_offline', '
   device_online', 'device_deleted', 'device_edited',

  -- Crop alerts
  'crop_selected', 'crop_deselected', 'crop_edited', 'crop_deleted'
   , 'crop_added', 'crop_edited',

  -- Irrigation alerts
  'manual_started', 'emergency_stop', 'manual_cancelled', '
   programmed_saved',
  'programmed_reminder', 'programmed_schedule', '
   programmed_cancelled', 'automatic_saved'
);


CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL,
  name VARCHAR(255) NOT NULL,
  role user_role NOT NULL DEFAULT 'visitante',
  token TEXT,
  verified BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT NOW()
);


CREATE TABLE crops (
  id SERIAL PRIMARY KEY,
  user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
  name VARCHAR(255) NOT NULL,
  description TEXT,
  image VARCHAR(255),
  category VARCHAR(255),
  growth_days INTEGER,
  air_humidity_min DECIMAL(5,2),
  air_humidity_max DECIMAL(5,2),
  soil_humidity_min DECIMAL(5,2),
  soil_humidity_max DECIMAL(5,2),
  temperature_max DECIMAL(5,2),
  session VARCHAR(255),
  created_at TIMESTAMP DEFAULT NOW(),
  selected BOOLEAN DEFAULT FALSE
);

CREATE TABLE devices (
  id SERIAL PRIMARY KEY,
```

```
  user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
  device_name VARCHAR(255) NOT NULL,
  enddevice_id VARCHAR(255) NOT NULL,
  app_eui VARCHAR(16) NOT NULL,
  dev_eui VARCHAR(16) NOT NULL,
  app_key VARCHAR(32) NOT NULL,
  ttn_region VARCHAR(10),
  ttn_app_id VARCHAR(100),
  ttn_access_key VARCHAR(255),
  is_active_communication BOOLEAN DEFAULT false,
  created_at TIMESTAMP DEFAULT NOW()
);


CREATE TABLE sensor_readings (
  id SERIAL PRIMARY KEY,
  device_id INTEGER REFERENCES devices(id) ON DELETE CASCADE,
  air_humidity DECIMAL(5,2),
  soil_humidity DECIMAL(5,2),
  temperature DECIMAL(5,2),
  received_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE irrigation_configs (
  id SERIAL PRIMARY KEY,
  user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
  crop_id INTEGER REFERENCES crops(id) ON DELETE CASCADE,
  mode_type irrigation_mode NOT NULL,
  is_active BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT NOW(),
  last_irrigation_at TIMESTAMP,
  duration_minutes INTEGER,
  UNIQUE(user_id, crop_id, mode_type)
);


CREATE TABLE programmed_settings (
  config_id INTEGER PRIMARY KEY REFERENCES irrigation_configs(id)
   ON DELETE CASCADE,
  start_datetime TIMESTAMP NOT NULL,
  frequency_type frequency_type NOT NULL,
  custom_days INTEGER[],
  notify_before_minutes INTEGER DEFAULT 5,
  notify_at_start BOOLEAN DEFAULT TRUE,
  notify_at_end BOOLEAN DEFAULT TRUE,
  last_execution TIMESTAMP,
  next_execution TIMESTAMP
);
```

```
CREATE TABLE automatic_settings (
  config_id INTEGER PRIMARY KEY REFERENCES irrigation_configs(id)
   ON DELETE CASCADE,
  air_humidity_min_threshold DECIMAL(5,2) NOT NULL,
  air_humidity_max_threshold DECIMAL(5,2) NOT NULL,
  soil_humidity_min_threshold DECIMAL(5,2) NOT NULL,
  soil_humidity_max_threshold DECIMAL(5,2) NOT NULL,
  temperature_max_threshold DECIMAL(5,2) NOT NULL,
  use_crop_thresholds BOOLEAN DEFAULT TRUE
);

CREATE TABLE pump_activations (
  id SERIAL PRIMARY KEY,
  irrigation_config_id INTEGER REFERENCES irrigation_configs(id),
  started_at TIMESTAMP NOT NULL,
  ended_at TIMESTAMP,
  duration_minutes INTEGER,
  status pump_status DEFAULT 'active',
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE alerts (
  id SERIAL PRIMARY KEY,
  user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
  alert_type alert_category NOT NULL,
  alert_subtype alert_subtype NOT NULL,
  severity alert_severity DEFAULT 'info',
  title VARCHAR(255) NOT NULL,
  message TEXT NOT NULL,
  is_resolved BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT NOW()
);
```

Listing 6.17: Script de generación de esquema de base de datos del sistema

6.5. Desarrollo del Módulo 4: Interfaz web - Visualización

Para el desarrollo del Módulo 4 se va a tener en cuenta el diseño propuesto en la sección 5.9. Implementaremos las páginas que conformarán la interfaz web y que permitirán interactuar con la funcionalidad del sistema. En base al diseño, se ha implementado una interfaz web con una navegación intuitiva, accesible, fácil de utilizar y responsiva.

Como ya mencionamos en la sección 6.3, el sistema está implementado siguiendo la arquitectura cliente-servidor. En esta sección nos encargaremos de implementar el lado de cliente.

6.5.1. Arquitectura y tecnologías del cliente

El cliente se ha desarrollado e implementado de tal manera que este se encargará de consumir las API'S REST proporcionadas por el servidor web. Esta separación permite escalabilidad y reutilización de una API por múltiples clientes si fuera necesario. El frontend o cliente se desarrolla como una SPA híbrida utilizando Vue.js 3 como framework principal, aprovechando su arquitectura basada en componentes que facilita la modularización, reutilización de código y mantenimiento de la aplicación.

Para optimizar el desarrollo y proporcionar funcionalidades avanzadas de enrutamiento, server-side rendering (SSR) y gestión de módulos, se ha adoptado por Nuxt.js 3 como metaframework, que actúa como una capa superior sobre el framework, mediante el cual, se simplifica la configuración y proporciona funcionalidades ya automatizadas, como por ejemplo, el enrutamiento automático. Al tratarse de una página web reactiva y con arquitectura SPA, se necesita de un sistema de gestión de estados global que permita actualizar la interfaz de usuario en respuesta a cambios en los datos, sin necesidad de recargar la página completa. Para ello utilizaremos Pinia que se trata de una librería que permite la gestión del estado global de la aplicación o lo que lo mismo, un store. En Vue.js, el store oficial es Vuex, pero lo reemplazaremos por Pinia por su facilidad de uso.

6.5.2. Principios de diseño adoptados

El lado de cliente o frontend sigue los principios de diseño orientado a componentes, donde gran parte de elementos de la interfaz es un componente Vue reutilizable y autocontenido. Se ha implementado el patrón Composition API de Vue 3 para mejorar la organización del código y facilitar la reutilización de lógica entre componentes.

Se han aplicado principios de diseño responsivo que garantiza una experiencia óptima en dispositivos móviles, tabletas y escritorio. La arquitectura implementa lazy loading y code splitting automático proporcionado por Nuxt.js para optimizar los tiempos de carga y mejorar el rendimiento general de la aplicación. Además Nuxt.js tiene integrados mecanismos de mejora de SEO. Entre otras características como optimización de imágenes [61].

6.5.2.1. Estructura de directorios del frontend

```
vivantia-frontend/
├── api/ ..... Módulos de comunicación con backend
├── assets/ ..... Recursos estáticos y estilos
│   ├── css/ ..... Hojas de estilo globales
│   ├── icons/ ..... Iconos SVG personalizados
│   └── scss/ ..... Variables y mixins SCSS
├── components/ ..... Componentes Vue reutilizables
│   ├── Admin/ ..... Componentes del panel administrativo
│   ├── Dashboard/ ..... Componentes del dashboard principal
│   ├── Crops/ ..... Gestión de cultivos
│   ├── Devices/ ..... Gestión de dispositivos IoT
│   └── Charts/ ..... Gráficas y visualizaciones
├── composables/ ..... Lógica reutilizable
├── layouts/ ..... Plantillas de estructura de páginas
├── middleware/ ..... Autenticación y control de acceso
├── pages/ ..... Sistema de enrutamiento automático de páginas
│   ├── auth/ ..... Páginas de autenticación
│   ├── admin/ ..... Panel administrativo
│   ├── cultivos/ ..... Páginas de cultivos
│   ├── modo/ ..... Configuración de riego
│   ├── alertas.vue/ ..... Página de alertas
│   ├── dashboard.vue/ ..... Página de la dashboard
│   ├── dispositivos.vue/ ..... Página de dispositivos
│   ├── index.vue/ .... Página inicial que redirije a otras páginas según ciertas condiciones
│   └── perfil.vue/ ..... Página del perfil del usuario
├── plugins/ ..... Extensiones de funcionalidad
├── public/ ..... Recursos públicos
├── static/ ..... Recursos estáticos
├── stores/ ..... Gestión de estado global con Pinia
├── utils/ ..... Funciones auxiliares y helpers
└── nuxt.config.ts ..... Configuración del metaframework
```

Figura 6.8: Estructura de directorios del cliente del sistema completo

De la estructura de directorios del frontend expuesta en la Figura B.2, podemos destacar los siguientes directorios

- **api/:** Contiene los módulos de comunicación con el servidor web, organizados por dominio funcional (AuthAPI, CropAPI, DeviceAPI, etc.). Centraliza todas las llamadas HTTP y proporciona una interfaz para la comunicación cliente-servidor.
- **assets/:** Almacena recursos estáticos como hojas de estilo CSS, iconos SVG personalizados, imágenes y variables SCSS. Incluye la configuración de estilos globales y recursos que requieren procesamiento durante la compilación.
- **components/:** Biblioteca de componentes Vue.js reutilizables organizados jerárquicamente por funcionalidad (Admin, Dashboard, Crops, Devices, etc.). Cada directorio agrupa componentes relacionados.
- **composables/:** Funciones composables que encapsulan lógica reutilizable. Incluye composables para gestión de modos de riego, datos de sensores y notificaciones toast.
- **layouts/:** Define las plantillas de layout que estructuran las páginas de la aplicación. Incluye el layout principal con sidebar y header, y un layout específico para páginas de autenticación.
- **middleware/:** Funciones de middleware que se ejecutan antes de renderizar páginas. Implementa lógica de autenticación, autorización por roles y protección de rutas.
- **pages/:** Sistema de enrutamiento automático de Nuxt.js donde cada archivo Vue representa una ruta. Organizado en subdirectorios que reflejan la estructura de navegación de la aplicación (auth, admin, cultivos, etc.).
- **stores/:** Stores de Pinia organizados por dominio (user, device, crop, irrigation, admin) que gestionan el estado global de la aplicación de manera modular.
- **utils/:** Funciones utilitarias y helpers compartidos que proporcionan funcionalidades comunes como formateo de datos y validaciones.

6.5.3. Estilización

Para la implementación de estilos se ha adoptado Tailwind CSS como framework principal, proporcionando un sistema de diseño consistente basado en utility classes, que consiste en clases constituidas a partir de un conjunto de reglas de css, estas permiten acelerar el desarrollo y garantizar la aplicación de estilos uniformes. Tailwind CSS facilita la implementación de diseño responsivo, gestión de colores y espaciado siguiendo principios de diseño modernos.

Como complemento a Tailwind CSS, se utiliza CSS vanilla y SCSS para estilos específicos que requieren mayor personalización o que no pueden ser expresados como se require mediante utility classes.

6.6. Pruebas

6.6.1. Pruebas Unitarias

En esta sección exploraremos las pruebas unitarias realizadas sobre cada módulo.

6.6.1.1. Módulo 1: Edge Computing - Captación de datos y acción sobre el actuador

6.6.1.1.1. Verificación de la inicialización de los sensores

Tabla 6.1: Caso de prueba PU-M1-01: Verificación de inicialización de sensores

Identificador	PU-M1-01
Descripción	Verificar la inicialización de sensores
Precondiciones	El dispositivo necesita ser alimentado previamente
Pasos a seguir	1. Observar la lectura del sensor DHT11 en el monitor serie. 2. Observar la lectura del sensor capacitive soil moisture en el monitor serie.
Resultado esperado	El sistema debe de mostrar los datos de temperatura y humedad ambiental y además la humedad en el suelo.

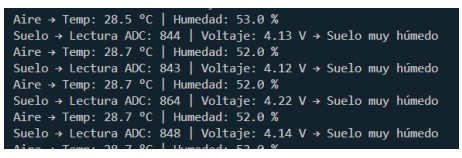


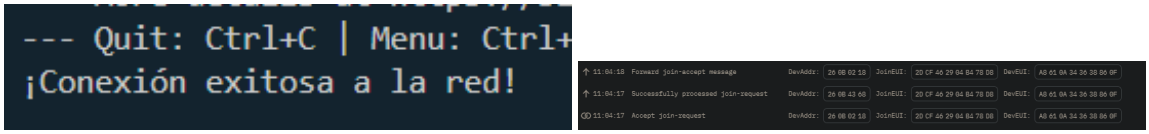
Figura 6.9: Correcto funcionamiento de los sensores

Figura 6.10: Caso de prueba PU-M1-01

6.6.1.1.2. Verificación del proceso de autenticación OTAA

Tabla 6.2: Caso de prueba PU-M1-02: Verificación del proceso de autenticación OTAA

Identificador	PU-M1-02
Descripción	Verificar que el proceso de autenticación OTAA ha sido éxitoso
Precondiciones	Se requiere que el módulo LoRa esté operativo, el dispositivo dado de alta en TTN y aportar credenciales OTAA válidas como en nuestro caso la appEUI y appKey en el fichero credenciales.h
Pasos a seguir	1. Reiniciar o reejecutar el código del dispositivo IoT para forzar nuevo proceso de join. 2. A modo de traza se tiene que mostrar el identificador único del dispositivo IoT en el monitor serie (DevEUI). 3. Iniciar proceso de join-request a través del método modem.joinOTAA(appEui, appKey), utilizando las credenciales appEUI y appKey. 4. Verificar en la consola de TTN si el dispositivo IoT se ha identificado correctamente en la red y que el servidor de red de TTN ha enviado join-accept al mismo.
Resultado esperado	Si las credenciales son correctas se debe de mostrar el mensaje "¡Conexión exitosa a la red!" en el monitor serie, a partir de este punto en cada uplink el modem LoraWAN cifrará el payload que se enviará a TTN. Si las credenciales son incorrectas se mostrará "Fallo al conectarse a la red, revisa sus credenciales" y entra en un bucle infinito



(a) ¡Conexión exitosa a la red! (b) Autenticación OTAA de TTN

Figura 6.11: Caso de prueba PU-M1-02

6.6.1.1.3. Verificación de la transmisión de datos de los sensores hacia TTN

Tabla 6.3: Caso de prueba PU-M1-03: Verificación de la transmisión de datos de los sensores hacia TTN

Identificador	PU-M1-03
Descripción	Verificar si la transmisión de datos de los sensores vía LoRaWAN hacia TTN ha sido exitosa (uplink)
Precondiciones	Se requiere que el módulo LoRa esté operativo, el gateway esté registrado en TTN y conectado a corriente. Además el proceso de autenticación OTAA ya debe de haberse dado previamente.
Pasos a seguir	1. Codificar los datos de los sensores en un payload de 6 bytes. Para que se cumplan las propiedades de bajo consumo de LoRaWAN 2. Usar el método modem.write(payload, 6) para transmistir los datos hacia TTN (uplink)
Resultado esperado	El sistema debe de mostrar en el monitor serie “Paquete enviado correctamente” y a modo de traza, el payload en formato hexadecimal.

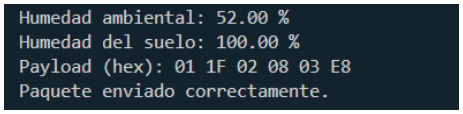


Figura 6.12: Correcto funcionamiento del envío de uplink

Figura 6.13: Caso de prueba PU-M1-03

6.6.1.1.4. Verificación del control de la bomba de riego mediante las órdenes de control (ON/OFF)

Tabla 6.4: Caso de prueba PU-M1-04: Verificación del control de la bomba de riego mediante las órdenes de control provenientes de TTN

Identificador	PU-M1-04
Descripción	Verificar el control de la bomba de riego mediante las órdenes de control provenientes de TTN (ON/OFF)
Precondiciones	Se requiere que el mecanismo de activación de riego y la propia bomba de riego ya estén instaladas.
Pasos a seguir	1. Enviar uplinks periódicos vacíos para desencolar órdenes de control de riego. 2. Desde TTN se envía el comando de control 0x01 (ON) desde TTN, el actuador (relé) activa el mecanismo de activación de la bomba de riego y esta se enciende. 3. Desde TTN se envía el comando de control 0x00 (OFF) desde TTN, el actuador (relé) desactiva el mecanismo de activación de la bomba de riego y esta se apaga.
Resultado esperado	Verificar en el monitor serie el mensaje "Bomba ENCENDIDA!" o "Bomba APAGADA!", y verificar en el relé que la luz que indica que el interruptor se ha activado o desactivado esté encendida o apagada respectivamente. Además de verificar que este absorbiendo agua la bomba de riego.

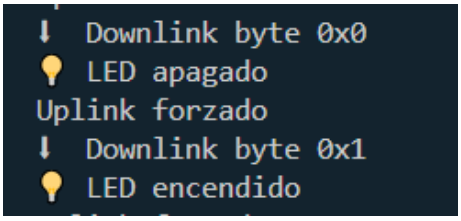


Figura 6.14: Correcto funcionamiento de recibdo de downlink

Figura 6.15: Caso de prueba PU-M1-04

Por otro lado en la siguiente imagen se muestra cuales LED's deben de estar encendidos cuando el microcontrolador recibe una orden de riego ya sea ON/OFF. Como se observa en la Figura 6.17, cuando el relé es energizado para activar la bomba de riego se enciende un LED rojo y en caso contrario se apagaría. Se puede ver en la imagen un LED verde fijo que indica que el relé está siendo alimentado y funciona correctamente.

Figura 6.16: Relé energizado

Figura 6.17: Caso de prueba PU-M1-04 - Relé

6.6.1.2. Módulo 2: Fog Computing - Recolección y preprocesamiento

6.6.1.2.1. Verificación de la recepción y decodificación de los datos de los sensores

Tabla 6.5: Caso de prueba PU-M2-01: Verificación de la correcta decodificación del payload en bytes de los datos de temperatura y humedad ambiental, y humedad del suelo a formato JSON. Así como comprobar la correcta recepción de datos desde el dispositivo IoT.

Identificador	PU-M2-01
Descripción	Verificar la correcta decodificación del payload en bytes de los datos de temperatura y humedad ambiental, y humedad del suelo a formato JSON. Así como comprobar la correcta recepción
Precondiciones	Se requiere que el payload formatter del uplink esté ya implementado en TTN y que el módulo 1 esté operativo.
Pasos a seguir	1. Revisar que el payload formatter para uplink esté correctamente implementado y guardados los cambios. 2. Revisar en la consola de TTN los mensajes recibidos, concretamente la secciónde “Live Data”. 3. Verificar el resultado del decodificador implementado en el payload formatter de TTN.
Resultado esperado	El decodificador debe de convertir correctamente los 6 bytes de los datos de los sensores a formato JSON, con el objetivo de ser reenviados al servidor web a través del webhook.

Figura 6.18: Correcta recepción y decodificación de los datos de los sensores

Figura 6.19: Caso de prueba PU-M2-01

6.6.1.2.2. Verificación del funcionamiento del webhook HTTP

Tabla 6.6: Caso de prueba PU-M2-02: Verificación del funcionamiento del webhook HTTP

Identificador	PU-M2-02
Descripción	Verificar el funcionamiento del webhook HTTP
Precondiciones	Se requiere que el módulo 1 esté operativo y enviando uplinks
Pasos a seguir	1. Configurar el webhook HTTP, asignándole un dominio y un endpoint HTTP donde reenviar la información hacia el servidor web. 2. Verificar en la consola de TTN si el webhook está operativo o no, cuando se envía un dato desde los sensores 3. Comprobar que los datos se están enviando hacia el webhook en el formato correcto.
Resultado esperado	En la consola de TTN se debe de mostrar los datos de los sensores enviados desde el dispositivo IoT, si está operativo no se mostrará ningún mensaje. En caso de que no esté operativo se mostrará un mensaje de error ("*fail webhook*").

Figura 6.20: Correcto funcionamiento del webhook HTTP

Figura 6.21: Caso de prueba PU-M2-02

6.6.1.2.3. Verificación de la recepción y envío de downlinks hacia el servidor web

Tabla 6.7: Caso de prueba PU-M2-03: Verificación de la recepción y envío de donwlinks hacia el servidor web

Identificador	PU-M2-03
Descripción	Verificar el encolado, desencolado y envío de downlinks con los datos de control de riego
Precondiciones	Se requiere que el módulo 1 esté operativo y enviando uplinks periódicos
Pasos a seguir	1. El servidor web envía una orden de control de riego ON/OFF (downlink). 2. El downlink se encola, esperando un uplink desde el dispositivo IoT. 3. TTN desencola el mensaje, al recibir un uplink tanto vacío como con los datos de los sensores. 4. El dispositivo IoT abre las ventanas de escucha para recibir los datos de control de riego desde TTN.
Resultado esperado	En la consola de TTN se debe de mostrar un mensaje que indique que los datos se han encolado y posteriormente desencolado al recibir un uplink, abriendo las ventanas de escucha para que el dispositivo pueda recibirlo.

Figura 6.22: Correcto funcionamiento del downlink

Figura 6.23: Caso de prueba PU-M2-03

6.6.1.3. Módulo 3.1: Cloud Computing - Recepción y postprocesamiento

El servidor web provee las funcionalidades de la aplicación a través de la exposición de una serie de API's. Cada API está formada por endpoints, que son puntos de acceso para cada una de estas funcionalidades. Para acceder a los endpoints, el cliente envía una petición HTTP usando un método determinado, que puede ser GET, PUT, POST y DELETE. El servidor web al recibir la petición ejecuta la lógica necesaria y devuelve una respuesta con el recurso solicitado o el resultado de la acción. Para probar el funcionamiento de un endpoint específico, existe una herramienta llamada POSTMAN[62], la cual permite enviar peticiones HTTP utilizando un método HTTP específico y visualizar de manera clara las respuestas devueltas por el servidor.

6.6.1.3.1. API de Autenticación

Tabla 6.8: Pruebas Unitarias - APIs de Autenticación

ID	Endpoint	Método	Descripción	Auth	Resultado Obtenido
PU-M3.1-01	`/api/auth/register`	POST	Registrar nuevo usuario	No	201: El usuario se creó correctamente, revisa tu email 400: El usuario ya existe
PU-M3.1-02	`/api/auth/login`	POST	Iniciar sesión	No	200: Devuelve token e información usuario logueado 400: Tu cuenta no ha sido confirmada aún / El password es incorrecto / El usuario no existe
PU-M3.1-03	`/api/auth/verify/:token`	GET	Verificar cuenta por email	No	200: Cuenta verificada 400: Token inválido o expirado
PU-M3.1-04	`/api/auth/forgot-password`	POST	Envío de un email con las instrucciones de cambio de contraseña	No	200: Hemos enviado un email con las instrucciones 404: El usuario no existe
PU-M3.1-05	`/api/auth/verify-token/:token`	GET	Verificar token para reset de contraseña	No	200: Token válido 400: Token inválido o expirado
PU-M3.1-06	`/api/auth/update-password/:token`	PUT	Actualizar contraseña con token	No	200: Passwrod modificado correctamente 400: Token inválido
PU-M3.1-07	`/api/auth/change-password/:id`	PUT	Cambiar contraseña (usuario autenticado)	Sí	200: Contraseña cambiada exitosamente 400: Token inválido / Usuario no encontrado / Contraseña actual incorrecta
PU-M3.1-08	`/api/auth/user`	GET	Obtener perfil de usuario autenticado	Sí	200: Datos del usuario 403: Token no valido
PU-M3.1-09	`/api/auth/admin`	GET	Verificar permisos de administrador	Sí	200: Usuario es admin 403: No es administrador
PU-M3.1-10	`/api/auth/logout`	POST	Cerrar sesión	Sí	200: Sesión cerrada exitosamente 401: Token inválido

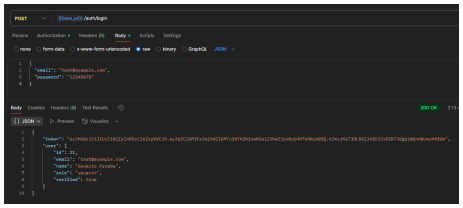
El caso de prueba de la Figura 6.24, tiene por objetivo verificar el correcto registro del usuario.

(a) Registro exitoso - Código de estado 201

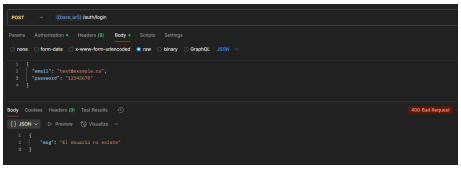
(b) El usuario ya existe, registro fallido - Código de estado 400

Figura 6.24: Caso de prueba PU-M3.1-01

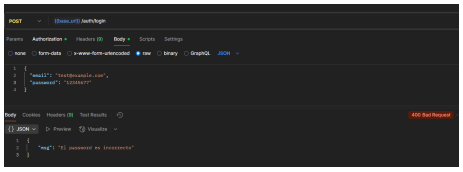
El caso de prueba de la Figura 6.25 tiene por objetivo verificar que el login se hace correctamente.



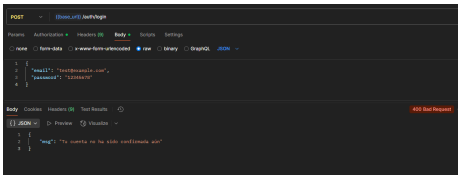
(a) Login exitoso - Código de estado 200



(b) El usuario no existe - Código de estado 400



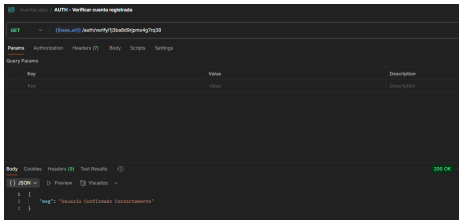
(c) El password es incorrecto - Código de estado 400



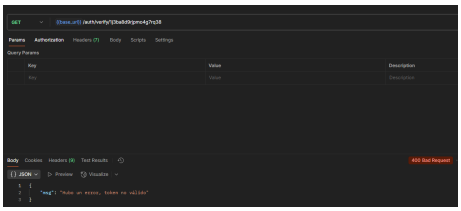
(d) Cuenta no confirmada - Código de estado 400

Figura 6.25: Caso de prueba PU-M3.1-02

El caso de prueba de la Figura 6.26 tiene por objetivo verificar que se realiza la verificación de la cuenta registrada por el usuario correctamente. Una vez verificada la cuenta el token se mascará como invalido y se almacenará en el localstorage para persistir la sesión del usuario.



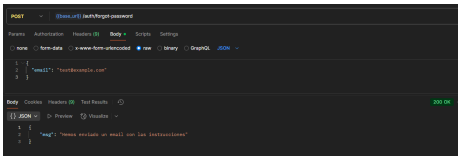
(a) Verificación de cuenta exitosa - Código de estado 200



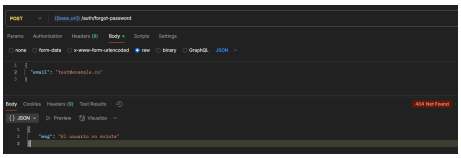
(b) El token ya no es válido - Código de estado 400

Figura 6.26: Caso de prueba PU-M3.1-03

El caso de prueba de la Figura 6.27 verifica si el correo de cambio de contraseña fue enviado correctamente.



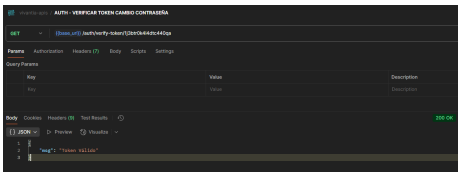
(a) Envío de un email con las instrucciones - Código de estado 200



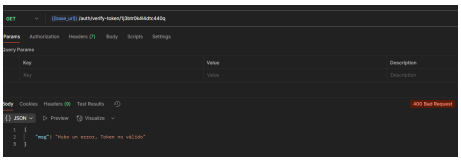
(b) El usuario no existe - Código de estado 404

Figura 6.27: Caso de prueba PU-M3.1-04

El caso de prueba de la Figura 6.28 verifica si el token para el reset de la contraseña es válido.



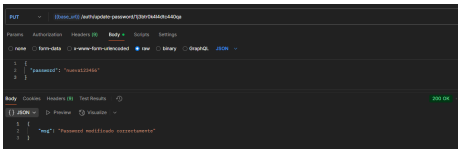
(a) Verificación token de reset de contraseña con éxito - Código de estado 200



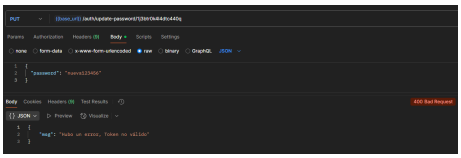
(b) El token es inválido - Código de estado 400

Figura 6.28: Caso de prueba PU-M3.1-05

El caso de prueba de la Figura 6.29 verifica si la contraseña es actualizada correctamente.



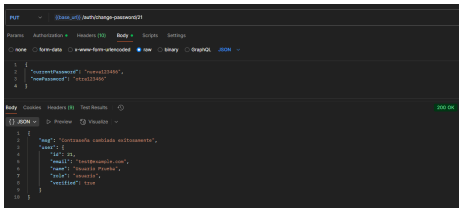
(a) Cambio de contraseña exitoso - Código de estado 200



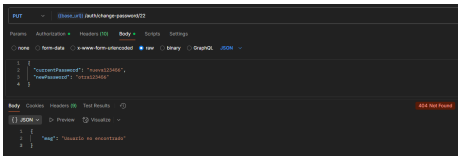
(b) El token es inválido - Código de estado 400

Figura 6.29: Caso de prueba PU-M3.1-06

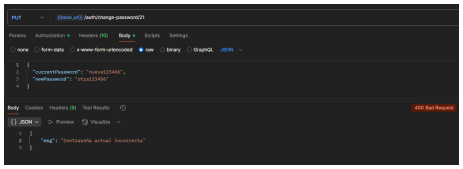
El caso de prueba de la Figura 6.30 verifica si la contraseña es actualizada correctamente con el usuario autenticado.



(a) Cambio de contraseña exitoso - Código de estado 200



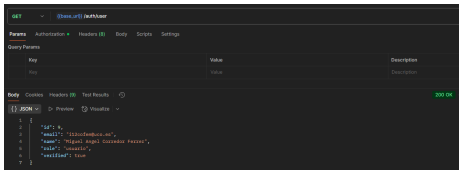
(b) El usuario no encontrado - Código de estado 400



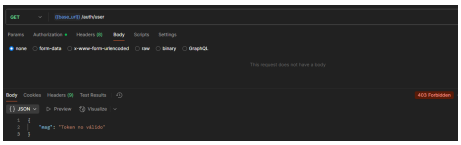
(c) El usuario no encontrado - Código de estado 400

Figura 6.30: Caso de prueba PU-M3.1-07

El caso de prueba de la Figura 6.31 verifica si se obtiene correctamente la información del usuario. Sólo puede utilizar este endpoint los usuarios.



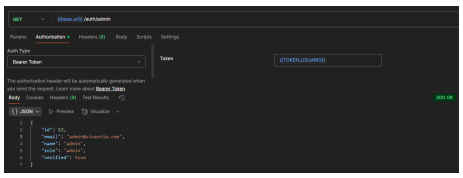
(a) Obtener información de usuario - Código de estado 200



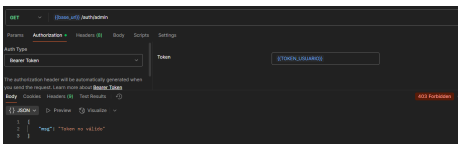
(b) Token inválido - Código de estado 403

Figura 6.31: Caso de prueba PU-M3.1-08

El caso de prueba de la Figura 6.32 verifica si se obtiene correctamente la información del admin. Sólo puede utilizar este endpoint el admin.



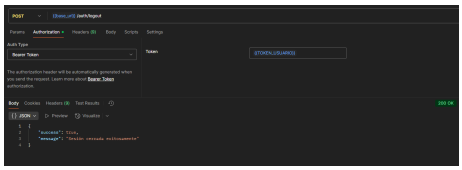
(a) Obtener información del admin - Código de estado 200



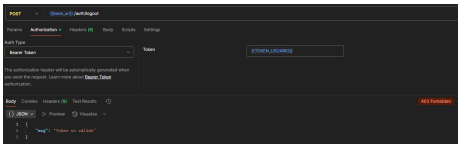
(b) Token inválido - Código de estado 403

Figura 6.32: Caso de prueba PU-M3.1-09

El caso de prueba de la Figura 6.33 verifica si se ha cerrado la sesión correctamente.



(a) Sesión cerrada exitosamente - Código de estado 200



(b) Token inválido - Código de estado 403

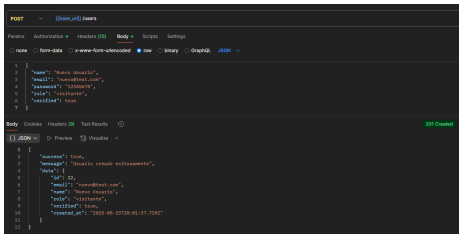
Figura 6.33: Caso de prueba PU-M3.1-10

6.6.1.3.2. API de gestión de usuarios

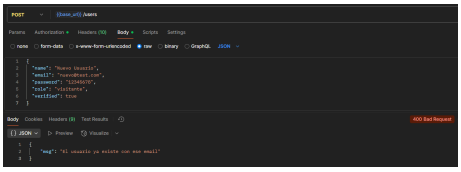
Tabla 6.9: Pruebas Unitarias - APIs de Gestión de Usuarios

ID	Endpoint	Método	Descripción	Auth	Resultado Obtenido
PU-M3.1-11	`/api/users`	POST	Crear usuario (Admin)	Sí	201: Usuario creado 403: Token inválido 400: El usuario ya existe con ese email
PU-M3.1-12	`/api/users`	GET	Obtener todos los usuarios (Admin)	Sí	200: Lista de usuarios 403: Token inválido
PU-M3.1-13	`/api/users/:id`	GET	Obtener usuario por ID	Sí	200: Datos del usuario 404: Usuario no encontrado
PU-M3.1-14	`/api/users/:id`	PUT	Actualizar usuario	Sí	200: Usuario actualizado 404: Usuario no encontrado
PU-M3.1-15	`/api/users/:id`	DELETE	Eliminar usuario	Sí	200: Usuario eliminado 404: Usuario no encontrado

El caso de prueba de la Figura 6.34 verifica si el admin puede crear usuarios correctamente.



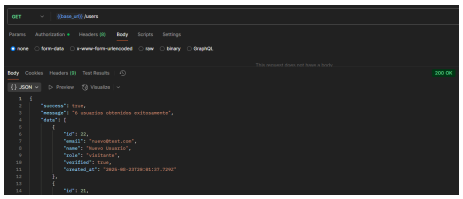
(a) Usuario creado por el administrador con éxito - Código de estado 200



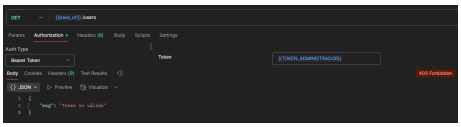
(b) El usuario ya existe con ese email - Código de estado 400

Figura 6.34: Caso de prueba PU-M3.1-11

El caso de prueba de la Figura 6.35 prueba que se obtienen todos los usuarios correctamente.



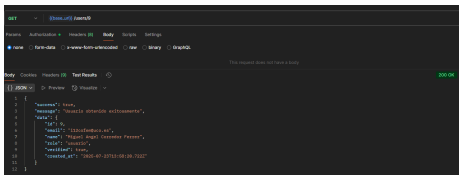
(a) Listar usuarios - Código de estado 200



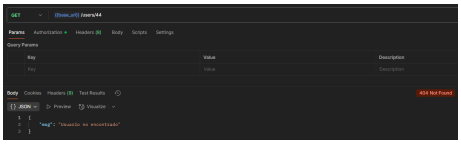
(b) Token inválido - Código de estado 403

Figura 6.35: Caso de prueba PU-M3.1-12

El caso de prueba de la Figura 6.36 verifica que se obtiene la información de un usuario específico correctamente.



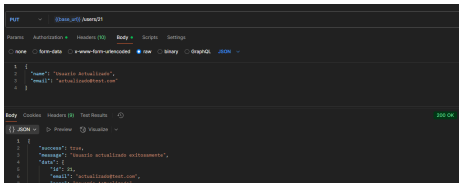
(a) Información del usuario obtenida con éxito - Código de estado 200



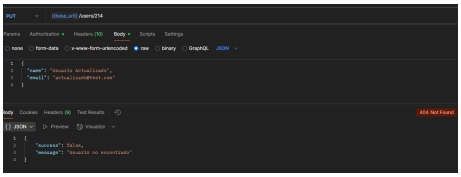
(b) Usuario no encontrado - Código de estado 404

Figura 6.36: Caso de prueba PU-M3.1-13

El caso de prueba de la Figura 6.37 verifica que se actualiza la información de un usuario específico correctamente.



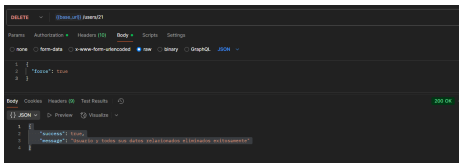
(a) Usuario actualizado con éxito - Código de estado 200



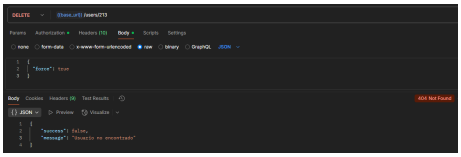
(b) Usuario no encontrado - Código de estado 404

Figura 6.37: Caso de prueba PU-M3.1-14

El caso de prueba de la Figura 6.38 verifica un usuario específico es eliminado con éxito por parte del administrador.



(a) Usuario eliminado con éxito - Código de estado 200



(b) Usuario no encontrado - Código de estado 404

Figura 6.38: Caso de prueba PU-M3.1-15

6.6.1.3.3. API de Dispositivos

Tabla 6.10: Pruebas Unitarias - APIs de Gestión de Dispositivos

ID	Endpoint	Método	Descripción	Auth	Resultado Obtenido
PU-M3.1-16	`/api/devices`	GET	Listar dispositivos usuario	Sí	200: Lista de dispositivos
PU-M3.1-17	`/api/devices`	POST	Crear nuevo dispositivo	Sí	201: Dispositivo creado 400: Credenciales TTN inválidas
PU-M3.1-18	`/api/devices/:id`	GET	Obtener dispositivo por ID	Sí	200: Datos del dispositivo 404: Dispositivo no encontrado
PU-M3.1-19	`/api/devices/:id`	PUT	Actualizar dispositivo	Sí	200: Dispositivo actualizado 404: Dispositivo no encontrado
PU-M3.1-20	`/api/devices/:id`	DELETE	Eliminar dispositivo	Sí	200: Dispositivo eliminado 404: Dispositivo no encontrado
PU-M3.1-21	`/api/devices/:id/activate`	PUT	Activar comunicación	Sí	200: Comunicación activada 404: Dispositivo no encontrado
PU-M3.1-22	`/api/devices/:id/deactivate`	PUT	Desactivar comunicación	Sí	200: Comunicación desactivada 404: Dispositivo no encontrado
PU-M3.1-23	`/api/devices/user/:user_id`	GET	Dispositivos por usuario	Sí	200: Lista de dispositivos 403: Sin permisos
PU-M3.1-24	`/api/devices/:id/latest-reading`	GET	Última lectura del dispositivo	Sí	200: Última lectura 404: Dispositivo no encontrado

El caso de prueba de la Figura 6.40 verifica que se estén obteniendo todos los dispositivos registrados por el usuario.

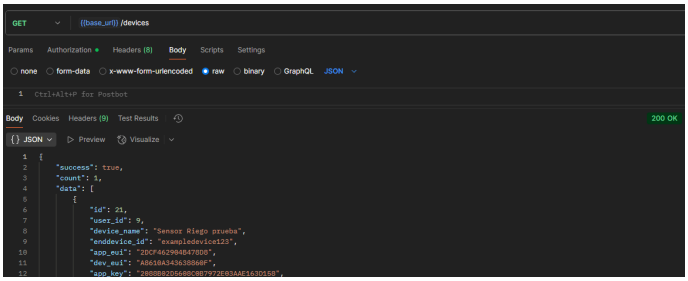
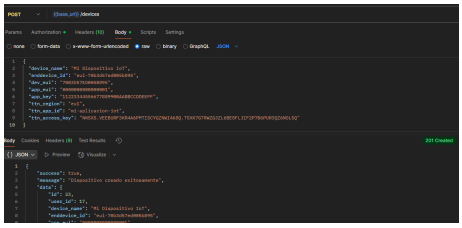


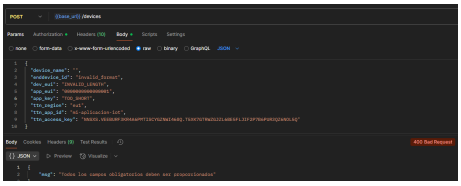
Figura 6.39: Listado de dispositivos mostrado con éxito - Código de estado 200

Figura 6.40: Caso de prueba PU-M3.1-16

El caso de prueba de la Figura 6.41 prueba si un usuario puede registrar un dispositivo correctamente.



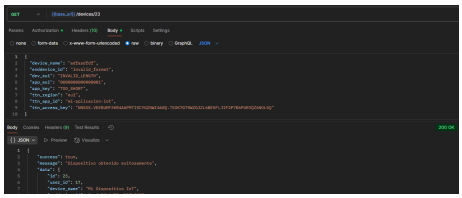
(a) Dispositivo creado con éxito - Código de estado 200



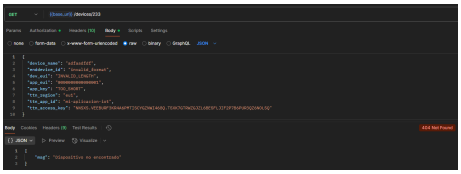
(b) Credenciales TTN inválidas - Código de estado 404

Figura 6.41: Caso de prueba PU-M3.1-17

El caso de prueba de la Figura 6.42 verifica que se esté obteniendo la información de un dispositivo concreto.



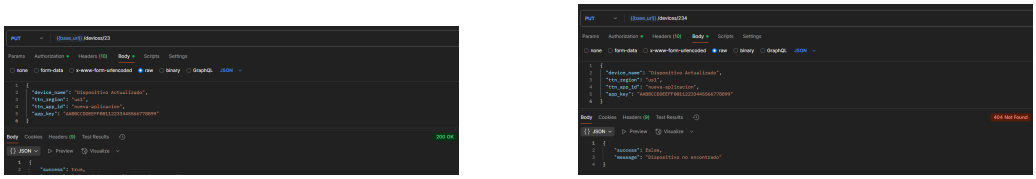
(a) Información del dispositivo obtenida con éxito - Código de estado 200



(b) Dispositivo no encontrado - Código de estado 404

Figura 6.42: Caso de prueba PU-M3.1-18

El caso de prueba de la Figura 6.43 verifica que la información del dispositivo se ha actualizado correctamente.

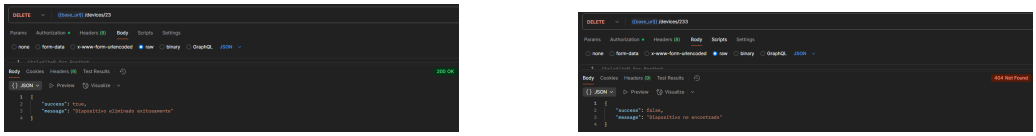


(a) Dispositivo actualizado con éxito - Código de estado 200

(b) Dispositivo no encontrado - Código de estado 404

Figura 6.43: Caso de prueba PU-M3.1-19

El caso de prueba de la Figura 6.45 verifica que el dispositivo se haya eliminado correctamente.

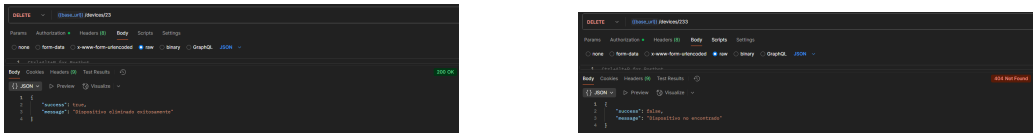


(a) Dispositivo eliminado con éxito - Código de estado 200

(b) Dispositivo no encontrado - Código de estado 404

Figura 6.44: Caso de prueba PU-M3.1-20

El caso de prueba de la Figura 6.45 verifica que el dispositivo se haya eliminado correctamente.



(a) Dispositivo eliminado con éxito - Código de estado 200

(b) Dispositivo no encontrado - Código de estado 404

Figura 6.45: Caso de prueba PU-M3.1-20

El caso de prueba de la Figura 6.47 verifica que el dispositivo se haya activado correctamente.

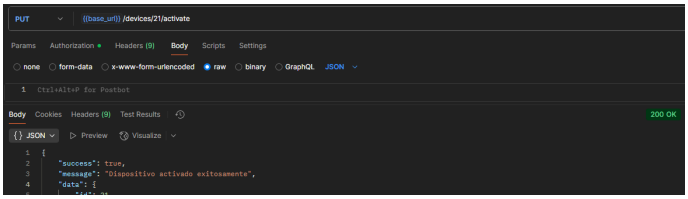
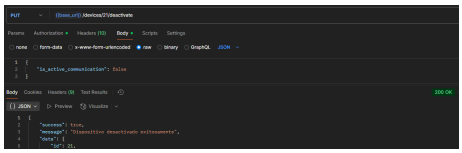


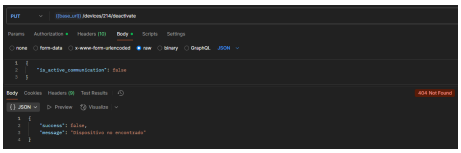
Figura 6.46: Dispositivo activado con éxito - Código de estado 200

Figura 6.47: Caso de prueba PU-M3.1-21

El caso de prueba de la Figura 6.48 verifica que el dispositivo se haya desactivado correctamente.



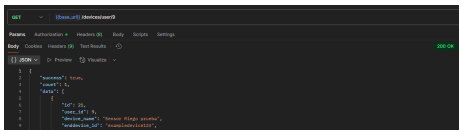
(a) Dispositivo desactivado con éxito - Código de estado 200



(b) Dispositivo no encontrado - Código de estado 404

Figura 6.48: Caso de prueba PU-M3.1-22

El caso de prueba de la Figura 6.49 verifica que se listan correctamente los dispositivo por usuario.

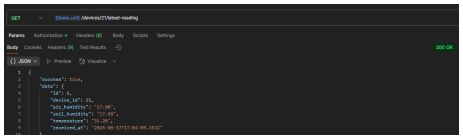


(a) Dispositivo desactivado con éxito - Código de estado 200

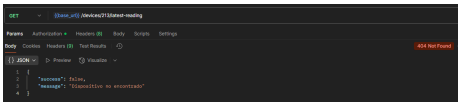
(b) No se tiene acceso a otros dispositivos de otros usuarios - Código de estado 403

Figura 6.49: Caso de prueba PU-M3.1-23

El caso de prueba de la Figura 6.50 verifica que se obtiene última lectura de los sensores para un dispositivo concreto.



(a) Última lectura de un dispositivo obtenida con éxito - Código de estado 200



(b) Dispositivo no encontrado - Código de estado 404

Figura 6.50: Caso de prueba PU-M3.1-24

6.6.1.3.4. API de cultivos

Tabla 6.11: Pruebas Unitarias - APIs de Gestión de Cultivos

ID	Endpoint	Método	Descripción	Auth	Resultado Obtenido
PU-M3.1-25	`/api/crops`	GET	Listar cultivos usuario	Sí	200: Lista de cultivos
PU-M3.1-26	`/api/crops`	POST	Crear nuevo cultivo	Sí	201: Cultivo creado 500: Error al crear un nuevo cultivo
PU-M3.1-27	`/api/crops/:id`	GET	Obtener cultivo por ID	Sí	200: Datos del cultivo 404: Cultivo no encontrado
PU-M3.1-28	`/api/crops/:id`	PUT	Actualizar cultivo	Sí	200: Cultivo actualizado 403: No es propietario
PU-M3.1-29	`/api/crops/:id`	DELETE	Eliminar cultivo	Sí	200: Cultivo eliminado 409: Cultivo en uso activo
PU-M3.1-30	`/api/crops/:id/select`	PUT	Seleccionar cultivo activo	Sí	200: Cultivo seleccionado 400: Solo uno activo por usuario
PU-M3.1-31	`/api/crops/:id/deselect`	PUT	Deseleccionar cultivo	Sí	200: Cultivo deseleccionado 404: Cultivo no encontrado
PU-M3.1-32	`/api/crops/user/:user_id`	GET	Cultivos por usuario	Sí	200: Lista de cultivos 403: Sin permisos
PU-M3.1-33	`/api/crops/categories`	GET	Obtener categorías de cultivos	No	200: Lista de categorías 500: Error del servidor

El caso de prueba de la Figura 6.52 verifica que todos los cultivos se estén listando correctamente.

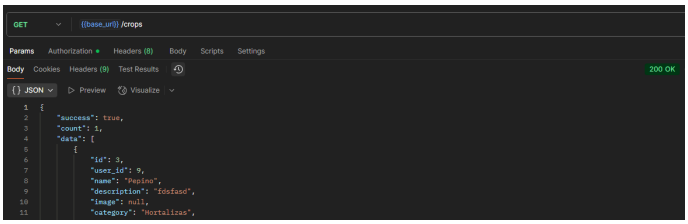
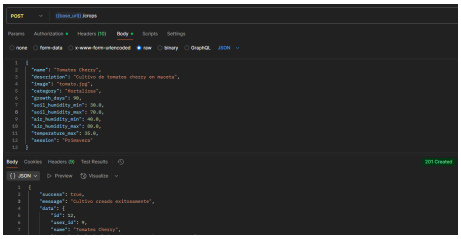


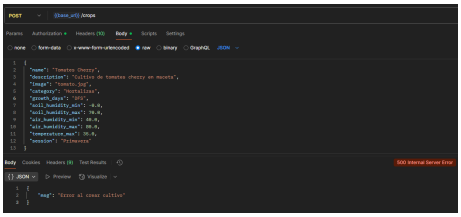
Figura 6.51: Todos los cultivos listados con éxito - Código de estado 200

Figura 6.52: Caso de prueba PU-M3.1-25

El caso de prueba de la Figura 6.53 prueba que la creación de un nuevo cultivo se efectúe correctamente.



(a) El cultivo se ha creado exitosamente - Código de estado 200

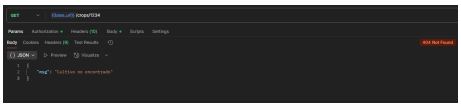


(b) Error al crear un cultivo - Código de estado 500

Figura 6.53: Caso de prueba PU-M3.1-26

El caso de prueba de la Figura 6.54 verifica que se obtiene la información de un cultivo especifico.

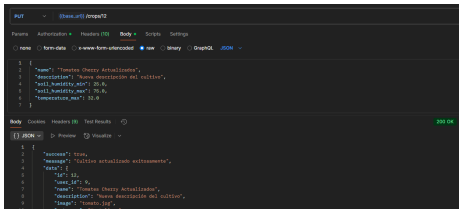
(a) Información del cultivo obtenida con éxito - Código de estado 200



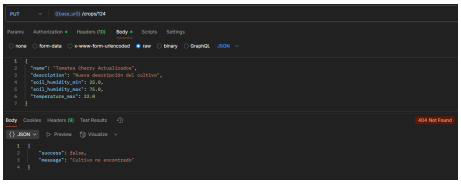
(b) Cultivo no encontrado - Código de estado 404

Figura 6.54: Caso de prueba PU-M3.1-27

El caso de prueba de la Figura 6.55 verifica que la información de un cultivo se haya actualizado correctamente.



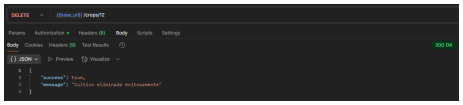
(a) Información del cultivo actualizada con éxito - Código de estado 200



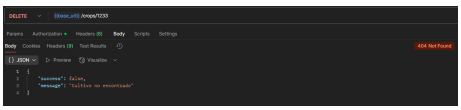
(b) Cultivo no encontrado - Código de estado 404

Figura 6.55: Caso de prueba PU-M3.1-28

El caso de prueba de la Figura 6.56 verifica que un cultivo se haya eliminado exitosamente.



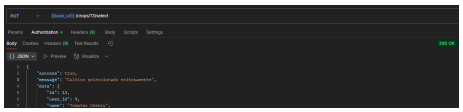
(a) Cultivo eliminado con éxito - Código de estado 200



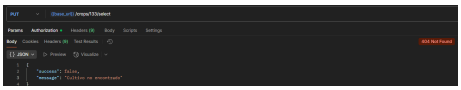
(b) Cultivo no encontrado - Código de estado 404

Figura 6.56: Caso de prueba PU-M3.1-29

El caso de prueba de la Figura 6.57 verifica que un cultivo se haya seleccionado exitosamente.



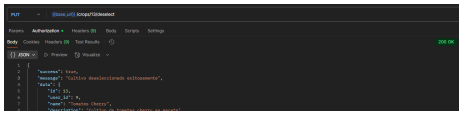
(a) Cultivo seleccionado con éxito - Código de estado 200



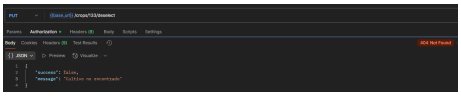
(b) Cultivo no seleccionado - Código de estado 404

Figura 6.57: Caso de prueba PU-M3.1-30

El caso de prueba de la Figura 6.58 verifica que un cultivo se haya deseleccionado exitosamente.



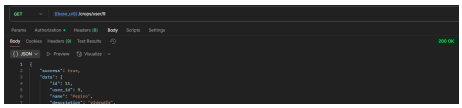
(a) Cultivo seleccionado con éxito - Código de estado 200



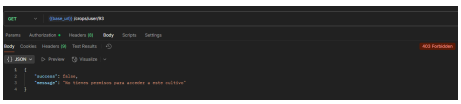
(b) Cultivo no seleccionado - Código de estado 404

Figura 6.58: Caso de prueba PU-M3.1-31

El caso de prueba de la Figura 6.59 verifica que un usuario esté listando todos los cultivos.



(a) Cultivos listados con éxito - Código de estado 200



(b) No tienes permiso para acceder a este cultivo - Código de estado 403

Figura 6.59: Caso de prueba PU-M3.1-32

El caso de prueba de la Figura 6.61 verifica que se obtienen todas las categoría de los cultivos.

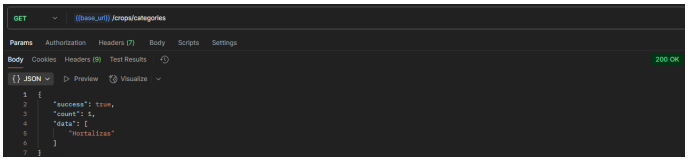


Figura 6.60: Obtener todas las categorías de los cultivos - Código de estado 200

Figura 6.61: Caso de prueba PU-M3.1-33

6.6.1.3.5. API de los datos de sensores

Tabla 6.12: Pruebas Unitarias - APIs de Datos de Sensores

ID	Endpoint	Método	Descripción	Auth	Resultado Obtenido
PU-M3.1-34	`/api/sensor-readings/device/:device_id`	GET	Lecturas por dispositivo	No	200: Lista de lecturas por dispositivo
PU-M3.1-35	`/api/sensor-readings`	POST	Crear lectura de sensor	No	201: Lectura creada exitosamente 500: Error al crear una lectura
PU-M3.1-36	`/api/sensor-readings/:id`	GET	Obtener lectura específica por ID	No	200: Datos completos de la lectura 404: Lectura no encontrada
PU-M3.1-37	`/api/sensor-readings/user/:user_id/active-device/latest`	GET	Última lectura del dispositivo activo	No	200: Última lectura con tendencias 404: Sin dispositivo activo o lecturas

El caso de prueba de la Figura 6.63 verifica que se obtengan las lecturas de sensores para un dispositivo determinado.

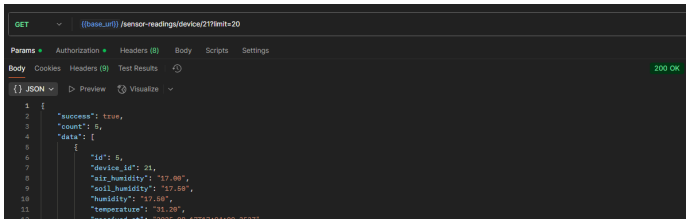
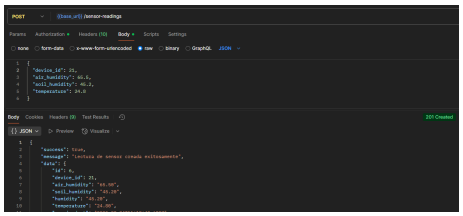


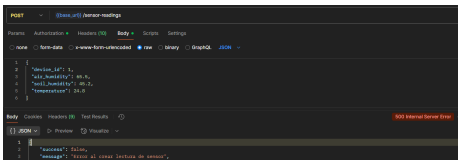
Figura 6.62: Lecturas de sensores obtenidas exitosamente - Código de estado 200

Figura 6.63: Caso de prueba PU-M3.1-34

El caso de prueba de la Figura 6.64 verifica que una lectura sea creada exitosamente.



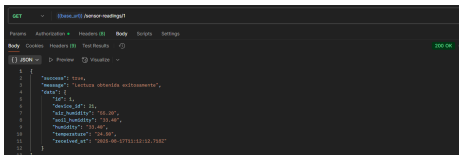
(a) Lectura creada con éxito - Código de estado 201



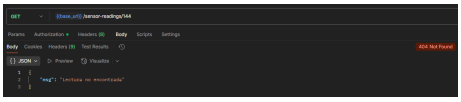
(b) Error al crear un nuevo cultivo - Código de estado 500

Figura 6.64: Caso de prueba PU-M3.1-35

El caso de prueba de la Figura 6.65 verifica que se haya obtenido una lectura específica por su ID correctamente.



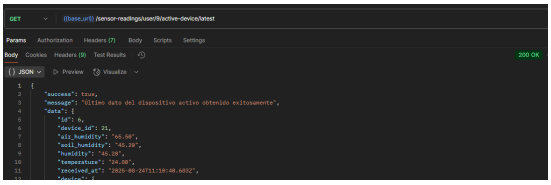
(a) Información de la lectura obtenida con éxito - Código de estado 200



(b) Lectura no encontrada - Código de estado 404

Figura 6.65: Caso de prueba PU-M3.1-36

El caso de prueba de la Figura 6.69 verifica que se obtenga última lectura del dispositivo actualmente activo para un usuario específico.



(a) Última lectura del dispositivo activo por el usuario obtenida con éxito - Código de estado 200

(b) No se encontró dispositivo activo para este usuario - Código de estado 404

Figura 6.66: Caso de prueba PU-M3.1-39

6.6.1.3.6. API de alertas

Tabla 6.13: Pruebas Unitarias - Endpoints de API de Alertas

ID	Endpoint	Método	Descripción	Auth	Resultado Obtenido
PU-M3.1-38	`/api/alerts`	GET	Listar alertas usuario	Sí	200: Lista de alertas 401: No autenticado
PU-M3.1-39	`/api/alerts/:id`	GET	Obtener alerta por ID	Sí	200: Datos de la alerta 404: Alerta no encontrada
PU-M3.1-40	`/api/alerts/:id/resolve`	PUT	Marcar alerta resuelta	Sí	200: Alerta marcada resuelta 404: Alerta no encontrada
PU-M3.1-41	`/api/alerts/:id`	DELETE	Eliminar alerta	Sí	200: Alerta eliminada 404: Alerta no encontrada
PU-M3.1-42	`/api/alerts/resolve-all`	PUT	Resolver todas alertas	Sí	200: Todas alertas resueltas 404: Sin alertas para resolver
PU-M3.1-43	`/api/alerts/user/:user_id`	GET	Alertas por usuario	Sí	200: Lista de alertas 400: No tienes permiso para ver estas alertas
PU-M3.1-44	`/api/alerts/admin/all`	GET	Obtener todas las alertas del sistema (Admin)	Sí	200: Todas las alertas del sistema 403: No es administrador

El caso de prueba de la Figura 6.68 verifica que todas las alertas se listen correc-

tamente.

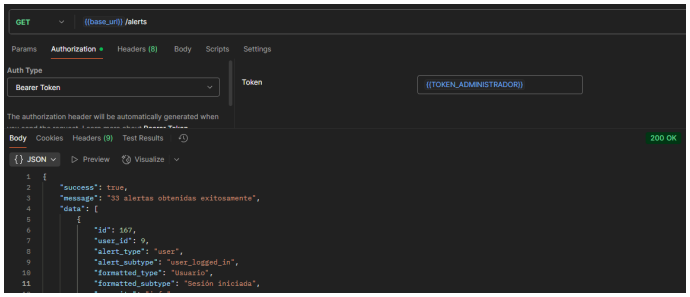
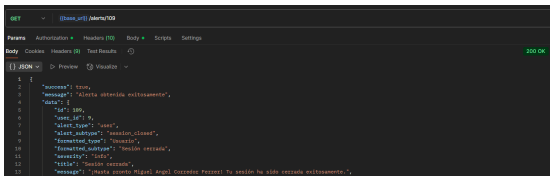


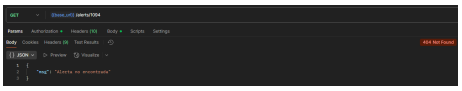
Figura 6.67: Alertas listadas con éxito - Código de estado 200

Figura 6.68: Caso de prueba PU-M3.1-38

El caso de prueba de la Figura 6.69 verifica que se obtenga la información de una alerta por su ID.



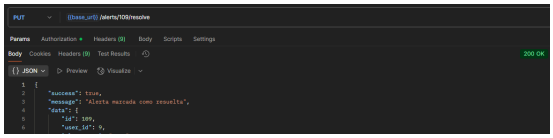
(a) Información de una alerta obtenida con éxito - Código de estado 200



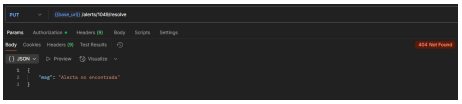
(b) Alerta no encontrada - Código de estado 404

Figura 6.69: Caso de prueba PU-M3.1-39

El caso de prueba de la Figura 6.70 verifica que se marque una alerta como resuelta.



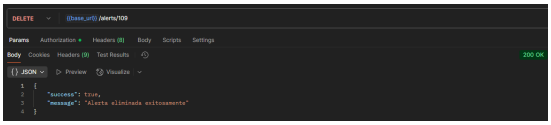
(a) Alerta marcada como resulta con éxito - Código de estado 200



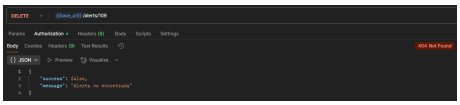
(b) Alerta no encontrada - Código de estado 404

Figura 6.70: Caso de prueba PU-M3.1-40

El caso de prueba de la Figura 6.71 verifica que una alerta se elimine correctamente.



(a) Alerta eliminada con éxito - Código de estado 200



(b) Alerta no encontrada - Código de estado 404

Figura 6.71: Caso de prueba PU-M3.1-41

El caso de prueba de la Figura 6.73 verifica que todas las alertas se resuelvan correctamente.

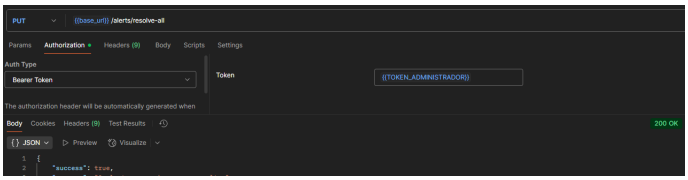
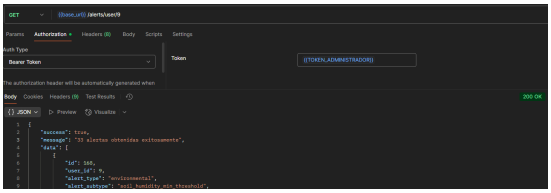


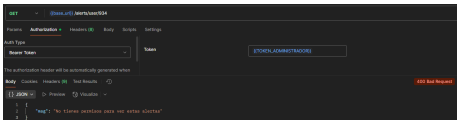
Figura 6.72: Alertas verificadas con éxito - Código de estado 200

Figura 6.73: Caso de prueba PU-M3.1-42

El caso de prueba de la Figura 6.112 obtener alertas para un usuario específico.



(a) Lista de alertas obtenidas con éxito - Código de estado 200



(b) No tienes permisos para ver estas alertas- Código de estado 400

Figura 6.74: Caso de prueba PU-M3.1-43

El caso de prueba de la Figura 6.76 verifica que se muestren todas las alertas del sistema.

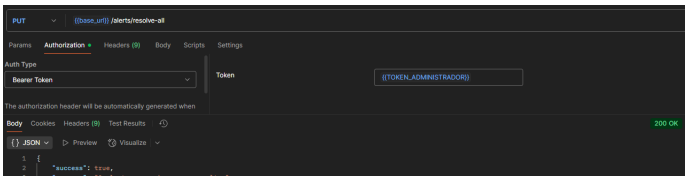


Figura 6.75: Todas las alertas del sistema listadas con éxito - Código de estado 200

Figura 6.76: Caso de prueba PU-M3.1-44

6.6.1.3.7. API de Configuración de riego

Tabla 6.14: Pruebas Unitarias - APIs de Control de Riego (Endpoints Principales)

ID	Endpoint	Método	Descripción	Auth	Resultado Obtenido
PU-M3.1-45	`/api/irrigation`	POST	Crear configuración de riego	Sí	201: Configuración creada 400: Datos inválidos
PU-M3.1-46	`/api/irrigation/:id`	GET	Obtener configuración por ID	Sí	200: Datos de configuración 404: Configuración no encontrada
PU-M3.1-47	`/api/irrigation/user/:user_id/active`	GET	Configuraciones activas por usuario	Sí	200: Configuraciones activas 403: Sin permisos
PU-M3.1-48	`/api/irrigation/:id/activate`	PUT	Activar configuración	Sí	200: Configuración activada 404: Configuración no encontrada
PU-M3.1-49	`/api/irrigation/:id/deactivate`	PUT	Desactivar configuración	Sí	200: Configuración desactivada 404: Configuración no encontrada
PU-M3.1-50	`/api/irrigation/pump-activation`	POST	Crear activación de bomba	Sí	201: Activación creada 409: Ya hay activación activa
PU-M3.1-51	`/api/irrigation/pump-activation/:id/pause`	PUT	Pausar activación	Sí	200: Activación pausada 400: No se puede pausar
PU-M3.1-52	`/api/irrigation/pump-activation/:id/resume`	PUT	Reanudar activación	Sí	200: Activación reanudada 400: No está pausada
PU-M3.1-53	`/api/irrigation/pump-activation/:id/complete`	PUT	Completar activación	Sí	200: Activación completada 404: Activación no encontrada

El caso de prueba de la Figura 6.76 verifica que esté creando una configuración de riego correctamente.

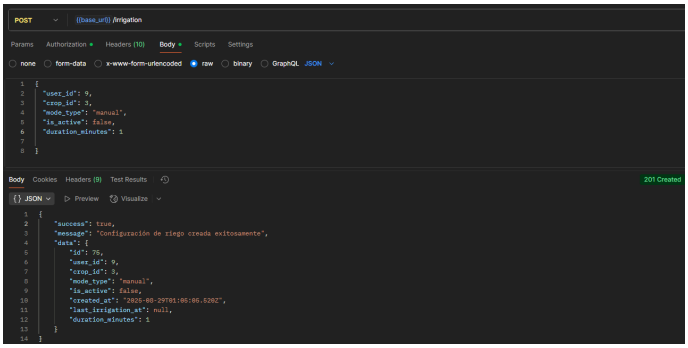
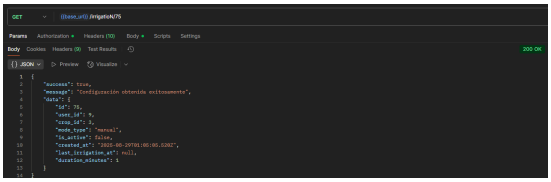


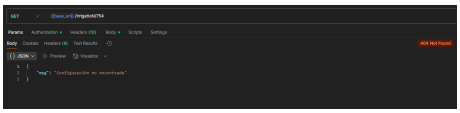
Figura 6.77: Configuración creada con éxito - Código de estado 200

Figura 6.78: Caso de prueba PU-M3.1-45

El caso de prueba de la Figura 6.79 verifica que se esté obteniendo una configuración concreta por su ID correctamente.



(a) Información de la configuración obtenida correctamente - Código de estado 200



(b) Configuración no encontrada - Código de estado 400

Figura 6.79: Caso de prueba PU-M3.1-46

El caso de prueba de la Figura 6.81 verifica que se estén obteniendo las configuraciones activas por usuario correctamente.

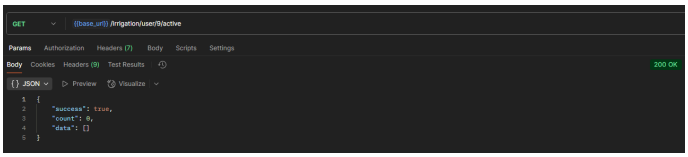


Figura 6.80: Configuración creada con éxito - Código de estado 200

Figura 6.81: Caso de prueba PU-M3.1-47

El caso de prueba de la Figura 6.83 verifica que se esté activando el riego correctamente en una configuración determinada.

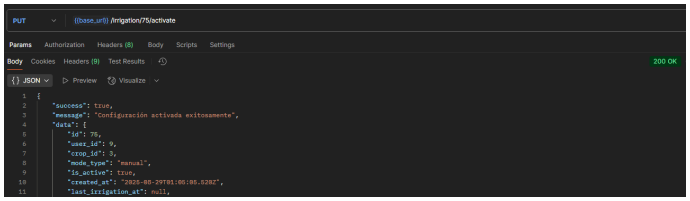


Figura 6.82: Riego activado correctamente - Código de estado 200

Figura 6.83: Caso de prueba PU-M3.1-48

El caso de prueba de la Figura 6.85 verifica que se esté activando el riego correctamente en una configuración determinada.

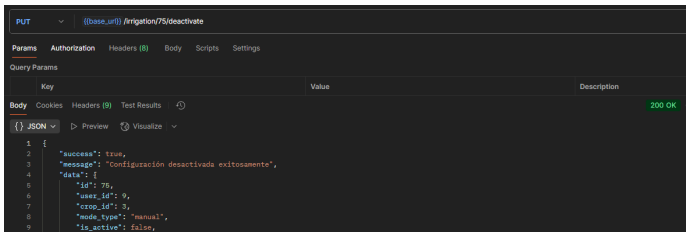


Figura 6.84: Riego desactivado correctamente - Código de estado 200

Figura 6.85: Caso de prueba PU-M3.1-49

El caso de prueba de la Figura 6.87 verifica que se esté creando una activación correctamente.

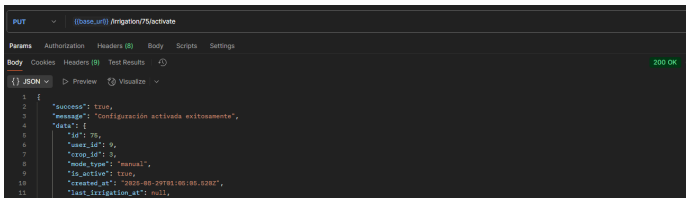


Figura 6.86: Riego desactivado correctamente - Código de estado 200

Figura 6.87: Caso de prueba PU-M3.1-50

El caso de prueba de la Figura 6.89 verifica que se haya pausado correctamente el riego.

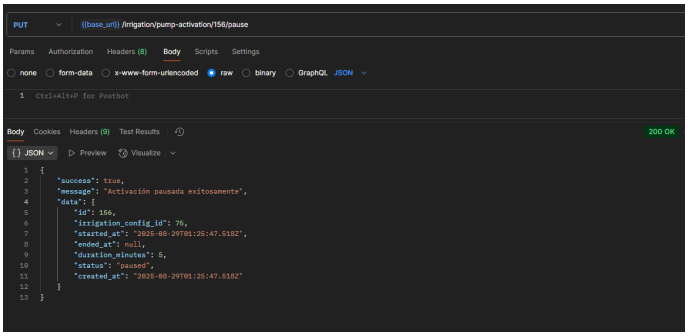


Figura 6.88: Riego pausado correctamente - Código de estado 200

Figura 6.89: Caso de prueba PU-M3.1-51

El caso de prueba de la Figura 6.91 verifica que se haya reanudado correctamente el riego.

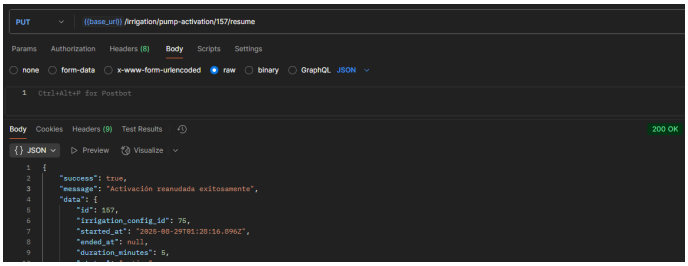


Figura 6.90: Riego reanudado correctamente - Código de estado 200

Figura 6.91: Caso de prueba PU-M3.1-52

El caso de prueba de la Figura 6.93 verifica que se haya completado correctamente el riego.

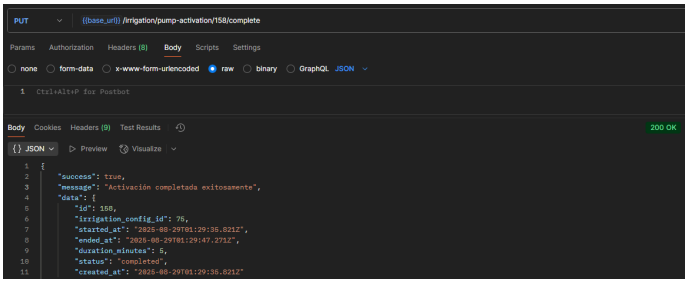


Figura 6.92: Riego completado correctamente - Código de estado 200

Figura 6.93: Caso de prueba PU-M3.1-53

6.6.1.4. Módulo 3.2: Cloud Computing - Almacenamiento de datos

6.6.1.4.1. Verificación de la correcta implementación de la tabla users

Tabla 6.15: Caso de prueba PU-M3.2-01: Verificación de la correcta implementación de la tabla users

Identificador	PU-M3.2-01
Descripción	Verificar que la tabla users se ha creado correctamente y tiene la estructura adecuada para almacenar la información sobre los usuarios
Precondiciones	Base de datos configurada y accesible
Pasos a seguir	1. Revisar que se haya definido correctamente la tabla en la base de datos 2. Comprobar las restricciones y claves primarias
Resultado esperado	La tabla debe de tener la estructura correcta, siguiendo el diseño de la Figura 5.10

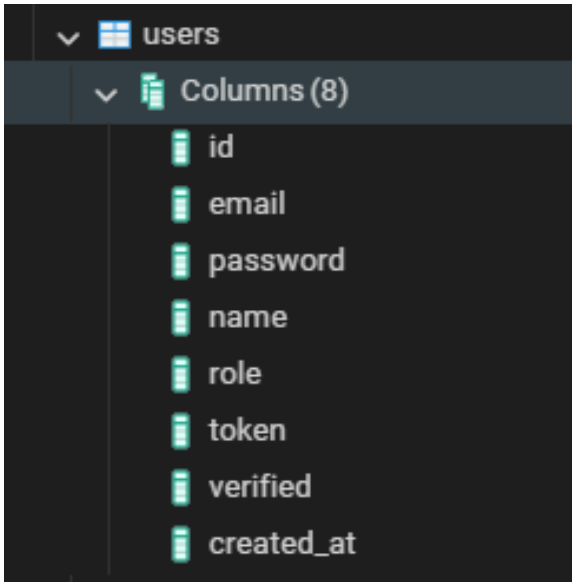


Figura 6.94: Correcta implementación de la tabla users

Figura 6.95: Caso de prueba PU-M3.2-01

6.6.1.4.2. Verificación de la correcta implementación de la tabla crops

Tabla 6.16: Caso de prueba PU-M3.2-02: Verificación de la correcta implementación de la tabla crops

Identificador	PU-M3.2-02
Descripción	Verificar que la tabla crops se ha creado correctamente y tiene la estructura adecuada para almacenar la información de los cultivos
Precondiciones	Base de datos configurada y accesible
Pasos a seguir	1. Revisar que se haya definido correctamente la tabla en la base de datos 2. Comprobar las restricciones y claves primarias
Resultado esperado	La tabla debe de tener la estructura correcta, siguiendo el diseño de la Figura 5.11

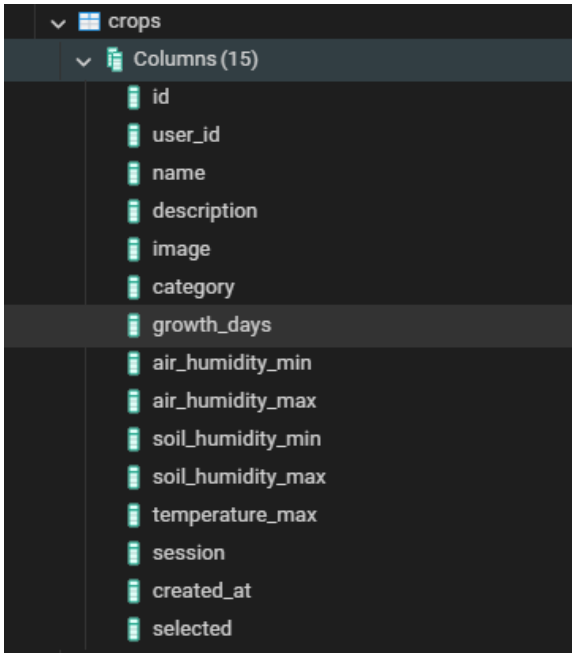


Figura 6.96: Correcta implementación de la tabla crops

Figura 6.97: Caso de prueba PU-M3.2-02

6.6.1.4.3. Verificación de la correcta implementación de la tabla devices

Tabla 6.17: Caso de prueba PU-M3.2-03: Verificación de la correcta implementación de la tabla devices

Identificador	PU-M3.2-03
Descripción	Verificar que la tabla devices se ha creado correctamente y tiene la estructura adecuada para almacenar la información de los dispositivos
Precondiciones	Base de datos configurada y accesible
Pasos a seguir	1. Revisar que se haya definido correctamente la tabla en la base de datos 2. Comprobar las restricciones y claves primarias
Resultado esperado	La tabla debe de tener la estructura correcta, siguiendo el diseño de la Figura 5.12

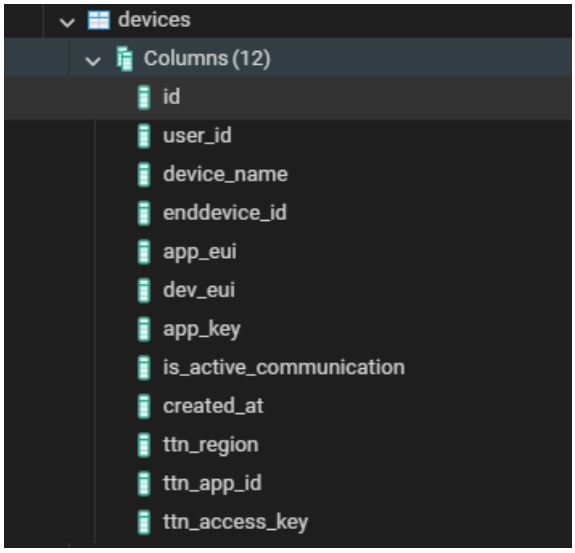


Figura 6.98: Correcta implementación de la tabla devices

Figura 6.99: Caso de prueba PU-M3.2-03

6.6.1.4.4. Verificación de la correcta implementación de la tabla irrigation_configs

Tabla 6.18: Caso de prueba PU-M3.2-04: Verificación de la correcta implementación de la tabla irrigation_configs

Identificador	PU-M3.2-04
Descripción	Verificar que la tabla irrigation_configs se ha creado correctamente y tiene la estructura adecuada para almacenar la información sobre las configuraciones de riego
Precondiciones	Base de datos configurada y accesible
Pasos a seguir	1. Revisar que se haya definido correctamente la tabla en la base de datos 2. Comprobar las restricciones y claves primarias
Resultado esperado	La tabla debe de tener la estructura correcta, siguiendo el diseño de la Figura 5.14

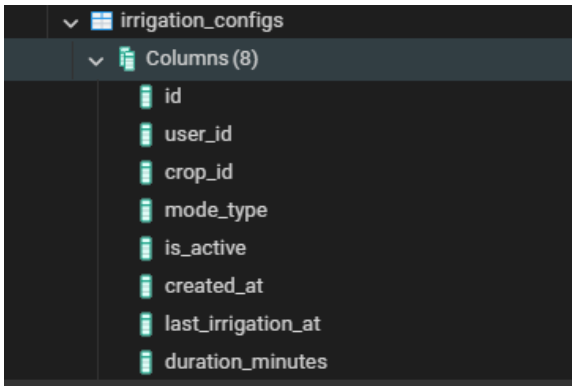


Figura 6.100: Correcta implementación de la tabla irrigation_configs

Figura 6.101: Caso de prueba PU-M3.2-04

6.6.1.4.5. Verificación de la correcta implementación de la tabla programmed_settings

Tabla 6.19: Caso de prueba PU-M3.2-05: Verificación de la correcta implementación de la tabla programmed_settings

Identificador	PU-M3.2-05
Descripción	Verificar que la tabla programmed_settings se ha creado correctamente y tiene la estructura adecuada para almacenar la información sobre las configuraciones de riego del modo programado
Precondiciones	Base de datos configurada y accesible
Pasos a seguir	1. Revisar que se haya definido correctamente la tabla en la base de datos 2. Comprobar las restricciones y claves primarias
Resultado esperado	La tabla debe de tener la estructura correcta, siguiendo el diseño de la Figura 5.15

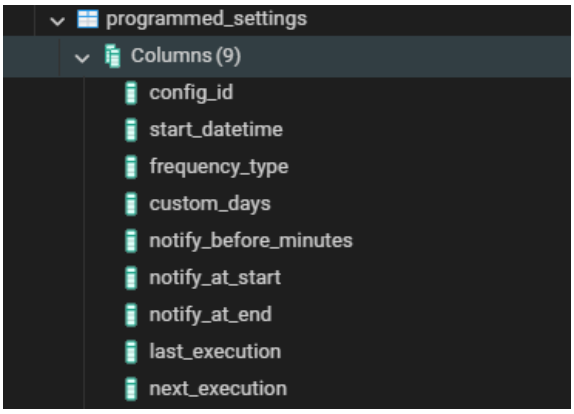


Figura 6.102: Correcta implementación de la tabla programmed_settings

Figura 6.103: Caso de prueba PU-M3.2-05

6.6.1.4.6. Verificación de la correcta implementación de la tabla automatic_settings

Tabla 6.20: Caso de prueba PU-M3.2-06: Verificación de la correcta implementación de la tabla automatic_settings

Identificador	PU-M3.2-06
Descripción	Verificar que la tabla automatic_settings se ha creado correctamente y tiene la estructura adecuada para almacenar la información sobre las configuraciones de riego del modo automático
Precondiciones	Base de datos configurada y accesible
Pasos a seguir	1. Revisar que se haya definido correctamente la tabla en la base de datos 2. Comprobar las restricciones y claves primarias
Resultado esperado	La tabla debe de tener la estructura correcta, siguiendo el diseño de la Figura 5.16

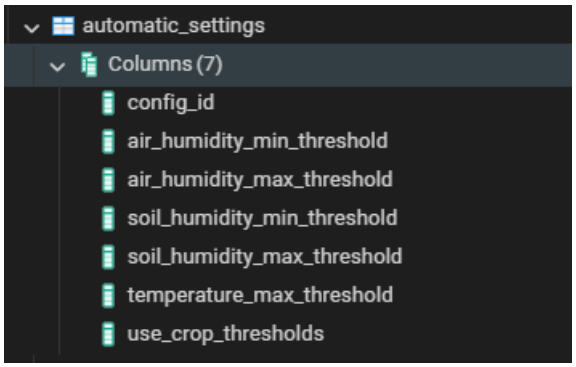


Figura 6.104: Correcta implementación de la tabla automatic_settings

Figura 6.105: Caso de prueba PU-M3.2-06

6.6.1.4.7. Verificación de la correcta implementación de la tabla pump_activations

Tabla 6.21: Caso de prueba PU-M3.2-07: Verificación de la correcta implementación de la tabla pump_activations

Identificador	PU-M3.2-07
Descripción	Verificar que la tabla pump_activations se ha creado correctamente y tiene la estructura adecuada para almacenar la información sobre las activaciones de riego
Precondiciones	Base de datos configurada y accesible
Pasos a seguir	1. Revisar que se haya definido correctamente la tabla en la base de datos 2. Comprobar las restricciones y claves primarias
Resultado esperado	La tabla debe de tener la estructura correcta, siguiendo el diseño de la Figura 5.17

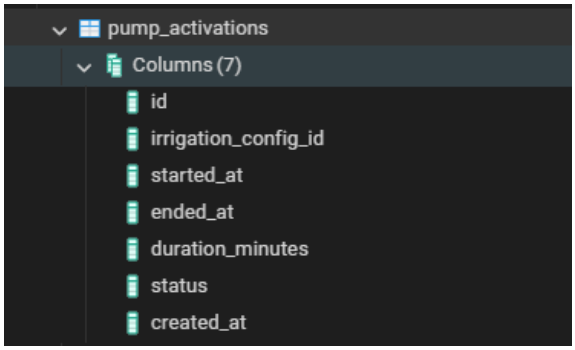


Figura 6.106: Correcta implementación de la tabla pump_activations

Figura 6.107: Caso de prueba PU-M3.2-07

6.6.1.4.8. Verificación de la correcta implementación de la tabla sensor_readings

Tabla 6.22: Caso de prueba PU-M3.2-08: Verificación de la correcta implementación de la tabla sensor_readings

Identificador	PU-M3.2-08
Descripción	Verificar que la tabla sensor_readings se ha creado correctamente y tiene la estructura adecuada para almacenar la información sobre los datos de los sensores enviados desde el dispositivo IoT
Precondiciones	Base de datos configurada y accesible
Pasos a seguir	1. Revisar que se haya definido correctamente la tabla en la base de datos 2. Comprobar las restricciones y claves primarias
Resultado esperado	La tabla debe de tener la estructura correcta, siguiendo el diseño de la Figura 5.13

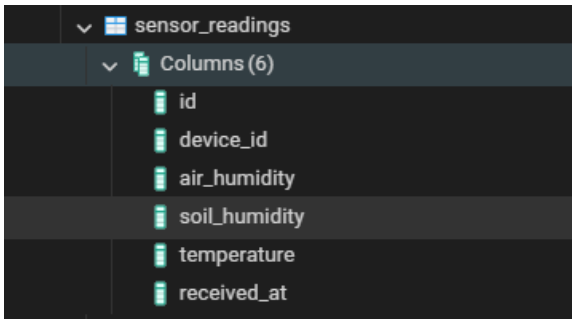


Figura 6.108: Correcta implementación de la tabla sensor_readings

Figura 6.109: Caso de prueba PU-M3.2-08

6.6.1.4.9. Verificación de la correcta implementación de la tabla alerts

Tabla 6.23: Caso de prueba PU-M3.1-09: Verificación de la correcta implementación de la tabla alerts

Identificador	PU-M3.2-09
Descripción	Verificar que la tabla alerts se ha creado correctamente y tiene la estructura adecuada para almacenar la información sobre las alertas del sistema
Precondiciones	Base de datos configurada y accesible
Pasos a seguir	1. Revisar que se haya definido correctamente la tabla en la base de datos 2. Comprobar las restricciones y claves primarias
Resultado esperado	La tabla debe de tener la estructura correcta, siguiendo el diseño de la Figura 5.18

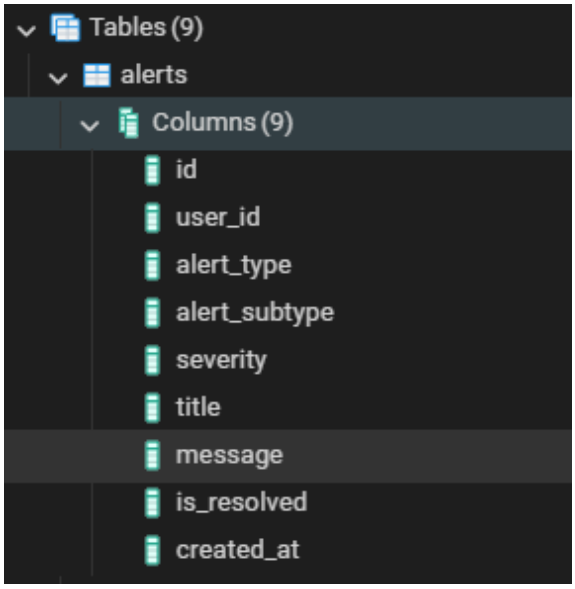


Figura 6.110: Correcta implementación de la tabla sql_alerts

Figura 6.111: Caso de prueba PU-M3.2-09

6.6.2. Pruebas de Integración

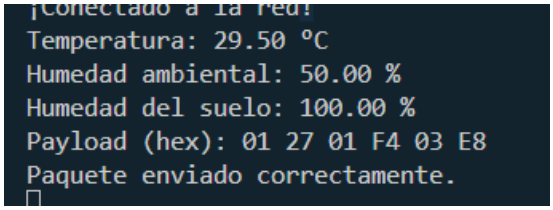
En esta sección analizaremos las pruebas que comprueban la interconexión entre los distintos módulos.

6.6.2.1. Integración del Módulo 1 y Módulo 2

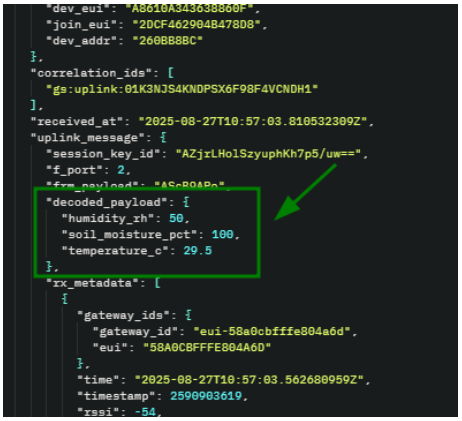
En esta prueba de integración se verificará que tanto el módulo 1 y módulo 2 funcionen correctamente en conjunto. Se comprobará que el módulo 1 se conecte al módulo 2 exitosamente con el objetivo de enviar lecturas de datos de sensores y recibir órdenes de riego.

Tabla 6.24: Caso de prueba PU-M1/M2-01: Verificación de la correcta integración entre el Módulo 1 y Módulo 2

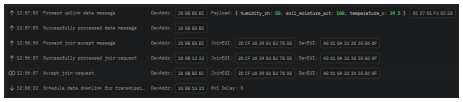
Identificador	PU-M1/M2-01
Descripción	Verificar que las lecturas tomadas del dispositivo IoT son enviadas al módulo 2 y que el módulo 2 envía las ordenes de riego al módulo 1
Precondiciones	Módulo 1 debe estar completamente operativo (sensores correctamente funcionando, actuador energizado, microcontrolador funcionando correctamente y gateway conectado) y el módulo 2 configurado (payload formatters configurados, API key no caducada, dispositivo y gateway dados de alta...)
Pasos a seguir	1. Alimentar módulo 1 con suficiente energía. 2. Comprobar que el módulo 2 ha sido configurado exitosamente 3. Capturar lectura de los sensores y comprobar en el módulo 2 la recepción. Además de verificar que la bomba de riego funcione correctamente cuando se envíe una orden de riego desde TTN.
Resultado esperado	Se espera que la lectura de los datos de los sensores se realice correctamente y aparezca en el panel de "Live data" TTN (módulo 2). Por otro lado también esperar, comprobar y también escuchar si la boomba de riego ha sido activada a través de la orden de riego proveniente de TTN.



(a) Datos enviados desde el dispositivo IoT



(b) Datos de los sensores enviados al live data correctamente



(c) Datos de los sensores en formato JSON

Figura 6.112: Caso de prueba PU-M1/M2-01

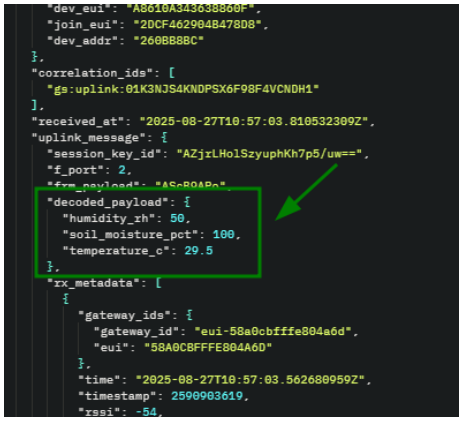
6.6.2.2. Integración del Módulo 2 y Módulo 3

En esta prueba de integración se verificará que tanto el módulo 2 cómo el módulo 3 funcionen correctamente en conjunto. Se comprobará que TTN reenvié correctamente los datos de los sensores al endpoint HTTP configurado en el webhook, que a su vez está definido en el módulo 3, específicamente en el módulo 3.1.

6.6.2.2.1. Integración del módulo 2 y módulo 3: Uplink

Tabla 6.25: Caso de prueba PU-M2/M3-01: Verificación de la correcta integración entre el Módulo 1 y Módulo 2 - Uplink

Identificador	PU-M2/M3-01
Descripción	Verificar el funcionamiento del webhook
Precondiciones	El webhook está configurado, activo y payloads formatters configurados
Pasos a seguir	1. Envío de de datos desde el módulo 1. 2. Decodificación de paquete recibido de bytes a JSON (conversión explícita). 3. Recolección y filtración de lecturas de datos de sensores e información de interés como el puerto y credenciales del dispositivo para posteriores validaciones 4. Reenviar lecturas al endpoint HTTP configurado en el webhook.
Resultado esperado	Recepción del paquete de datos (lectura de sensores + credenciales dispositivo + puerto) por parte del módulo 3 a través del webhook



(a) Datos de los sensores enviados al live data correctamente

(b) Datos de los sensores recibidios y postprocesados en el servidor web correctamente

Figura 6.113: Caso de prueba PU-M2/M3-01

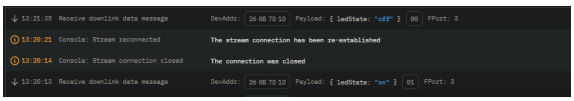
6.6.2.2.2. Integración del módulo 2 y módulo 3: downlink

Por otro lado tenemos otra prueba de integración que verificará que el módulo 2 recibirá los datos de control de riego emitidos desde el módulo 3, específicamente módulo 3.1.

Tabla 6.26: Caso de prueba PU-M2/M3-02: Verificación de la correcta integración entre el Módulo 1 y Módulo 2 - downlink

Identificador	PU-M2/M3-02
Descripción	Verificar que el módulo 2 reciba datos de control de riego desde el módulo 3.1
Precondiciones	La API Key debe estar generada y no caducada.
Pasos a seguir	1. Envío de datos de control de riego desde módulo 3.1. 2. codificación de paquete recibido de base64 a bytes (conversión ímplicita) . 3. Decodificación del paquete recibido de bytes a JSON, no tiene repercusión sobre la funcionalidad pero sirve a modo de traza para saber si se ha mandado un on/off. 4. Envío de orden de control del módulo 2 al módulo 1 tras recibir un uplink.
Resultado esperado	Recepción de alguna orden de control ya sea on/off desde el módulo 3.1

(a) Downlink enviado desde el módulo 3



(b) Downlink mostrado en Live Data de TTN

Figura 6.114: Caso de prueba PU-M2/M3-02

6.6.2.3. Integración del módulo 3.1 y módulo 3.2

A pesar de que la integración del módulo 3.1 (servidor web) y módulo 3.2 (almacén de datos) ya se sobreentiende puesto que son el mismo módulo. Sin embargo, hay que probar si estos sub-módulos se integran correctamente. No vamos a entrar en detalle sobre como interactúa cada una de una de las funcionalidades con el módulo 3.2 puesto que ya esto se ha probado exhaustivamente en la sección 6.6.1.3 del como cada una de las API's del sistema interactúan con la base de datos del sistema. Simplemente haremos una prueba sobre unas de las funcionalidades principales de la aplicación que es la dashboard para comprobar que se está almacenando correctamente las lecturas de los sensores en el módulo 3.2.

Tabla 6.27: Caso de prueba PU-M3.1/M3.2-01: Verificación de la correcta integración entre el Módulo 3.1 y Módulo 3.2

Identificador	PU-M3.1/M3.2-01
Descripción	Verificar que el módulo 3.1 (servidor web) ejecuta las acciones necesarias para almacenar la información en el módulo 3.2 (almacén de datos)
Precondiciones	El servidor web debe de estar operativo y la base de datos ya creada en PostSQL.
Pasos a seguir	1. El servidor web recibe los datos de los sensores a través del webhook configurado en TTN. 2. El servidor web verifica que el puerto desde donde llegan los datos es el 2 y además las credenciales del dispositivo desde dónde se envían coinciden con las del dispositivo actualmente activo. 3. El servidor guarda los datos de los sensores en la tabla sensorReadings.
Resultado esperado	La información debe de estar correctamente almacenada en el módulo 3.2

Figura 6.115: Preprocesamiento y guardado en la base de datos de los datos de los sensores

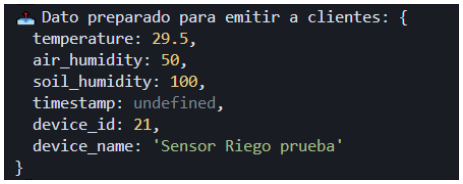
Figura 6.116: Caso de prueba PU-M3.1/M3.2-01

6.6.2.4. Integración del módulo 3 y módulo 4

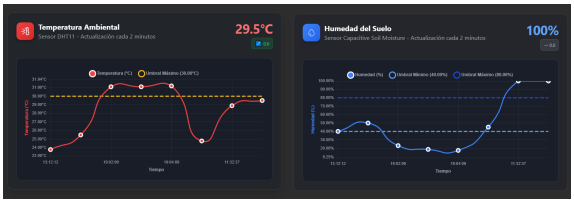
Cómo se ha comentado en la sección 6.6.2.3 el sistema cuenta con una gran cantidad de funcionalidades que ya han sido probadas en su mayoría en la sección 6.6.1.3. Por lo que probar una por una y su integración con el módulo 4 (interfaz web), sería alargar la presente sección excesivamente. Por lo que, funcionalidades relacionadas con la gestión de cultivos, gestión de usuarios, gestión de alertas, gestión de riegos y gestión de dispositivos se sobreentienden que funcionan. Por lo que vamos a probar lo esencial, es decir, el cómo se integra el módulo 3 y el módulo 4 para mostrar gráficamente las lecturas obtenidas desde los sensores y cómo se integra también para que gráficamente se pueda enviar la orden de riego al servidor web y el servidor web a TTN.

Tabla 6.28: Caso de prueba PU-M3/M4-01: Verificación de la correcta integración entre el Módulo 3 y módulo 4 - Uplink

Identificador	PU-M3/M4-01
Descripción	Verificar que el módulo 3 (servidor web) envía la lectura de los sensores al cliente o módulo 4 (interfaz web) correctamente y esta se gráfica correctamente
Precondiciones	El servidor web e interfaz web deben de estar operativas en puertos diferentes.
Pasos a seguir	1. El servidor web obtienen los datos de los sensores del almacén de datos. 2. El servidor web expone la API para la obtención de las últimas 20 lecturas de los sensores. 3. El usuario desde la aplicación web consume la API y se muestran gráficamente los datos.
Resultado esperado	Las lecturas de los sensores por parte del dispositivo se deben de mostrar gráficamente en la dashboard del usuario.



(a) Emitir lectura de sensores desde el servidor web al cliente



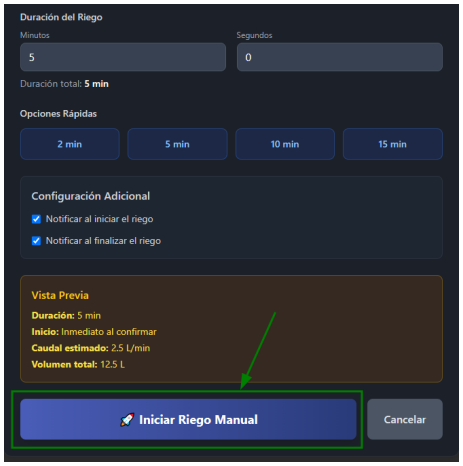
(b) Graficar lecturas de sensores en el lado del cliente

Figura 6.117: Caso de prueba PU-M3/M4-01

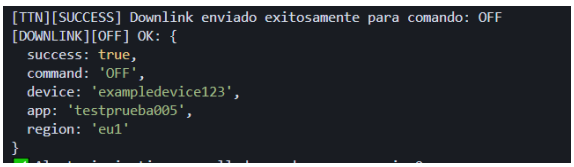
En la siguiente prueba de integración se verificará que el usuario desde la interfaz web pueda emitir una orden de riego hacia el servidor web y posteriormente el servidor web consultar la API de TTN para encolar el mensaje.

Tabla 6.29: Caso de prueba PU-M3/M4-02: Verificación de la correcta integración entre el Módulo 3 y módulo 4 - Downlink

Identificador	PU-M3/M4-02
Descripción	Verificar que el usuario que interactúa con el módulo 4 (interfaz web) emita una orden de riego
Precondiciones	El servidor web e interfaz web deben de estar operativas en puertos diferentes. Además la API Key no debe de estar caducada
Pasos a seguir	1. El usuario desde la interfaz web selecciona un modo de riego. 2. En cualquiera de los 3 modos de riego, cuando este comience o se detenga la interfaz web enviará al servidor web la orden de control de riego correspondiente on/off. 3. El servidor web consumirá la API expuesta por TTN para encolar el downlink con los datos de control de riego.
Resultado esperado	Orden de riego on/off encolada en TTN y lista para ser enviada al módulo 1.



(a) Inicio de riego en la interfaz web



(b) Recepción petición de riego en el servidor web, consume la API y TTN encola el mensaje

Figura 6.118: Caso de prueba PU-M3/M4-02

Capítulo 7

Conclusión

Este Trabajo de fin de grado ha sentado las bases teóricas y prácticas para montar un sistema de gestión de riego inteligente autónomo. Un sistema el cual puede funcionar de manera remota sin intervención de un agricultor, permitiendo un riego eficiente mediante la integración de sensores y microcontroladores con conectividad LoRaWAN. Por otro lado, proporciona un sistema de monitorización en tiempo real con el objetivo de ayudar al agricultor a que tenga un control exhaustivo sobre las condiciones climáticas y del suelo sobre el cultivo que se esta regando.

El proyeco combina tecnologías recientes como la arquitectura distribuida Edge-Fog-Cloud Computing, logrando que los datos captados en el espacio agrario donde se coloque el dispositivo se procesen, filtren y almacenen de forma eficiente para posteriormente ser mostrados en tiempo real en una dashboard y utilizados en la toma de decisiones del riego. La comunicación bidireccional con TTN (uplinks y downlinks) asegura que no solo se recopile información de temperatura y humedad ambiental y humedad del suelo, sino que también se puedan ejecutar acciones remotas como la activación o desactivación de la bomba de riego.

Para la implementación del sistema de gestión de riego inteligente se ha optado por 2 modos de operación de riego, modo manual, programado y automático.

- **Manual:** Ideal para riegos puntuales o de emergencia.
- **Programado:** Ideal para establecer rutinas de riego regulares
- **Automático:** Ideal para una gestión inteligente del cultivo en base a condiciones de riego

En definitiva, se trata de un proyecto cuyo objetivo es modernizar el sector agrícola mediante la tecnología IoT con el fin de reducir el consumo de agua y

hacer el riego de cultivos lo más eficiente posible. Debido a que este proyecto es un prototipo funcional se pueden introducir numerosas mejoras que han quedado pendientes para implementarlas en un futuro, las cuales veremos en la siguiente sección

7.1. Futuras mejoras

1. Implementación de una inteligencia artificial que calcule la cantidad de agua necesaria para un riego óptimo.

2. Diseño e implementación de una Printed Circuit Board (PCB) [63] para reducir el número de cables y optimizar el espacio del sistema en la caja estanca que lo protege.

3. Despliegue en cloud del módulo 3.

4. Implementación de un Large Language Model (LLM) [64] a partir del cual mediante una aplicación móvil puedas enviarle una imagen y que este te diga cuales son los umbrales óptimos de riego de cada cultivo. Además de proporcionarte información varía de ese cultivo.

5. Implementación de un sistema de notificaciones inteligente en el que un bot [65] de whatsapp o telegram informe sobre los eventos acontecidos en el sistema.

6. Implementación del cálculo de la métrica Vapor Pressure Deficit (VPD) [66] en función de los parámetros de temperatura y humedad ambiental para determinar las condiciones óptimas de humedad del suelo. Esta métrica estima el estrés hídrico de la planta y por tanto, permite optimizar la decisión de riego en función de las condiciones reales de transpiración del cultivo.

Bibliografía

[1] Internet de las cosas, 2025. URL https://www.redhat.com/es/topics/internet-of-things/what-is-iot. Consultado 10–08–2025.

[2] Mahamadou Tembely, Sarhan M. Musa, and otros. Internet of services. *International Journal of Advanced Research in Computer Science and Software Engineering*, 8(5):678, June 2018. doi: 10.23956/ijarcsse.v8i5.678.

[3] Mercado de IoT - Tamaño, crecimiento y tendencias, 2024. URL https://www.mordorintelligence.com/es/industry-reports/internet-of-things-iot-market. Consultado 01–08–2025.

[4] El impacto Del Internet De Las Cosas (IoT) En las Empresas , 2025. URL https://www.fi-group.es/es/internet-de-las-cosas-iot-en-empresas. Consultado 01–08–2025.

[5] EL LIDERAZGO DE SINGAPUR COMO CIUDAD INTELIGENTE , 2025. URL https://www.andinalinksmartcities.com/el-liderazgo-de-singapur-como-ciudad-inteligente/. Consultado 01–08–2025.

[6] ¿Cómo se distribuye el agua dulce y salada?, 2025. URL https://www.fundacionaquae.org/wiki/agua-dulce-salada/. Consultado 01–08–2025.

[7] El agua en la agricultura, 2025. URL https://www.bancomundial.org/es/topic/climate-resilient-irrigation. Consultado 01–08–2025.

[8] La agricultura, el "elefante en la habitación" que consume el 80 % del agua en la España de las sequías, 2024. URL https://www.rtve.es/noticias/20240217/agricultura-consume-80-agua-espana-sequias/15972242.shtml. Consultado 01–08–2025.

[9] Tecnología LoRA y LoRAWAN, 2025. URL https://www.catsensors.com/es/lorawan/tecnologia-lora-y-lorawan. Consultado 03–08–2025.

[10] Bomba de agua acomplamiento directo, Caudal Máximo 0.4l/min, Alimentación 3-6V, 2025. URL https://es.rs-online.com/web/p/bombas-de-agua/8170716?searchId=154ca0b8-111a-4e2a-bddc-b2c0fabd4575&gb=s. Consultado 10–08–2025.

[11] Cómo utilizar el sensor DHT11 para medir la temperatura y humedad con Arduino, 2025. URL https://programarfacil.com/blog/arduino-blog/sensor-dht11-temperatura-humedad-arduino/. Consultado 03–08–2025.

[12] Sensor de Humedad de Suelo Capacitivo v1.2, 2025. URL https://naylampmechatronics.com/sensores-temperatura-y-humedad/538-sensor-de-humedad-de-suelo-capacitivo-v1.html. Consultado 03–08–2025.

[13] ¿Qué es FIWARE?, 2025. URL https://catedratelefonica.unex.es/fiware-el-estandar-que-necesita-el-internet-de-las-cosas/. Consultado 10–08–2025.

[14] El Relé: para qué es, para qué sirve y qué tipos existen, 2025. URL https://www.seas.es/blog/automatizacion/el-rele-para-que-es-para-que-sirve-y-que-tipos-existen/. Consultado 10–08–2025.

[15] ¿Qué es Edge Computing, explicado de manera sencilla?, 2025. URL https://telefonicatech.com/blog/edge-computing-que-es. Consultado 03–08–2025.

[16] ¿Qué es Fog Computing y en qué se diferencia del Edge Computing?, 2025. URL https://www.arsys.es/blog/fogcomputing. Consultado 03–08–2025.

[17] ¿Qué es el Cloud Computing?, 2025. URL https://www.ibm.com/es-es/think/topics/cloud-computing. Consultado 03–08–2025.

[18] Sistema de riego BRESSER Smart Garden Smart Home, 2025. URL https://www.bresser.es/p/sistema-de-riego-bresser-smart-garden-smart-home-7510100. Consultado 10–08–2025.

[19] Amazon, 2025. URL https://developer.amazon.com/es-ES/alexa. Consultado 10–08–2025.

[20] Google Assistant, 2025. URL https://assistant.google.com/intl/es_es/. Consultado 10–08–2025.

[21] Programador de riego Solem LoRa de 9V, 4 estaciones de riego, 2025. URL https://www.leroymerlin.es/productos/programador-de-riego-solem-lora-de-9v-4-estaciones-de-riego-89721014.html?highlightedOfferCode=45b001589b080b645bf1429f97d74fd28d0d5377. Consultado 10–08–2025.

[22] ¿Qué es bluetooth?, 2025. URL https://www.lenovo.com/es/es/glossary/what-is-bluetooth/. Consultado 10–08–2025.

[23] MKR WAN 1310, 2025. URL https://docs.arduino.cc/hardware/mkr-wan-1310/. Consultado 03–08–2025.

[24] Gateway LoRaWAN, 2025. URL https://s-connect.es/Productos/Red-industrial/Gateway-LoRaWAN/. Consultado 03–08–2025.

[25] The Things Network, 2025. URL https://www.thethingsnetwork.org/. Consultado 03–08–2025.

[26] PostgreSQL, 2025. URL https://www.postgresql.org/. Consultado 10–08–2025.

[27] Uplink, 2025. URL https://www.thethingsindustries.com/docs/integrations/payload-formatters/javascript/uplink/. Consultado 03–08–2025.

[28] HTTP, 2025. URL https://developer.mozilla.org/es/docs/Web/HTTP. Consultado 03–08–2025.

[29] Downlink, 2025. URL https://www.thethingsindustries.com/docs/integrations/payload-formatters/javascript/downlink/. Consultado 03–08–2025.

[30] API Reference TTN, 2025. URL https://www.thethingsnetwork.org/docs/applications/mqtt/api/. Consultado 03–08–2025.

[31] ¿Qué son las Single-page application (SPA)? El desarrollo elegido por Gmail y LinkedIn, 2025. URL https://digital55.com/blog/que-son-single-page-application-spa-desarrollo-elegido-por-gmail-linkedin/. Consultado 03–08–2025.

[32] Nuxt.js, 2025. URL https://nuxt.com/. Consultado 03–08–2025.

[33] Vue.js, 2025. URL https://vuejs.org/. Consultado 03–08–2025.

[34] Over the Air Activation (OTAA), 2025. URL https://www.thethingsindustries.com/docs/hardware/devices/adding-devices/manual/otaa/. Consultado 10–08–2025.

[35] Todo lo que necesitas saber sobre Base64, 2025. URL https://blog.logto.io/es/todo-sobre-base64. Consultado 03–08–2025.

[36] Introduction to JSON Web Tokens, 2025. URL https://www.jwt.io/introduction. Consultado 03–08–2025.

[37] Qué es C++ y su importancia en la programación moderna, 2025. URL https://openwebinars.net/blog/que-es-cpp/. Consultado 10–08–2025.

[38] PlatformIO: Introducción, 2025. URL https://www.linkedin.com/pulse/platformio-introducci%C3%B3n-diego-hinojosa-c%C3%B3rdova/. Consultado 10–08–2025.

[39] ¿Qué es Visual Studio Code y cuáles son sus ventajas?, 2025. URL https://www.arsys.es/blog/que-es-visual-studio-code-y-cuales-son-sus-ventajas. Consultado 10–08–2025.

[40] MKRWAN Library, 2025. URL https://registry.platformio.org/libraries/arduino-libraries/MKRWAN. Consultado 10–08–2025.

[41] ¿Qué es JavaScript?, 2025. URL https://developer.mozilla.org/es/docs/Learn_web_development/Core/Scripting/What_is_JavaScript. Consultado 10–08–2025.

[42] Node.js, 2025. URL https://nodejs.org/es. Consultado 03–08–2025.

[43] Express.js, 2025. URL https://expressjs.com/. Consultado 03–08–2025.

[44] ¿Qué es Tailwindcss?, 2025. URL https://openwebinars.net/blog/que-es-tailwind-css-y-por-que-deberias-usarlo/. Consultado 10–08–2025.

[45] ¿Qué es css?, 2025. URL https://lenguajecss.com/css/introduccion/que-es-css/. Consultado 10–08–2025.

[46] ¿Qué es una API REST?, 2025. URL https://www.redhat.com/es/topics/api/what-is-a-rest-api. Consultado 10–08–2025.

[47] npm, 2025. URL https://www.arsys.es/blog/que-es-npm-javascript-para-principiantes. Consultado 10–08–2025.

[48] ¿Qué es https?, 2025. URL https://www.cloudflare.com/es-es/learning/ssl/what-is-https/. Consultado 10–08–2025.

[49] ¿Qué es bcrypt?, 2025. URL https://www.cloudflare.com/es-es/learning/ssl/what-is-https/. Consultado 10–08–2025.

[50] Caja de conexión estanca IP55 170x220x85 mm, 2025. URL https://www.leroymerlin.es/productos/caja-de-conexion-estanca-ip55-170x220x85-mm-82049186.html. Consultado 10–08–2025.

[51] Download Window 10 Home, 2025. URL https://www.microsoft.com/es-es/software-download/windows10. Consultado 10–08–2025.

[52] Overleaf, 2025. URL https://es.overleaf.com/. Consultado 10–08–2025.

[53] Requisitos funcionales: ¿Qué son y cómo funcionan?, 2025. URL https://www.gluo.mx/blog/requisitos-funcionales-que-son-y-como-funcionan. Consultado 03–08–2025.

[54] Requerimientos no funcionales: Ejemplos, 2025. URL https://www.pmoinformatica.com/2015/05/requerimientos-no-funcionales-ejemplos.html. Consultado 03–08–2025.

[55] ¿Qué es el lenguaje unificado modelado?, 2025. URL https://www.lucidchart.com/pages/es/que-es-el-lenguaje-unificado-de-modelado-uml. Consultado 10–08–2025.

[56] Device Classes, 2025. URL https://www.thethingsnetwork.org/docs/lorawan/classes/. Consultado 10–08–2025.

[57] ¿Qué es un diagrama de secuencia en UML?, 2025. URL https://www.lucidchart.com/pages/es/diagrama-de-secuencia. Consultado 10–08–2025.

[58] ¿Qué es Modelo-Vista-Controlador?, 2025. URL https://victorroblesweb.es/2013/11/18/tutorial-mvc-en-php-nativo/. Consultado 10–08–2025.

[59] vivantia-tfg-microcontrolador, 2025. URL https://github.com/miguelangelcorredorferrer/vivantia-tfg-microcontrolador. Consultado 10–08–2025.

[60] Mailtrap, 2025. URL https://mailtrap.io/es/. Consultado 10–08–2025.

[61] ¿Qué es Nuxt.js y cuáles son sus características?, 2025. URL https://www.arsys.es/blog/que-es-nuxt-js-y-cuales-son-sus-caracteristicas. Consultado 10–08–2025.

[62] postman, 2025. URL https://www.postman.com/. Consultado 10–08–2025.

[63] ¿Qué es un PCB o Placa de Circuito Impreso?, 2025. URL https://resources.altium.com/es/p/what-is-a-pcb. Consultado 10–08–2025.

[64] ¿Qué es un LLM (modelo de lenguaje de gran tamaño)?, 2025. URL https://aws.amazon.com/es/what-is/large-language-model/. Consultado 10–08–2025.

[65] ¿Qué es un bot?, 2025. URL https://aws.amazon.com/es/what-is/bot/. Consultado 10–08–2025.

[66] ¿Qué es VPD (Déficit de Presión de Vapor)?, 2025. URL https://drygair.com/es/blog-es/vpd-vapor-pressure-deficit/. Consultado 10–08–2025.

[67] Repositorio Github del servidor Web, 2025. URL https://github.com/miguelangelcorredorferrer/vivantia-tfg-backend. Consultado 10–08–2025.

[68] Repositorio Github del frontend o cliente, 2025. URL https://github.com/miguelangelcorredorferrer/vivantia-tfg-frontend. Consultado 10–08–2025.

[69] Applications, 2025. URL https://www.thethingsnetwork.org/docs/applications/. Consultado 10–08–2025.

[70] ngrok, 2025. URL https://ngrok.com/. Consultado 10–08–2025.

[71] PgAdmin, 2025. URL https://www.pgadmin.org/. Consultado 10–08–2025.

Anexo A

Manual de Usuario

A.1. Introducción

Este manual proporciona una guía completa para la correcta utilización del sistema de gestión inteligente de riego, al que se le ha dado el nombre de Vivantia. El documento presenta instrucciones detalladas que permitirán tanto a usuarios técnicos como finales, implementar y operar el sistema de manera eficaz.

En las siguientes secciones el usuario de la aplicación encontrará información exhaustiva sobre los componentes del sistema, los procedimientos de configuración inicial, las instrucciones de uso diario y las tareas de mantenimiento necesarias. El manual está estructurado de forma progresiva, comenzando con los requisitos fundamentales y avanzando hacia funcionalidades más específicas.

El sistema se compone de cuatro módulos principales que trabajan en conjunto para proporcionar las funcionalidades necesaria para la gestión inteligente de riego. Este documento describe como utilizar e implementar cada componente para que el sistema funcione de forma óptima.

A.2. Requisitos del sistema

A.2.1. Requisitos Técnicos

En este apartado se detallan los requisitos técnicos que el sistema necesita para poder ser instalado y ejecutado sin ningún contratiempo.

A.2.1.1. Módulo 1: Edge Computing - Captación de datos y acción sobre el actuador

- Arduino MKR WAN 1310, equipado con módulo LoRa y LoRaWAN, para capturar, transmitir y recibir datos.
- HUAREW 5V El aislamiento del optoacoplador del módulo de relé de 1 canal activación, como interruptor de la bomba de riego.
- Visual Studio Code como IDE, junto con extensión PlatformIO.
- Librería MKRWAN en su última versión.
- Librería oficial de Arduino en su última versión.
- Librería DHT Sensor de Adafruit en su última versión.
- D260BLX-V Brushless Micropump Range como bomba de riego
- Gemtek / MiniHub v1.5 Wifi Indoor LoRaWAN Gateway, necesario para garantizar el envío de datos de Arduino MKR WAN 1310 mediante LoRa y LoRaWAN.
- Sensor de humedad y temperatura ambiental DHT11.
- Sensor de humedad del suelo Capacitive Soil Moisture v1.2.
- Acceso al repositorio del código del microcontrolador [59]

A.2.1.2. Módulo 2: Fog Computing - Recolección y preprocesamiento

- Acceso con una cuenta a la platforma The Things Network, previamente dada de alta.
- Configuración del decoder Uplink/Downlink Javascript personalizado.
- Configuración del webhook para la transmisión de información al siguiente módulo.
- Creación de la API Key para el uso de la API de The Things Network.

A.2.1.3. Módulo 3.1: Cloud Computing - Recepción y postprocesamiento

- Entorno de ejecución Javascript de Node.js en su última versión.
- Framework Express.js en su última versión para el desarrollo de la API REST.
- Cliente PostgreSQL para Node.js (pg) en su última versión.
- Middleware de autenticación JWT para la gestión de sesiones seguras.
- Librería bcryptjs para el hash seguro de contraseñas.
- Middleware cors para el manejo de solicitudes cross-origin.
- Librería dotenv para la gestión de variables de entorno.
- Librería nodemailer para el envío de emails de verificación y notificaciones.
- Librería date-fns para el manejo de fechas y zonas horarias.
- Visual Studio Code como entorno de desarrollo integrado IDE.
- Acceso al repositorio github del servidor web [67].

A.2.1.4. Módulo 3.2: Cloud Computing - Almacenamiento de datos

- Sistema de gestión de base de datos relacional PostgreSQL en su última versión.
- pgAdmin como interfaz gráfica de usuario (GUI) para la administración de PostgreSQL.

A.2.1.5. Módulo 4: Interfaz Web - Visualización

- Entorno de ejecución Node.js en su última versión (v18 o superior).
- Framework Nuxt.js 3 en su última versión para el desarrollo de la aplicación SPA.
- Framework Vue.js 3 en su última versión para el desarrollo de componentes reactivos.
- Librería Tailwind CSS para el diseño y estilizado de la interfaz de usuario.
- Librería PostCSS para el procesamiento de estilos CSS.

- Librería Autoprefixer para la compatibilidad cross-browser de CSS.
- Librería @nuxtjs/tailwindcss para la integración de Tailwind CSS con Nuxt.js.
- Librería Chart.js para la visualización de gráficos y datos en tiempo real.
- Librería vue-chartjs para la integración de Chart.js con Vue.js.
- Librería date-fns para el manejo y formateo de fechas en el frontend.
- Librería Prettier para el formateo automático del código.
- Visual Studio Code como entorno de desarrollo integrado IDE.
- Acceso al repositorio de github del frontend [68].
- Navegador web moderno con soporte para JavaScript ES6+ (Chrome, Firefox, Safari, Edge).
- Sistema de gestión de paquetes npm para la instalación de dependencias.

A.2.2. Requisitos Finales

En este apartado se describen los requisitos necesarios que el usuario final debe disponer para que el sistema desarrollado pueda funcionar correctamente.

A.2.2.1. Módulo 1: Edge Computing - Captación de datos y acción sobre el actuador

Teniendo en cuenta que tenemos el dispositivo con el código ya introducido y configurado. Los requisitos que el usuario final tiene que satisfacer para que este módulo funcione correctamente son:

- Alimentación eléctrica para el dispositivo IoT y gateway de soporte, mediante:
  - Batería externa de 20000 mAh (incluida en el sistema) para autonomía del dispositivo.
- Batería externa de 20000 mAh (incluida en el sistema) para autonomía del dispositivo.
  - Router doméstico o empresarial
  - Dispositivo móvil actuando como punto de acceso (hotspot)

En el caso particular desarrollado se ha usado alimentación mediante batería externa para el dispositivo IoT y el gateway conectado a una red WiFi doméstica.

A.2.2.2. Módulo 2: Fog Computing - Recolección y preprocesamiento

Este módulo no requiere de instalación puesto que está completamente configurado en la nube mediante TTN. El usuario final no requiere de configuración adicional, ya que:

- Los dispositivos tienen que estar dados de alta y configurados en TTN.
- Los decoders para Uplink/Downlink están implementados en lenguaje Javascript.
- El webhook que reenvía los datos al servidor web del módulo 3.1 está configurado.
- La API Key está creada y disponible para uso.

A.2.2.3. Módulo 3.1: Cloud Computing - Recepción y postprocesamiento

- Se requiere la última versión de PostgreSQL

A.2.2.4. Módulo 3.2: Cloud Computing - Almacenamiento de datos

No quiere de configuración por parte del usuario final, ya que:

- La base de datos PostgreSQL está configurada automáticamente.
- Las tablas y relaciones están creadas.
- Los permisos de acceso están establecidos.

A.2.2.5. Módulo 4: Interfaz Web - Visualización

Para acceder al sistema de visualización, el usuario final necesita:

- Navegador web moderno (chrome, Firefox, Safari, Edge) con Javascript habilitado
- Conexión a Internet para acceder a la aplicación desplegada en cloud (cambiar por vercel)
- Registrarse en la plataforma

A.2.2.6. Funcionamiento del Sistema Completo

Para utilizar el sistema completo, el usuario debe

1. Asegurar la alimentación completa del módulo 1 (dispositivo + Gateway).
2. Conectar el gateway a Internet vía WiFi.
3. Esperar aproximadamente 1-2 minutos para que el sistema de riego se inicie y comience a mandar datos y estar disponible para la recepción de datos desde TTN.
4. Acceder a la interfaz web usando el navegador.
5. Registrarse en el sistema
6. El sistema está listo para utilizar.

A.3. Instalación y configuración para el usuario técnico

Para facilitar la comprensión de este capítulo vamos a dividir la explicación en módulos

A.3.1. Módulo 1: Edge Computing - Captación de datos y acción sobre el actuador

A.3.1.1. Montaje del dispositivo

En la Figura 6.1a podemos observar como se deben de implementar los componentes en el sistema. Por otro lado y para evitar dudas, en la Figura 5.26 podemos observar un esquema más visual dónde se especifica el conexionado aún más.

A.3.1.2. Entorno de programación y código del programa

En primer lugar, se debe de instalar el entorno Visual Studio Code como IDE, para ello tenemos que acceder a su página oficial.

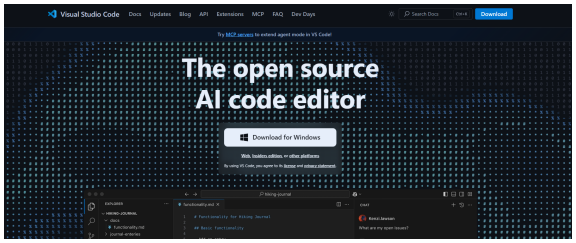
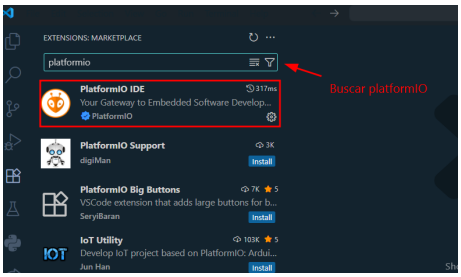


Figura A.1: Visual Studio Code Download

En la Figura A.1 observamos la página oficial de Visual Studio Code y simplemente tendremos que pulsar "Download for Windows" para descargar el ejecutable para nuestro sistema operativo.

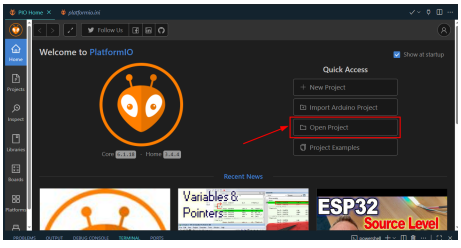
Una vez tengamos el IDE, debemos de descargar el plugin PlatformIO para posteriormente cargar el código desde el repositorio GitHub del proyecto. En este caso debemos de descargarnos el contenido de la carpeta del proyecto de código del microcontrolador [59].

(a) Seleccionar Extensiones



(b) Buscador de extensiones

(c) Instalar PlatformIO



(d) Platform Open project

Figura A.2: Preparación del entorno en VScode para la programación del microcontrolador

La descarga del proyecto está representado en la Figura A.3.

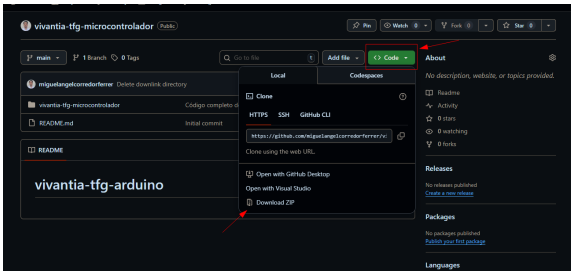
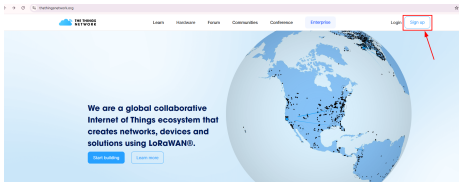


Figura A.3: Descargar código del microcontrolador

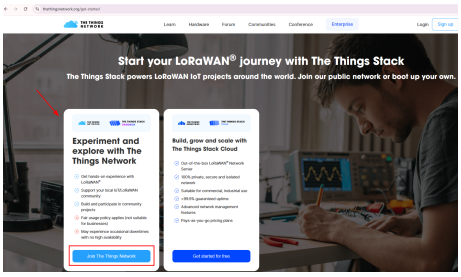
A.3.2. Módulo 2: Fog Computing - Recolección y preprocesamiento

A.3.2.1. Registro y login en el entorno TTN

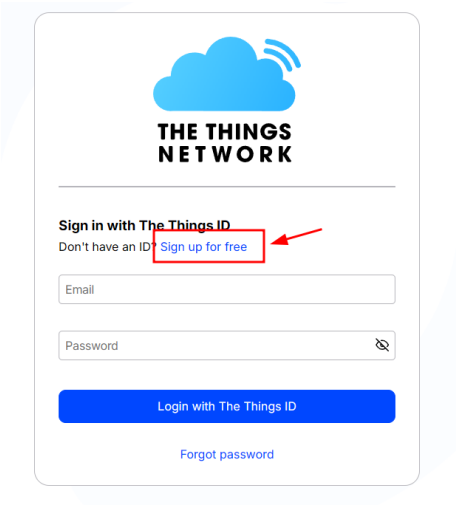
En este apartado se mostrará paso por paso como registrarse en el entorno TTN y una vez creada la cuenta acceder a la consola de TTN. En la Figura A.4 vemos paso por paso como registrarse en TTN



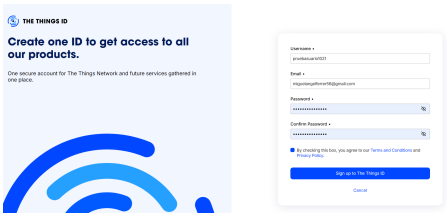
(a) Registro de cuenta en TTN



(b) Seleccionar el plan de TTN



(c) Registro como cuenta gratis en TTN

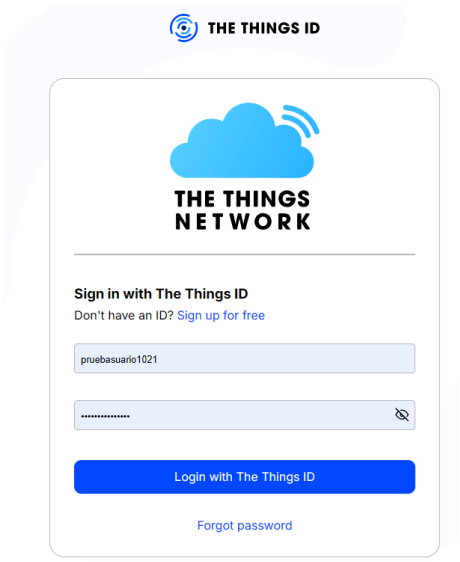


(d) Formulario de registro de TTN

Figura A.4: Registro en TTN

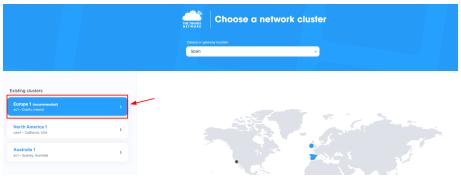
A.3.2.2. Login y acceso a TTN

Una vez el usuario se haya conectado en la plataforma, podrá loguearse y acceder a la consola de TTN introduciendo el cluster en el que va a operar el servidor de red de TTN. Esto verse representado en la Figura A.5



(a) Login en TTN

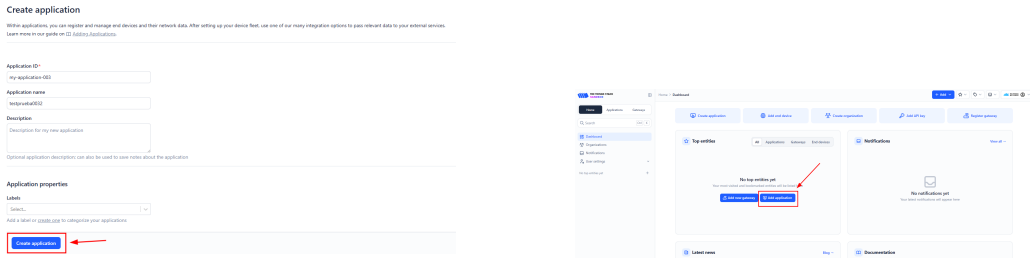
(b) Seleccionar consola de TTN



(c) Seleccionar el cluster (europeo)

Figura A.5: Login y acceso al servidor de red de TTN

Una vez el usuario se haya logueado, accederemos a la consola de TTN donde se creará el primer Application [69]. Un Application es una entidad lógica que agrupa y organiza nodos LoRaWAN o dispositivos IoT que comparten un mismo propósito o servicio. En resumen, es la capa lógica en TTN que recibe los datos de los dispositivos y los reenvía al servidor web de la aplicación a través de un webhook. Esto se ve representado en la Figura A.6.



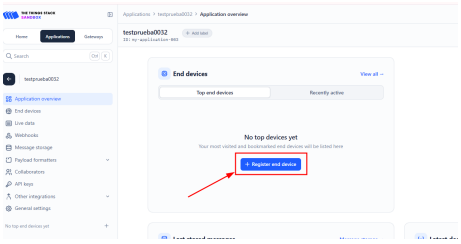
(a) Crear aplicación

(b) Añadir aplicación

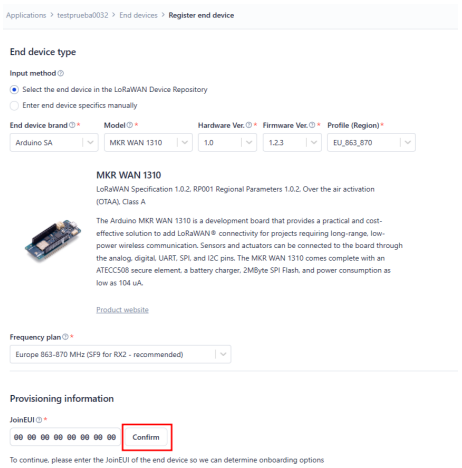
Figura A.6: Crear una aplicación

A.3.2.3. Dar de alta un dispositivo en TTN

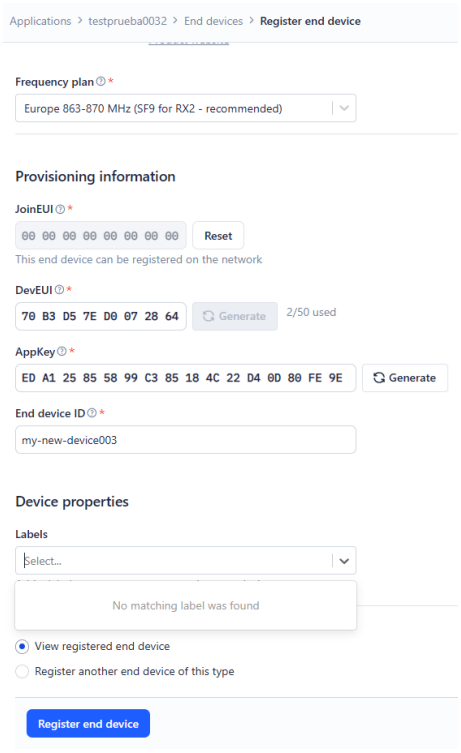
En la Figura A.7 observamos los pasos para dar de alta el dispositivo IoT, en la subFigura A.7b podemos ver que lo primero que se hace para dar de alta un dispositivo en TTN es seleccionar el tipo de dispositivo que es, posteriormente se selecciona el tipo de frecuencia junto con el joineui que será distinto al especificado en el manual, ya que este será un identificador de la aplicación concreta del dispositivo y debe ser único (en nuestro caso será el 0000000000000000). El resto de campos se generarán automáticamente como se puede observar en la subFigura A.7c. Con esto ya estaría el dispositivo dado de alta en TTN.



(a) Registro del dispositivo IoT



(b) Registro del dispositivo IoT y configuración)(1)

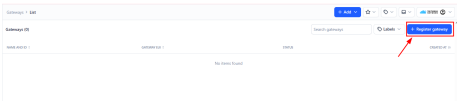


(c) Registro del dispositivo IoT y configuración)(2)

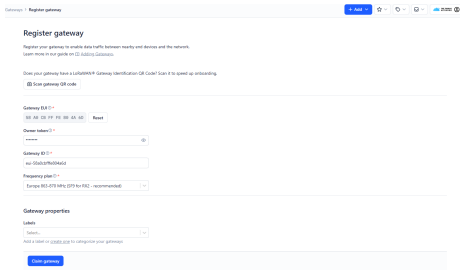
Figura A.7: Dar de alta un dispositivo IoT

A.3.2.4. Registro del gateway

Después de haber dado de alta el dispositivo IoT, tenemos que registrar también el gateway. En la Figura A.8 podemos observar el proceso de registro del gateway paso por paso.



(a) Registrar Gateway



(b) TTN formulario para registrar gateway

Figura A.8: Registrar gateway en TTN

Como se puede observar en la subfigura A.26b se deben de completar una serie de campos para registrar el gateway. Estos campos son:

1. **Gateway EUI:** Valor 1 de la Figura A.9
2. **Owner Token:** Valor 2 de la Figura A.9
3. **Gateway Id:** Escogido por el usuario y debe ser único
4. **Frecuency plan** Depende del país, en Europa utilizamos Europe 868.1 Mhz

Figura A.9: Etiqueta de información específica del gateway

Después de dar de alta el gateway en TTN, nos queda un último paso en la configuración de este. Este último paso consiste en asignarle una conexión WiFi al gateway para que pueda servir como puerta de enlace LoRaWAN hacia TTN. La

WiFi será la que deseemos ya sea una red fija o los datos móviles de un teléfono inteligente. Para ello tenemos que aimentar el dispositivo ya sea mediante alimentación directa a la corriente eléctrica o mediante una batería portátil al puerto USB-C que tiene integrado. Posteriormente presionaremos el botón de reset ubicado en la parte inferior, que se muestra en la Figura A.10 durante 5 segundos hasta que el LED parpadee de verde a rojo.

Figura A.10: Botón de reset

Una vez que parpadee de verde a rojo debemos presionar durante 10 segundos el botón de setup ubicado en la parte superior, que se muestra en la Figura A.11. El gateway comenzará a parpadear en rojo.

Figura A.11: Botón Setup del gateway

Tras esto el gateway se mostrará como punto de acceso WiFi disponible llamado MINHUB-xxxxxx (donde xxxxxx es el GatewayEUI) y nos debemos de conectar a este con la contraseña del gateway que viene en la ficha técnica (valor 2 en Figura A.9). Cuando estemos conectados a la red accederemos a esta dirección 192.168.4.1 en cualquier navegador y configuramos la red WiFi que deseemos para proporcionar conectividad al gateway. Una vez completado marcamos la opción “Save and Reboot” y el dispositivo estará listo para usar. En la Figura A.12 podemos ver

la pantalla de configuración que se nos muestra cuando accedemos a la dirección 192.168.4.1.

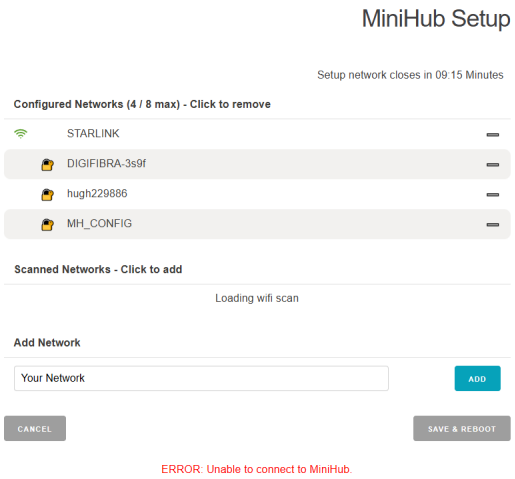
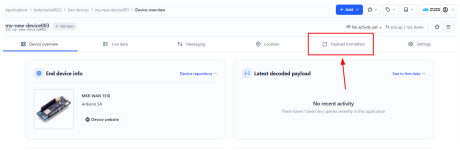


Figura A.12: Pantalla configuración Wi-Fi - Minihub

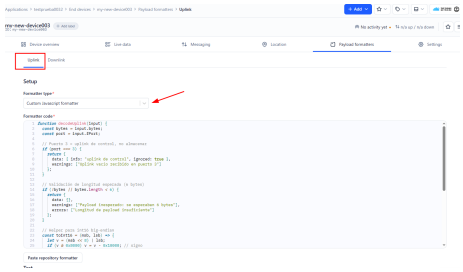
Una vez configurado el gateway las credenciales disponibles en credenciales.h en el código del microcontrolador, estarán disponibles para que el dispositivo se valide en el servidor de TTN mediante autenticación OTAA y comience el envío de uplinks y recibo de uplinks.

A.3.2.5. Configuración de los payloads formatter

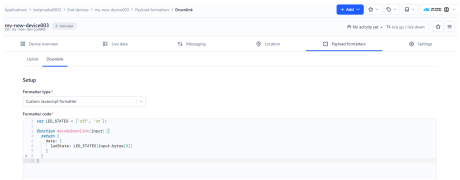
Para realizar este proceso simplemente debemos seguir los pasos de la Figura A.13, en la que desde la consola de TTN accedemos a la sección del dispositivo IoT y configuramos la función de decodificación de mensajes Uplink y Downlinks.



(a) Seleccionar payload formatter



(b) TTN uplink payload formatter

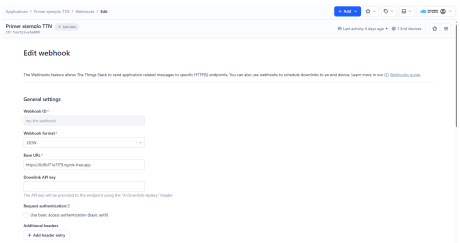


(c) TTN downlink payload formatter

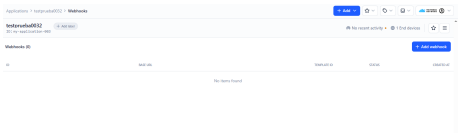
Figura A.13: Configurar payload formatters Uplinks/Downlinks

A.3.2.6. Configuración del webhook

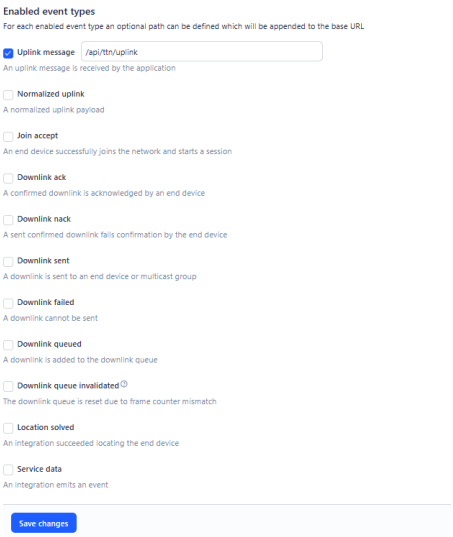
Para configurar el webhook partimos desde la consola de TTN y accedemos directamente a la funcionalidad webhook, donde eligiremos “Custom webhook” y añadiremos uno personalizado. El proceso de configuración se puede ver en la Figura A.14



(a) Registrar webhook



(b) Configuración del webhook



(c) Añadir API para uplink message

Figura A.14: Dar de alta un webhook

En la subfigura A.26a vemos el formulario destinado a la configuración del webhook. Estos campos son:

1. **Webhook ID:** Identificador único para el webhook (ej: sistema-riego-vivantia)
2. **Webhook format:** Formato de los datos a enviar, típicamente JSON.
3. **Base URL:** Se trata del dominio de servidor web, donde se reenviarán las lecturas de los sensores
4. **Filter event data:** Filtros para seleccionar qué eventos específicos se enviarán:
   - up.uplink_message.decoded_payload - Datos decodificamos de las lecturas de los sensores

- end_device_ids - credenciales del dispositivo que envía las lecturas de los sensores
- up.uplink_message.f_port - Puerto hacia donde se envían las lecturas de los sensores

5. **Enabled event types:** Tipos de eventos habilitados, como Uplink message dónde configuraremos el endpoint HTTP hacia donde TTN enviará una petición HTTP POST para enviar las lecturas de los sensores en formato JSON al servidor web. Con esto especificamos la ruta a la que dirigiremos la información

Una vez completado todos los pasos anteriormente comentados el módulo 2 estará completamente instalado.

A.3.3. Módulo 3.1: Cloud Computing - Recepción y post-procesamiento

El módulo 3 tendría que estar desplegado en cloud pero como en este proyecto se va ha trabajado en local y no se tiene previsto poner el proyecto en producción al ser un proyecto con fines educativos no es necesario desplegarlo en cloud. El usuario tendrá que realizar los siguientes pasos para desplegar el proyecto en local:

1. Descargar código desde el repositorio del servidor web
2. Instalar dependencias del servidor web a través de npm
3. Ejecutar el servidor web en local a través npm
4. Asignar un dominio al servidor web temporal usando la herramienta ngrok [70]
5. Asignar las variables de entorno necesarias para ejecutar el servidor web

A.3.3.1. Descargar código del servidor web desde repositorio

En este paso simplemente lo que tendremos que hacer es irnos al repositorio github del servidor web [67] y descargar el código desde ahí, para luego descomprimirlo en nuestro ordenador portátil o PC. Este proceso puede observarse en la Figura A.15

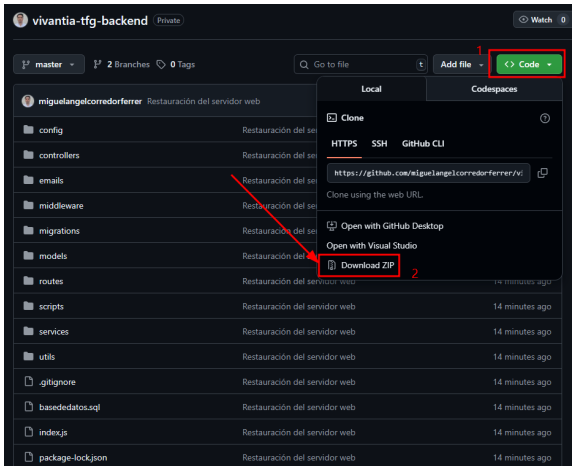
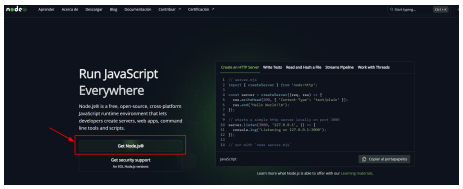


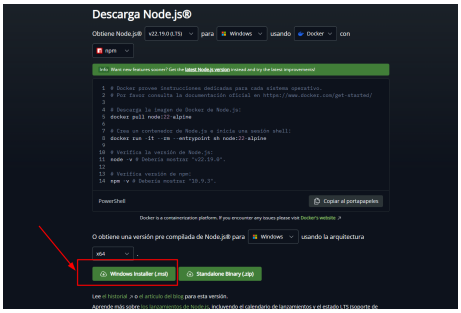
Figura A.15: Descarga del código del servidor web

A.3.3.2. Instalar dependencias del servidor web

Para hacer uso del servidor web una vez descomprimido el directorio, tendremos que abrir una terminal e instalar las depedencias del servidor web a través del gestor de paquetes npm. No sin antes instarla Node.js desde la página oficial [42]. El proceso de descarga de Node.js se puede observar en la Figura A.16



(a) Instalar Nodejs



(b) Botó de descarga para window de Nodejs

Figura A.16: Proceso de descarga de Node.js

Una vez instalado Node.js, tendremos disponible el gestor de paquetes npm, con el que, podremos instalar todas las dependencias del servidor web. Para ello debemos de dirigirnos a la carpeta donde se encuentre el servidor web y abrir una terminal sobre esta y ejecutar el siguiente comando.

```
npm install
```

Listing A.1: Instalación de dependencias con npm

Puede tardar un tiempo puesto que se tienen que instalar todas las dependencias, el resultado final será el mostrado en la Figura A.17

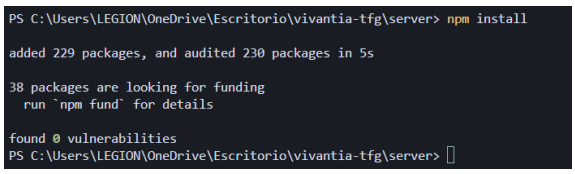


Figura A.17: Instalar dependencias del servidor web

A.3.3.3. Ejecutar el servidor web en local a través de npm

Una vez instaladas las dependencias, estamos listos para ejecutar el servidor web. Para ello, desde la terminal que se ha abierto anteriormente en el directorio del servidor web, vamos a ejecutar el siguiente comando

```
npm run dev
```

Listing A.2: Ejecución del servidor web

Al ejecutar este comando el servidor web se desplegará en *localhost* en el puerto *3000*. Con ello el servidor web quedará totalmente funcional y disponible para conexión con el cliente. Puede observarse este comportamiento en la Figura A.18

Figura A.18: Despliegue del servidor web en localhost

A.3.3.4. Asignar un dominio temporal al servidor web

Cómo vimos en la sección 6.2.3 es necesario que el servidor web disponga de un dominio a la hora de configurar el webhook. Debido a que TTN no puede realizar peticiones HTTP POST a un dominio como es el nuestro caso localhost que solo forma parte de nuestra propia computadores y no es accesible desde el exterior. Para ello haremos uso de la herramienta ngrok [70] que nos ayudará a asignar un dominio temporal a nuestro servidor web reemplazando `http://localhost:3001/`. En primer lugar, nos registraremos en la plataforma de ngrok. El proceso de registro puede verse en la Figura A.19.

(a) Pantalla de inicio de ngrok (b) Registro con cuenta de github en ngrok

Figura A.19: Proceso de registro en ngrok

En la Figura A.20 podemos ver los 2 pasos que hay que hacer para asignar un dominio temporal con ngrok. El primero es introducir el siguiente comando para generar el token de autenticación de ngrok, único para cada usuario. Y el segundo comando permite generar el dominio temporal para el *localhost*.

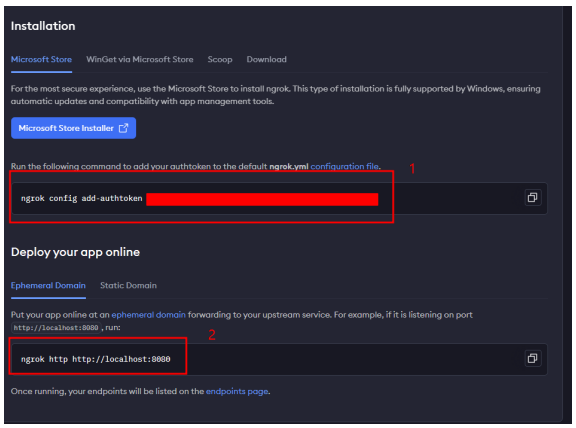


Figura A.20: Asignar dominio temporal al servidor web

Para asignar el dominio temporal en nuestro caso tenemos que desde la misma terminal que abrimos anteriormente, introducir el siguiente comando:

```
ngrok http http://localhost:3001/
```

Listing A.3: Generar dominio para el servidor web

En la Figura A.21 se puede ver el dominio temporal asignado.

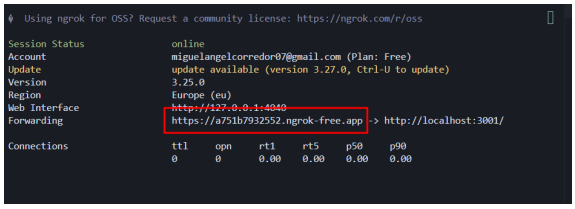


Figura A.21: Asignación del dominio temporal

Hay que recordar que cada vez que el servidor se apague o deje de ejecutarse por alguna razón hay que volver a asignar un dominio temporal.

A.3.3.5. Asignar las variables de entorno necesarias

Para que nuestro servidor web funcione correctamente, es necesario configurar las variables de entorno. En el servidor web habrá 6 variables de entorno. La primera variable entorno es la URL del frotend o el cliente a través del cual el servidor web o también backend recibirá solicitudes HTTP, ante estas solicitudes, el servidor web generará respuestas al cliente. Las otras 4 variables de entorno son utilizadas en la configuración del servicio de email, y por último, la variable de entorno *JWT_SECRET* es la clave secreta destinada a cifrar la cifrar la sesión y la autenticación del usuario.

- **FRONTEND_URL:** `http://localhost:3000`
- **EMAIL_HOST:** sandbox.smtp.mailtrap.io
- **EMAIL_PORT:** 2525
- **EMAIL_USER:** b5a898786729e1
- **EMAIL_PASS:** 7b59b7481f060b
- **JWT_SECRET:** jwt_secret_key

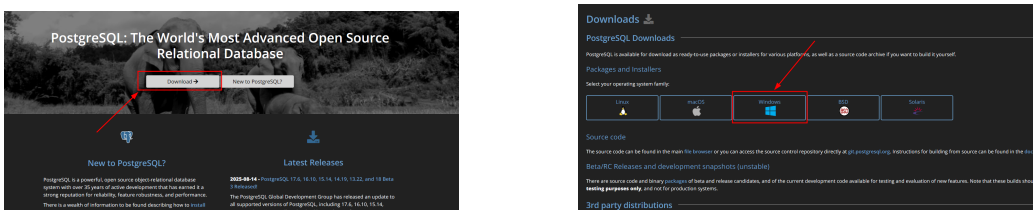
A.3.4. Módulo 3.2: Cloud Computing - Almacenamiento de datos

Para la instalación y configuración del almacén de datos, se deberá de seguir los siguientes pasos:

1. Descargar e instalar PostgreSQL junto con PgAdmin en su última versión
2. Crear server en local para alojar la base de datos
3. Creación de la base de datos de Vivantia
4. Creación de las tablas de la base de datos

A.3.4.1. Descargar e instalar PostgreSQL junto con PgAdmin en su última versión

En este paso descargaremos el sistema gestor de base de datos PostgreSQL [26] para gestionar la base de datos del sistema y además una herramienta gráfica llamada PgAdmin[71], que va a permitir gestionar la base de datos mediante una interfaz visual intuitiva. En la Figura A.22 se describe el proceso de instalación de PostgreSQL.



(a) Botón de descarga de PostgreSQL (b) Descargar PostgreSQL para window

Figura A.22: Proceso de descarga de PostgreSQL

Una vez descargado PostgreSQL, tras aceptar los términos y terminar el proceso de instalación. PgAdmin vendrá instalado por defecto, y simplemente tendremos que acceder él. La interfaz de PgAdmin puede verse en la Figura A.23.

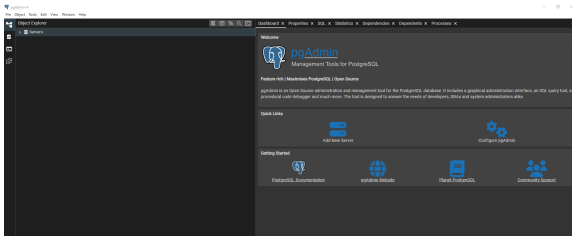
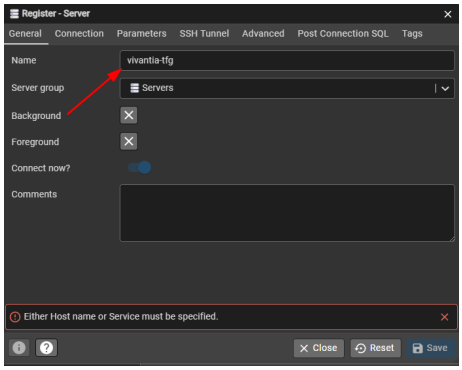


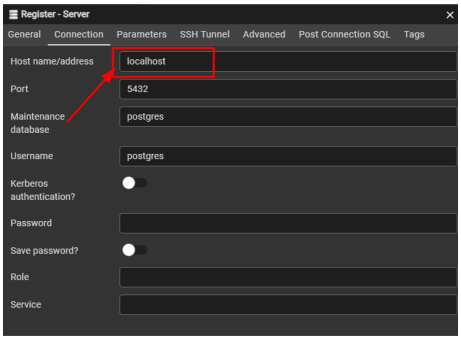
Figura A.23: Pantalla de inicio de PgAdmin

A.3.4.2. Crear server en local para alojar la base de datos

En la pantalla de inicio de PgAdmin, habrá una entidad llamada *Server*. A esta entidad le daremos click derecho *Register* → *Server*. Para crear el server donde alojar la base de datos, se deben de introducir las credenciales de la Figura A.26:



(a) Registrar servidor en PostgreSQL



(b) Registra conexión del servidor en PostgreSQL

Figura A.24: Proceso de creación del server en PostgreSQL

A.3.4.3. Creación de la base de datos de Vivantia

Una vez creado el servidor, vamos a crear la base de datos de Vivantia que se va a alojar en él. Para ello daremos click derecho en servidor creado *Create* → *Database* para luego crear la base de datos en base a una serie de credenciales como puede verse en la Figura A.25,

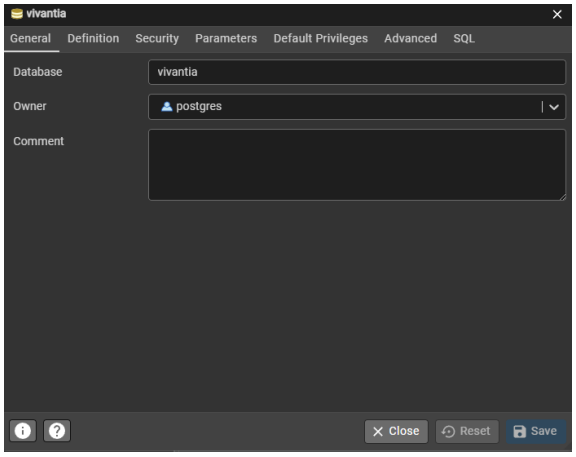
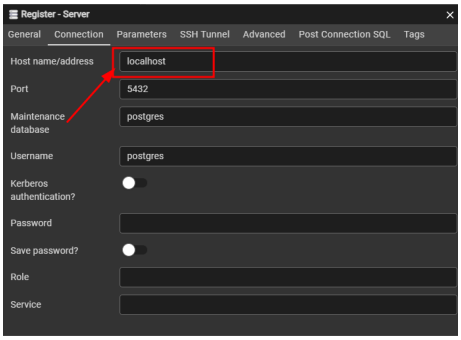


Figura A.25: Crear base de datos en el server

A.3.4.4. Creación de las tablas de la base de datos

Por último, una vez creada la base de datos, vamos a ejecutar el script para la creación de tablas de la base de datos. Para ello le daremos click derecho a la base de datos creada y pulsaremos *CREATE SCRIPT*. Se abrirá un fichero donde copiaremos el script de la base de datos disponible en el respositorio del servidor web A.15. Posteriormente ejecutaremos el script y las tablas quedarán correctamente implementadas. El proceso de creación de las tablas en la base de datos puede observarse en la Figura A.27

(a) Descargar script de base de datos



(b) Registra conexión del servidor en PostgreSQL

Figura A.26: Proceso de creación del server en PostgreSQL

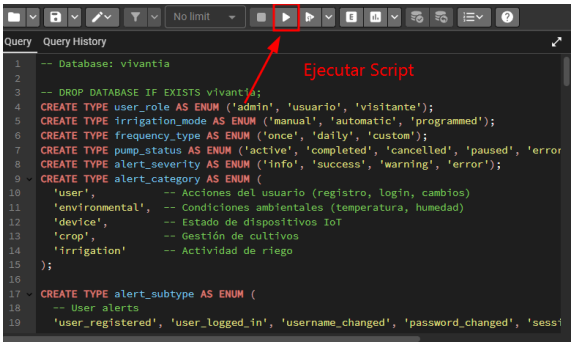


Figura A.27: Crear base de datos en el server

Finalmente la base de datos quedaría configurada de la forma en la que puede observarse en la Figura A.28

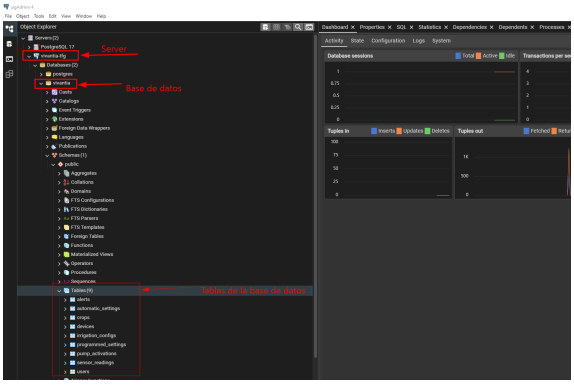


Figura A.28: Crear base de datos en el server

A.3.4.5. Registrar administrador del sistema

Tras crear las tablas en la tabla usuario se debe de insertar un usuario administrador por defecto para que se puedan acceder a las funcionalidades del mismo, a través del rol de administrador el usuario puede realizar cualquier operación sobre los usuarios, dispositivos y cultivos del sistema. Además de contar con un log del sistema donde se informará de todos los eventos acontecidos en el sistema por parte del propio administrador y de los usuarios verificados. Como este proyecto no ha sido desplegado en cloud ya que no va a pasar a producción, al instalar el sistema por primera vez se deberá de registrar un usuario con las siguientes credenciales:

- **name:** admin
- **email:** admin@vivantia.com

- **password:** adminVivantia

Una vez registrado el usuario, se debe de actualizar su rol a "admin" con el siguiente script

```
UPDATE users
SET role = 'admin'
WHERE name = 'admin';
```

Listing A.4: Actualizar rol de usuario a admin

De esta manera estará disponible el usuario administrador del sistema y se podrá hacer uso de sus funcionalidades.

A.3.5. Módulo 4: Interfaz Web - Visualización

La configuración e instalación del módulo 4 es prácticamente igual e incluso más sencillo que la del módulo 3.2, visto en la sección A.3.3. Como se comentó en esta sección como no se tiene previsto pasar a producción el proyecto, el cliente o frontend del proyecto se desplegará el local. Para desplegar el cliente en local, el usuario tendrá que realizar los siguientes pasos:

1. Descargar el código desde el repositorio del frontend del proyecto
2. Instalar dependencias del servidor web a través de npm
3. Ejecutar el servidor web en local a través de npm
4. Asignar las variables de entorno necesarias para ejecutar el frontend

A.3.5.1. Descargar el código desde el repositorio del frontend

El primer paso, será acceder al repositorio del frontend [68]. Una vez accedido al repositorio del frontend debemos de descargar el código, para luego descomprimirlo en nuestro ordenador portátil o PC. Este proceso puede verse en la Figura A.30

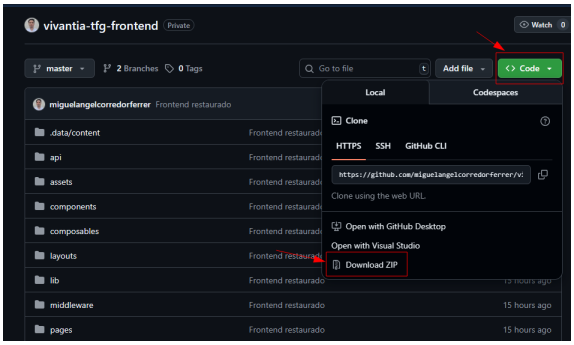


Figura A.29: Descargar del código del frontend

A.3.5.2. Instalar dependencias del frontend

Al haber instalado en la sección A.3.3.2 Node.js ya tendremos disponible el gestor de paquetes npm, por lo que, únicamente tendremos que abrir una terminal sobre el directorio descomprimido y ejecutar el siguiente comando.

```
npm install
```

Listing A.5: Instalación de dependencias con npm

Puede tardar un tiempo puesto que se tienen que instalar todas las dependencias, el resultado final será el mostrado en la Figura

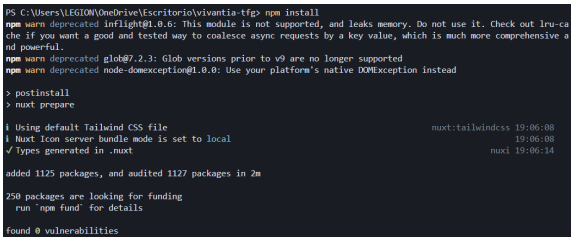


Figura A.30: Instalar dependencias del servidor web

A.3.5.3. Ejecutar el frontend en local a través de npm

Una vez instaladas las dependencias, estamos listos para ejecutar el servidor web. Para ello, desde la terminal que se ha abierto anteriormente en el directorio del frontend, vamos a ejecutar el siguiente comando

```
npm run dev
```

Listing A.6: Ejecución del frontend del proyecto

Al ejecutar este comando el frontend se desplegará en *localhost* en el puerto *3001*. Con ello el frontend quedará totalmente funcional y disponible para conectarse con el servidor web. Puede observarse este comportamiento en la Figura A.32

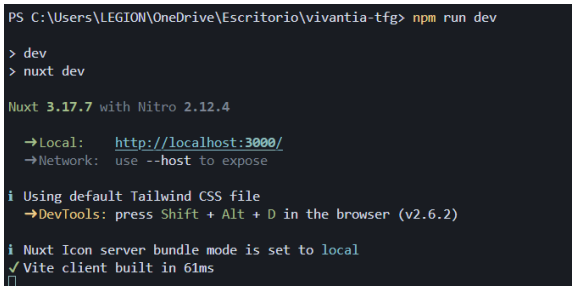


Figura A.31: Frontend ejecutado

A.3.5.4. Asignar las variables de entorno necesarias

Para que nuestro servidor web funcione correctamente, es necesario configurar las variables de entorno. En el frontend únicamente habrá una variable de entorno. Esta variable de entorno guarda la ruta hacia la API'S disponibles en el servidor web.

- **NUXT_PUBLIC_API_URL:** `http://localhost:3001/api`

Finalmente el frontend quedará totalmente configurado y listo para utilizarse, accediendo a `http://localhost:3000/`

Figura A.32: Pantalla renderizada en el frontend

A.4. Desinstalación Usuario Técnico

A.4.1. Módulo 1: Edge Computing - Captación de datos y acción sobre el actuador

La desinstalación del módulo 1 es bastante simple básicamente tendremos que restaurar el microcontrolador Arduino para que no se reinicie con nuestro programa por defecto, para ello la única manera existente consiste en cargar un programa vacío como el de la Listing A.7. De esta manera el microcontrolador volverá a restaurarse de fábrica.

```
void setup() {
}

void loop() {
}
```

Listing A.7: Programa vacío de Arduino

Por otra parte, tenemos que reiniciar el gateway con valores de fábrica (sin acceso a redes WiFi), para ello simplemente presionamos el botón de *reset* de la Figura A.10.

A.4.2. Módulo 2: Fog Computing - Recolección y preprocesamiento

A.4.2.1. Eliminación del dispositivo dado de alta en TTN

Para eliminar el dispositivo o dispositivos de la plataforma TTN, basta con acceder a la consola y en el apartado de información de cada uno de los dispositivos dados de alta, seleccionar la opción de eliminarlos. Tal y como se indica en la Figura A.33

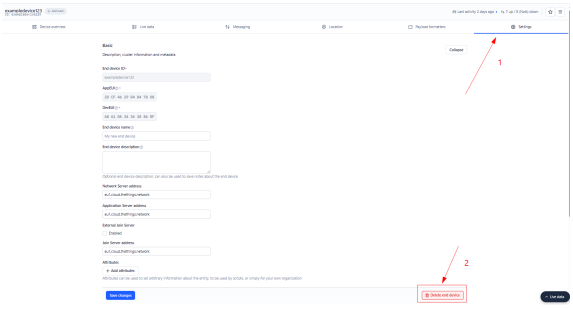


Figura A.33: Desinstalar dispositivos dados de alta

A.4.2.2. Eliminación del gateway dado de alta en TTN

Para la desinstalación del gateway simplemente nos dirigiremos a la sección de Gateway, seleccionamos nuestro gateway registrado en TTN y en la esquina superior derecha habrá un botón de ajustes donde se abrirá un desplegable con la opción de eliminar el gateway, como se puede ver en la Figura4 A.34

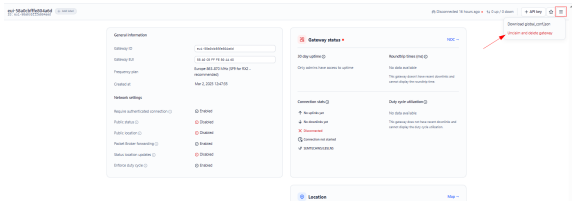


Figura A.34: Desinstalarel gateway de TTN

A.4.2.3. Eliminación de los payloads formatters

Para eliminar los payloads formatters simplemente tendremos que acceder a cada uno y donde ponga Formatter type ponerlo en none. De esta manera no habrá ningún payload formatter disponible. Este proceso puede verse en la Figura A.13

Figura A.35: Desinstalar payloads formatters

A.4.2.4. Eliminación del webhook

Para eliminar el webhook simplemente accederemos a la configuración del mismo y en la propia configuración habrá un botón que indica la posibilidad de borrarlo. Puede observarse más claramente en la Figura A.36

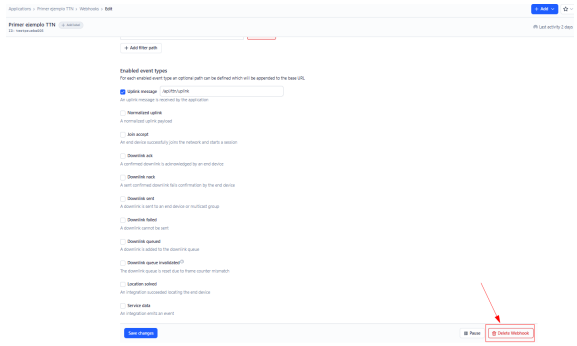


Figura A.36: Desinstalar webhook

A.4.2.5. Eliminación de cuenta de TTN

Por último, para eliminar la cuenta dada de alta en TTN, accederemos al perfil del usuario y los ajustes del perfil de usuario habrá un botón para eliminar la cuenta de usuario. Puede observarse más claramente en la Figura A.37

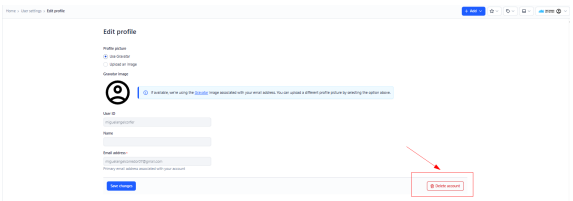


Figura A.37: Desinstalar cuenta de usuario

A.4.3. Módulo 3.1: Cloud Computing - Recepción y post-procesamiento

La desinstalación del móduo 3.1 es bastante sencilla. Para desinstalar Nodejs simplemente el usuario se dirigirá al panel de control del sistema operativo Window y desinstalar Nodejs. Si la máquina tiene un sistema operativo Linux se deberá de introducir los siguientes comandos

```
sudo apt remove nodejs
sudo apt purge nodejs
sudo apt autoremove
```

Listing A.8: Desinstalar Nodejs

Por otro lado, para cancelar la ejecución del servidor web, simplemente nos dirigiremos a la terminal desde donde lo hemos lanzado y pulsaremos *Ctrl+C* para cancelar la ejecución.

A.4.4. Módulo 3.2: Cloud Computing - Almacenamiento de datos

Para la desinstalación del módulo 3.2, simplemente ejecutaremos un script de eliminación de la base de datos localizado en el repositorio del servidor web [67].

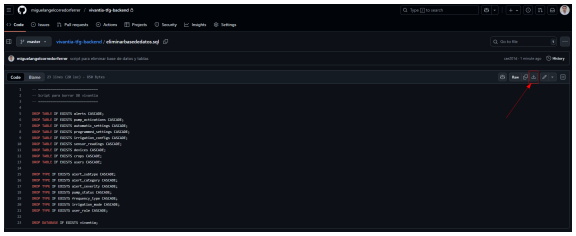


Figura A.38: Descargar el script para la eliminación de la base de datos y sus tablas

Una vez ejecutado este script en PgAdmin, el módulo 3.2 o almacén de datos quedará totalmente desinstalado.

A.4.5. Módulo 4: Interfaz Web - Visualización

Habiendo desinstalado previamente Nodejs tal y como se ha indicado en la sección A.4.3. Para cancelar la ejecución del frontend se tiene que seguir el mismo proceso que para cancelar la ejecución del servidor web. Nos dirigiremos a la terminal donde se ha lanzado el frontend y pulsaremos *Ctrl+C* para cancelar la ejecución.

A.5. Guía Usuario Final

Esta sección del manual está especialmente dirigida al usuario final que hará uso del sistema completo. Se asume que este ha sido previamente configurado por un usuario técnico y está listo para su uso.

A.5.1. Preparación y verificación del sistema

A.5.1.1. Verificación inicial del sistema

Antes de comenzar para comenzar a tomar lecturas de los sensores y que el sistema esté listo para recibir ordenes de riego, asegúrese de que el sistema esté completamente preparado:

1. El dispositivo IoT debe estar alimentado:

- Compruebe que la batería portátil está conectada al dispositivo.
- Compruebe que el dispositivo muestra signos de actividad (LED del microcontrolador encendido, LED del relé encendido)
- Asegúrese de que todos los cables de alimentación y componentes están firmemente conectados.
- Compruebe que el LED del microcontrolador con la etiqueta de CHRG parpadee repetidamente en un tono rojo. Esto indica que el dispositivo está alimentado

2. El gateway tiene conexión a internet
   - Puede estar conectado a su red WiFi doméstica
   - Puede usar datos móviles
   - Verifique que el LED del gateway indique conexión exitosa
3. La interfaz web está activa y podemos acceder:
   - Verificar que se cuenta con conexión a internet
   - Verificar que se puede acceder a la aplicación web, en caso contrario contactar con un usuario técnico.

Se entiende que el usuario técnico se puso en contacto con el final para configurar esta parte, en caso contrario se deben seguir los pasos referentes a la configuración del gateway explicados en la sección de instalación y configuración para el usuario técnico.

A.6. Uso del módulo 1 para la gestión de riego y captación de datos de los sensores

A.6.1. Estado inicial del módulo 1

El sistema hardware que forma el módulo 1 estará continuamente en funcionamiento mientras esté alimentado, la secuencia de inicialización es la siguiente:

1. **Encendido inicial:** Al conectar la batería portátil al módulo 1
2. **Autodiagnóstico:** El dispositivo IoT del módulo 1 mostrará LED amarillo durante 30-60 segundos mientras:
   - Se establece conexión con la red LoRaWAN

- Se verifica la comunicación con el gateway
- Se diagnóstica y verifica todos los sensores internos

3. **Estado operativo:** Una vez que el dispositivo IoT del módulo 1 haya completado todas las verificaciones, el LED cambiará a verde fijo
4. **Listo para tomar lecturas de sensores y recibir ordenes de riego**: El sistema hardware del módulo 1 está completamente preparado para realizar captura de lecturas de datos y recibir ordenes de riego a través de la red LoRaWAN

Al inicializar el dispositivo pueden ocurrir situaciones inesperadas, para ello es importante comprender el funcionamiento de los indicadores LED en la fase de inicialización del dispositivo IoT del módulo 1.

- **LED verde fijo:** Dispostivo IoT listo para captar lecturas de sensores y recibir ordenes de riego - Sistema hardware inicializado correctamente
- **LED CHRG no parpadea:** Esto puede indicar un error en la alimentación o que el dispositivo IoT recibe alimentación pero no la suficiente para mantener en funcionamiento el sistema hardware completo - Consulte la sección de resolución de problemas
- **LED amarillo prolongado (aproximadamente 2 minutos)** Problema de conectividad - verifique el gateway
- **Sin LEDS:** Problema de alimentación - verifique conexiones y carga de batería

A.6.2. Realización de mediciones de temperatura y humedad con los sensores

Para realizar mediciones de temperatura ambiental, humedad ambiental y humedad del suelo, primeramente el usuario se tiene que dirigir a un lugar de cultivo, ya sea un jardín, huerto o cualquier otro espacio destinado a la agricultura con la caja donde se encuentra implementado todo el sistema hardware. Posteriormente el usuario debe de enterrar en el suelo el sensor capacitive soil moisture para que este comience a tomar mediciones correctamente. Por otro lado el sensor DHT11 estará colocado en el exterior de la caja tomando mediciones. No hará falta intervención del usuario para su configuración.

A.6.2.1. Ejecución del riego

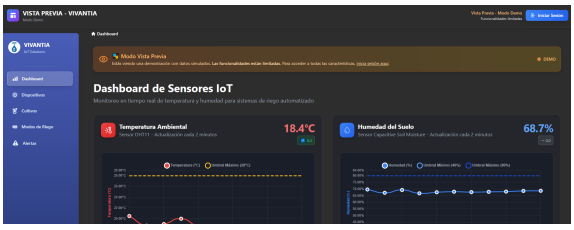
Para la ejecución efectiva del riego, el usuario deberá de colocar la tubería de extracción de agua de la bomba de riego en un recipiente con agua y la tubería de expulsión de agua en el espacio donde se quiera regar y donde esté colocado el capactive soil moisture. Es necesario tener en cuenta separar la tubería donde se expulsará el agua a una distancia de aproximadamente 20-30 centímetros del capacitive soil moisture, puesto que de lo contrario, estaría continuamente húmedo y tomaría lecturas imprecisas. También las tuberías de riego han sido correctamente etiquetadas, la tubería más larga sirve para extraer el agua de un recipiente y ha sido etiquetada como "Absorver" y la tubería más corta sirve para expulsar el agua, esta ha sido etiquetada como "Expular". Estas son visibles en cada una de las tuberías.

A.7. Uso de la interfaz web

A.7.1. Acceder a demo del sistema (Visitante)

Si usted es un usuario, cuyo propósito es simplemente previsualizar las funcionalidades del sistema para saber si los servicios proveídos por Vivantia le son útiles y por tanto decidir registrarse en el sistema. Se le proporciona una funcionalidad demo a la cual se accede desde la página de autenticación, sin necesidad de autenticarse en el sistema. Se accede a ella a través de "Vista previa" y al acceder a la página demo se mostrará la dashboard del sistema cargada con datos simulados. Para acceder al resto de funcionalidades el usuario debe de registrarse en el sistema.

(a) Botón para acceder a la Demo



(b) Pantalla Demo cargada con datos simulados

Figura A.39: Demo del sistema

A.7.2. Acceso a gestión de usuarios (Administrador)

Cunado usted se loguee en el sistema como usuario, se le redirigirá automáticamente a la página "Gestión de Usuarios" donde tendrá acceso a todos los usuarios del sistema y podrá realizar cualquier acción sobre ellos, modificar información del usuario, eliminarlo, añadir nuevos usuarios y editarlos. Se le proporciona un filtro para facilitar la búsqueda de usuarios y guías para saber cuántos usuarios hay registrados en el sistema, cuántos administradores y cuántos usuarios se han filtrado. Las funcionalidades pueden verse en la Figura A.40

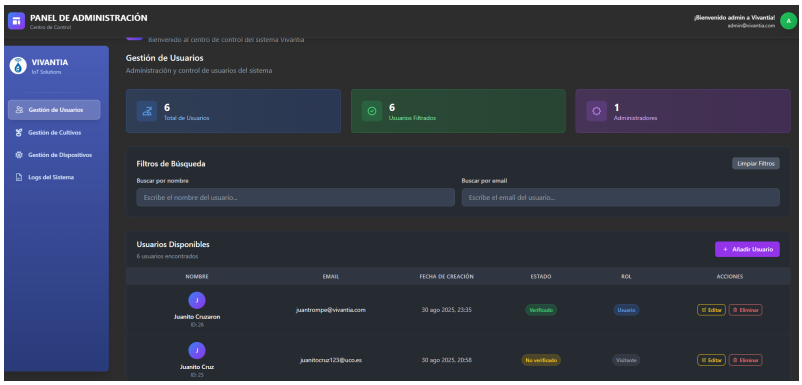


Figura A.40: Gestión de usuarios (Administrador)

A.7.3. Acceso a gestión de cultivos (Administrador)

En la página de gestión de cultivos tendrá acceso a todos los cultivos registrados en el sistema por los usuarios. Podrá realizar cualquier acción sobre ellos, modificar información del cultivo, añadir cultivo, eliminarlo, seleccionarlo y ver su información. Se proporciona un filtro para facilitar la búsqueda de cultivos y guías para saber cuántos cultivos hay registrados en el sistema, cuántos cultivos hay seleccionados actualmente y cuántos cultivos han sido filtrados para búsqueda. Las funcionalidades pueden verse en la Figura A.41.

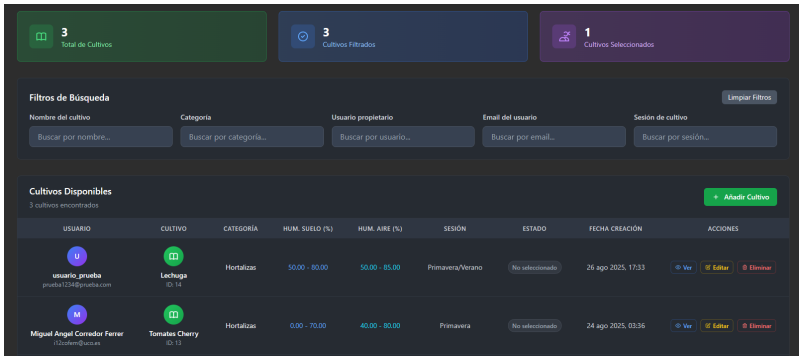


Figura A.41: Gestión de cultivos (Administrador)

A.7.4. Acceso a gestión de dispositivos (Administrador)

En la página de gestión de dispositivos se tendrá acceso a todos los dispositivos registrados en el sistema por los usuarios. Podrá realizar cualquier acción sobre ellos, modificar información del dispositivo, añadir dispositivo, eliminarlo y activar/desactivar comunicación a la base de datos. Se proporciona un filtro para facilitar la búsqueda de cultivos y guías para saber cuántos dispositivos hay actualmente registrados en el sistema, cuántos dispositivos hay activos y cuántos dispositivos se han filtrado para búsqueda. Las funcionalidades pueden verse en la Figura A.42.

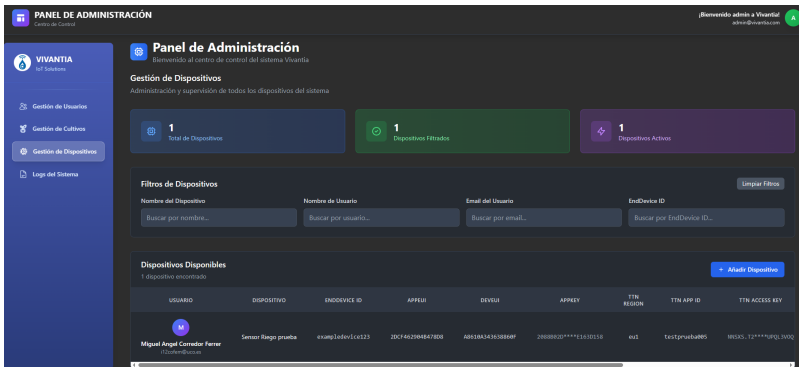


Figura A.42: Gestión de dispositivos (Administrador)

A.7.5. Acceso a logs del sistema (Administrador)

Esta página tiene como objetivo mostrar en forma de notificación todos los eventos provocados por el propio administrador y por los usuarios registrados. Usted como administrador podrá ver que tipo de alerta se trata y a que usuario está asociada,

podrá ver que alertas no han sido resueltas, resolverlas y ademas podrá eliminar todas o eliminar una alerta en particular. Se le proporciona guías para indicar cuantas alertas ha generado el sistema, cuáles son críticas y filtros para buscar una alerta en concreto o grupos de alertas. En la esquina superior izquierda de cada alerta puede observar un punto naranja parpadeante que indica que esa alerta aun no ha sido resuelta. Puede observarse las funcionalidades de esta página en la siguiente Figura A.43

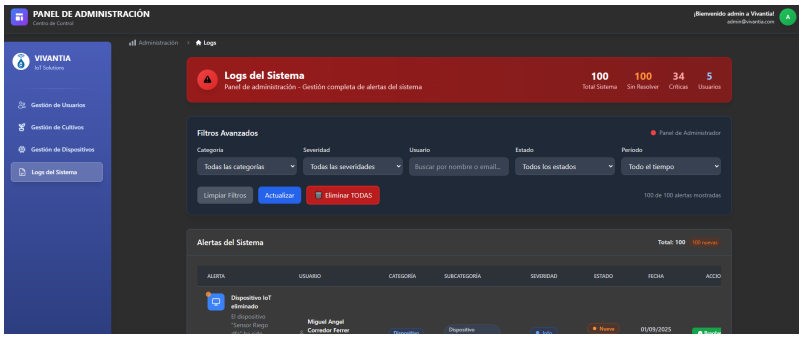


Figura A.43: Logs del sistema (Administrador)

A.7.6. Acceso y autenticación

Acceda a la URL del proyecto `http://localhost:3000/` desde cualquier dispositivo con navegador web y conexión a internet. A continuación se mostrará el panel de autenticación del sistema, representado en la Figura A.44.

Figura A.44: Panel de autenticación

Se le presentan varias opciones de autenticación, si usted quiere ingresar el sistema, en primer lugar debe de registrarse en el sistema y posteriormente autenticarse.

Para ello seleccionará la opción de registro, representado en la Figura A.45

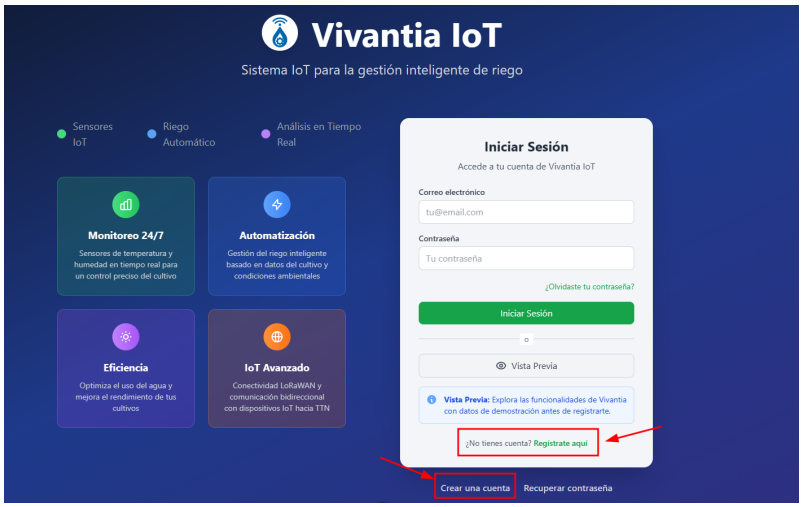


Figura A.45: Panel de autenticación - Acceso al registro

Una vez seleccionada esta opción se abrirá un formulario, donde usted tendrá que poner sus datos para darse de alta en el sistema, como se ve en la Figura A.46

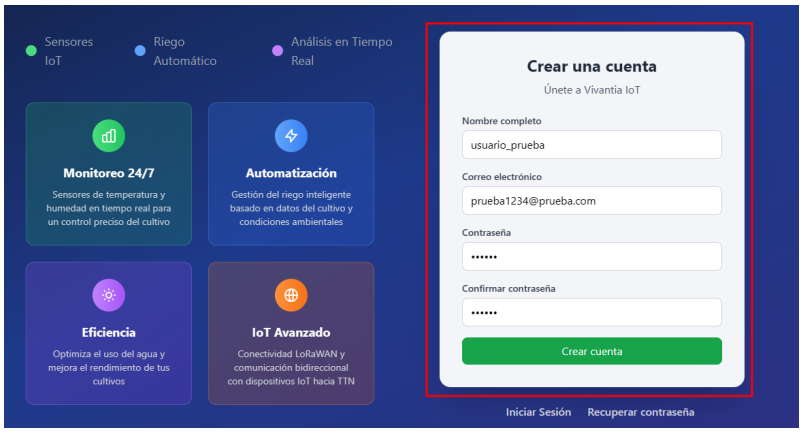


Figura A.46: Panel de autenticación - Formulario de registro

Una vez registrado en el sistema, le aparecerá el siguiente panel informativo donde se informará de que usted debe de revisar su bandeja de entrada de su correo electrónico para verificar su cuenta.

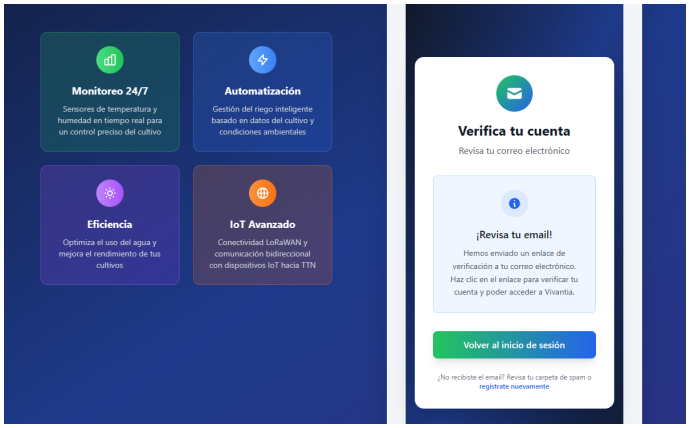


Figura A.47: Panel de autenticación - Revise su email

Al revisar su correo deberá de recibir un email con un botón para confirmar su cuenta y se redirigirá al panel de autenticación, una vez que ya esté verificado. Podrá acceder al sistema logueandose

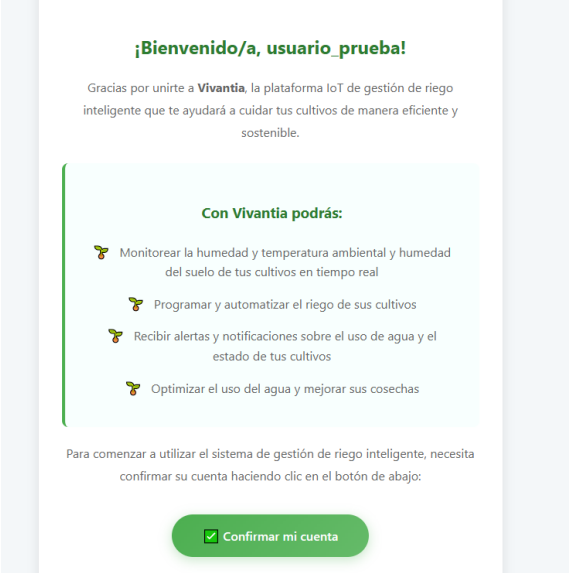
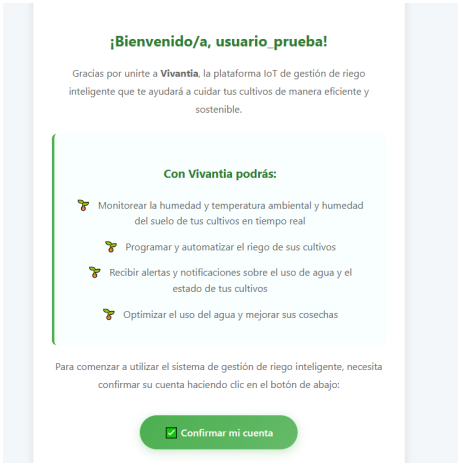
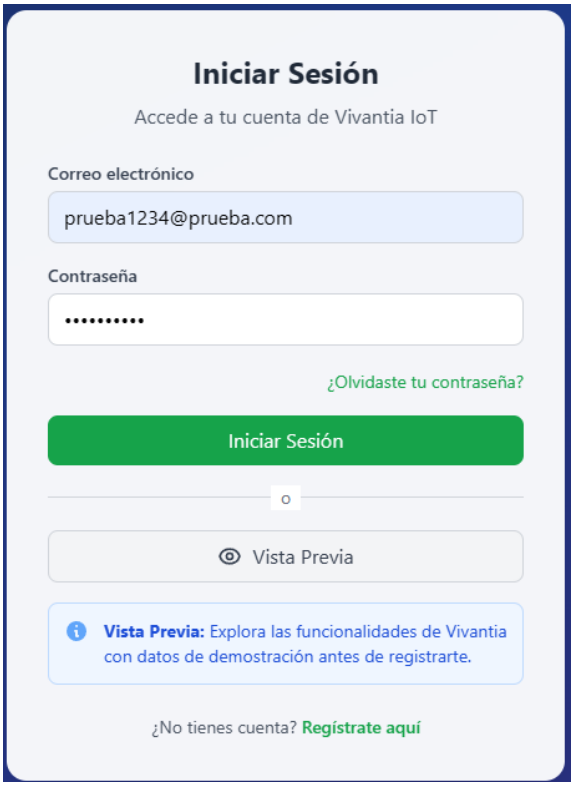


Figura A.48: Email de verificación de cuenta



(a) Email de verificación de cuenta

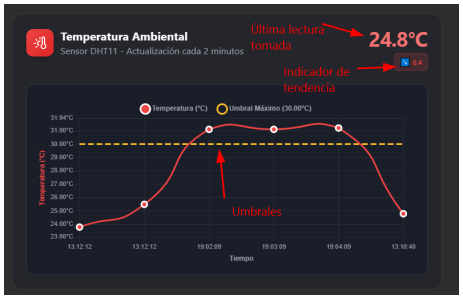


(b) Login en el sistema

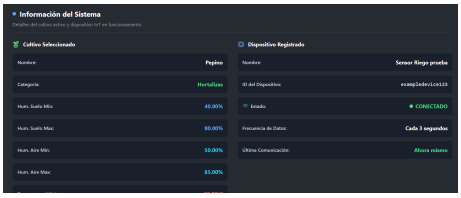
Figura A.49: Proceso de login

Una vez su cuenta ya ha sido verificada y se ha logueado, a la primera página que se redirigirá será a la de dashboard. En esta página se mostrarán las gráficas que representarán la evolución de los datos de temperatura ambiental, humedad ambiental y humedad del suelo tomados por el dispositivo IoT, en el caso de que se hayan tomado lecturas sino estarán vacías. Además se mostrarán paneles de informativos sobre cultivo actualmente seleccionado, dispositivo cuya comunicación este actualmente activa y también información varia como el estado de la bomba de riego o la última vez que usted inicio sesión en el sistema. En las gráficas puede ver varios elementos, en primer lugar en la esquina superior derecha se puede apreciar un indicador del último valor tomado por el dispositivo IoT para ese parámetro. Por debajo de este, se encuentra un indicador de tendencia, que indica cuanto a subido o bajado el último valor tomado para ese parámetro con respecto a su valor anterior. Por último vemos la gráfica trazada con los valores que van llegando al sistema para ese parámetro junto con una línea discontinua horizontal que representan los umbrales. Los umbrales son aquellos valores límites que indican, en el caso de temperatura y humedad ambiental, si las condiciones del suelo van a cambiar

próximamente. Mientras que los umbrales de humedad del suelo indican si el cultivo debe regarse o no.



(a) Componentes de una gráfica



(b) Paneles de información sobre cultivo seleccionado y dispositivo activo

Figura A.50: Dashboard

A.7.7. Gestión de dispositivos

En la página "Dispositivos" usted puede registrar en el sistema el dispositivo IoT que previamente ha dado de alta en TTN. Introduzca todas las credenciales y pulse añadir dispositivo.

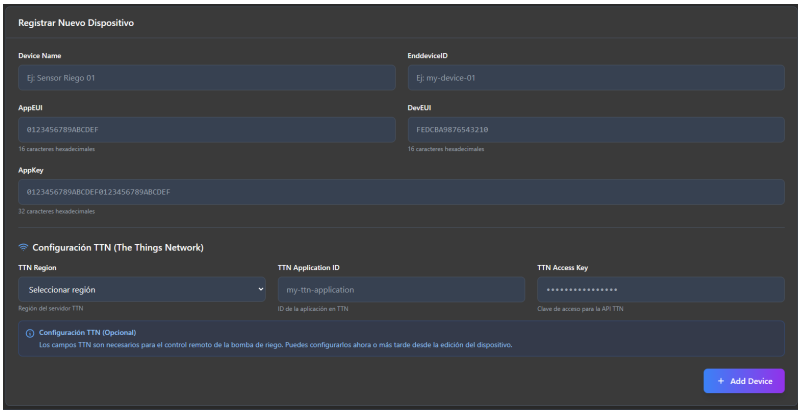


Figura A.51: Formulario para el registro de un dispositivo en el sistema

En la parte inferior de la página puede ver que se mostrarán en forma de tabla los dispositivos que usted ha registrado en el sistema, acompañado de una funcionalidad para filtrar dispositivos, en caso de que cuente con múltiples dispositivos y quiera encontrarlos de forma sencilla.

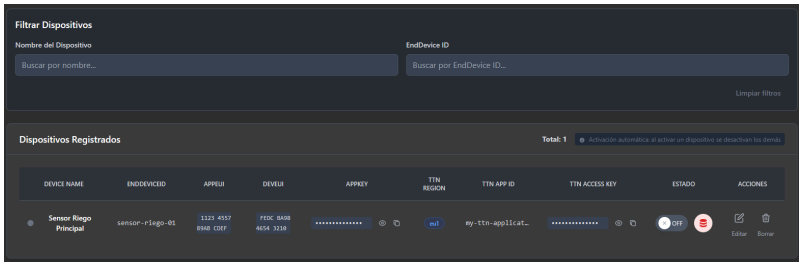


Figura A.52: Filtrar Dispositivos y Dispositivos Registrados

En cada uno de los dispositivos registrados, el usuario puede llevar a cabo varias acciones como son activar la comunicación del dispositivo lo que implicaría dejar de permitir que las lecturas de los sensores tomadas por el dispositivo IoT se almacenaran en la base de datos. Se le permite también modificar las credenciales del dispositivo IoT y eliminarlo del sistema.

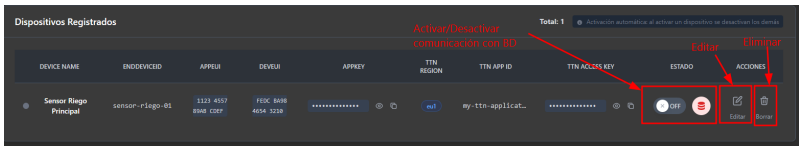
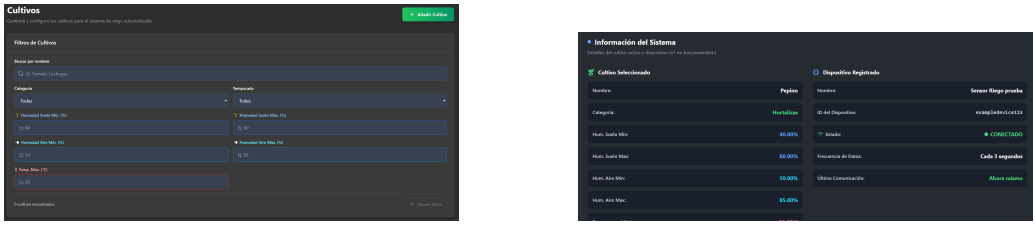


Figura A.53: Operaciones sobre los dispositivos registrados

A.7.8. Gestión de cultivos

En la página “Cultivos” usted puede registrar los cultivos que tenga considerados regar, pero únicamente se podrá seleccionar un cultivo de entre todos los que ha registrado. En primer lugar puede ver un panel de filtros con el cual puede usted filtrar cultivos que optarán para riego o sobre los cuales quiera hacer alguna operación en concreto como eliminar, ver información relacionada a este o incluso editarlo. Junto con el panel de filtros en la parte superior derecha se puede ver un botón “Añadir Cultivo” que tendrá como próposito registrar un cultivo en el sistema. Al pulsarlo se abrirá un formulario donde usted introducirá los datos de ese cultivo.



(a) Filtros de cultivo y añadir cultivo (b) Formulario para registrar cultivo en el sistema

Figura A.54: Filtrar y añadir cultivos

Una vez registrado el cultivo podemos ver en la parte inferior de la página de "Cultivos" puede ver que se mostrarán en forma de tabla todos los cultivos disponibles en el sistema con toda la información asociada a estos. Por otro lado, por cada cultivo registrado se podrán realizar una serie de acciones, como seleccionar el cultivo que se va a regar, editar información del cultivo y eliminarlo.

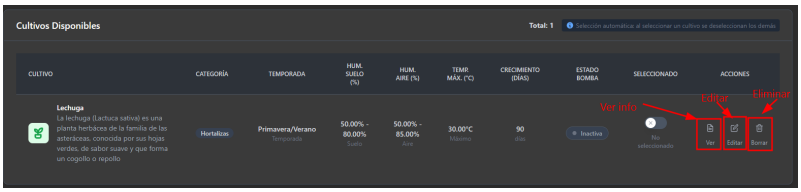


Figura A.55: Operaciones sobre cultivos registrados

A.7.9. Modos de riego

En esta página es donde puede encontrar la funcionalidad principal del sistema, se divide en 2 partes. La parte superior de la página consta de un conjunto de paneles informativos que sirven como guía para comprobar el modo actual seleccionado, el dispositivo activo, el cultivo en riego etc...

En la parte inferior se encuentran los 3 modos a elegir. Como puede verse en la Figura A.60

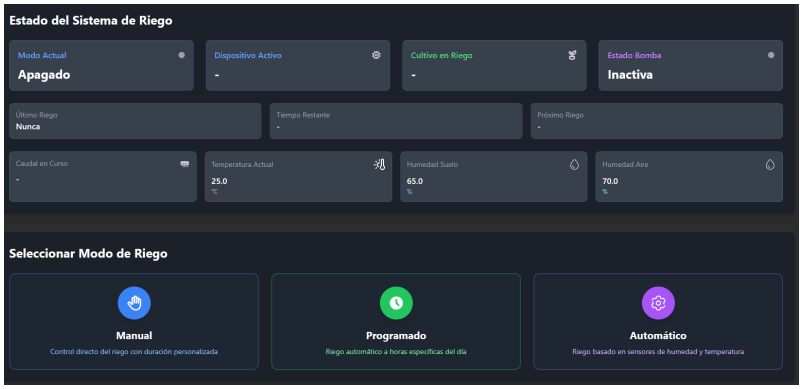
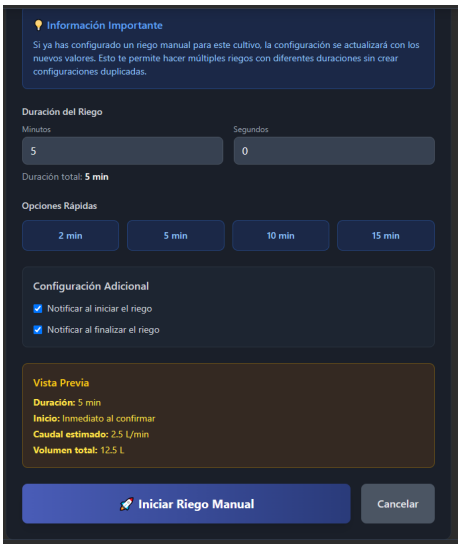
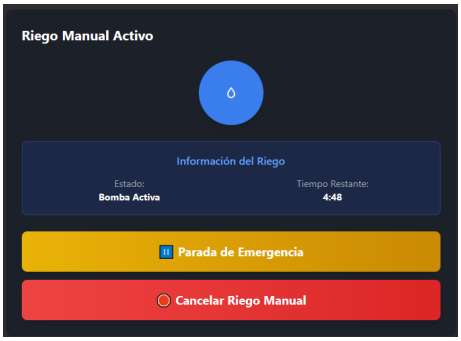


Figura A.56: Modos de riego

Los 3 modos de riego puede comprobar, que funcionan de manera similar pero con pequeños matices. El modo manual es el más simple de todos básicamente bastaría con ajustar el tiempo de duración de riego y pulsar iniciar riego manual. Una vez que se inicie el riego puede cancelar el riego o pausarlo/reanudarlo.



(a) Configurar modo manual

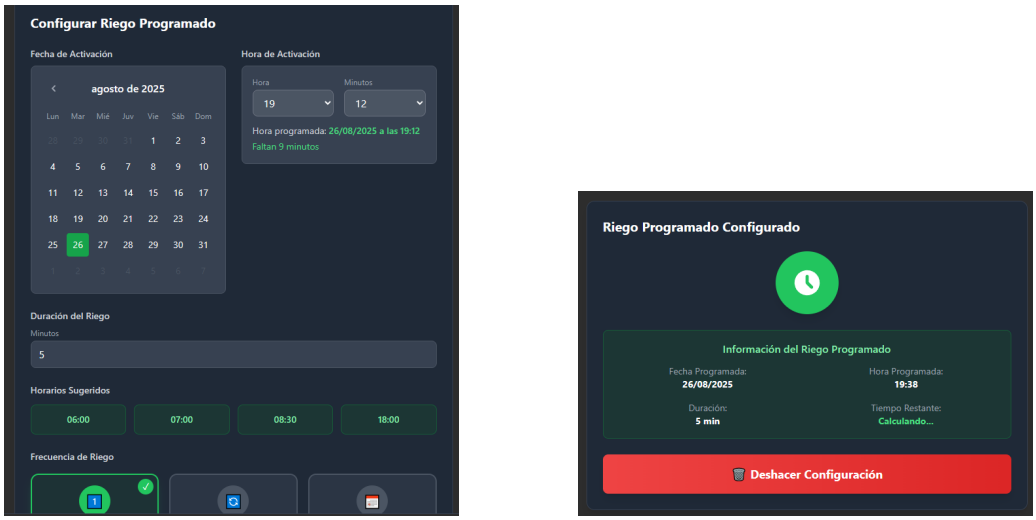


(b) Pausar/reanudar y cancelar riego

Figura A.57: Modo de riego Manual

Con respecto a las imágenes es necesario aclarar varios aspectos, si usted elige un modo de riego hasta que no se cancele el riego o no cancele la configuración (programado o automático) no se podrá elegir otros modos. Por otro lado el panel de activación de riego es el mismo para el resto de modos. Con este se podrá pausar/reanudar y cancelar riego.

En el modo programado, usted podrá programar el riego a una hora y fecha específica, por lo que guardará la configuración y se esperará al momento del riego. Si es necesario puede cancelar la configuración antes de que llegue la fecha de riego y se desbloquearían el resto de modos.



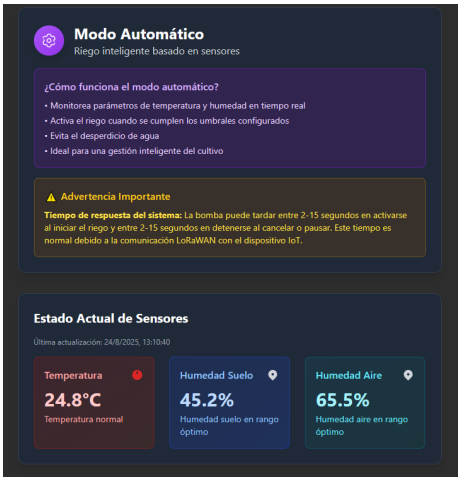
(a) Configurar modo de riego programado (b) Deshacer configuración de riego

Figura A.58: Modo de riego programado

Por último, el modo automático funciona exactamente igual que el modo programado. La diferencia es que las lecturas de los sensores que van llegando desde el dispositivo IoT se van comparando con los umbrales de humedad del suelo del cultivo seleccionado. Para que el riego se efectúe o por el contrario se pare, se deben de cumplir ciertas condiciones. Estas condiciones son las siguientes:

- Para que el riego se efectúe, Se debe de cumplir la siguiente condición
  - Humedad suelo $\leq$ mínimo
- Para que el riego se pare, se debe de cumplir una de estas condiciones
  - Humedad del suelo en rango óptimo
  - Humedad >máximo

A modo adicional, si el usuario requiere cambiar los umbrales de la humedad de riego, puede editarlos yendo a la página de Cultivos y pulsando “Editar” en el cultivo correspondiente.



(a) Configurar modo automático

(b) Modo automático esperando que se cumplan condiciones de riego

Figura A.59: Modo de riego automático

A.7.10. Gestión de alertas

Como puede ver la página de "Alertas" es bastante sencilla, cuenta con una serie de filtros, para filtrar las alertas que considere necesarias y tres botones. Un botón para limpiar filtros, otro botón para Marcar todo como resuelto y limpiar todas las alertas antiguas. Cuando reciba una alerta esta no estará resuelta y se marcará con un punto naranja parpadeante en la esquina superior izquierda de la alerta. Además de marcarse como Nueva en el estado de la alerta. Si lo considera necesario puede marcar individualmente una alerta como resuelta o borrarla individualmente

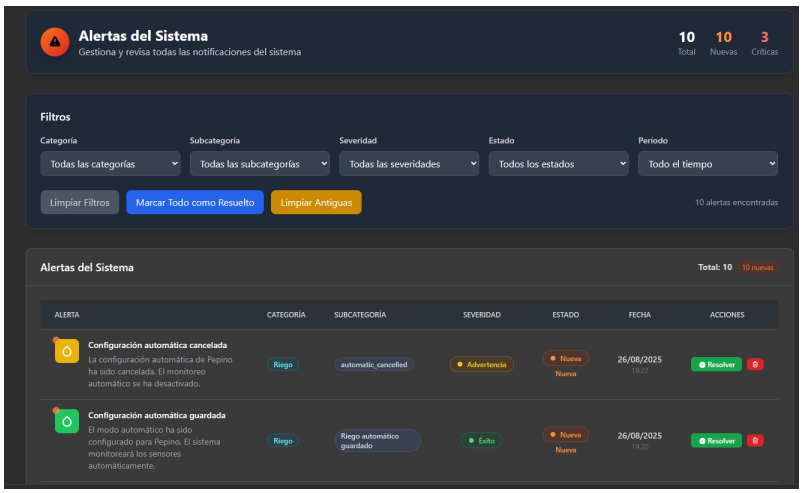


Figura A.60: Alertas

A.8. Resolución de Problemas Comunes

A.8.1. Problemas con el módulo 1

Tabla A.1: Problemas comunes del módulo 1

Problema	**Causa probable**	**Solución**
Dispositivo no enciende ningún LED	Batería agotada	Recargue la bateria portátil o verifique conexiones
LED CHRC no enciende	Batería defectuosa	Cambie la batería o consulte al usuario técnico
LED amarillo prolongado al inicio	Gateway sin conexión	Verifique conexión Wi-Fi del gateway
LED's del relé apagados	Conexiones incorrectas o relé defectuoso	Cambie el relé o consulte al usuario técnico
Bomba de riego no enciende	Fallo en las conexiones o bomba defectuosa	Consulte al usuario técnico
Valores de los sensores erráticos	Sensores defectuosos o conexiones incorrectas	Consulte al usuario técnico o en el caso del sensor de humedad de suelo cambie de terreno

A.8.2. Problemas con la interfaz web

Tabla A.2: Problemas comunes del módulo 4

Problema	**Causa probable**	**Solución**
No se puede acceder al sistema	No se verificó la cuenta	Mire en la bandeja de entrada de su correo electrónico
Al cambiar de una cuenta a otra se solapa contenido	No se han limpiado correctamente los datos	Recargue la página con F5 o cambie de página y vuelva a entrar
Contraseña rechazada	Error de escritura	Verifique que se cumple con el formato correcto
No se muestran las lecturas de los sensores en la dashboard	Fallo de autenticación del dispositivo	Verifique que ha introducido las credenciales correctas del dispositivo IoT dado de alta en TTN y que la API Key no esté caducada
Bomba de riego tarda en encender	Lentitud al desencolar el downlink	Espere unos segundos hasta que se desencole el downlink enviado cuando llegue un uplink periódico

Anexo B

Manual de código

Debido a que el código implementado del proyecto cuenta con una gran cantidad de líneas de código, explicar lo que hace cada fichero provocaría que el documento fuera excesivamente extenso, por lo que, en las próximas secciones simplemente se mostrarán los repositorios github donde se almacena el código del proyecto. Además se dará una explicación de que hace cada directorio del repositorio y se puntualizará ciertos aspectos importantes a tener en cuenta.

B.1. Repositorio github del servidor web

La implementación en código del servidor web se encuentra en el siguiente repositorio de Github:

`https://github.com/miguelangelcorredorferrer/vivantia-tfg-backend` [67]

La estructura de los directorios del servidor web viene descrita en el siguiente diagrama

B.1.1. Estructura de directorios del backend

```
vivantia-backend/
├── config/ ..... Configuración de base de datos y servicio de email
├── controllers/ ..... Controladores para la gestión de las API REST
├── emails/ ..... Plantillas de correo electrónico
├── middleware/ ..... Interceptores de peticiones y protección de rutas
├── models/ ..... Modelos de datos
├── node_modules/ ..... Dependencias de Node.js
├── routes/ ..... Definición de rutas API REST
├── services/ ..... Servicios de lógica de negocio
├── utils/ ..... Funciones auxiliares
├── basededatos.sql ..... Script de creación de base de datos
├── eliminarbasededatos.sql ..... Script de eliminación de BD
├── index.js ..... Fichero principal para la ejecución del servidor web
├── package-lock.json ..... Archivo de bloqueo de dependencias
├── package.json ..... Configuración del proyecto Node.js
└── tsconfig.json ..... Configuración de TypeScript
```

Figura B.1: Estructura de directorios del servidor web

Es necesario destacar que en cada uno de estos directorios habrá una explicación detallada de que hace cada fichero en un fichero *README*.Por otro lado, si se requiere una explicación más extensa del código del repositorio explicado con diagramas y demás elementos visuales, se recomienda acceder a la siguiente URL

`https://deepwiki.com/miguelangelcorredorferrer/vivantia-tfg-backend`

Al sustituir en la URL *github* por *deepwiki*, se generará mediante inteligencia artificial una documentación extensa y bien fundamentada con diagramas, del código del repositorio del servidor web.

B.2. Repositorio github del cliente o frontend

La implementación en código del servidor web se encuentra en el siguiente repositorio de Github:

`https://github.com/miguelangelcorredorferrer/vivantia-tfg-frontend` [68]

La estructura de los directorios del frontend o cliente viene descrita en el siguiente diagrama

B.2.1. Estructura de directorios del frontend

```
vivantia-tfg/
├── api/ ..... Llamadas a las API's definidas en el servidor web o backend
├── assets/ ..... Recursos de estilos de las paginas
├── components/ ..... Componentes Vue reutilizables
├── composables/ ..... Lógica reutilizable
├── layouts/ ..... Plantillas de estructura de páginas
├── lib/ ..... Librerías y utilidades auxiliares
├── middleware/ ..... Autenticación y control de acceso
├── pages/ ..... Sistema de enrutamiento automático de páginas
├── plugins/ ..... Extensiones de funcionalidad
├── public/ ..... Recursos públicos
├── static/ ..... Recursos estáticos
├── stores/ ..... Gestión de estado global con Pinia
├── utils/ ..... Funciones auxiliares y helpers
├── eslint.config.mjs ..... Configuración de ESLint
├── nuxt.config.ts ..... Configuración del metaframework
├── package-lock.json ..... Archivo de bloqueo de dependencias
├── package.json ..... Configuración del proyecto y dependencias
├── postcss.config.js ..... Configuración de PostCSS
├── tailwind.config.js ..... Configuración de Tailwind CSS
└── tsconfig.json ..... Configuración de TypeScript
```

Figura B.2: Estructura de directorios del cliente del sistema completo

Es necesario destacar que en cada uno de estos directorios habrá una explicación detallada de que hace cada fichero en un fichero *README*. Por otro lado, si se requiere una explicación más extensa del código del repositorio explicado con diagramas y demás elementos visuales, se recomienda acceder a la siguiente URL

`https://deepwiki.com/miguelangelcorredorferrer/vivantia-tfg-frontend`

Al sustituir en la URL *github* por *deepwiki*, se generará mediante inteligencia artificial una documentación extensa y bien fundamentada con diagramas, del código del repositorio del frontend.

B.3. Repositorio github del microcontrolador

La implementación en código del microcontrolador se encuentra en el siguiente repositorio de Github:

`https://github.com/miguelangelcorredorferrer/vivantia-tfg-microcontrolador` [59]

El repositorio del microcontrolador únicamente contendrá un directorio compuesto por un conjunto de subdirectorios creados por defecto al crear el proyecto en PlatformIO. El código estará disponible en el subdirectorio *src*.

Al sustituir en la URL *github* por *deepwiki*, se generará mediante inteligencia artificial una documentación extensa y bien fundamentada con diagramas, del código del repositorio del microcontrolador.

`https://deepwiki.com/miguelangelcorredorferrer/vivantia-tfg-microcontrolador`